



Università degli Studi di Napoli
FEDERICO II

A.A 2023-2024

BASI DI DATI 1

INDICE

0 INTRODUZIONE	9
1 INTRODUZIONE ALLE BASI DI DATI	10
1.1 Tipi di database e applicazioni	10
1.2 Cos'è un database?	10
1.3 Funzionalità dei sistemi di basi di dati	11
1.3.1 Natura autodescrittiva di un sistema di basi di dati	11
1.3.2 Isolamento tra programmi e dati, astrazione dei dati	11
1.3.3 Supporto per viste multiple dei dati	11
1.3.4 Condivisione dei dati e processo transazionale	11
2 ARCHITETTURA DI UN DATABASE	13
2.1 Astrazione e modelli	13
2.1.1 Categorie di modelli di dati	13
2.2 Altre tipologie di modelli dei dati	14
2.3 Schemi, istanze, stato di una base di dati	14
2.4 Architettura a tre livelli	14
2.4.1 Indipendenza dei dati	15
2.5 I linguaggi del DBMS	15
3 LA PROGETTAZIONE CONCETTUALE	17
3.1 Il ciclo di vita del Software	17
3.1.1 Progettazione e ciclo di vita	17
3.2 Definizione del problema	17
3.3 Il modello Entità - Relazioni	18
3.3.1 Le entità	18
3.3.2 Gli attributi	19
3.3.3 Le relazioni e associazioni	20
3.3.4 Cardinalità degli attributi	22
3.3.5 Generalizzazioni	22
3.4 Il modello UML	23
3.4.1 Le classi	23
3.4.2 Le associazioni	23
3.4.3 Molteplicità	24
3.4.4 Generalizzazioni	25
3.4.5 Aggregazione e composizione	26
4 IL MODELLO RELAZIONALE	27
4.1 Concetti del modello relazionale	27
4.2 Definizione del modello relazionale	27
4.2.1 Caratteristiche delle relazioni	28
4.3 Vincoli e schemi	28
4.3.1 Vincoli di tupla e di dominio	29
4.3.2 Vincoli di chiave o di valore NULL	29
4.3.3 Schemi del database relazionale	29
4.3.4 Vincoli di integrità e chiavi esterne	29
5 LA PROGETTAZIONE LOGICA	31

5.1	Ristrutturazione del modello concettuale	31
5.1.1	Analisi delle ridondanze	31
5.1.2	Eliminazione delle generalizzazioni	34
5.1.3	Scelta degli identificatori principali	38
5.2	Traduzione verso il modello relazionale	38
5.2.1	Mapping dei tipi di entità regolari	38
5.2.2	Codifica delle associazioni	38
5.2.3	Sommario delle regole di mapping	40
5.3	Documentazione a supporto della progettazione	40
5.3.1	Il dizionario delle classi	40
5.3.2	Il dizionario delle associazioni	41
5.3.3	Il dizionario dei vincoli	41
5.3.4	Il dizionario delle interrogazioni	41
6	ALGEBRA E CALCOLO RELAZIONALE	43
6.1	Operazioni relazionali unarie: selezione e proiezione	43
6.1.1	L'operazione di selezione	43
6.1.2	L'operazione di proiezione	45
6.1.3	Sequenze di operazioni e operazione di ridenominazione	46
6.2	Operazioni insiemistiche dell'algebra relazionale	47
6.2.1	Le operazioni di unione, intersezione e differenza	47
6.2.2	L'operazione di prodotto cartesiano	47
6.3	Operazioni relazionali binarie	48
6.3.1	L'operazione di Theta Join	48
6.3.2	Le operazioni di Join Naturale	49
6.3.3	Operazioni di Join esterno	49
6.4	Operazioni relazionali aggiuntive	50
6.4.1	Funzioni aggregate e raggruppamento	50
6.4.2	Operazioni di chiusura ricorsiva	50
6.4.3	Un insieme completo di operazioni dell'algebra relazionale	51
6.5	Esempi di interrogazioni in algebra relazionale	51
7	LO STANDARD SQL	52
7.1	Tipi di dati e definizione dei dati in SQL	52
7.1.1	Il concetto di schema	52
7.1.2	Il comando CREATE TABLE in SQL	52
7.1.3	Domini e tipi di dati per gli attributi in sql	53
7.2	Specificazione dei vincoli di base	54
7.2.1	Specificazione di vincoli e di valori predefiniti per gli attributi	54
7.2.2	Vincoli di chiave e di integrità referenziale	55
7.2.3	Specificazione di vincoli sulle tuple	56
7.3	Istruzioni di modifica degli schemi in SQL	56
7.3.1	Il comando DROP	57
7.3.2	Il comando ALTER	57
8	INTERROGAZIONI FONDAMENTALI IN SQL	59
8.1	La struttura SELECT-FROM-WHERE	59
8.1.1	Assegnazione di alias	60
8.1.2	Utilizzo dell'asterisco	60
8.2	Le tabelle come insiemi	60
8.2.1	Operazioni insiemistiche	60
8.3	La clausola WHERE	61
8.3.1	La clausola WHERE	61
8.3.2	Confronti di sottostringhe e operatori aritmetici	61
8.3.3	IN e BETWEEN	62
8.4	Gestire i risultati delle interrogazioni	62
8.4.1	Ordinare i risultati: la clausola ORDER BY	62
8.4.2	Funzioni di aggregazione in SQL	63
8.4.3	Le clausole HAVING e GROUP BY	64
9	QUERY AVANZATE: LE SUBQUERIES	65
9.1	Interrogazioni nidificate	65
9.1.1	Le clausole ANY e ALL	66

9.1.2	SELECT innestate nella clausola FROM	67
9.2	Inferrogazioni nidificate correlate	67
9.2.1	Quantificazione esistenziale: la funzione EXISTS	67
10	JOIN E PRODOTTO CARTESIANO IN SQL	70
10.1	La clausola JOIN	70
10.1.1	La clausola NATURAL JOIN	71
10.1.2	Le operazioni di OUTER JOIN	71
10.2	Il cross join o prodotto cartesiano	71
10.2.1	Il self-join	72
11	DML: DATA MANIPULATION LANGUAGE	74
11.1	Il comando INSERT	74
11.2	Il comando DELETE	75
11.3	Il comando UPDATE	75
12	ASSERZIONI E TRIGGER	77
12.1	Le asserzioni	77
12.1.1	Sintassi delle asserzioni	77
12.2	I trigger	78
12.2.1	Parametri di un trigger	78
12.2.2	Modifica dei Trigger	79
12.2.3	La clausola BEFORE	80
12.2.4	La clausola INSTEAD OF	80
13	LINGUAGGI PROCEDURALI PER SQL: ORACLE PL/SQL	82
13.1	Introduzione	82
13.2	Struttura di base di un blocco PL/SQL	82
13.2.1	Dichiarazione delle variabili	84
13.3	Blocco di un programma	85
13.3.1	Assegnazioni ed espressioni	85
13.3.2	Costrutti condizionali	86
13.3.3	Costrutti iterativi	86
13.4	Blocchi non anonimi: procedure e funzioni	88
13.4.1	Creare una procedura in PL/SQL	88
13.4.2	Creare una funzione in PL/SQL	89
13.5	Uso dei cursori	90
13.5.1	Dichiarazione dei cursori	90
13.5.2	Cursori semplici: schemi di iterazione	90
13.5.3	Cursori vincolati	91
13.6	Gestione delle eccezioni	91
13.6.1	Eccezioni di sistema	91
13.6.2	Eccezioni utente	92
13.7	SQL dinamico	93
13.7.1	Modalità di interazione	94
13.7.2	Prepare ed Execute	94
13.7.3	Execute Immediate	94
13.7.4	Cursori per l'SQL dinamico	96
13.8	Tabelle di sistema e gestione degli errori	97
14	INTRODUZIONE A POSTGRESQL	98
14.1	I tipi di dato In PostgreSQL	98
14.1.1	Tipi booleani	98
14.1.2	Caratteri	98
14.1.3	Numerici	99
14.1.4	Tipi di dati temporali	99
14.2	Amministrare un database PostgreSQL	100
14.2.1	Comandi principali psql	100
14.2.2	Tabelle di sistema	102
14.3	Procedimenti utili in PostgreSQL	103
14.3.1	Come confrontare due tabelle	103
14.3.2	Come generare un numero randomico	104

14.4 Funzioni in PostgreSQL	105
14.4.1 Funzioni standard	105
14.4.2 Funzioni per la gestione delle stringhe	106
14.4.3 Funzioni per la gestione delle date	106
15 LINGUAGGI PROCEDURALI PER SQL: POSTGRES PL/PgSQL	108
15.1 Introduzione	108
15.1.1 Il dollar quoting	108
15.1.2 Struttura dei blocchi procedurali	109
15.1.3 Sottoblocchi	109
15.2 Variabili e costanti	110
15.2.1 Variabili	110
15.2.2 Costanti	113
15.3 Statement di base	113
15.3.1 L'istruzione SELECT INTO	113
15.3.2 Non fare nulla	114
15.4 Strutture di controllo	115
15.4.1 If statement	115
15.4.2 Loop statement	116
15.4.3 While statement	117
15.4.4 For loop	117
15.4.5 Exit	119
15.4.6 Continue	119
15.5 Funzioni user-defined	120
15.5.1 L'istruzione Create Function	120
15.5.2 Modalità di passaggio dei parametri	122
15.5.3 Funzioni che restituiscono una tabella	123
15.5.4 Eliminare una funzione	123
15.5.5 L'istruzione Create Procedure	124
15.6 Cursori	124
15.6.1 Gestione dei cursori	124
15.6.2 Aprire un cursore	125
15.6.3 Usare i cursori	126
15.6.4 Muovere il cursore	127
15.6.5 Cancellare o aggiornare un cursore	127
15.6.6 Chiudere un cursore	127
15.6.7 Restituire un cursore	128
15.7 Gestione delle eccezioni	128
15.7.1 Lanciare errori e riportare messaggi	128
15.7.2 La clausola exception	129
15.8 Funzioni triggers	130
15.8.1 Introduzione ai trigger PostgreSQL	130
15.8.2 L'istruzione Create Trigger	130
15.8.3 L'istruzione Drop Trigger	134
15.8.4 L'istruzione Alter Trigger	134
15.9 SQL dinamico con PL/pgSQL	134
15.9.1 L'istruzione Execute	134
15.9.2 Esecuzione di istruzioni DDL in SQL dinamico	136
15.9.3 Eseguire istruzioni DML in SQL dinamico	136
15.9.4 Il retrieve dinamico	136
15.9.5 Il sort dinamico	137
15.10 Transazioni in PL/pgSQL	137
15.10.1 Cos'è una transazione?	137
15.10.2 Iniziare una transazione	138
15.10.3 Committare una transazione	138
15.10.4 Ripristinare una transazione	139
15.10.5 Usare le transazioni all'interno delle procedure	139
15.11 Porting da Oracle PL/SQL	140
15.11.1 Le differenze tra PL/SQL e PL/pgSQL	140
16 ESERCIZI ED APPROFONDIMENTI	142
16.1 Algebra relazionale	142
16.2 Esercizi SQL	147
16.2.1 Attività sportive studentesche	147

16.2.2	Rappresentanti e fatture ai clienti	150
16.2.3	Interrogazioni semplici	154

FRAMMENTI DI CODICE

■ Codice 7.1	Esempio di utilizzo della clausola <code>CREATE SCHEMA</code>	52
■ Codice 7.2	Sintassi della clausola <code>CREATE TABLE</code>	53
■ Codice 7.3	Definizione di un dominio in SQL	54
■ Codice 7.4	Abilitazione e disabilitazione di un vincolo	58
■ Codice 8.1	Sintassi della clausola <code>SELECT</code>	59
■ Codice 8.2	Prima query in SQL	59
■ Codice 8.3	Sintassi per l'assegnazione di alias alle tabelle	60
■ Codice 8.4	Utilizzo dell'asterisco in SQL	60
■ Codice 8.5	La clausola <code>SELECT DISTINCT</code>	60
■ Codice 8.6	Gli operatori insiemistici in SQL	60
■ Codice 8.7	Utilizzo della clausola <code>IS [NOT] NULL</code>	61
■ Codice 8.8	L'operatore <code>LIKE</code>	61
■ Codice 8.9	Sintassi della clausola <code>BETWEEN</code>	62
■ Codice 8.10	La clausola <code>IN</code>	62
■ Codice 8.11	La clausola <code>ORDER BY</code>	62
■ Codice 8.12	La clausola <code>COUNT</code>	63
■ Codice 8.13	La funzione <code>SUM</code>	63
■ Codice 8.14	Funzione <code>MAX</code>	63
■ Codice 8.15	Funzione <code>MIN</code>	63
■ Codice 8.16	Funzione <code>AVG</code>	63
■ Codice 11.1	Sintassi della clausola <code>INSERT INTO</code>	74
■ Codice 11.2	Sintassi della clausola <code>DELETE</code>	75
■ Codice 11.3	Esempio di utilizzo della clausola <code>WHERE</code> con query annidata	75
■ Codice 11.4	Sintassi della clausola <code>UPDATE</code>	75
■ Codice 12.1	Sintassi assertion	77
■ Codice 12.2	Sintassi generale per la definizione di un trigger	79
■ Codice 12.3	Esempio di trigger in PostgreSQL che utilizza la clausola <code>INSTEAD OF</code>	80
■ Codice 13.1	Struttura di un blocco PL/SQL	82
■ Codice 13.2	Blocchi annidati in PL/SQL	83
■ Codice 13.3	Hello World.sql	84
■ Codice 13.4	Esempio di utilizzo della <code>SELECT INTO</code>	84
■ Codice 13.6	Dichiarazione di una variabile di tipo <code>ROWTYPE</code>	85
■ Codice 13.5	Esempio di dichiarazioni ancorate	85
■ Codice 13.7	Assegnazione diretta in PL/SQL	85
■ Codice 13.8	Assegnazione tramite <code>SELECT INTO</code>	85
■ Codice 13.9	Gestione delle eccezioni in PL/SQL	86
■ Codice 13.10	Esempio di costrutto condizionale in PL/SQL	86
■ Codice 13.11	Esempio di ciclo <code>for</code> in PL/SQL	87
■ Codice 13.12	Esempio di loop semplice in PL/SQL	87
■ Codice 13.13	Esempio di ciclo <code>while</code> in PL/SQL	88
■ Codice 13.14	Sintassi per la definizione di una procedura in PL/SQL	88
■ Codice 13.15	Esempio di procedura in PL/SQL	88
■ Codice 13.16	Definizione di una funzione in PL/SQL	89
■ Codice 13.17	Dichiarazione di cursori in PL/SQL	90
■ Codice 13.18	Utilizzo dei cursori in PL/SQL	90
■ Codice 13.19	Esempio di utilizzo di un cursore parametrico	91
■ Codice 13.20	Sintassi delle eccezioni in PL/SQL	91
■ Codice 13.21	Esempio di SQL dinamico in PL/SQL	93
■ Codice 13.22	Utilizzo di una funzione dinamica	93
■ Codice 13.23	Esempio di utilizzo del comando <code>PREPARE</code>	94
■ Codice 13.24	Uso delle tabelle di sistema in PL/SQL	97
■ Codice 14.1	Tabelle di sistema in PostgreSQL	102

▣ Codice 14.2	Descrizione delle colonne di una tabella	102
▣ Codice 14.3	Confrontare due tabelle in PostgreSQL	103
▣ Codice 14.4	Confrontare due tabelle con join esterno	104
▣ Codice 15.1	Dichiarazione di una funzione in PL/pgSQL	108
▣ Codice 15.2	Utilizzo del dollar quoting all'interno di una funzione PL/pgSQL	108
▣ Codice 15.3	Utilizzo del dollar quoting all'interno delle stored procedures in PL/pgSQL	108
▣ Codice 15.4	Esempio di struttura di un blocco PL/pgSQL	109
▣ Codice 15.5	Dichiarazione e inizializzazione delle variabili in PL/pgSQL	110
▣ Codice 15.6	Utilizzo del parametro %type in PL/pgSQL	111
▣ Codice 15.7	Utilizzo delle row variables in PL/pgSQL	112
▣ Codice 15.8	Utilizzare una variabile di tipo record all'interno di una istruzione select into	112
▣ Codice 15.9	Utilizzare una variabile di tipo record all'interno di un ciclo for	113
▣ Codice 15.10	Esempio di utilizzo delle costanti in PL/pgSQL	113
▣ Codice 15.11	Sintassi dell'istruzione SELECT INTO	113
▣ Codice 15.12	Esempio di utilizzo dell'istruzione SELECT INTO	114
▣ Codice 15.13	Record nullo	114
▣ Codice 15.14	L'istruzione NULL	114
▣ Codice 15.15	Esempio di blocco if then in PL/pgSQL	115
▣ Codice 15.16	Esempio di utilizzo di un blocco if then else	115
▣ Codice 15.17	Esempio di blocco if then elsif	115
▣ Codice 15.18	Calcolo della sequenza di Fibonacci	116
▣ Codice 15.19	Esempio di ciclo while	117
▣ Codice 15.20	Esempio di ciclo for	117
▣ Codice 15.21	Esempio di ciclo for al contrario	117
▣ Codice 15.22	Utilizzo del ciclo for per iterare su un ResultSet	117
▣ Codice 15.23	Utilizzo di ciclo for per iterare sul resultset di una query dinamica	118
▣ Codice 15.24	Esempio di utilizzo dell'istruzione exit in PL/pgSQL	119
▣ Codice 15.25	Esempio di istruzione continue	120
▣ Codice 15.26	Trovare l'ennesima occorrenza di una sottostringa in una stringa	120
▣ Codice 15.27	Istruzione create function in PL/pgSQL	121
▣ Codice 15.28	Dichiarazione di un cursore in PostgreSQL	125
▣ Codice 15.29	Utilizzo di un cursore mediante keyword FETCH	126
▣ Codice 15.30	Esempio di utilizzo dei cursori	127
▣ Codice 15.31	Funzione che restituisce il nome di un cursore	128
▣ Codice 15.32	Esempio di utilizzo dell'istruzione raise	128
▣ Codice 15.33	Sollevamento di un errore in PL/pgSQL	129
▣ Codice 15.34	Creazione tabella employees	131
▣ Codice 15.35	Creazione tabella employee_audits	131
▣ Codice 15.36	Creazione funzione	132
▣ Codice 15.37	Creazione del trigger	132
▣ Codice 15.38	Inserimento dati	132
▣ Codice 15.39	Esempio di utilizzo dell'argomento tg_op in un trigger PL/pgSQL	133
▣ Codice 15.40	Esempio di query costruita dinamicamente in PL/pgSQL	135
▣ Codice 15.41	Esecuzione di un'istruzione DML sfruttando l'SQL dinamico in PL/pgSQL	136
▣ Codice 15.42	Esempio di retrieve dinamico sfruttando l'SQL dinamico in PL/pgSQL	136
▣ Codice 15.43	Esempio di sort dinamico sfruttando l'SQL dinamico in PL/pgSQL	137
▣ Codice 15.44	Esempio di Transazione in PostgreSQL	138
▣ Codice 15.45	Trasferimento di denaro fra due conti mediante una transazione	138
▣ Codice 15.46	Esempio di rollback di una transazione	139
▣ Codice 15.47	Esempio di transazione all'interno di una procedura	139

INTRODUZIONE

ESTRATTO

Il seguente documento rappresenta una raccolta di appunti, riorganizzata e migliorata nelle vesti grafiche, del corso di Basi Di Dati 1 della docente Mara Sangiovanni, per il corso di laurea in Informatica A.A 2023-2024, della Federico II di Napoli.

Si tiene a specificare che questo documento non ha lo scopo di **sostituire libri di testo**, note del professore o lezioni frontali, nonostante siano tutte fonti utilizzate per la stesura, ma quello di integrare al materiale più complesso una rilettura degli argomenti da un punto di vista meno tecnico e quindi più lento nel soffermarsi nei passaggi critici.

Si ringraziano  **Valentino Bocchetti** e  **Pasquale Miranda** ideatori e gestori del progetto *Unina Docs*,

Si ringraziano inoltre tutti i revisori che hanno contribuito al documento con consigli e proposte di modifica, invitando a dare feedback in caso di errori.

Si ringraziano in fine i lettori nella speranza che questo lavoro sia stato utile.

Redazione a cura di  **Giorgio Di Fusco**

INTRODUZIONE ALLE BASI DI DATI

1.1

TIPI DI DATABASE E APPLICAZIONI



Le attività di raccolta, organizzazione e conservazione dei dati costituiscono uno dei principali compiti dei sistemi informatici. Le rubriche, i saldi dei conti correnti bancari o l'elenco degli iscritti a una facoltà universitaria e gli esiti dei loro esami sono esempi di dati indispensabili a gestire alcune attività umane.

I sistemi informatici garantiscono che questi dati vengano conservati in modo permanente su dispositivi per la loro memorizzazione, aggiornati per riflettere rapidamente le loro variazioni e resi accessibili alle interrogazioni degli utenti. Si pensi, per esempio, all'interrogazione sulla disponibilità di spesa sulle carte di credito, effettuata tramite cellulare mediante le applicazioni dell'apposita banca.

Queste poche applicazioni sono esempio di quelli che vengono chiamati **database tradizionali**, nei quali l'informazione che viene conservata e visualizzata è di tipo testuale o numerico. Con il passare degli anni però, con la proliferazione dei social media come Facebook, Twitter e TikTok, è stato necessario sviluppare enormi database che potessero immagazzinare dati non tradizionali, come post, tweets, immagini e video. Questi nuovi tipi di database, spesso chiamati anche sistemi di conservazione dei **big data**, o **sistemi NOSQL**, sono stati creati per l'apposita funzione di conservare i dati per le applicazioni rivolte ai social media.

Oltre ai dati di tipo multimediale, lo sviluppo tecnologico delle applicazioni moderne ha reso necessario lo sviluppo di database rivolti al salvataggio di dati come i **Geographic information systems (GIS)**, fondamentali per applicazioni come Google Maps, oppure la conservazione dei dati genetici, biometrici e sanitari.

1.2

COS'È UN DATABASE?



Un **database** è una collezione di *dati interconnessi*, memorizzati su supporto permanente, a cui diversi utenti e programmi possono accedere. Per **dati** si intendono informazioni note che possono essere memorizzate e che hanno un **significato intrinseco** per chi deve interagire con essi. Consideriamo i nomi, i numeri e gli indirizzi presenti in un registro. Questi sono esempi di dati che oggi vengono conservati nei cellulari, i quali hanno un apposito database per la loro conservazione e consultazione. Un database rappresenta alcuni aspetti del mondo reale, spesso chiamato **mini-mondo**. I cambiamenti che avvengono nel mini-mondo vengono riflessi nel database.

Nel momento in cui andiamo a creare, quindi, un sistema software per la gestione, creazione e manutenzione di una base di dati, stiamo andando a costruire un **DataBase Management System (DBMS)**. Esso è basato su un ben preciso **Modello dei Dati**¹. Ad esempio il software MySQL gestisce il DB, mentre nel file .mdb troviamo i vari dati. Un *sistema di gestione di basi di dati* (in inglese **DataBase Management System**, abbreviato con **DBMS**) è un sistema software in grado di gestire collezioni di dati che siano grandi, condivise e persistenti, assicurando la loro affidabilità e privacy. Come ogni prodotto informatico, un DBMS deve essere efficiente ed efficace. Un DBMS è basato su di uno specifico **modello di dati** (Gerarchico, Relazionale, NOSQL, a grafo). Un **sistema di base di dati** è l'insieme del DBMS e dei dati. Il DBMS consente quindi di definire un particolare database in termini di:

- Tipi di dati
- Strutture
- Vincoli

¹esempi di vari tipi: gerarchico, relazionale, NOSql, a grafo, ...



- Le basi di dati sono *grandi*. Di conseguenza i DBMS devono prevedere una gestione dei dati in memoria secondaria. L'azione del costruire e caricare il contenuto iniziale del database su un mezzo di memorizzazione (storage) secondario è detta **popolazione del database**.
- Le basi di dati sono *condivise*, nel senso che applicazioni e utenti diversi devono poter accedere a dati comuni. In questo modo si riduce la *ridondanza dei dati*, poiché si evitano ripetizioni e conseguentemente si riduce anche la possibilità di *inconsistenze*: se esistono varie copie degli stessi dati, è possibile che queste, in qualche momento, non siano uguali. Per garantire l'accesso condiviso ai dati da parte di molti utenti che operano contemporaneamente, il DBMS dispone di un meccanismo detto **controllo di concorrenza**.
- Le basi di dati sono *persistenti*, cioè hanno un tempo di vita che non è limitato a quello delle singole esecuzioni dei programmi che le utilizzano.
- I DBMS garantiscono *affidabilità*, cioè la capacità del sistema di conservare sostanzialmente intatto il contenuto della base di dati in caso di malfunzionamenti hardware e software. A questo scopo i DBMS forniscono specifiche funzionalità di *salvataggio e ripristino*.
- I DBMS garantiscono la *privatezza dei dati*. Ciascun utente, opportunamente riconosciuto, viene abilitato a svolgere solo determinate azioni sui dati, attraverso *meccanismi di autorizzazione*.
- I DBMS sono *efficienti*, cioè capaci di svolgere le operazioni utilizzando un insieme di risorse (tempo e spazio) che sia accettabile per gli utenti.
- I DBMS consentono inoltre di manipolare il database attraverso una serie di interrogazioni, dette **query**, per la visualizzazione, modifica e accesso alle informazioni.

1.3.1 ■ Natura autodescrittiva di un sistema di basi di dati

Una delle caratteristiche principali di un database è che questo non contiene soltanto sé stesso ma anche una *descrizione completa della struttura del database e dei suoi vincoli*. Questa descrizione è conservata nel **catalogo del DBMS**, il quale contiene informazioni quali la struttura di ogni file, il tipo e il formato di salvataggio di ogni elemento così come i vari vincoli su ogni dato. Le informazioni che sono conservate nel catalogo vengono chiamati **metadati** e descrivono la struttura del database primario.

1.3.2 ■ Isolamento tra programmi e dati, astrazione dei dati

Nella gestione tradizionale dei file la struttura dei file è strettamente connessa con l'applicativo. In questo modo ogni cambiamento alla struttura di un file richiede una modifica diretta di tutto il programma che ne fa uso. Al contrario un DBMS non richiede una modifica del genere. Questa proprietà è detta **indipendenza dato-programma**. La caratteristica che permette questa indipendenza è l'**astrazione dei dati**. Un DBMS fornisce agli utenti una **rappresentazione concettuale dei dati**, ma non include molti dei dettagli su come i dati sono memorizzati o su come sono implementate le operazioni.

Informalmente, un modello di dati è un tipo di astrazione dei dati che viene utilizzata per fornire questa rappresentazione concettuale. Il modello di dati utilizza concetti logici, come gli oggetti, le loro proprietà e le loro interrelazioni, che possono essere più facili da capire per la maggior parte degli utenti rispetto ai concetti di memorizzazione del computer. Pertanto, il modello di dati nasconde i dettagli di memorizzazione e di implementazione che non interessano alla maggior parte degli utenti di database.

1.3.3 ■ Supporto per viste multiple dei dati

Un database ha tipicamente molti tipi di utenti, ognuno dei quali può richiedere una diversa prospettiva o vista diversa del database. Una vista può essere un sottoinsieme del database oppure può contenere dati virtuali che derivano dai file del database ma che non sono esplicitamente memorizzati. Alcuni utenti potrebbero non aver bisogno di sapere se i dati a cui fanno riferimento sono memorizzati o derivati. Un DBMS multiutente i cui utenti hanno una varietà di applicazioni distinte deve fornire la possibilità di definire viste multiple.

1.3.4 ■ Condivisione dei dati e processo transazionale

Un DBMS multiutente, come dice il nome, deve consentire a più utenti di accedere al database contemporaneamente. Questo è essenziale se si vogliono integrare e mantenere in un unico database i dati di più applicazioni in un unico database. Il DBMS deve includere un software di controllo della concorrenza per garantire che diversi utenti che cercano di aggiornare gli stessi dati

lo facciano in modo controllato, in modo che il risultato degli aggiornamenti sia corretto.

Per esempio, quando diversi addetti alle prenotazioni cercano di assegnare un posto su un volo di linea, il DBMS deve garantire che ogni posto sia accessibile a un solo agente alla volta per l'assegnazione a un passeggero. Questo tipo di applicazioni sono generalmente chiamate **applicazioni di elaborazione delle transazioni online (OLTP)**. Un ruolo fondamentale del software DBMS multiutente è quello di garantire che le transazioni concorrenti funzionino in modo corretto ed efficiente.

Il concetto di **transazione** è diventato centrale in molte applicazioni di database. Una transazione è un *programma* o un *processo in esecuzione* che include uno o più accessi al database, come la lettura o l'aggiornamento di record del database che devono essere eseguite in modo atomico, ovvero indivisibile. Ogni transazione è in grado di eseguire un accesso al database logicamente corretto se eseguito nella sua interezza senza interferenze da parte di altre transazioni.

Il DBMS deve far rispettare diverse proprietà delle transazioni, dette **ACID**:

- **Atomicità:** Tutte le modifiche ai dati vengono eseguite come se fossero un'unica operazione. Ovvero, tutte le modifiche vengono eseguite, oppure nessuna. Ad esempio, in un'applicazione che trasferisce fondi da un conto a un altro, la proprietà di atomicità garantisce che, se un addebito viene effettuato con successo da un conto, il corrispondente accredito viene effettuato sull'altro conto.
- **Consistenza:** I dati si trovano in uno stato coerente quando una transazione inizia e quando termina. Ad esempio, in un'applicazione che trasferisce fondi da un conto a un altro, la proprietà consistency garantisce che il valore totale dei fondi in entrambi i conti sia lo stesso all'inizio e alla fine di ogni transazione.
- **Isolamento:** Lo stato intermedio di una transazione è invisibile alle altre transazioni. Di conseguenza, le transazioni che vengono eseguite simultaneamente sembrano essere serializzate. Ad esempio, in un'applicazione che trasferisce fondi da un conto a un altro, la proprietà di isolamento garantisce che un'altra transazione veda i fondi trasferiti in un conto o nell'altro, ma non in entrambi o in nessuno dei due.
- **Durabilità:** Dopo che una transazione è stata completata con successo, le modifiche ai dati persistono e non vengono annullate, anche in caso di guasto del sistema. Ad esempio, in un'applicazione che trasferisce fondi da un conto a un altro, la proprietà durability garantisce che le modifiche apportate a ciascun conto non vengano annullate.

File system	Databases
Adatto a sistemi piccoli, relativamente economico, pochi files	Adatto a sistemi complessi, relativamente costoso, molti files.
Definizione e struttura dei dati contenuta nei programmi applicativi	Astrazione dei dati, separazione fra programmi e dati, viste multiple dei dati.
Dati ridondanti, possibilità di inconsistenza, dati isolati.	Ridondanze ridotte o assenti, consistenza dei dati, dati condivisi.
Progettazione iniziale minima	Progettazione iniziale vasta e complessa
Sicurezza limitata	Sicurezza rigorosa
Sistemi di recupero e backup primitivi	Sistemi di recupero e backup sofisticati
In generale per utenti singoli	Utenti multipli in accesso concorrente
Controlli di integrità lasciati al programmatore	Controllo di integrità dei dati intrinseco e robusto

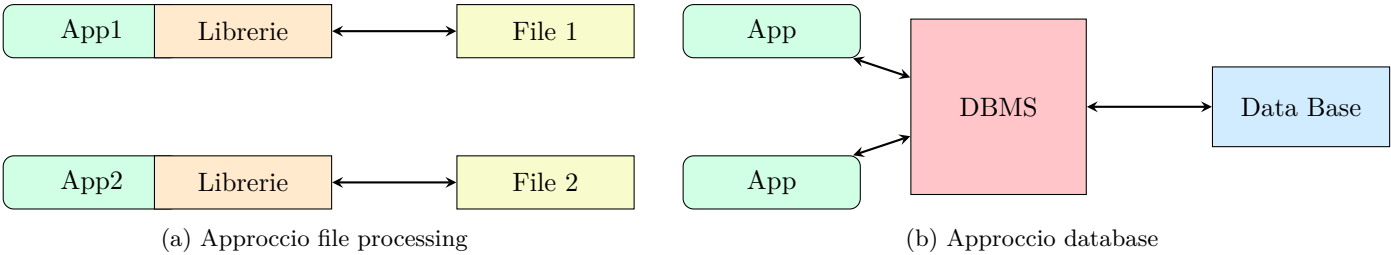


Figura 1.1

ARCHITETTURA DI UN DATABASE

2.1

ASTRAZIONE E MODELLI



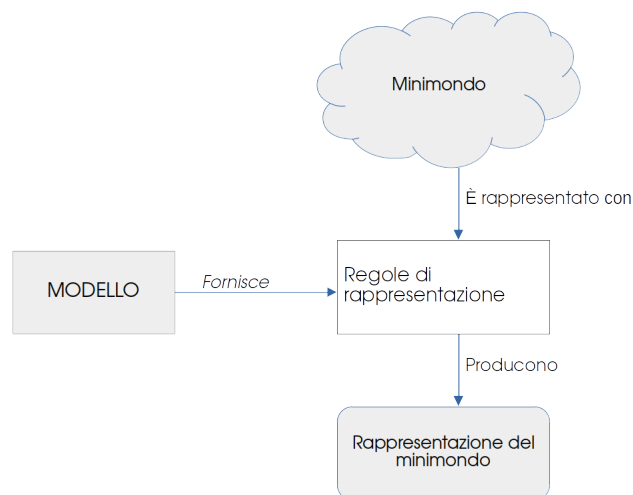
Una delle caratteristiche fondamentali delle basi di dati è quella che queste forniscono un livello di astrazione dei dati con i quali stiamo lavorando. L'**astrazione dei dati** permette di nascondere all'utente tutti i dettagli dell'implementazione.

Un **modello dei dati** è un insieme di concetti che sono usati per descrivere:

- La **struttura** della base di dati (tipi, entità, relazioni);
- Le **operazioni di base** per manipolare le strutture (inserimento, query, update);
- I **vincoli** che la base di dati deve rispettare.

Oltre a questo, un modello dei dati deve permettere di rappresentare una certa realtà e fornire un insieme di strutture simboliche per descrivere la rappresentazione della stessa come rappresentato in figura 2.1.

Figura 2.1: Il modello astratto dei dati



2.1.1 ■ Categorie di modelli di dati

1. **Concettuali o ad alto livello:** forniscono un modello astratto molto vicino al modo in cui l'utente percepisce il dato.
2. **Fisici o di basso livello:** forniscono dettagli sul metodo di salvataggio dei dati in memoria secondaria.
3. **Implementabili o rappresentazionali:** forniscono concetti che possono essere facilmente comprensibili dagli utenti finali che non sono molto distanti dal modo in cui il dato è memorizzato in memoria.
4. **Autodescrittivi:** forniscono una descrizione unitaria dei dati e dei metadati.

I modelli di dati concettuali fanno uso di concetti quali entità, attributi e relazioni. Un'**entità** rappresenta un oggetto del mondo reale, materiale o immateriale, preso dal mini-mondo preso in esame. Gli **attributi** rappresentano le proprietà che vogliamo descrivere dell'entità. Una **relazione** tra due o più entità rappresenta un'associazione tra entità. Il più comune data model di

2.2

ALTRE TIPOLOGIE DI MODELLI DEI DATI

Un **modello dei dati** è un insieme di concetti utilizzati per organizzare i dati di interesse e descriverne la struttura in modo che essa risulti comprensibile a un elaboratore. Il *modello relazionale* dei dati permette di definire tipi per mezzo del concetto matematico di **relazione**, che consente di organizzare i dati in insiemi di record a struttura fissa. Oltre al modello relazionale sono stati definiti altri quattro tipi di modelli:

- Il **modello gerarchico**, basato sull'uso delle strutture ad albero, definito durante la prima fase di sviluppo dei DBMS ma tuttora utilizzato;
- Il **modello reticolare**, basato sull'uso di grafi, sviluppato successivamente al modello gerarchico.
- Il **modello a oggetti**, che estende alle basi di dati il paradigma di programmazione a oggetti.
- Il **modello XML**, in cui i dati vengono presentati assieme alla loro descrizione e non devono sottostare a un'unica struttura logica.

2.3

SCHEMI, ISTANZE, STATO DI UNA BASE DI DATI

In una base di dati è importante distinguere tra la *descrizione* del database e il database propriamente detto. La descrizione di un database è chiamata **schema del database**, che viene specificato durante la progettazione del database e non si prevede che cambi frequentemente. Uno schema visualizzato è chiamato **diagramma dello schema**. Ogni oggetto nello schema, come nella tabella 2.1, è chiamato **costrutto di schema**.

Studente			
Nome	Cognome	DataNascita	Matricola

Tabella 2.1: Esempio di costrutto di schema.

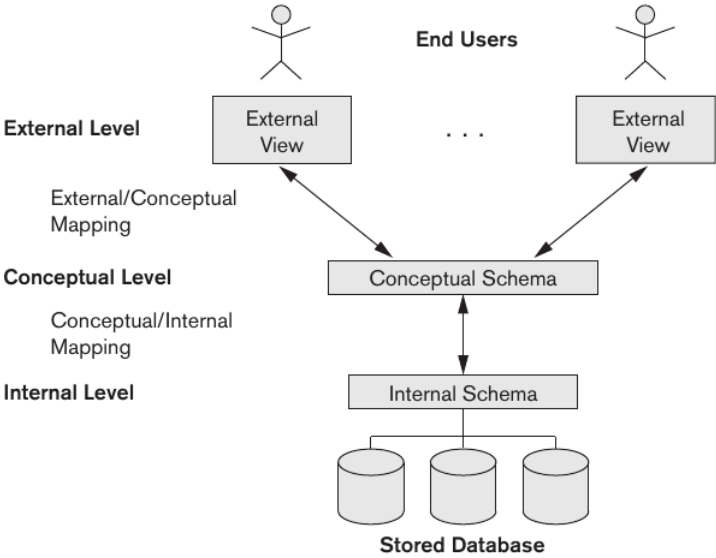
Mentre lo schema non cambia frequentemente, i dati presenti in un database possono essere aggiornati molto velocemente. L'insieme dei dati presenti in un preciso istante nel database è chiamato **stato del database** o **snapshot**. Viene anche chiamato *insieme corrente* delle **occorrenze** o **istanze** nel database.

Quando **definiamo** un nuovo database stiamo specificando al DBMS il suo schema. A questo punto, lo stato corrispondente del database è detto **stato vuoto**, senza dati. Otteniamo lo **stato iniziale** quando iniziamo a popolare il database con i vari record di dati. Da questo momento in poi, ogni volta che si ha un'operazione di aggiornamento, si otterranno nuovi stati della base di dati. In qualsiasi istante di tempo la base di dati ha un proprio **stato corrente**.

2.4

ARCHITETTURA A TRE LIVELLI

Figura 2.2: L'architettura a tre livelli



L'obiettivo dello schema a tre livelli, illustrato in figura 2.2 è quello di dividere le applicazioni utente dal database fisico. Con questa architettura, gli schemi possono essere definiti come segue:

1. Il **livello interno** ha uno **schema interno** che descrive la struttura fisica del database.
2. Il **livello concettuale** ha uno **schema concettuale** che descrive la struttura dell'intero database per una comunità di utenti. Lo schema concettuale nasconde i dettagli della struttura fisica di memorizzazione e si concentra sulla descrizione delle entità, dei tipi, delle relazioni, delle operazioni e dei vincoli.
3. Il **livello esterno** include una serie di **schemi esterni** o **user views**. Ciascun schema esterno descrive la parte del database che un determinato gruppo di utenti è abilitato all'utilizzo nascondendo il resto.

Gli schemi sono solo descrizioni dei dati, i dati esistono solo a livello fisico. All'interno dell'architettura a tre livelli ogni gruppo di utenti fa riferimento al proprio schema esterno. Di conseguenza, il DBMS deve trasformare una richiesta specificata su uno schema esterno in una richiesta sullo schema concettuale e poi in una richiesta sullo schema interno per l'elaborazione del database memorizzato. Se la richiesta è un recupero di database, i dati estratti dal database memorizzato devono essere riformattati per corrispondere alla vista esterna dell'utente. I processi di trasformazione delle richieste e i risultati tra i livelli sono chiamati **mappature**.

2.4.1 ■ Indipendenza dei dati

L'architettura a tre schemi può essere utilizzata per spiegare ulteriormente il concetto di indipendenza dei dati, che può essere definita come la capacità di cambiare lo schema ad un livello del sistema di base di dati senza dover modificare lo schema del livello superiore. Possiamo definire due tipi di indipendenza dei dati:

1. L'**indipendenza logica** è la capacità di cambiare lo schema concettuale senza dover cambiare gli schemi esterni o gli applicativi che si interfacciano al sistema di basi di dati.
2. L'**indipendenza fisica** è la capacità di cambiare lo schema interno senza dover cambiare lo schema fisico.

2.5

I LINGUAGGI DEL DBMS



Una volta aver completato la **progettazione** della base di dati e **scelto uno specifico DBMS** per poterla implementare, il primo passo è quello di specificare gli schemi concettuali e fisici e il mapping fra i due. Un DBMS deve fornire ad ogni categoria di utenti delle **interfacce** e dei **linguaggi** adeguati per poterne usufruire. In molti DBMS in cui non viene mantenuta una rigida separazione dei livelli viene utilizzato un unico linguaggio, chiamato **data definition language (DDL)** che viene utilizzato dai progettisti di database per definire entrambi gli schemi. Il DBMS avrà un compilatore la cui funzione è quella di elaborare le dichiarazioni DDL per identificare le descrizioni dei costrutti dello schema e memorizzare la descrizione dello schema nel catalogo del DBMS.

Nei DBMS in cui viene mantenuta una chiara separazione tra il livello concettuale e quello interno, il DDL viene utilizzato per specificare solo lo schema concettuale. Un altro linguaggio, lo **storage definition language (SDL)**, viene utilizzato per specificare lo schema interno. Le mappature tra i due schemi possono essere specificate in uno dei due linguaggi. Nella maggior parte dei DBMS relazionali non esiste un linguaggio specifico che svolga il ruolo di SDL. Al contrario, lo schema interno è specificato da una combinazione di funzioni, parametri e specifiche relative alla memorizzazione dei file. Queste permettono all'**amministratore della base di dati (DBA)** di controllare le scelte di indicizzazione e la mappatura dei dati nella memoria.

Per una vera architettura a tre schemi, sarebbe necessario un terzo linguaggio, il **View Definition Language (VDL)** per specificare le viste dell'utente e le loro mappature allo schema concettuale, ma nella maggior parte dei DBMS il DDL è usato per definire sia lo schema concettuale che quello esterno. Nei DBMS relazionali, **SQL** viene utilizzato nel ruolo di VDL per definire le viste utente o applicazioni come risultati di *query predefinite*.

Una volta che gli schemi del database sono stati compilati e il database è stato popolato con i dati, gli utenti devono essere in grado di poter manipolare il database. Le manipolazioni tipiche includono il **recupero**, l'**inserimento**, la **cancellazione** e la **modifica** dei dati. Il DBMS fornisce una serie di operazioni o un linguaggio chiamato **data manipulation language (DML)** per questi scopi.

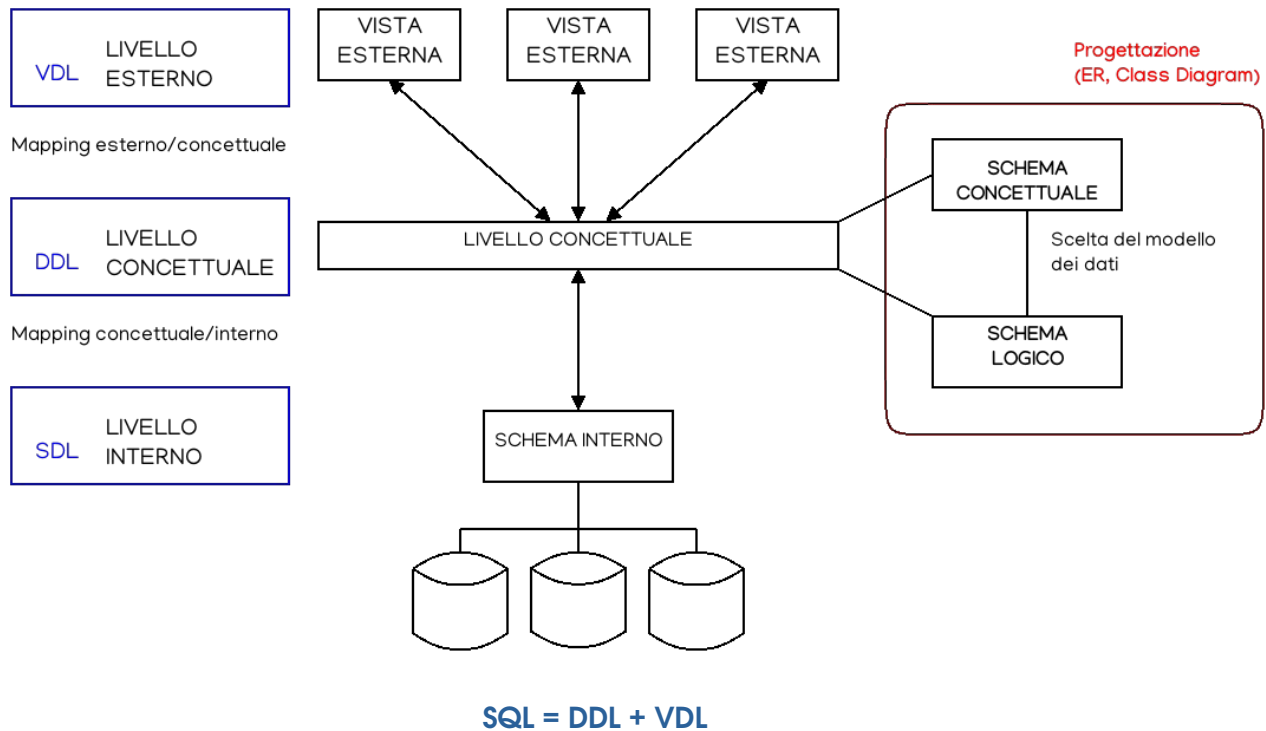


Figura 2.3: Architettura e linguaggi del DBMS

Esistono due tipi di linguaggi DML:

1. DML di **basso livello** o procedurali : sono incapsulati (embedded) in linguaggi general-purpose e chiamati anche **record-at-a-time** poiché recuperano solo un record alla volta.
2. DML di **alto livello** o non-procedurali: possono essere specificati interattivamente da terminale, essere inglobati in un linguaggio di programmazione general-purpose e sono detti **set-at-a-time** o **set-oriented** poiché possono specificare e recuperare molti record con un singolo statement DML. Sono detti anche dichiarativi perché una query di un DML high-level spesso specifica quale dato deve essere ritrovato piuttosto che come ritrovarlo.

LA PROGETTAZIONE CONCETTUALE

3.1

IL CICLO DI VITA DEL SOFTWARE



3.1.1 ■ Progettazione e ciclo di vita

La progettazione di un database è una di quelle attività del processo di sviluppo dei **sistemi informativi**. Il sistema informativo è un componente (sottosistema) di un'**organizzazione** che gestisce, quindi acquisisce, elabora, memorizza e distribuisce, **informazioni** d'interesse, ovvero per i fini degli scopi dell'organizzazione.

Il sistema di sviluppo può essere visto come un vero e proprio **ciclo vitale**, che si compone di 6 fasi:

1. **Pianificazione**: si definiscono gli obiettivi del sistema, le risorse necessarie e le tempistiche di sviluppo.
2. **Analisi**: si analizzano i requisiti del sistema, ovvero le funzionalità che il sistema deve avere.
3. **Design**: si progetta il sistema, ovvero si definiscono le componenti del sistema e le loro interazioni.
4. **Implementazione**: si implementa il sistema, ovvero si scrive il codice.
5. **Test & Integrazione**: si testa il sistema e si verifica che funzioni correttamente.
6. **Manutenzione**: si mantiene il sistema, ovvero si correggono i bug e si aggiungono nuove funzionalità.

Una volta arrivati al punto **6**, si torna al punto **1**, in quanto il sistema informativo è un sistema dinamico, che deve essere aggiornato e migliorato continuamente. Adesso andremo a soffermarci maggiormente sul punto **2** e **3**, ovvero l'analisi e la progettazione di un database.

3.2

DEFINIZIONE DEL PROBLEMA



La **progettazione di un database** è una delle attività del processo di sviluppo dei **sistemi informativi**. Un sistema informativo è una componente di una organizzazione che gestisce le informazioni di interesse. La progettazione concettuale di una basi di dati è il processo di formulazione di un modello dettagliato del database capace di sintetizzare la visione degli utenti e la visione dei progettisti. Questi modelli contengono tutte le **scelte progettuali a livello logico e fisico** e i parametri fisici per la generazione del data definition language (DDL), che può essere utilizzato per l'implementazione del database.

Modellare i dati significa costruire una rappresentazione semplificata della realtà osservata o di un problema aziendale, individuandone gli elementi caratterizzanti e i legami intercorrenti tra essi. La progettazione di un modello di dati avviene a livelli diversi:

- il **livello concettuale** (o **esterno**) rappresenta la realtà dei dati e le relazioni tra essi attraverso uno schema. Nella maggioranza dei casi, chi progetta il database è una persona che possiede competenze in tale campo, piuttosto che competenze nel dominio applicativo dei dati da memorizzare. Pertanto, i dati da memorizzare devono essere definiti in cooperazione con un *esperto di dominio*, che è consapevole di quali dati devono essere memorizzati nel nuovo sistema. Questo processo è uno di quelli comunemente considerati parte dell'**analisi dei requisiti**, e richiede che il progettista abbia le competenze necessarie per sfruttare al meglio le conoscenze di dominio applicativo. Questo avviene perché spesso chi progetta il database non riesce esprimere con chiarezza quali sono i requisiti del sistema siccome non è abituato a pensare in termini dei dati che devono essere memorizzati. Tali dati possono essere rilevati tramite un'apposita *specifica dei requisiti*.

- il **livello logico** rappresenta il modo attraverso il quale i dati sono organizzati negli archivi elettronici: descrive quindi la composizione e il formato dei dati nel loro aspetto di struttura logica di dati. Il livello logico viene derivato dal livello concettuale applicando alcune semplici regole di trasformazione. Nel caso di un database relazionale gli oggetti memorizzati sono tabelle che immagazzinano i dati in righe e colonne. In un database ad oggetti, invece, gli oggetti memorizzati corrispondono direttamente agli oggetti usati dalla programmazione orientata agli oggetti per scrivere le applicazioni che vogliono gestire e accedere ai dati.
- il **livello fisico** rappresenta l'effettiva installazione degli archivi elettronici: esso indica l'ubicazione dei dati nelle memorie di massa (dischi). Il livello fisico è quindi l'implementazione del livello logico sui supporti per la registrazione fisica dei dati: partizioni, puntatori, blocchi fisici, cluster, indici.

Schema concettuale

Lo schema concettuale è una descrizione concisa dei requisiti e include descrizioni dettagliate delle:

- **entità** ovvero gli elementi informativi di base;
- i loro **tipi**;
- le **associazioni**, ovvero il modo in cui sono interrelate le entità;
- i **vincoli**, le regole che i dati memorizzati nel database dovrebbero soddisfare per garantire la qualità e la coerenza dell'informazione.

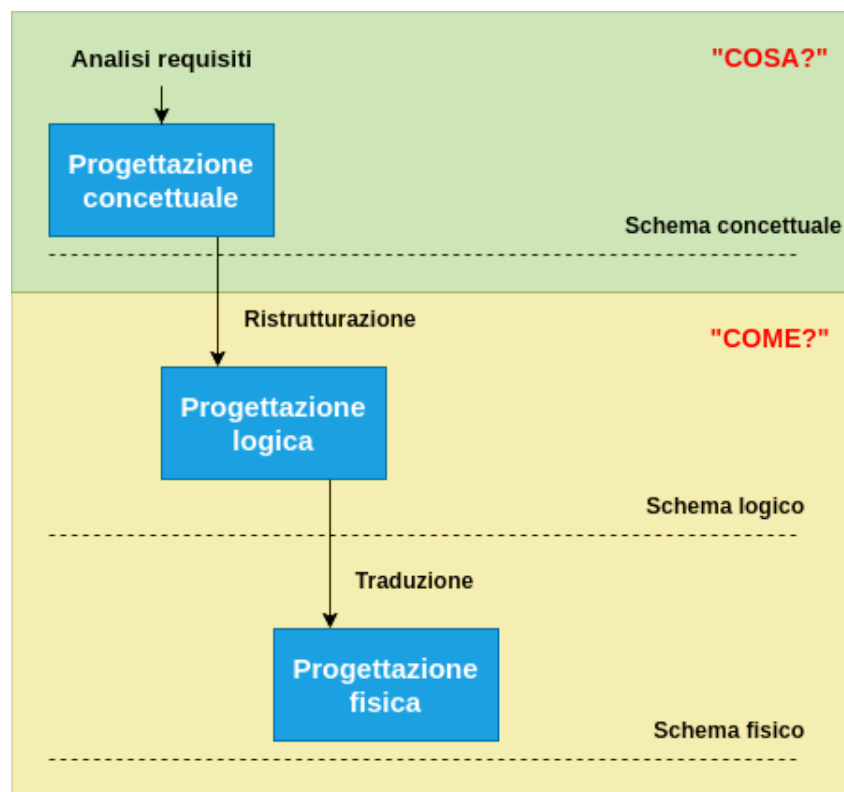


Figura 3.1: Fasi della progettazione di un database

3.3

IL MODELLO ENTITÀ - RELAZIONI



Il modello *entità-relazioni* è un modello concettuale di dati e fornisce una serie di strutture, dette **costrutti**, utilizzate nella descrizione del minimondo. Questi costrutti vengono utilizzati per definire **schemi** che descrivono l'organizzazione e la struttura delle occorrenze¹ dei dati, ovvero dei valori assunti dai dati al variare del tempo. Nel modello E-R i dati vengono descritti in termini di **entità**, **relazioni** e **attributi**.

3.3.1 Le entità

Il concetto che sta alla base della modellazione concettuale è quello di **entità**, ovvero una cosa o un oggetto del mondo reale che siamo interessati a descrivere nella nostra base di dati. Ogni entità ha dei propri **attributi**, ovvero delle qualità particolari che le descrivono. Un'entità particolare, anche detta **istanza**, avrà un valore definito per ciascun attributo. Nella rappresentazione grafica le entità sono identificate con un rettangolo contenente all'interno il nome dell'entità mentre gli attributi sono inseriti

¹In genere si utilizza il termine **istanza** invece di *occorrenza*.

singolarmente all'interno di ellissi collegate tramite delle linee all'entità cui si riferiscono. Ad esempio, l'entità PERSONA rappresentato nella figura 3.2 potrà essere descritta attraverso il suo nome, il cognome, la data di nascita e il codice fiscale.

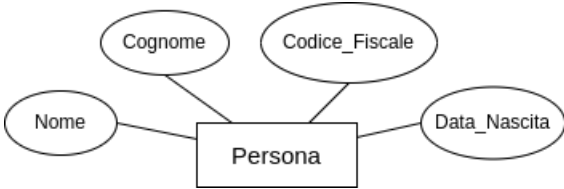


Figura 3.2: Esempio di rappresentazione di un'entità nel modello ER

3.3.2 ■ Gli attributi

Descrivono le proprietà elementari di entità o relazioni che sono di interesse ai fini dell'applicazione. Un attributo associa a ciascuna occorrenza di entità un valore appartenente ad un insieme, detto **dominio**, che contiene i valori ammissibili per l'attributo. Nel modello ER sono presenti diversi tipologie di attributi: *semplici* o *composti*, a *valore singolo* o *multiplo*, *memorizzati* o *derivati*.

Attributi semplici e composti. Gli attributi composti possono essere divisi in parti più piccole che rappresentano attributi più basilari con un significato indipendente. Gli attributi che sono invece atomici vengono chiamati semplici. Nell'esempio in figura 3.3 l'entità Impiegato ha tre attributi: Nome, Sesso e DataNascita. Sesso e DataNascita sono attributi semplici, Nome invece essendo un attributo composto viene ulteriormente suddiviso tra *Nome*, *Cognome* e *Iniziali*.



Figura 3.3: Rappresentazione nel modello ER degli attributi composti

Attributi singoli e attributi multipli. Molti attributi hanno un singolo valore; attributi del genere vengono chiamati **singoli**. Ad esempio l'attributo Nome dell'entità Persona è un attributo singolo. Attributi che invece possono avere più valori vengono chiamati **multipli**. Un attributo a più valori può avere limiti inferiori e superiori per limitare il numero di valori consentiti per ogni singola entità. Gli attributi multi-valore sono rappresentati tramite una doppia ellisse come mostrato nella figura 3.4.

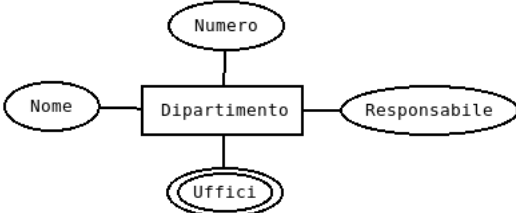


Figura 3.4: Esempio di attributo multi-valore

Attributi memorizzati e attributi calcolati. In alcuni casi, due o più attributi possono essere interrelati. Ad esempio gli attributi Età e DataNascita di una persona. Per una particolare istanza di una persona l'attributo Età può essere determinato eseguendo la sottrazione tra la data corrente e la data di nascita della persona. Questi tipi di attributi sono definiti come **attributi calcolati** oppure **derivati** dall'attributo DataNascita, che invece viene dichiarato **attributo memorizzato**. Per rappresentare un attributo derivato si usa tratteggiare l'ellisse.

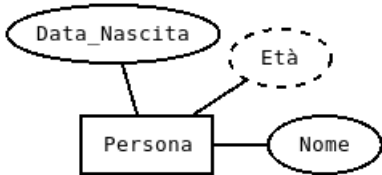


Figura 3.5: Esempio di attributo derivato in ER

Attributi nulli. In alcuni casi, una entità particolare potrebbe non avere un attributo da poter essere inserito all'interno del database. In questi casi si usa inserire il valore nullo, **NULL**, che rappresenta un'informazione mancante in quanto inapplicabile o sconosciuta.

Tipo di entità

Un database contiene solitamente gruppi di entità che sono simili. Per esempio, un'azienda che impiega centinaia di dipendenti potrebbe voler informazioni simili su ciascuno di essi. Queste entità di dipendenti condividono gli stessi attributi, ma ogni entità ha i propri valori per ogni attributo. Un **tipo di entità** definisce un *insieme* (o set) di entità che hanno gli stessi attributi. Ogni tipo di entità nel database è descritto dal suo nome e dai suoi attributi.

Un *tipo di entità* descrive lo schema o l'**intensione** di un insieme di entità che condividono la stessa struttura. L'insieme delle entità di un particolare tipo di entità viene raggruppato in un insieme di entità, chiamato anche **estensione** del tipo di entità.

Identificatori delle entità

Vengono spesso specificati per ciascuna entità di uno schema e descrivono gli attributi che permettono di identificare in *maniera univoca* le istanze delle entità. In molti casi, uno o più attributi di una entità sono sufficienti a individuare un identificatore: si parla in questo caso di **identificatore interno** detto anche **chiave**. Nella notazione diagrammatica del modello E-R le chiavi vengono rappresentate attraverso la *sottolineatura* degli attributi chiave. Dire che un attributo è una chiave per una particolare entità significa richiedere che questo sia unico per ogni tipo di entità.

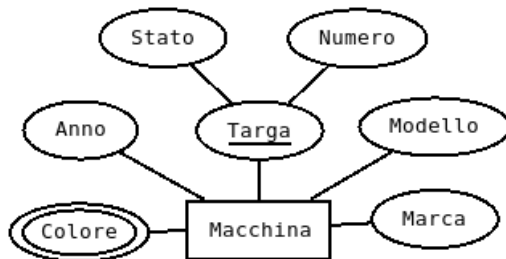


Figura 3.6: Rappresentazione di un identificatore interno nel modello ER

I tipi di entità che non hanno un attributo chiave sono chiamati **entità deboli**. Le entità appartenenti ad un tipo di entità debole si identificano associandole ad altri tipi di entità, chiamati **identificanti** e chiamiamo **relazione identificante** il tipo di relazione che associa un tipo di entità debole all'entità identificante. Un tipo di entità debole ha sempre un *vincolo di partecipazione totale* all'interno della relazione identificante poiché un tipo debole non può essere identificato senza un identificante.

Un tipo di entità debole di solito ha una **chiave parziale**, ovvero un attributo che può identificare univocamente l'entità che sono associate all'entità identificante. Nei diagrammi ER sia le entità deboli che le proprie relazioni identificanti sono contraddistinte da rettangoli e rombi doppi.

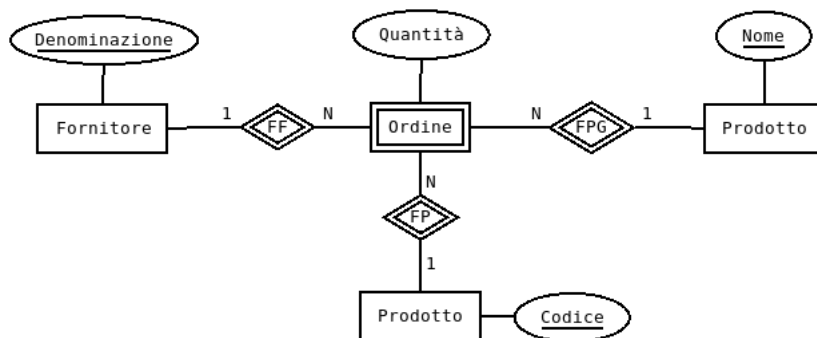


Figura 3.7: Esempio di entità debole

3.3.3 Le relazioni e associazioni

L'**associazione** (in inglese **relationship**) è un legame logico che stabilisce un'interazione tra entità. Più precisamente, un **tipo di relazione** R tra n tipi di entità $E_1, \dots, E_n$ definisce un insieme di associazioni- o **insieme relazione**- tra le entità di questi tipi di entità. Matematicamente, l'insieme relazione R è l'insieme delle **istanze delle relazioni** r_i dove ciascun r_i associa n entità individuali $(e_1, \dots, e_n)$ e ogni entità e_j in r_i è un membro dell'insieme di entità E_j , $1 \leq j \leq n$. Quindi, una relazione è un sottoinsieme del prodotto cartesiano degli insiemi di entità $E_1 \times E_2 \times \dots \times E_n$. Ciascun tipo di entità **partecipa** nella relazione di tipo R . Analogamente, ogni singola istanza $e_1, \dots, e_n$ partecipano all'istanza della relazione $r_i = (e_1, e_2, \dots, e_n)$.

La rappresentazione grafica convenzionalmente usata per indicare un'associazione è una linea che unisce le due entità interessate con l'aggiunta, a metà della linea, di un rombo che contiene la descrizione dell'associazione come mostrato in figura 3.8.



Figura 3.8: Simbolismo ER per le associazioni

Le associazioni hanno un **grado**. Il grado è dato dal numero di entità che partecipano all'associazione. L'associazione tra *Employee* e *Department* è un'associazione di grado 2, detta associazione binaria ma, naturalmente, esistono associazioni di grado 1, 3 e di grado superiore. Un esempio viene mostrato in figura 3.9: la relazione *Fornitura* tra tre entità *Fornitore*, *Prodotto* e *Dipartimento* descrive il fatto che un fornitore rifornisce un dipartimento di un certo prodotto.

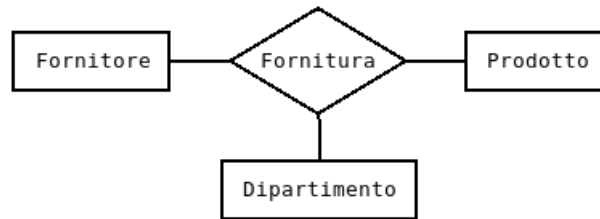


Figura 3.9: Esempio di relazione ternaria nel modello E-R

È anche possibile avere relazioni **ricorsive**, ovvero relazioni tra una entità e se stessa. Per esempio, in figura 3.8 la relazione *Collega* sull'entità *Impiegato* connette coppie di impiegati che lavorano insieme mentre la relazione *Successione* sull'entità *Sovrano* associa ad ogni sovrano di una dinastia il suo immediato successore. Va osservato che, a differenza della prima relazione, la relazione *Successione* non è simmetrica. In questo caso è necessario stabilire dei **ruoli**, ovvero degli *identificatori* che descrivono il modo in cui l'entità viene coinvolta nella relazione.

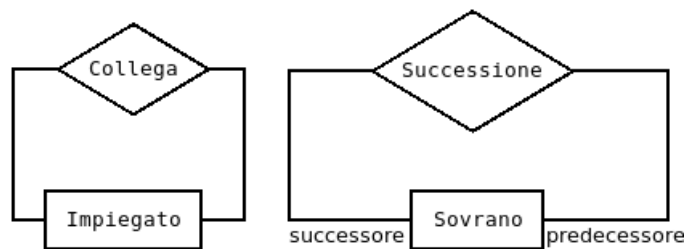


Figura 3.10: Esempi di relazioni ricorsive nel modello E-R

Cardinalità delle relazioni

Vengono specificate per ciascuna partecipazione di entità a una relazione e descrivono il *numero minimo e massimo* di occorrenze di relazione a cui una istanza dell'entità può partecipare. Dicono quindi quante volte, in una relazione tra entità, un'istanza di una di queste entità può essere legata a istanze delle altre entità coinvolte. La notazione per descrivere la cardinalità delle relazioni è la seguente:

- Per la cardinalità minima, zero o uno; nel primo caso si dice che la partecipazione dell'entità relativa è *opzionale*, nel secondo si dice che la partecipazione è *obbligatoria*.
- Per la cardinalità massima, uno o molti (N); nel primo caso la partecipazione dell'entità relativa può essere vista come una funzione (parziale se la cardinalità minima vale zero) che associa ad una istanza dell'entità una sola (o nessuna) istanza dell'altra entità che partecipa alla relazione; nel secondo c'è invece un'associazione con un numero arbitrario di occorrenze dell'altra entità.

Osservando le cardinalità massime, è possibile classificare le relazioni binarie in base al tipo di corrispondenza che viene stabilita tra le istanze delle entità coinvolte.

- Le relazioni aventi cardinalità massima pari a uno per entrambe le entità coinvolte definiscono una corrispondenza uno a uno tra le istanze di tali entità e vengono quindi denominate **relazioni uno a uno**.
- Le relazioni aventi cardinalità massima pari a uno e l'altra con cardinalità massima pari a N sono denominate **relazioni uno a molti**.
- Le relazioni aventi cardinalità massima pari a N per entrambe le entità coinvolte vengono denominate **relazioni molti a molti**.

Un esempio di associazione 1:1 è la relazione *Gestisce* mostrata in figura 3.11. Questo diagramma descrive una situazione in cui un singolo impiegato può gestire (o meno) al massimo un solo dipartimento e un dipartimento deve avere al massimo un solo manager.



Figura 3.11: Una relazione 1:1, Gestisce

L'associazione *LavoraSu* mostrata in figura 3.12 è invece un esempio di relazione molti a molti poiché un singolo impiegato può lavorare su più progetti e un singolo progetto può vedere più impiegati coinvolti in esso.



Figura 3.12: Esempio di relazione M:N, LavoraSu

Relazioni con attributi

Così come le entità, anche le relazioni possono avere degli attributi. Ad esempio, per tenere traccia delle ore settimanali che un impiegato impegna lavorando su un singolo progetto è possibile pensare di introdurre un attributo *Tempo* alla relazione *LavoraSu* vista in figura 3.12. Analogamente si potrebbe introdurre il giorno in cui un manager ha iniziato a gestire un dipartimento attraverso un attributo *GiornoInizio* da inserire alla relazione *Gestisce*.

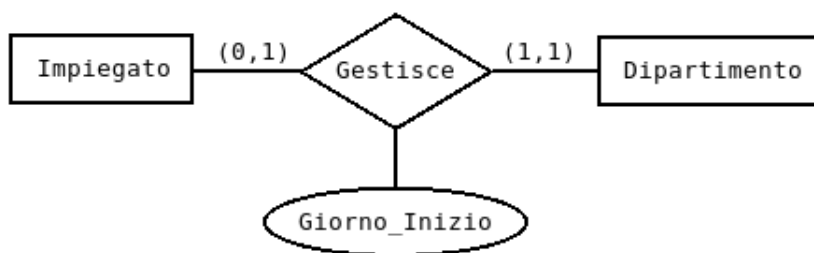


Figura 3.13: Esempio di attributo di una relazione

3.3.4 Cardinalità degli attributi

Così come le partecipazioni delle istanze alle associazioni si possono specificare le cardinalità anche per gli attributi di entità o relazioni. Queste descrivono il *numero minimo e massimo* di valori dell'attributo associati a ogni istanza di entità o relazione. Nella maggior parte dei casi, la cardinalità di un attributo è pari a (1,1) e viene omessa.

È possibile vedere un attributo come una funzione che associa a ogni istanza di entità un solo valore dell'attributo. Sia A un attributo di un tipo di entità E , il cui insieme di valori è V : $A : E \rightarrow \mathcal{P}(V)$ è la funzione che associa un insieme di valori all'istanza E , dove $\mathcal{P}(V)$ è l'insieme delle parti di V . Il valore per un certo attributo può essere quindi anche nullo, essendo $\emptyset \in \mathcal{P}(V)$, oppure possono esistere diversi valori di un certo attributo per una istanza dell'entità. Queste situazioni si rappresentano associando all'attributo in questione una cardinalità minima pari a zero nel primo caso, e una cardinalità massima pari a molti (N) nel secondo.

In maniera simile alle partecipazioni delle occorrenze di entità alle relazioni, diremo che un attributo con cardinalità minima pari a zero è **opzionale** per la relativa entità o relazione, mentre è **obbligatorio** se la cardinalità minima è pari a 1. Diremo che un attributo è **multi-valore** se la sua cardinalità massima è pari a N .

3.3.5 Generalizzazioni

Rappresentano legami logici tra un'entità E , detta entità **genitore**, e una o più entità $E_1, \dots, E_n$ dette entità **figlie** di cui E è più generale, nel senso che le comprende come caso particolare. Si dice in questo caso che E è **generalizzazione** di $E_1, \dots, E_n$ e che le entità $E_1, \dots, E_n$ sono **specializzazioni** dell'entità E . Per esempio, l'entità **PERSONA** è una generalizzazione delle entità **UOMO** e **DONNA**.

Ogni proprietà dell'entità genitore è anche una proprietà delle entità figlie. Il viceversa non è valido. Questa proprietà delle generalizzazioni è nota sotto il nome di **ereditarietà**.

Il modello E-R non prevede costrutti per la rappresentazione grafica delle generalizzazioni, per questo motivo viene estesa attraverso il modello **E-E-R (Extended / Enhanced Entity Relationship model)**.

Le generalizzazioni possono essere classificate sulla base di due proprietà tra loro ortogonali:

- Una generalizzazione è detta **totale** se ogni occorrenza dell'entità genitore è una occorrenza di almeno una delle entità figlie, altrimenti è **parziale**. Quando la specializzazione è totale si usa una doppia linea per collegare l'entità padre all'entità figlia. Una sola linea starà ad indicare invece la specializzazione di tipo parziale.
- Una generalizzazione è **disjointed** se ogni occorrenza dell'entità genitore è al più un'occorrenza di una delle entità figlie, altrimenti è **overlapped**. Graficamente si rappresenta inserendo un cerchio con l'indicazione del tipo di specializzazione tra l'entità padre e l'entità figlia: d (disjoint), o (overlap).
- In questo tipo di modello viene introdotta inoltre il costrutto dell'**unione** quando una entità figlia può essere solo una delle entità di cui è composta.

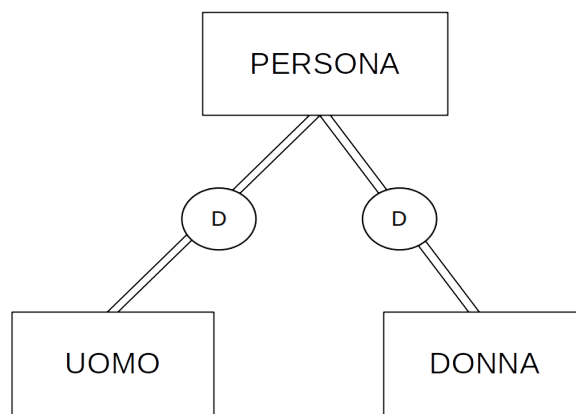


Figura 3.14: Esempio di specializzazione totale e disgiunta

3.4

IL MODELLO UML



UML (Unified Modeling Language) è un linguaggio grafico per la modellazione di applicazioni software basate sulla programmazione orientata agli oggetti. Per il suo successo, UML viene utilizzato per la rappresentazione concettuale di una base di dati. In particolare vengono utilizzati i **class diagram** che descrivono le classi di oggetti di interesse per l'applicazione e le relazioni che intercorrono tra di esse. Molti dei costrutti del modello Entità-Relazioni sono riconducibili a nozioni usate nei class diagram.

3.4.1 Le classi

Sono le componenti principali dei diagrammi delle classi e corrispondono alle entità del modello E-R. Una classe viene rappresentata in UML da un rettangolo contenente, in alto, il nome della classe e, al suo interno, gli attributi a essa associati.

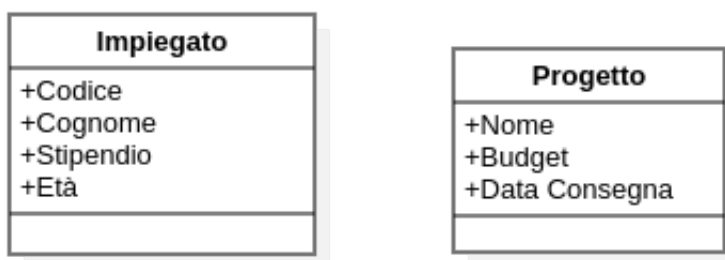


Figura 3.15: Rappresentazione di classi in UML

3.4.2 Le associazioni

Corrispondono alle relazioni del modello E-R e vengono rappresentate come indicato negli esempi in figura 3.16.

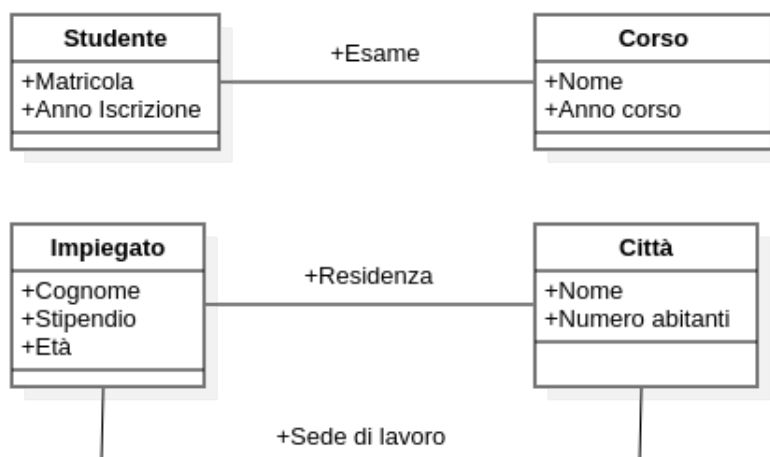


Figura 3.16: Associazioni binarie in UML

Le associazioni binarie si rappresentano con semplici linee che congiungono le classi coinvolte. Il nome della relazione viene generalmente posto sulla linea. Come per le relazioni del modello E-R, si possono definire più associazioni tra le medesime classi ed è anche possibile associare ruoli alle classi coinvolte in un'associazione. Non è invece possibile assegnare attributi alle associazioni. Per fare questo si fa uso delle **classi di associazione** che descrivono proprietà di un'associazione e vengono collegate, mediante una linea tratteggiata, all'associazione da descrivere.

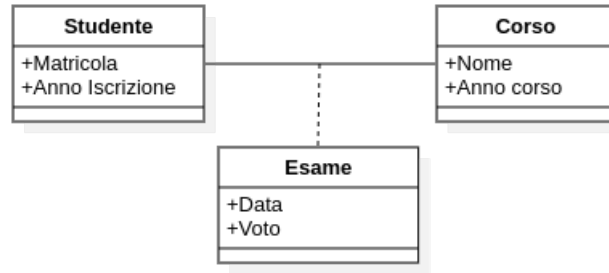


Figura 3.17: Una classe di associazione in UML

Se l'associazione è n-aria si adotta la stessa notazione grafica del modello E-R: l'associazione viene rappresentata da un rombo e da linee che congiungono il rombo con le classi che partecipano all'associazione come in figura 3.18. Le associazioni n-arie si usano molto di rado nei diagrammi delle classi e, quando si incontrano, viene sempre suggerito di **reificarle**, ovvero di trasformare l'associazione in una classe legata alle classi originarie con associazioni binarie.

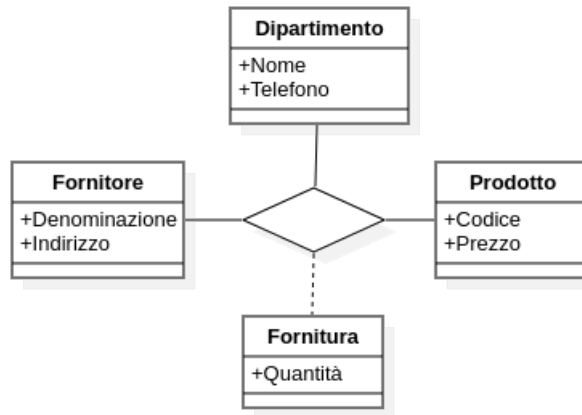


Figura 3.18: Una associazione ternaria in UML

3.4.3 ■ Molteplicità

Con un diagramma delle classi è possibile indicare le cardinalità di partecipazione (qui denominate **molteplicità**) delle classi alle associazioni, secondo le stesse modalità delle cardinalità del modello E-R, ovvero come coppie di valori che specificano la cardinalità minima e massima di partecipazione di un oggetto della classe all'associazione. Le convenzioni adottate nei due formalismi sono però diverse.

- La cardinalità minima viene separata dalla massima non da una virgola ma da due punti
- La cardinalità “molti” viene rappresentata dal simbolo *
- Quando si specifica solo * si intende 0..* ovvero (0,N), mentre con un semplice 1 si denota la coppia di cardinalità 1..1
- In un'associazione binaria, le cardinalità di partecipazione minima e massima di una classe non vengono riportate accanto alla classe stessa, ma accanto all'altra classe che partecipa all'associazione. In altre parole, rispetto a uno schema E-R, le cardinalità delle associazioni binarie nei diagrammi delle classi risultano invertite.

Nella figura 3.19 la molteplicità 0..1 nell'associazione *Vendita* tra le classi *Ordine* e *Fattura* indica che un ordine può avere una fattura associata o nessuna, mentre la molteplicità 1 indica che una fattura ha uno e un solo ordine associato. L'assenza di molteplicità nell'associazione *Residenza* tra *Persona* e *Città*, dalla parte della classe *Città*, sottintende 1..1 cioè il fatto che una persona è residente esattamente in una città. La molteplicità * indica invece che ogni città ha molti residenti ma non può averne nessuno.

Usando la medesima sintassi, è possibile associare molteplicità anche agli attributi delle classi.

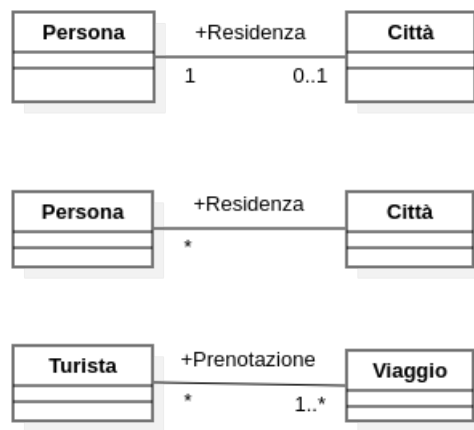


Figura 3.19: Associazioni con molteplicità in UML

3.4.4 ■ Generalizzazioni

Esiste in UML la possibilità di definire generalizzazioni, con modalità molto simili a quelle del modello E-R. Ad esempio la figura 3.20 rappresenta, in termini di UML, lo schema presentato nella figura 3.14. Come viene mostrato le classi figlie della generalizzazione vengono collegate con frecce separate alla classe genitore, ma è ammesso anche unire tali linee. Eventuali proprietà delle generalizzazioni possono essere rappresentate con vincoli, usando la medesima sintassi descritta per gli attributi.

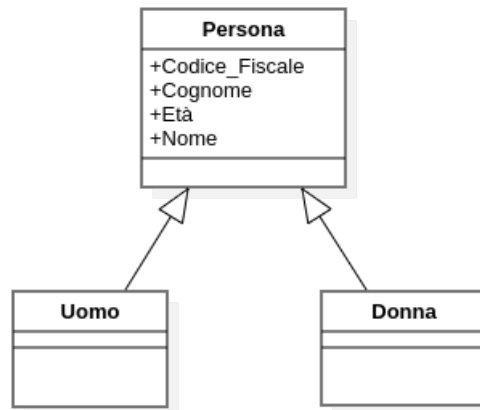
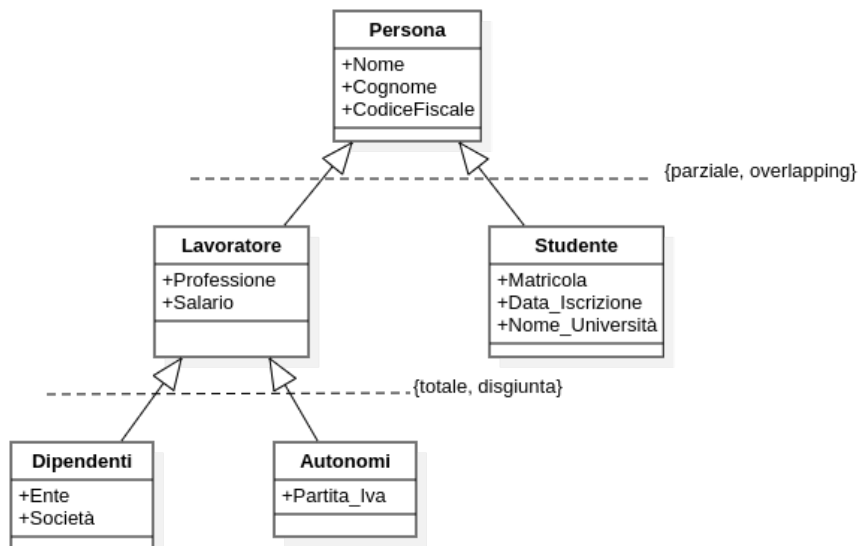


Figura 3.20: Generalizzazione in UML

In modo analogo al modello ER è possibile descrivere in UML costrutti come la specializzazione *parziale/totale* e quella di tipo *overlap/disgiunta*.



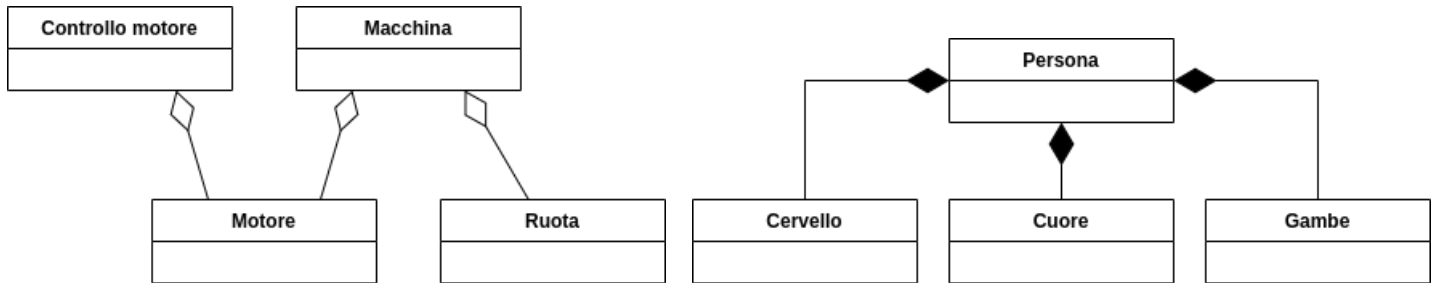
3.4.5 ■ Aggregazione e composizione

Uno strumento interessante di UML è la possibilità di specificare associazioni che sono *aggregazioni* e *composizioni di concetti*. Queste associazioni definiscono una relazione tra un *concetto composto* e uno o più concetti che ne costituiscono una sua *parte*.

L'**aggregazione** e la **composizione** sono due sottoinsiemi delle associazioni; tuttavia, la **composizione** è un ulteriore sottoinsieme della relazione di aggregazione. In parole semplici, l'**aggregazione** può essere definita come quando gli oggetti di una classe possono accedere alle entità di un'altra classe. Mentre la **composizione** rappresenta una relazione bilaterale e l'oggetto fa parte di un solo composto alla volta.

L'**aggregazione** è un'associazione in cui le entità sono configurate insieme per creare un oggetto più complesso. Attraverso di essa si descrive un gruppo di oggetti e il loro collegamento comune. Definisce un unico punto di controllo chiamato **aggregato**, e il gruppo di oggetti che rappresenta l'**insieme**. In un diagramma UML, i dati fluiscono dall'**aggregato (classe madre)** verso l'**insieme (classe figlia)** con una forma di diamante (non riempita) vicino alla prima classe. Il classificatore di parti, o le entità di assemblaggio, possono essere collegate a più di un aggregato ed esistere indipendentemente.

Le **composizioni** sono un tipo di associazione che rappresentano una relazione bidirezionale e altamente dipendente tra due classi diverse. Se la composizione viene cancellata, anche le altre entità ad essa collegate vengono cancellate. Esso rappresenta la **dipendenza tra il genitore e il figlio**. In un diagramma UML si disegna una linea tra la classe genitore e la classe figlio con una forma di diamante riempito vicino alla classe genitore.



(a) Questo esempio di aggregazione mostra che la macchina ha sempre bisogno di una ruota e di un motore per funzionare correttamente, mentre il motore e la ruota non hanno sempre bisogno di una macchina. Possono essere usati con un autobus, una bicicletta, macchine, tosaerba e molto altro. Allo stesso modo, il motore è necessario per un controllo del motore, ma il motore può esistere come entità indipendente ed essere associato anche ad altri veicoli.

(b) Esempio di composizione: una persona ha gambe, un cuore e un cervello. Se una persona viene a mancare, anche tutte le entità composte saranno scartate. A differenza dell'aggregazione, le entità non possono esistere indipendentemente e sono una "parte della" persona.

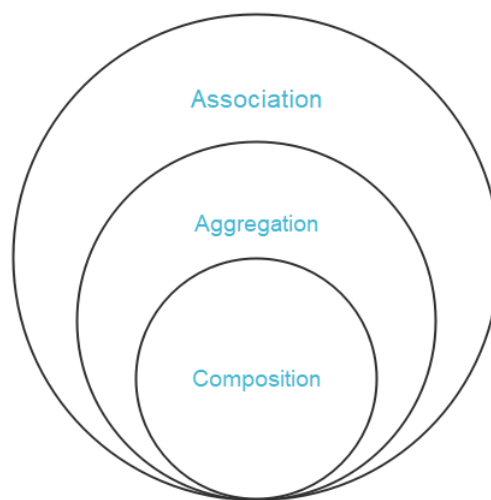


Figura 3.23

IL MODELLO RELAZIONALE

Il modello relazionale dei dati fu formulato nel 1970 da Edgar Frank “Ted” Codd¹, ricercatore della IBM, ed ebbe subito una rapida diffusione grazie alla sua semplicità e alle sue solide basi matematiche. Il modello usa il concetto di *relazione matematica*, che ha una naturale rappresentazione come una **tabella**, e si fonda interamente sulla teoria degli insiemi e l'algebra relazionale.

4.1

CONCETTI DEL MODELLO RELAZIONALE



Il modello relazionale rappresenta il database come una collezione di *relazioni*. Quando una relazione è vista come una tabella, ogni riga rappresenta una collezione di *dati aggregati*. Il nome della tabella e i nomi delle colonne sono usati per aiutare ad interpretare il significato dei valori in ciascuna cella.

Nella terminologia formale del modello relazionale, una *riga* viene chiamata **tupla** oppure **ennupla**, il *nome di una colonna* viene chiamato **attributo** mentre la tabella viene chiamata **relazione**. Il *tipo di dato* che descrive il tipo di valore che può apparire in ciascuna colonna è rappresentato da un **dominio** di valori possibili.

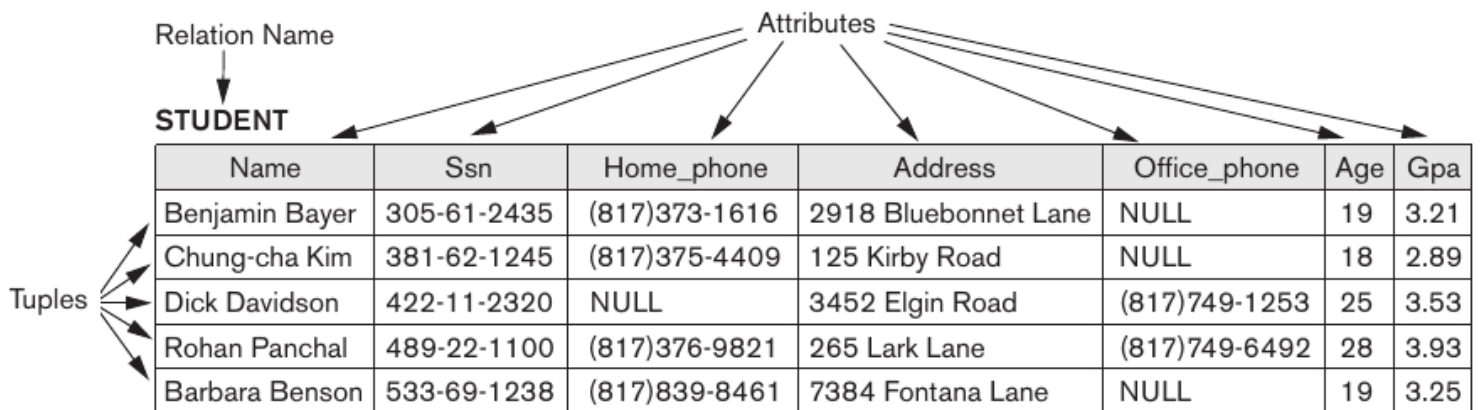


Figura 4.1: Attributi e tuple della relazione Student

4.2

DEFINIZIONE DEL MODELLO RELAZIONALE



Come già anticipato, il modello relazionale si basa sul concetto di relazione come definita nella teoria degli insiemi.

Si dice che r è una **relazione** sugli insiemi di valori $D_1, \dots, D_n$, non necessariamente distinti, se è un sottoinsieme del prodotto cartesiano $D_1 \times D_2 \times \dots \times D_n$. In simboli:

$$r \subseteq D_1 \times D_2 \times \dots \times D_n$$

Gli insiemi $D_1, \dots, D_n$ sono chiamati **domini** della relazione, ovvero insieme di valori atomici. Per *atomico* si intende il fatto che ciascun valore nel dominio è indivisibile. Un metodo comune attraverso il quale si specifica un dominio è quello di indicare un tipo di dato dai quali i valori possono essere estratti. È anche utile specificare un nome al dominio, per aiutare ad interpretare meglio i valori.

¹Portland, 23 agosto 1923 – Williams Island, 18 aprile 2003

Alcuni esempi di domini possono essere:

- *NumeriTelefonici*: l'insieme di numeri telefonici composti da 10 cifre
- *Nomi*: l'insieme delle stringhe di caratteri rappresentanti nomi propri di persone
- *Voti*: l'insieme dei numeri interi compresi tra 18 e 30

Questi sono esempi di *definizioni logiche* di domini. È possibile specificare anche un **tipo di dato** o **formato** per ogni dominio. Ad esempio, il formato per il dominio *NumeriTelefonici* può essere dichiarato come una stringa della forma $((ddd)ddd-dddd)$, dove d è un numero.

Uno **schema relazionale** R , detto anche **schema logico** di R , denotato da $R(A_1, A_2, \dots, A_n)$ è composto da un nome di una relazione R e una lista di attributi, $A_1, \dots, A_n$. Ogni **attributo** A_i è il nome di un ruolo esercitato da qualche dominio D nello schema relazionale R . D è chiamato **dominio** di A_i ed è denotato con $\text{dom}(A_i)$.

Uno schema relazionale viene utilizzato per descrivere una relazione; R è detto **nome** di questa relazione. Il **grado** (o **arità**) di una relazione è il numero di attributi n del suo schema relazionale.

Esempio

La relazione *Studenti* composta dagli attributi:

- Matricola
- Cognome
- Nome
- Data di nascita

si rappresenta secondo il seguente **schema logico**:

Studenti			
Matricola	Cognome	Nome	Data di nascita

Secondo la definizione data:

1. Una relazione è un insieme di ennuple; pertanto, non è definito nessun ordinamento tra le ennuple (in un insieme l'ordine degli elementi è irrilevante) e le ennuple sono tra loro distinte;
2. Più precisamente, una relazione è un insieme di ennuple ordinate (cioè l' i -esimo valore di ogni ennupla appartiene all' i -esimo dominio); pertanto l'ordinamento tra i domini di una relazione è, dal punto di vista insiemistico, rilevante.

4.2.1 ■ Caratteristiche delle relazioni

Valori nulli

Ogni valore in una tupla è atomico; ovvero non può essere diviso in componenti più piccole. Per questo motivo gli attributi composti e multi-valore non sono ammessi nel modello relazionale. Per questo motivo il modello relazionale viene anche chiamato **piatto**.

Un concetto importante è quello dei valori NULL, i quali sono utilizzati per rappresentare i valori degli attributi che possono essere sconosciuti o non possono essere applicati alla tupla. In questi casi si inserisce il valore NULL all'interno della cella.

Notazione del modello relazionale

Si usa a seguente notazione nella rappresentazione degli schemi relazionali:

- Uno schema relazionale R di grado n è denotato da $R(A_1, \dots, A_n)$
- Le lettere maiuscole Q, R, S denotano i nomi delle relazioni
- Le lettere t, u, v denotano le tuple

4.3

VINCOLI E SCHEMI

In un database relazionale esistono tipicamente molte relazioni e le tuple in esse contenute sono spesso associate in qualche modo. Lo **stato del database** corrisponde all'insieme di tutti gli stati delle sue relazioni in un momento in particolare. Ci sono spesso molte restrizioni o **vincoli** sui valori attuali nello stato del database. Questi vincoli sono derivati dalle regole del mini-mondo che il database rappresenta.

Distinguiamo due categorie:

1. Un vincolo è **intrarelazionale** se il suo soddisfacimento è definito rispetto a singole relazioni della base di dati
2. Un vincolo è **interrelazionale** se coinvolge più relazioni.

4.3.1 Vincoli di tupla e di dominio

I **vincoli di tupla** sono un tipo di vincolo intrarelazionale che esprimono condizioni sui valori di ciascuna tupla, indipendentemente dalle altre tuple.

Una possibile sintassi per esprimere vincoli di questo tipo è quella che permette di definire espressioni booleane con atomi che confrontano valori di attributo o espressioni aritmetiche su valori di attributo.

Un vincolo definito con riferimento a singoli valori viene detto **vincolo di dominio**, in quanto impone una restrizione sul dominio dell'attributo.

4.3.2 Vincoli di chiave o di valore NULL

Nel modello relazionale, una relazione è definita attraverso un *insieme di tuple*. Per definizione, tutti gli elementi di un insieme sono distinti, quindi tutte le tuple in una relazione devono essere distinte. Questo significa che due tuple non possono avere la stessa combinazione di valori per tutti i loro attributi. Per distinguere una tupla da un'altra si individua un **sottoinsieme di attributi** dello schema relazionale per il quale due tuple in uno stato della relazione r in R non possano avere gli stessi valori. Chiamato questo sottoinsieme SK , allora per ogni coppia distinta di tuple t_1 e t_2 in uno stato di relazione r di R si ha il seguente vincolo:

$$t_1[SK] \neq t_2[SK]$$

Ogni sottoinsieme di attributi di questo tipo viene chiamato **superchiave** dello schema relazionale. Una superchiave SK specifica un **vincolo di unicità**: due tuple distinte non possono avere gli stessi valori per SK . Ogni relazione ha almeno una superchiave di default: l'insieme di tutti i suoi attributi. Una superchiave può avere attributi ridondanti, per questo motivo è più comodo ricercare il sottoinsieme più piccolo che garantisca il vincolo di unicità senza ridondanza. Una **chiave** K di uno schema relazionale R è una superchiave di R con le seguenti proprietà:

1. Due distinte tuple in ciascuno stato della relazione non può avere gli stessi valori per tutti gli attributi nella chiave. Questo garantisce che K è una superchiave.
2. È una **superchiave minimale**, ovvero una superchiave dalla quale sono stati rimossi tutti gli attributi ridondanti.

In generale, uno schema relazionale può avere più di una singola chiave. In questo caso, ciascuna chiave viene chiamata **chiave candidata**. Tra le chiavi candidati se ne sceglierà una che verrà chiamata **chiave primaria**. La chiave primaria è la chiave candidata che verrà usata per identificare in maniera univoca le tuple nella relazione. All'interno degli schemi logici le chiavi primarie vengono evidenziate attraverso la sottolineatura degli attributi identificanti.

Esempio

Nel caso dello schema della relazione *Studenti* si ha che l'attributo *Matricola* rappresenti una superchiave ma anche una superchiave minimale, in quanto contiene un solo attributo. Quindi sarà la chiave primaria per la relazione:

Studenti			
<u>Matricola</u>	Cognome	Nome	Data di nascita

4.3.3 Schemi del database relazionale

Uno **schema di un database relazionale** S è un insieme di schemi relazionali $S = \{R_1, \dots, R_m\}$ e un insieme di **vincoli di integrità** IC . Uno **stato di un database relazionale** DB di S è un insieme di stati di relazioni $DB = \{r_1, \dots, r_m\}$ tale che ciascun r_i è uno stato di R_i e tale che gli r_i stati di relazioni soddisfano i vincoli di integrità specificati in IC .

4.3.4 Vincoli di integrità e chiavi esterne

Il **vincolo di integrità di entità** afferma che nessuna chiave primaria può avere un valore NULL. Questo perché la chiave primaria viene usata per identificare le tuple in una relazione. Avere un valore NULL in una chiave primaria significa non poter identificare qualche tupla.

I vincoli di chiave e i vincoli di integrità vengono specificati sulle singole relazioni. Il **vincolo di integrità referenziale** viene specificato tra due relazioni ed è utilizzato per mantenere la consistenza tra le tuple nelle due relazioni. Il vincolo di integrità referenziale obbliga una tupla presente in una relazione che è associata ad un'altra relazione a riferirsi ad una *tupla esistente* della seconda relazione.

Per definire il vincolo di integrità referenziale si introduce il concetto di **chiave esterna**. Un insieme di attributi FK in uno schema relazionale R_1 è una **chiave esterna** di R_1 che è **associata** alla relazione R_2 se soddisfa le seguenti regole:

1. Gli attributi in FK hanno gli stessi domini degli attributi nella chiave primaria PK di R_2 ; gli attributi FK **si riferiscono** alla relazione R_2 .
2. Un valore di FK in una tupla t_1 di uno stato corrente $r_1(R_1)$ o si trova come valore di PK per qualche tupla t_2 nello stato corrente $r_2(R_2)$ o è NULL. Nel primo caso si avrà

$$t_1[FK] = t_2[PK]$$

e si dice che la tupla t_1 **referenzia** la tupla t_2 .

In questo caso, R_1 viene chiamata **relazioni referente** ed R_2 è la **relazione referenziata**. Se queste due condizioni sono tenute, si dice che vale un **vincolo di integrità referenziale**.

LA PROGETTAZIONE LOGICA

Una volta finita la fase della progettazione concettuale e selezionato un particolare modello per la rappresentazione logica dei dati si entra nella fase della **progettazione logica**. Questa si distingue in due fasi:

1. **Ristrutturazione** del modello concettuale e sua ristrutturazione;
2. **Mapping** del modello concettuale ristrutturato in schema logico.

In questo capitolo ci concentreremo sulla seconda fase della progettazione: il **mapping** dallo schema concettuale ristrutturato allo schema logico.

5.1

RISTRUTTURAZIONE DEL MODELLO CONCETTUALE



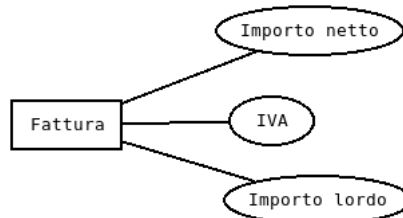
La fase di ristrutturazione di uno schema concettuale si può suddividere in una serie di passi da effettuare in sequenza:

- **Analisi delle ridondanze:** si decide se eliminare o mantenere eventuali ridondanze presenti nello schema.
- **Eliminazione delle generalizzazioni:** tutte le generalizzazioni presenti nello schema vengono sostituite da altri costrutti.
- **Partizionamento/accorpamento di entità e associazioni:** si decide se è opportuno partizionare concetti dello schema (entità e/o associazioni) in più concetti o, viceversa accorpate concetti separati in un unico concetto.
- **Scelta degli identificatori principali:** si seleziona un identificatore per quelle entità che ne hanno più di uno.

5.1.1 ■ Analisi delle ridondanze

Una ridondanza in uno schema concettuale corrisponde alla presenza di un dato che può essere derivato da altri dati. I casi più frequenti sono i seguenti:

1. Attributi derivabili da altri attributi della stessa entità o associazione. Ad esempio nell'entità *Fattura* uno degli attributi è deducibile dagli altri attraverso una operazione di somma o differenza.



2. Attributi derivabili da attributi di altre entità (o associazioni), di solito attraverso funzioni aggregative. L'attributo **Importo totale** dell'entità *Acquisto* si può derivare, attraverso l'associazione *Composizione*, dall'attributo Prezzo dell'entità *Prodotto*, sommando i prezzi dei prodotti di cui un acquisto è composto.

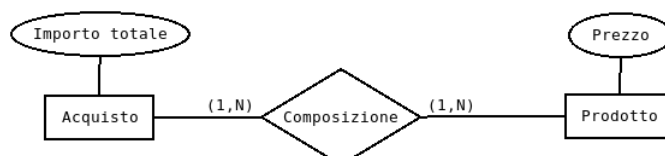
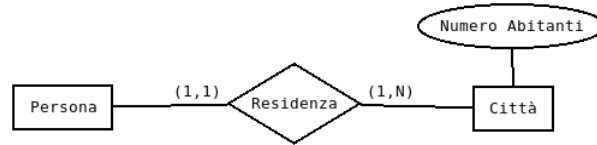
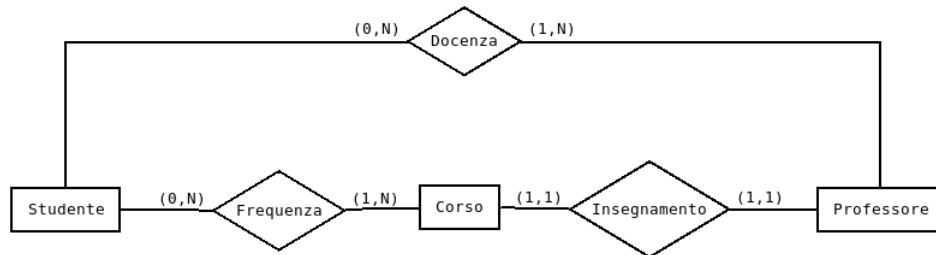


Figura 5.1

3. Attributi derivabili da operazioni di conteggio di occorrenze. L'attributo Numero di abitanti di una città può essere derivato contando le occorrenze della associazione *Residenza* a cui tale città partecipa.



4. Associazioni derivabili dalla composizione di altre associazioni in presenza di cicli. L'associazione *Docenza* tra studenti e professori può essere derivata dalle associazioni *Frequenza* e *Insegnamento*. Va precisato però che la presenza di cicli non genera a priori una ridondanza.



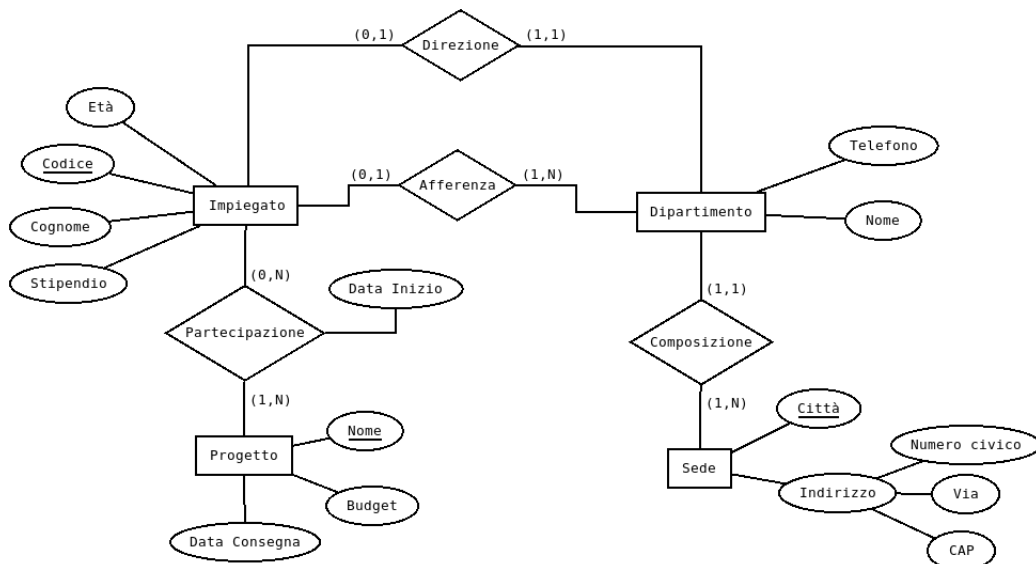
La presenza di un dato derivato presenta un vantaggio e alcuni svantaggi. Il vantaggio è la *riduzione degli accessi necessari* per calcolare il dato derivato, gli svantaggi sono una *maggiore occupazione di memoria* e la *necessità di effettuare operazioni aggiuntive per mantenere il dato derivato aggiornato*.

La decisione di mantenere o eliminare una ridondanza va presa confrontando il costo di esecuzione delle operazioni che coinvolgono il dato ridondante e relativa occupazione di memoria, nei casi di presenza e assenza della ridondanza.

- Il **costo di una operazione** viene valutato in termini di numero di occorrenze di entità e associazioni che mediamente vanno visitate per rispondere a una operazione sulla base di dati;
- L'**occupazione di memoria** viene valutata in termini dello spazio di memoria (misurato per esempio in numero di byte) necessario per memorizzare i dati descritti dallo schema.

Per studiare questi parametri è necessario conoscere, oltre allo schema, le seguenti informazioni:

- il **volume dei dati**, vale a dire
 - numero di occorrenze di ogni entità e associazione nello schema;
 - dimensioni di ciascun attributo (di entità o associazione).
- le **caratteristiche delle operazioni**. Vale a dire:
 - tipo delle operazioni (interattiva o *batch*¹);
 - frequenza (numero medio di esecuzioni in un certo intervallo di tempo);
 - dati coinvolti (entità e/o associazioni).



¹Operazioni pianificate da altri programmi in particolari momenti della giornata

Prendiamo in considerazione lo schema concettuale rappresentato in figura 5.2. Trattandosi di uno schema riguardante dati sul personale di un'azienda, le operazioni possibili potrebbero essere:

1. Assegnare un impiegato a un progetto
2. Trovare i dati di un impiegato, del dipartimento nel quale lavora e dei progetti ai quali partecipa
3. Trovare i dati di tutti gli impiegati di un certo dipartimento
4. Per ogni sede, trovare i suoi dipartimenti con il cognome del direttore e l'elenco degli impiegati del dipartimento.

Sebbene un'analisi delle prestazioni che fa riferimento a un numero ristretto di operazioni può sembrare riduttiva rispetto al reale carico della base di dati, va notato che le operazioni sulle basi di dati seguono la cosiddetta **regola "ottanta-venti"**. In base a questa regola, l'80% del carico è generato dal 20% delle operazioni.

Il volume dei dati e le caratteristiche generali delle operazioni vengono descritte facendo uso delle **tavole dei volumi** e delle **tavole delle operazioni**. Nella *tavola dei volumi* vengono riportati tutti i concetti dello schema (entità e relazioni) con il volume previsto a regime. Nella *tavola delle operazioni* riportiamo, per ogni operazione, la frequenza prevista e un simbolo che indica se l'operazione è interattiva (I) o batch (B). Nella tavola dei volumi, il numero delle occorrenze delle associazioni dipende da due parametri: il numero di occorrenze delle entità coinvolte nelle associazioni e il numero (medio) di partecipazioni di una occorrenza di entità alle occorrenze di associazioni. Il secondo parametro dipende a sua volta dalle cardinalità delle associazioni.

Per esempio, il numero di occorrenze dell'associazione *Composizione* è pari al numero dei dipartimenti, perché le cardinalità ci dicono che un dipartimento appartiene a una sola sede. Il numero di occorrenze dell'associazione *Afferenza* è invece poco meno del numero degli impiegati, perché dalle cardinalità si evince che ci sono impiegati che non afferiscono a nessun dipartimento. Infine, assumendo che un impiegato partecipa in media a tre progetti, abbiamo $2000 \times 3 = 6000$ occorrenze per l'associazione *Partecipazione* (e quindi $6000/500 = 12$ impiegati in media su ogni progetto).

Tavola dei volumi		
Concetto	Tipo	Volume
Sede	E	10
Dipartimento	E	80
Impiegato	E	2000
Composizione	R	80
Afferenza	R	1900
Direzione	R	80
Partecipazione	R	6000

Tavola delle operazioni		
Operazioni	Tipo	Frequenza
Op.1	I	50/giorno
Op.2	I	100/giorno
Op.3	I	10/giorno
Op.4	B	2/settimana

Tabella 5.1: Esempio di tavole dei volumi e delle operazioni

Per calcolare il costo di ogni operazione andrà costruita successivamente, per ogni operazione, la relativa **tavola degli accessi** basata su uno **schema di navigazione**. Lo schema di navigazione è la parte dello schema concettuale interessata dall'operazione, estesa con delle frecce che indicano in che modo l'operazione "naviga" i dati. Per ogni entità e associazione interessata dall'operazione si riporta il *numero di istanze interessate* e il *tipo di accesso* (L: lettura; S: scrittura). Il numero delle istanze si ricava dalla tavola dei volumi.

Tavola degli accessi			
Concetto	Costrutto	Accessi	Tipo
Impiegato	E	1	L
Afferenza	R	1	L
Dipartimento	E	1	L
Partecipazione	R	3	L
Progetto	E	3	L

Tabella 5.2: Esempio di tavola degli accessi

È importante fare la distinzione tra accesso in lettura e in scrittura perché, generalmente, le operazioni di scrittura sono più onerose di quelle in lettura. Considerando lo schema su *Persone e Città* visto nell'esempio 5.1 e supponiamo che faccia riferimento a una applicazione anagrafica di una regione italiana per la quale sono state definite le seguenti operazioni principali:

1. Memorizza una nuova persona con la relativa città di residenza;
2. Stampa tutti i dati di una città (incluso il numero di abitanti).

Supponiamo inoltre che per questa applicazione i dati di carico siano quelli riportati nella tabelle riportanti nella Tabella 5.3.

Tavola dei volumi		
Concetto	Tipo	Volume
Città	E	200
Persona	E	1 000 000
Residenza	R	1 000 000

Tavola delle operazioni		
Operazione	Tipo	Frequenza
Operazione 1	I	500/giorno
Operazione 2	I	2/giorno

Figura 5.3: Tavole dei volumi e delle operazioni per lo schema dell’esempio 5.1

A questo punto proviamo a valutare gli *indici di prestazione*² in caso di presenza del dato ridondante (attributo NUMERO DI ABITANTI nell’entità *Città*).

Assumendo che il numero degli abitanti di una città richieda 4 byte abbiamo che il dato ridondante richiede $4 \times 200 = 800$ byte, ovvero meno di 1 KB di memoria aggiuntiva. Passiamo ora alla stima del costo delle operazioni costruendo le tavole degli accessi in caso di presenza e assenza della ridondanza. Come descritto nella tavola 5.3 l’operazione 1 richiede un accesso in scrittura all’entità *Persona* (per memorizzare una nuova persona), un accesso in scrittura all’entità *Residenza* (per memorizzare una nuova coppia persona-città) e infine un accesso in lettura (per cercare la città d’interesse) e uno in scrittura (per incrementare di uno il numero degli abitanti di quella occorrenza) all’entità *Città*, il tutto ripetuto per 500 volte al giorno, per un totale di 1500 accessi in scrittura e 500 accessi in lettura. Il costo dell’operazione 2 è praticamente trascurabile perché richiede un solo accesso in lettura all’entità *Città* da ripetere due volte al giorno. Supponendo che un accesso in scrittura abbia un costo doppio rispetto ad un accesso in lettura, abbiamo un totale di 3500 accessi al giorno in caso di presenza di dato ridondante.

Consideriamo ora il caso in cui il dato ridondante sia assente. Per l’Operazione 1 abbiamo bisogno di un accesso in scrittura all’entità *Persona* e un accesso in scrittura all’associazione *Residenza* (non c’è infatti bisogno di accedere all’entità *Città* per aggiornare il dato derivato), per un totale di 1000 accessi in scrittura al giorno. Per l’Operazione 2 abbiamo invece bisogno di un accesso in lettura all’entità *Città* (per avere i dati della città), che possiamo trascurare, e di 5000 accessi in lettura all’associazione *Residenza* in media (ottenuto dividendo il numero di persone per il numero di città) per calcolare il numero di abitanti di questa città, per un totale di 10000 accessi in lettura al giorno. Contando doppi gli accessi in scrittura abbiamo un totale di 12000 accessi al giorno in caso di dato ridondante assente. Quindi, circa 8500 accessi giornalieri in più rispetto al caso di dato ridondante presente contro un risparmio di un solo KB. Questo dipende dal fatto che gli accessi in lettura necessari per calcolare il dato derivato sono molto di più degli accessi in scrittura necessari per mantenerlo aggiornato. Possiamo quindi concludere che conviene, in questo caso, mantenere il dato ridondante.

Tavole degli accessi in presenza di ridondanza			
Operazione 1			
Concetto	Costrutto	Accessi	Tipo
Persona	E	1	S
Residenza	R	1	S
Città	E	1	L
Città	E	1	S
Operazione 2			
Concetto	Costrutto	Accessi	Tipo
Città	E	1	L

Tavole degli accessi in assenza di ridondanza			
Operazione 1			
Concetto	Costrutto	Accessi	Tipo
Persona	E	1	S
Residenza	R	1	S
Operazione 2			
Concetto	Costrutto	Accessi	Tipo
Città	E	1	L
Residenza	R	5000	L

Tabella 5.3: Tavole degli accessi per lo schema su dati anagrafici

5.1.2 ■ Eliminazione delle generalizzazioni

Dato che i sistemi tradizionali per la gestione delle basi di dati non consentono di rappresentare direttamente una generalizzazione, risulta spesso necessario trasformare questo costrutto in altri costrutti del modello E-R o UML. Per rappresentare una generalizzazione esistono essenzialmente tre alternative possibili.

Accorpamento verso l’alto

Si consideri il caso di uno schema concettuale come in figura 5.4.

²Parliamo di indici di prestazione e non di prestazione perché, in realtà, le prestazioni di una base di dati non sono valutabili in maniera precisa in sede di programmazione logica, in quanto dipendenti anche da parametri fisici che non si tengono in considerazione a questo livello.

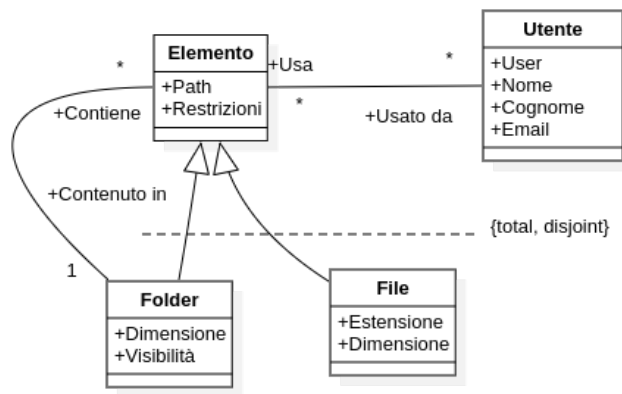


Figura 5.4: Esempio di schema concettuale di un filesystem

La classe *Elemento* ha due sottoclassi figlie *Folder* e *File*. Per rimuovere la generalizzazione è possibile pensare di eliminarle e spostare le loro proprietà (attributi e partecipazioni ad associazioni e generalizzazioni) all'entità genitore *Elemento*. A tale entità viene aggiunto un ulteriore attributo discriminante che servirà a distinguere il "tipo" di una sua istanza. Per farlo, in UML, è possibile inserire una enumerazione contenente le varie tipologie di specializzazioni (disgiunte o sovrapposte). Successivamente si *ricontrollano le opzionalità* degli attributi per tener conto del fatto che sono diventati opzionali. Infatti se un *Elemento* è di tipo FOLDER non avrà senso avere un valore per l'attributo DIMENSIONE_FILE. Si ottiene quindi uno schema concettuale ristrutturato come in figura 5.4.

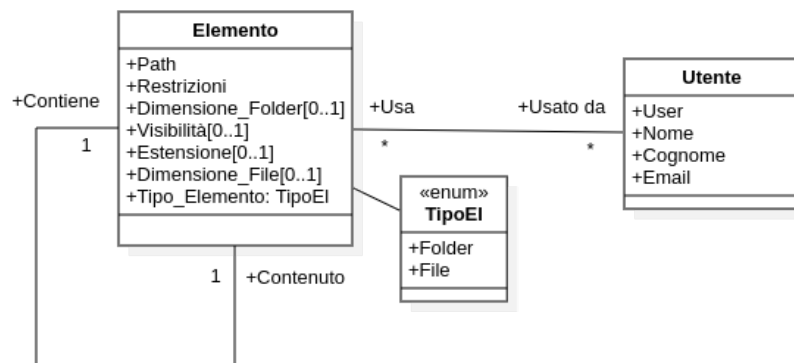


Figura 5.5: Esempio di accorpamento verso l'alto

Alla fine di una ristrutturazione del genere sarà necessario poi introdurre **vincoli di integrità**:

- Solo l'elemento di tipo Folder può avere associazioni di tipo "contiene";
- Se TipoEl è Folder allora Estensione e Dimensione dovranno essere NULL;
- Se TipoEl è File allora Dimensione_Folder e Visibilità devono essere NULL.

Questa tecnica va bene anche per casi partial/overlapping. Vanno però modificate le cardinalità delle associazioni coinvolte e specificati opportunamente i valori dell'attributo discriminante. Ad esempio il modello concettuale nella figura 5.6 vedrà l'inserimento di una enumerazione con quattro attributi:

- **Tipo Lavoratore**: per le persone che lavorano soltanto
- **TipoStudente**: per le persone che studiano soltanto
- **TipoStudenteLavoratore**: per descrivere la sovrapposizione dei due tipi
- **NoStudenteNoLavoratore**: per descrivere la parzialità della specializzazione

Sostituzione della generalizzazione con associazioni

In questo caso la specializzazione si trasforma in due associazioni uno a uno che legano rispettivamente l'entità genitore con le entità figlie. Non ci sono trasferimenti di attributi o associazioni e le entità figlie sono identificate esternamente dall'entità padre. Nello schema ottenuto vanno aggiunti però dei vincoli di integrità: ogni occorrenza della classe padre non può partecipare contemporaneamente ad entrambe le associazioni; inoltre, se la generalizzazione è totale, ogni occorrenza della classe padre deve partecipare a solo una associazione. Nel caso dell'esempio già visto nella figura 5.4 si ottiene quindi una ristrutturazione come in figura 5.8. La scelta tra le varie alternative può essere fatta in maniera analoga a quanto fatto per i dati derivati, considerando vantaggi e svantaggi di ognuna delle scelte possibili relativamente alla occupazione di memoria e al costo delle operazioni coinvolte.

Partizionamento/accorpamento di concetti

Entità, classi e associazioni possono essere partizionati o accorpati per garantire una maggiore efficienza delle operazioni in base al seguente principio: gli accessi si riducono separando attributi di uno stesso concetto che vengono acceduti da operazioni diverse e raggruppando attributi di concetti diversi che vengono acceduti dalle medesime operazioni. Si possono usare le stesse tecniche viste per l'analisi delle generalizzazioni.

Partizionamento di entità

Un esempio di partizionamento viene mostrato in figura 5.9: l'entità IMPIEGATO viene sostituita da due entità, collegate da un'associazione uno a uno, che descrivono rispettivamente i dati anagrafici degli impiegati e i dati relativi alla loro retribuzione. Questa ristrutturazione è conveniente se le operazioni che coinvolgono frequentemente l'entità originaria richiedono, per un impiegato, o solo informazioni di carattere anagrafico, o solo informazioni relative alla sua retribuzione.

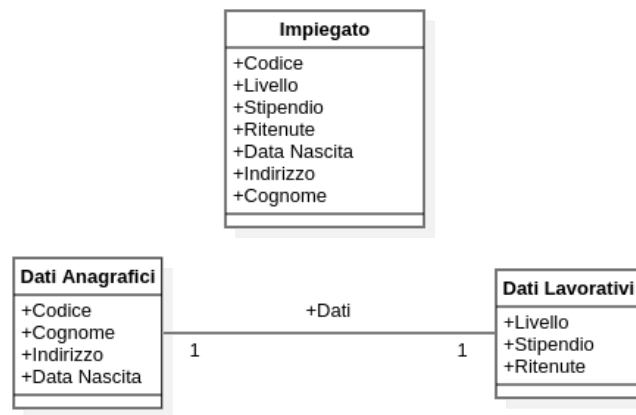


Figura 5.9

Un partizionamento di questo tipo è un esempio di **decomposizione verticale** di una entità. La decomposizione verticale non è unica, esiste anche la **decomposizione orizzontale** che consiste nel duplicare le colonne come nel caso dell'accorpamento verso il basso della classe padre nelle sottoclassi.

Eliminazione degli attributi strutturati e multi-valore

Un particolare tipo di partizionamento che è opportuno trattare a parte è quello che riguarda l'eliminazione di attributi strutturati e multi-valore. Questa ristrutturazione si rende necessaria perché, come per le generalizzazioni, il modello relazionale non permette di rappresentare in maniera diretta questi tipi di attributo. Prendiamo l'esempio mostrato in Figura 5.10.

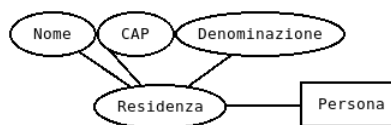


Figura 5.10: Residenza è un attributo strutturato perché composto da tre attributi

Per la rimozione degli attributi strutturati si può procedere in tre maniere:

- Creare una nuova entità per l'attributo strutturato e inserire un'associazione tra la vecchia entità e la nuova (vedi Figura 5.11a).
- Raggruppare tutto all'interno dell'entità (vedi Figura 5.11b).
- Rimuovere l'attributo strutturato e considerarlo atomico (vedi Figura 5.11c).

Come nel caso del partizionamento/accorpamento delle entità non esiste una scelta migliore. Tutto dipende dal tipo di accesso che viene fatto sull'entità e i suoi attributi. Nel caso della prima scelta si separano e si riducono le dimensioni delle entità,

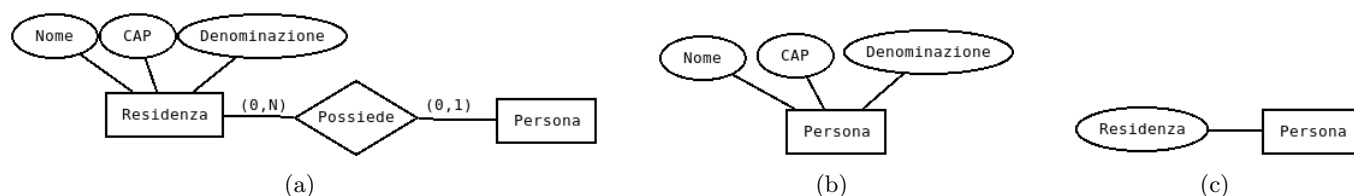


Figura 5.11

prevedendo di accedere solo raramente in modo contemporaneo ad entrambi i dati. Nel secondo caso si prevede un recupero ripetuto di tutte le informazioni mentre si usa una stringa unica quando non si è interessati nell'accesso ai singoli campi. Non è detto comunque che non si possa spaccettare l'informazione in maniera programmatica dall'esterno per ottenere le singole informazioni che altrimenti non sarebbero più accessibili.

5.1.3 Scelta degli identificatori principali

La scelta degli identificatori principali è essenziale nelle traduzioni verso il modello relazionale perché, come discusso nel Capitolo 4, in questo modello le chiavi vengono usate per stabilire legami tra dati in relazioni diverse. Inoltre, i sistemi di gestione di basi di dati richiedono generalmente di specificare una *chiave primaria* sulla quale vengono costruite automaticamente delle strutture ausiliarie, dette **indici**, per il reperimento efficiente dei dati. I criteri per la scelta delle chiavi sono i seguenti:

- Gli attributi con valori nulli non possono essere delle chiavi primarie.
- Un identificatore composto da uno o pochi attributi è da preferire a identificatori composti da molti attributi.
- Meglio avere un identificatore interno piuttosto che uno esterno.
- Un identificatore che viene utilizzato da molte operazioni per accedere alle occorrenze è da preferire rispetto agli altri.

A questo punto, se nessuno degli identificatori candidati soddisfa tali requisiti, è possibile pensare di introdurre una **chiave surrogata**: questo attributo conterrà valori speciali (detti **codici**) generati appositamente per identificare le occorrenze delle entità.

5.2

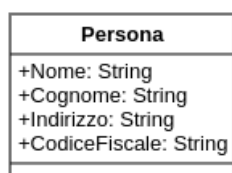
TRADUZIONE VERSO IL MODELLO RELAZIONALE



5.2.1 Mapping dei tipi di entità regolari

Per ogni tipo di entità regolare (forti) E nello schema concettuale (UML o ER) viene creata una relazione R che include tutti gli attributi semplici di E . Successivamente si seleziona una chiave primaria per R . Se la chiave scelta per E è composta allora l'insieme degli attributi semplici sarà la chiave primaria di R . Le relazioni che vengono create dal mapping dei tipi di entità sono spesso chiamate **relazioni di identità** poiché ogni tupla rappresenta un'istanza di una entità.

Ad esempio il class diagram rappresentante l'entità persona mostrato in figura:



viene mappato nel seguente schema relazionale:

Persona			
Nome	Cognome	Indirizzo	CodiceFiscale

5.2.2 Codifica delle associazioni

Si distinguono tre casi:

1. Associazioni molti a molti
2. Associazioni uno a molti

Codifica delle associazioni molti a molti

Per ciascuna relazione R di tipo molti a molti viene creata una nuova relazione, chiamata **tabella ponte** per rappresentare R . Questa nuova relazione conterrà le chiavi esterne delle relazioni referenziate; la loro *combinazione* formerà la chiave primaria della tabella ponte. Vengono incluse inoltre eventuali attributi della relazione descritti nel modello concettuale mediante classi di associazione o come semplici attributi nel modello ER. Si prenda ad esempio la relazione *Segue* tra le entità *Studente* e *Corso*. Come mostrato nel class diagram 5.12 uno studente può seguire più esami mentre un corso può essere seguito da più studenti.

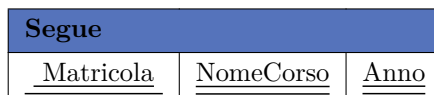


Figura 5.12: Rappresentazione concettuale di una relazione molti a molti

Dopo aver eseguito il mapping delle classi ed avere individuato le chiavi primarie si arriva ai seguenti schemi logici:



Definiamo una nuova relazione (la tabella ponte):



Codifica delle associazioni uno a molti

Per ciascuna associazione binaria R di tipo uno a molti si identificano quelle che sono le entità forti e deboli (ovvero quelle che necessitano delle informazioni provenienti dall'associazione per essere complete) che partecipano nell'associazione. Solitamente l'entità che partecipa con una molteplicità uno (sul lato della N nel caso dei diagrammi ER) sono quelle deboli. Per tradurre quindi la sua associazione con quella forte basterà includere la chiave esterna riferita alla chiave primaria dell'entità forte oltre agli eventuali attributi semplici dell'associazione. Prendiamo ad esempio l'associazione *Lavora* raffigurato sia in UML che in ER nella figura 5.13.

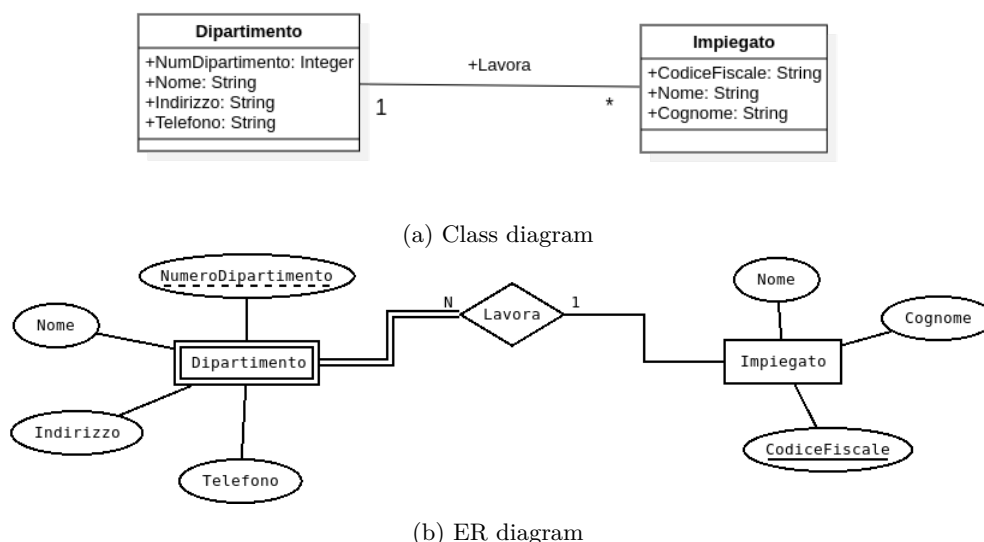
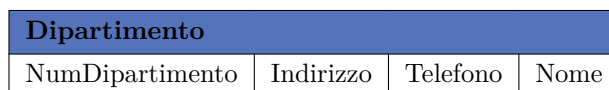


Figura 5.13: Associazione uno a molto: *Lavora*

Una volta aver fatto il mapping dell'entità forte DIPARTIMENTO e individuato la sua chiave primaria NumDipartimento:



Si procede con la traduzione dell'associazione uno a molti, basta inserire una chiave esterna all'interno dello schema logico dell'entità debole IMPIEGATO.

Impiegato			
Nome	Cognome	CodiceFiscale	NumDipartimento

Non avrebbe avuto senso il contrario, ovvero inserire una chiave esterna che referenziasse la chiave primaria di IMPIEGATO, perché in questo modo ogni dipartimento avrebbe avuto un unico impiegato.

Codifica delle associazioni uno a uno

Per ciascuna relazione binaria 1:1 R nel modello concettuale si identificano le relazioni S e T che corrispondono alle entità che partecipano in R. Ci sono tre possibili approcci per eseguire il mapping di queste associazioni:

1. Considerare una delle due entità come debole
2. Considerare entrambe le entità come deboli
3. Unire le due entità in un'unica relazione
4. Sostituire l'associazione 1:1 con una terza associazione di tipo 1:N e tradurre separatamente le due associazioni

5.2.3 ■ Sommario delle regole di mapping

Tabella 5.4: Regole di mapping nella traduzione da schema concettuale a fisico

Regola 1	Le classi vengono mappate in schemi relazionali .
Regola 2	Gli attributi delle classi vengono mappati in attributi di relazioni .
Regola 3	Gli oggetti vengono mappati in tuple di una o più relazioni.
Regola 4	Ogni oggetto è univocamente identificato. Se l'attributo identificante è definito esplicitamente nel modello concettuale allora questo sarà mappato in una chiave primaria nello schema relazionale. In caso contrario si crea una chiave surrogata.
Regola 5	Il mapping delle associazioni molti a molti implica la creazione di una tabella ponte ovvero un nuovo schema relazionale contenente gli attributi dell'associazione e le chiavi esterne per ciascuno schema coinvolto.
Regola 6	Il mapping delle associazioni uno a molti implica l'introduzione, nello schema relazionale dell' entità debole , della chiave esterna dell'entità forte.
Regola 7	Il mapping delle associazioni uno a uno ha due soluzioni in generale. La prima corrisponde alla fusione degli attributi delle classi coinvolte in uno schema comune. La seconda soluzione consiste nel considerare come debole una delle due classi e introdurre in questa la chiave esterna della seconda classe.
Regola 8	Le associazioni di aggregazione e composizione hanno un impatto minimo nel processo di mapping. La loro traduzione consisterà nell'introduzione con SQL di un vincolo ("CASCADE") nella definizione delle operazioni di inserimento e/o rimozione delle tuple.
Regola 9	Il mapping delle generalizzazioni prevede tre soluzioni. La prima consiste nell'accorpamento della classe padre nelle classi figlie. La seconda nell'accorpamento delle classi figlie nella classe padre mentre la terza soluzione prevede la sostituzione della generalizzazione con delle associazioni.

5.3

DOCUMENTAZIONE A SUPPORTO DELLA PROGETTAZIONE



I modelli ER e UML, nonostante forniscano strumenti di modellazione molto espressivi che permettono di descrivere, con efficacia e facilità, situazioni anche molto complesse non sono però sufficienti, da soli, a rappresentare nel dettaglio tutti gli aspetti di un'applicazione. Scrivere solo il nome di una classe ad esempio non è sufficiente per comprenderne il significato. Nel caso di schemi concettuali particolarmente complessi può accadere inoltre di non riuscire a rappresentare in maniera comprensibile ed esaustiva i vari concetti. Risulta indispensabile quindi corredare ogni progettazione con una documentazione di supporto che possa servire a facilitare l'interpretazione dello schema stesso e a descrivere proprietà dei dati rappresentati che non possono essere espressi direttamente dai costrutti del modello.

5.3.1 ■ Il dizionario delle classi

Contiene in forma tabellare la descrizione dettagliata ed esaustiva di ogni classe. Contiene in particolare:

- Nome della classe
- Descrizione testuale della classe

- Elenco degli attributi con descrizione degli attributi e la loro cardinalità (totale/parziale)

5.3.2 ■ Il dizionario delle associazioni

Contiene in forma tabellare la descrizione dettagliata ed esaustiva di ogni associazione. Contiene in particolare:

- Nome dell'associazione
- Descrizione testuale dell'associazione
- Elenco delle classi associate, caratteristiche di cardinalità, descrizione dei ruoli
- Riferimento ad eventuali classi di associazione collegate e loro attributi

5.3.3 ■ Il dizionario dei vincoli

Contiene in forma tabellare la descrizione dettagliata ed esaustiva dei vincoli non espressi nel modello concettuale. Vanno specificati:

- Nome
- Tipo del vincolo
 - **Di dominio:** sul singolo attributo
 - **Di tupla:** su più valori di una stessa tupla
 - **Intrarelazionali:** vincoli su istanze della stessa classe
 - **Interrelazionali:** vincoli su istanze di classi diverse
- Descrizione del vincolo

5.3.4 ■ Il dizionario delle interrogazioni

Contiene in forma tabellare la descrizione dettagliata ed esaustiva delle interrogazioni previste, ed in particolare:

- Nome
- Descrizione
- Frequenza qualitativa
- Eventuali attori

Tabella 5.5: Esempio di dizionario delle classi

Classe	Descrizione	Attributi
ClassDiagram	Descrittore di un Class Diagram UML	Id_ClassDiagram (<i>integer</i>): Chiave tecnica. Identifica univocamente ciascuna istanza di Class Diagram
		Nome (<i>String</i>): Nome associato al Class Diagram
		Autore (<i>String</i>): Nome dell'autore del Class Diagram
		Data (<i>Date</i>): Data della creazione del class diagram
Classe	Descrittore di una classe appartenente ad un Class Diagram	Descrizione (<i>String</i>): Descrizione facoltativa del Class Diagram
		Id_Classe : Chiave tecnica. Identifica univocamente ciascuna istanza di Classe.
		Nome : Nome associato alla classe.
		Descrizione : Descrizione facoltativa della classe.
Dipendenza	Descrittore di una dipendenza tra due classi	Tipo : Identifica il tipo della classe.
		Dipendenza (<i>String</i>): Indica il tipo di dipendenza.

Tabella 5.6: Esempio di dizionario delle associazioni

Associazione	Descrizione	Classi coinvolte
Specializzazione	Esprime una relazione gerarchica che intercorre tra due classi	Classe[0..1]ruolo(generale) : Indica la classe generalizzata
		Classe[0..*]ruolo(specializzata) : Indica le classi che specializzano la classe.
Dipendenza	Esprime una relazione di dipendenza che intercorre tra due classi	Classe[0..*]ruolo(superiore) : Indica le classi da cui dipendono altre.
		Classe[0..*]ruolo(dipendente) : Indica le classi che dipendono da altre.
Realizzazione	Esprime l'implementazione di un'interfaccia da parte di una classe	Classe [0..*]ruolo(implementa) : Indica le classi che implementano le interfacce.
		Interfaccia[0..*]ruolo (implementate) : Indica le interfacce implementate.
Partecipazione	Esprime la partecipazione di una classe ad un'associazione	Classe [0..1]ruolo(è posseduta) : Indica la classe (di tipo "Associazione") posseduta da un'associazione.
		Associazione[0..1]ruolo(possiede) : Indica l'associazione che possiede la classe di associazione.

Vincolo	Tipo	Descrizione
noCycleClass	Intrarelazionale	Non possono esistere relazioni gerarchiche cicliche tra classi.
noCycleInterface	Intrarelazionale	Non possono esistere relazioni gerarchiche cicliche tra interfacce.
unique-CDName	Intrarelazionale	Il nome di un Class Diagram deve essere univoco.
noSelfSpecialClass	Tupla	Una classe non può specializzare se stessa.
AggrIsBin	Interrelazionale	Un'associazione di tipo "Aggregazione" deve essere binaria.

Tabella 5.7: Esempio di dizionario dei vincoli

ALGEBRA E CALCOLO RELAZIONALE

L'insieme principale di operazioni per il modello relazionale è l'**algebra relazionale**. Le operazioni dell'algebra consentono all'utente di specificare le *interrogazioni fondamentali*. Il risultato di un'interrogazione è una **nuova relazione**, che può essere formata a partire da una o più relazioni. Una sequenza di operazioni di algebra relazionale forma un'**espressione dell'algebra relazionale**, il cui esito sarà ancora una relazione che rappresenta il risultato di un'interrogazione della base di dati.

Mentre l'algebra definisce un insieme di operazioni per il modello relazionale, il **calcolo relazionale** fornisce una *notazione dichiarativa* di livello più alto per specificare interrogazioni relazionali. Un'espressione di calcolo relazionale crea una nuova relazione, che è specificata in termini di variabili che assumono valori sulle righe delle relazioni memorizzate nella base di dati (**calcolo su tuple**) o sulle colonne delle relazioni memorizzate (**calcolo su domini**).

Mentre in un'espressione dell'algebra relazionale l'ordine mediante il quale si compongono le interrogazioni è importante, nelle espressioni del calcolo relazionale non c'è un ordine delle operazioni per specificare come recuperare il risultato delle interrogazioni. Questa è la caratteristica principale che distingue l'algebra relazionale e il calcolo relazionale. Il calcolo relazionale è importante perché ha una solida base nella logica matematica e perché il linguaggio SQL per RDBMS ha alcuni dei suoi fondamenti nel calcolo relazionale su tuple¹.

6.1

OPERAZIONI RELAZIONALI UNARIE: SELEZIONE E PROIEZIONE

6.1.1 ■ L'operazione di selezione

L'operazione di **selezione** è usata per selezionare un *sottoinsieme* di tuple di una relazione che soddisfano una **condizione di selezione**. L'operazione è indicata con:

$$\sigma_{\langle \text{condizione di selezione} \rangle}(R)$$

dove il simbolo σ è usato per denotare l'operazione di selezione, e la condizione di selezione è un'**espressione booleana** specificata sugli attributi della relazione R . La relazione risultante dall'operazione di selezione è costituita solo dalle tuple della relazione di partenza che soddisfano la condizione di selezione. La relazione ottenuta ha gli *stessi attributi* di R , ne conserva quindi lo schema, mentre la cardinalità può essere minore o uguale alla tabella di partenza.

L'espressione booleana specificata in *<condizione di selezione>* è costituita da un certo numero di **clausole** della forma:

<nome di attributo><operazione di confronto> <valore costante>

oppure:

<nome di attributo> <operazione di confronto> <nome di attributo>

o

<nome di attributo> {IS [NOT] NULL}

dove *<nome di attributo>* è il nome di un attributo di R , *<operazione di confronto>* è di solito uno degli operatori $\{=, <, \leq, >, \geq, \neq\}$, e *<valore costante>* è un valore costante del dominio dell'attributo. Le clausole possono essere unite arbitrariamente tramite gli operatori booleani AND, NOT, OR per formare una condizione di selezione generale.

¹SQL si basa sul calcolo relazionale su tuple, ma incorpora anche alcune operazioni dell'algebra relazionale come si vedrà nel Capitolo 7.

Esempio

Sia dato ad esempio il seguente schema relazionale:

Studente			
Matricola	Nome	Cognome	Genere

istanziato come segue:

Tabella 6.1: Schema relazionale *Studenti*

Matricola	Nome	Cognome	Sesso
100	Mario	Rossi	M
200	Giulia	Rossi	F
300	Ciro	Esposito	M
400	Maria	Esposito	F

Se volessimo *selezionare* dalla tabella soltanto gli studenti di sesso femminile basta fare una operazione di selezione ponendo come condizione di selezione: “*Sesso=F*”, ovvero:

$$\sigma_{\text{Sesso}=F}(\textit{Studenti})$$

ottenendo così il seguente risultato:

Matricola	Nome	Cognome	Sesso
200	Giulia	Rossi	F
400	Maria	Esposito	F

Gli operatori di confronto nell'insieme $\{=, <, \leq, >, \geq, \neq\}$ si applicano quando gli attributi appartengono a domini nei quali è possibile stabilire una **relazione d'ordine**.

Logica a tre valori

Cosa succede quando all'interno di una riga è presente un *valore nullo*? Come dovrebbe comportarsi una condizione di selezione che vede al suo interno operatori booleani? La logica a due valori non potrebbe rispondere a questi problemi, per questo motivo all'interno dei DBMS viene introdotta una logica polivalente in cui sono presenti più valori di verità rispetto ai canonici **vero** e **falso**². Introdotto quindi il valore **unknown** (U) si ottengono le tavole di verità presenti nella Tabella 6.2.

A and B		B		
		F	U	V
A	F	F	F	F
	U	F	U	U
	V	F	U	V

(a)

A or B		B		
		F	U	V
A	F	F	U	V
	U	U	U	V
	V	V	V	V

(b)

A	not A
V	F
F	V
U	U

(c)

Tabella 6.2: Tavole di verità nella logica a tre valori

²Nella logica a tre valori non vale quindi il principio del terzo escluso

Esempio

Si consideri la seguente tabella:

Esami	Matricola	codCorso	Voto	Lode
	29323	483	28	NO
	39654	729	30	SI
	29323	913	26	NO
	35467	913	30	NO
	31283	729	30	NO

L'operazione di selezione:

$$\sigma(\text{Voto}=30) \text{ AND } (\text{Lode}=\text{NO})(\text{Esami})$$

avrà come risultato la seguente tabella:

Esami	Matricola	codCorso	Voto	Lode
	35467	913	30	NO
	31283	729	30	NO

La selezione:

$$\sigma(\text{CodCorso}=729) \text{ OR } (\text{Voto}=30)(\text{Esami})$$

ha come risultato:

Esami	Matricola	codCorso	Voto	Lode
	39654	729	30	SI
	31283	729	30	NO

6.1.2 L'operazione di proiezione

Se si pensa ad una relazione come a una tabella, l'operazione di selezione seleziona alcune righe della tabella scartandone altre. L'operazione di **proiezione**, al contrario, seleziona determinate *colonne* della tabella e scarta le altre. Mentre l'operazione di selezione può essere vista come una *partizione orizzontale*, la proiezione rappresenta una *partizione verticale* della relazione. Il formato generale dell'operazione di proiezione è:

$$\Pi_{\langle \text{lista di attributi} \rangle}(R)$$

Il risultato dell'operazione di proiezione ha solo gli attributi specificati nella **<lista di attributi>**, e nello stesso ordine con cui compaiono nella lista. Perciò il suo **grado** è uguale al numero di attributi in **<lista attributi>**.

Se la lista di attributi comprende solo attributi non-chiave di R, è probabile che si presentino tuple duplicate; l'operazione di proiezione *rimuove tutte le tuple duplicate*, in modo che il suo risultato sia un insieme di tuple e perciò una relazione valida: ciò è noto come **eliminazione dei duplicati**.

Esempio

Consideriamo nuovamente la Tabella 6.1 e calcoliamo:

$$\Pi_{\text{Cognome}}(\text{Studenti})$$

Il risultato che si ottiene sarà una tabella con sole due colonne e senza duplicati del tipo:

Cognome
Rossi
Esposito

Composizione delle operazioni di selezione e proiezione

Le operazioni di selezione e di proiezione hanno come operando una sola tabella e possono essere applicate anche in successione: in questo modo vengono effettuate interrogazioni sulle relazioni ottenute come risultato di un'interrogazione precedente.

Prendiamo ad esempio la seguente tabella:

Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
9553	Milano	Napoli	44
5698	Bianchi	Torino	64
2568	Rossi	Roma	55

Vogliamo sapere *Matricola* e *Cognome* che guadagnano più di 62. Allora dalla tabella andremo a selezionare tutte le tuple che soddisfano la condizione:

$$Stipendio > 62$$

e poi andremo a fare una proiezione sulle colonne *Matricola* e *Cognome*. Cioè:

$$\Pi_{Matricola, Cognome}(\sigma_{Stipendio > 62}(Impiegati))$$

Ottenendo così:

Matricola	Cognome
5998	Neri
5698	Bianchi

Scambiare selezione e proiezione non è sempre possibile e produce spesso gravi **errori**. Infatti la tabella risultante dall'operazione di proiezione può non avere gli attributi necessari per applicare una selezione con predicati che coinvolgono tali attributi. Quindi in generale non vale:

$$\Pi_x(\sigma_{condizione}(R)) \neq \sigma_{condizione}(\Pi_x(R))$$

In Algebra relazionale quindi è buona prassi applicare prima la selezione e poi la proiezione:

“Quando costruisco una interrogazione, prima penso a cosa selezionare e da dove. Infine proietto.”

6.1.3 Sequenze di operazioni e operazione di ridenominazione

Le relazioni risultanti illustrate nei paragrafi precedenti *non hanno alcun nome*. In generale, è probabile che si vogliano eseguire più operazioni di algebra relazionale una dopo l'altra. In questo caso è possibile scrivere le operazioni come una singola **espressione dell'algebra relazionale** nidificando le operazioni, oppure applicare una operazione alla volta e creare relazioni che danno il risultato intermedio. In quest'ultimo caso serve dare un *nome intermedio* alle relazioni che danno il risultato intermedio.

Per associare un nome ad una relazione risultante da una operazione relazionale basta porre:

$$\text{Nome Schema Risultante} \leftarrow \text{Espressione algebra relazionale}$$

Per ridenominare gli attributi di una relazione si elencano semplicemente i nomi dei nuovi attributi tra parentesi:

$$R(\text{Lista attributi}) \leftarrow \Pi_{\text{lista attributi da ridenominare}}(\text{Relazione sorgente})$$

È possibile anche definire un'operazione di **ridenominazione** che può ridenominare il nome di una relazione o i nomi degli attributi, o entrambi. L'operazione di ridenominazione generale, quando viene applicata a una relazione R di grado n , è indicata da una delle seguenti tre forme:

$$\rho_S(B_1, B_2, \dots, B_n)(R)$$

oppure

$$\rho_S(R)$$

oppure

$$\rho_S(B_1, B_2, \dots, B_n)(R)$$

dove il simbolo ρ è usato per indicare l'operatore di ridenominazione, S è il nome nuovo della relazione, e $B_1, B_2, \dots, B_n$ sono i nuovi nomi degli attributi. La prima espressione dà un nome nuovo sia alla relazione sia ai suoi attributi, la seconda solo alla relazione, e la terza solo agli attributi. Se gli attributi di R sono $A_1, A_2, \dots, A_n$ in quest'ordine, allora ogni A_i è ridenominato come B_i .



6.2.1 Le operazioni di unione, intersezione e differenza

Molte operazioni sugli insiemi vengono usate per fondere gli elementi di due insiemi in modi diversi; fra queste in particolare l'**unione**, l'**intersezione** e la **differenza** (chiamata anche **minus**). Sono operazioni binarie le quali, quando adattate a schemi relazionali, devono ricevere in ingresso schemi **compatibili**, ovvero avere lo stesso **tipo di tuple**; questa condizione è detta **compatibilità all'unione**.

Due relazioni $R(A_1, \dots, A_n)$ e $S(B_1, \dots, B_k)$ sono **compatibili all'unione** se

- $n = k$
- $dom(A_i) = dom(B_i) \quad \forall \quad 1 \leq i \leq n$

Quindi R ed S hanno lo stesso numero di attributi e che ogni coppia di attributi corrispondenti ha lo stesso dominio.

È possibile definire queste tre operazioni su due relazioni R ed S compatibili all'unione in questo modo:

- **Unione:** il risultato di questa operazione, indicato con $R \cup S$, è una relazione che comprende tutte le tuple che sono in R o in S oppure sia in R sia in S. Le tuple duplicate vengono eliminate.
- **Intersezione:** il risultato, indicato con $R \cap S$, è una relazione che comprende tutte le tuple che sono sia in R sia in S.
- **Differenza (o Minus):** il risultato, indicato con $R \setminus S$, è una relazione che comprende tutte le tuple che sono in R ma non in S.

Studente	Nome_Batt	Cognome
	Susan	Yao
	Ramesh	Shah
	Johnny	Kohler

Assistente	Nome_Batt	Cognome
	John	Smith
	Ricardo	Browne
	Susan	Yao
	Ramesh	Shah

(a) Due relazioni compatibili all'unione

$S \cup A$	Nome_Batt	Cognome
	Susan	Yao
	Ramesh	Shah
	Johnny	Kohler
	John	Smith

(b)

$S \cap A$	Nome_Batt	Cognome
	Susan	Yao
	Ramesh	Shah

(c)

$S \setminus A$	Nome_Batt	Cognome
	Ramesh	Shah

(d)

$A \setminus S$	Nome_Batt	Cognome
	John	Smith
	Ricardo	Browne

(e)

Tabella 6.3: Le operazioni insiemistiche unione, intersezione e differenza

Sia l'unione che l'intersezione sono **operazioni commutative**, cioè:

$$R \cup S = S \cup R$$

$$R \cap S = S \cap R$$

L'operazione di differenza invece *non è commutativa* come si vede dalla Tabella 6.3 confrontando la tabella (d) e la tabella (e). In generale

$$R \setminus S \neq S \setminus R$$

6.2.2 L'operazione di prodotto cartesiano

Si consideri l'operazione di **prodotto cartesiano** - noto anche come **prodotto incrociato (cross product)** o **join incrociato (cross join)** - indicato con $\times$. Si tratta di un'operazione binaria, ma le relazioni a cui si applica non devono essere *compatibili all'unione*. Infatti, il prodotto cartesiano di due relazioni produce una nuova relazione combinando ogni tupla di una delle due

relazioni con ogni tupla dell'altra relazione. In generale il risultato di

$$R(A_1, A_2, \dots, A_n) \times B(B_1, B_2, \dots, B_m)$$

è una relazione Q con $n + m$ attributi

$$Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$$

presi in quest'ordine. La relazione risultante Q contiene una tupla per ogni possibile combinazione di tuple. Perciò, se R ha n_R tuple e se S ne ha m_S allora $R \times S$ avrà $n_R \cdot m_S$ tuple. Il prodotto cartesiano produce tuple con tutti gli attributi propri delle due relazioni. È possibile poi effettuare una selezione delle sole tuple logicamente collegate delle due relazioni specificando una appropriata condizione di selezione.

Esempio

Si considerino ad esempio le due tabelle R ed S di grado 2 e 3 rispettivamente e contenenti 2 tuple ciascuna. Il prodotto cartesiano di R ed S sarà una tabella di grado 5 e contenente 4 tuple:

R		S			R × S				
A ₁	A ₂	B ₁	B ₂	B ₃	A ₁	A ₂	B ₁	B ₂	B ₃
a	a	c	d	d	a	a	c	d	d
a	b	c	d	e	a	a	c	d	e
					a	b	c	d	d
					a	b	c	d	e

Va osservato il fatto che il prodotto cartesiano produce tuple con tutti gli attributi propri delle due relazioni con il rischio di **agganciare informazioni non correlate** e **perdere la coerenza dei dati**. È responsabilità dell'utente di assicurarsi di eseguire sulle relazioni solo operazioni significative.

6.3

OPERAZIONI RELAZIONALI BINARIE

6.3.1 L'operazione di Theta Join

L'operazione di **theta join**, indicata con $\bowtie_\theta$, è usata per unire *tuple logicamente collegate*, provenienti da due relazioni, in tuple singole. È molto importante per ogni base di dati relazionale che abbia più di una relazione, perché consente di eseguire associazioni tra relazioni.

Esempio

Si considerino i seguenti schemi relazionali:

$S = \text{Studente (Matricola, Nome, Cognome)}$

$E = \text{Esame (Matr_Stud, Voto, Esame)}$

Studente	Matricola	Nome	Cognome	Esame	Matr_Stud	Voto	Esame
	100	Thomas	Andersen		100	28	BDD
	200	Robin	Hobb		100	30	PR1
	300	Emmet	Brown		200	25	BDD

Se volessimo vedere gli studenti che hanno sostenuto esami non avrebbe molto senso effettuare il prodotto cartesiano di $S \times E$ dato che produrrebbe la **combinazione lineare** di tutte le tuple. Va da sé che bisogna porre una condizione nella selezione delle tuple che vogliamo. In questo caso uno studente è legato ad un'esame e viceversa mediante la propria matricola. Quindi ponendo:

$$\Pi_{Nome, Cognome} \left(\sigma_{Matricola=Matr_Stud}(S \times E) \right) \tag{6.1}$$

si ottiene la tabella:

Nome	Cognome
Thomas	Andersen
Robin	Hobb

La 6.1 si può esprimere mediante l'operazione di **join**, si scriverà quindi:

$$\Pi_{Nome, Cognome} \left(S \bowtie_{Matricola=Matr_Stud} S \right) \tag{6.2}$$

La forma di un'operazione di **join** su due relazioni $R(A_1, A_2, \dots, A_n)$ e $S(B_1, \dots, B_m)$ è:

$$R \bowtie_{\langle \text{condizione di join} \rangle} S$$

Il risultato del join è una relazione Q con $n + m$ attributi $Q(A_1, \dots, A_n, B_1, \dots, B_m)$. Q presenta una tupla in corrispondenza di tuple - una di R e una di S - *quando la combinazione soddisfa la condizione di join*. Questa è la differenza principale tra l'operazione di Join e il Prodotto Cartesiano. Ogni combinazione di tuple per la quale la condizione di join ha valore vero è inserita nella relazione risultante Q *come singola tupla risultante dall'unione*.

Si ha così che la join vista nell'esempio risulta:

Matricola	Nome	Cognome	Matr_Stud	Voto	Esame
100	Thomas	Andersen	100	28	BDD
100	Thomas	Andersen	100	30	PR1
200	Robin	Hobb	200	25	BDD

Tabella 6.4: Operazione di join: $S \bowtie_{\text{Matricola}=\text{Matr_Stud}} S$

Un'operazione di join con una condizione di join generica è detta anche **theta join** e si indica con il simbolo $\bowtie_\theta$. Le tuple i cui attributi di join sono nulli **non compaiono** nel risultato. In questo senso l'operazione di join non conserva necessariamente tutte le informazioni presenti nelle relazioni partecipanti.

Esempio

Trovare nome e cognome degli studenti che non hanno fatto esami. L'insieme di tutti gli studenti è dato da:

$$TuttiStudenti \leftarrow \Pi_{Nome, Cognome}(Studenti)$$

Mentre quello degli studenti che hanno sostenuto almeno un esame si ricava effettuando una join tra STUDENTI ed ESAMI ponendo come condizione la corrispondenza delle matricole:

$$AlmenoUnEsame \leftarrow \Pi_{Nome, Cognome}(Studenti \bowtie_{\text{Matricola}=\text{Matr_Stud}} Esami)$$

6.3.2 Le operazioni di Join Naturale

Una theta join che prevede condizioni di join con solo confronti di uguaglianza è detto **Equijoin**. Dato che, in ogni coppia di attributi con valori uguali, uno degli attributi è superfluo, è stata ideata una nuova operazione, detta **join naturale**, indicata semplicemente con $\bowtie$ senza specificare i nomi degli attributi sui quali andare a verificare la condizione di uguaglianza. La definizione standard di join naturale richiede che i due attributi di join abbiano lo stesso nome in entrambe le relazioni. Se le cose non stanno così, bisogna prima applicare un'operazione di ridenominazione. La relazione risultante di una Natural Join vede compressa in una sola colonna i due attributi con lo stesso nome delle due relazioni partecipanti. Se non ci sono attributi che corrispondono e non è stata effettuata nessuna ridenominazione allora l'operazione restituisce un prodotto cartesiano. Osserviamo quindi che:

- 1. Nel caso di chiavi primarie e chiavi esterne il vincolo di integrità referenziale garantisce della correttezza delle informazioni che andranno ad essere giuntate.
- 2. Bisogna essere sicuri che non ci siano attributi omonimi oltre a quelli che si vogliono far corrispondere.

6.3.3 Operazioni di Join esterno

L'operazione di join è usata per combinare dati provenienti da più relazioni, in modo tale che informazioni logicamente collegate possano essere presentate in una sola tabella. Queste operazioni sono anche note come **join interni** o **inner join**. Bisogna notare però che all'interno di queste viste, le tuple che non soddisfano la condizione espressa o con valori nulli negli attributi del join non verranno visualizzate. Questo produce una **perdita delle informazioni**. Per questo motivo, quando si desidera mantenere nel risultato del join tutte le tuple della prima relazione o della seconda, si usano le operazioni di **join esterno** o **outer join**.

Studente

Matricola	Nome	Cognome
100	Thomas	Andersen
200	Robin	Hobb
300	Emmet	Brown

Esame

Matr_Stud	Voto	Esame
100	28	BDD
100	30	PR1
200	25	BDD

Una possibile operazione di join esterno è la **left outer join**. Questa operazione equivale ad una normale theta join con l'unica

differenza che prende le righe dell'operando a sinistra che sono state escluse nel matching. Si avrà così che:

$$S \bowtie_{Matricola=Matr_Stud} E$$

restituisce come risultato la tabella 6.5.

Matricola	Nome	Cognome	Matr_Stud	Voto	Esame
100	Thomas	Andersen	100	28	BDD
100	Thomas	Andersen	100	30	PR1
200	Robin	Hobb	200	25	BDD
300	Emmet	Brown	NULL	NULL	NULL

Tabella 6.5: Esempio di left outer join tra STUDENTE ed ESAME

In modo analogo si possono definire le operazioni di **join esterno destro (right outer join)** indicato con $\bowtie\right$ e il **join esterno completo (full outer join)** indicato con $\bowtie\rightleftarrows$.

Esempio

Trovare nome e cognome degli studenti che non hanno fatto esami:

$$NessunEsame \leftarrow \Pi_{Nome, Cognome}(\sigma_{Matr_Stud=NULL}(Studenti \bowtie Esami))$$

6.4

OPERAZIONI RELAZIONALI AGGIUNTIVE



Alcune comuni richieste a una base di dati non possono essere espresse con le operazioni basilari dell'algebra relazionale descritte nelle sezioni precedenti.

6.4.1 Funzioni aggregate e raggruppamento

Il primo tipo di richiesta che non può essere espressa nell'algebra relazionale di base consiste nello specificare **funzioni aggregate** matematiche su collezioni di valori della base di dati. Funzioni comuni applicate a collezioni di valori numerici sono:

- SUM
- COUNT
- AVERAGE
- MAX
- MIN

Un altro tipo comune di richiesta prevede il *raggruppamento* delle tuple presenti in una relazione sulla base del valore di alcuni loro attributi, e quindi l'applicazione di una funzione aggregata indipendentemente a ogni gruppo. Per specificare queste richieste è possibile definire un'operazione di **funzione aggregata**, usando il simbolo $\mathfrak{F}$, nel modo seguente:

$$\underbrace{B_1, \dots, B_k}_{\text{criterio di raggruppamento}} \quad \mathfrak{F}_{\langle \text{listadi funzioni} \rangle}(R)$$

il **criterio di raggruppamento** è una lista di attributi della relazione R che *individua i raggruppamenti* in base al valore degli attributi. In ogni gruppo avrà attributi con **valore uguale**. Il criterio di raggruppamento può essere anche vuoto. In questo caso si vuole operare sull'intera tabella.

6.4.2 Operazioni di chiusura ricorsiva

Un altro tipo di operazione che di solito non può essere specificata nell'algebra relazionale di base è la **chiusura ricorsiva**. Questa operazione è applicata a un'associazione ricorsiva tra tuple dello stesso tipo. Per questo motivo quando si lavora con operazioni binarie che operano sulla stessa relazione si può usare la tecnica del **self join** che prevede la:

- Ridenominazione degli attributi della seconda tabella
- Join delle due tabelle

6.4.3 ■ Un insieme completo di operazioni dell'algebra relazionale

Si può dimostrare che l'insieme di operazioni dell'algebra relazionale $\{\sigma, \Pi, \cup, \setminus, \times\}$ è un insieme **completo**, cioè ogni altra operazione dell'algebra relazionale può essere espressa come una *sequenza di operazioni di questo insieme*.

Esempio

L'operazione di intersezione può essere espressa usando l'unione e la differenza nel modo seguente:

$$R \cap S = (R \cup S) \setminus ((R \setminus S) \cup (S \setminus R))$$

Come si è visto in 6.1, un'operazione di join può essere espressa con un prodotto cartesiano seguito da un'operazione di selezione:

$$R \bowtie_{\langle \text{condizione} \rangle} S = \sigma_{\langle \text{condizione} \rangle}(R \times S)$$

Analogamente, un join naturale può essere specificato con un prodotto cartesiano preceduto da una ridenominazione e seguito dalle operazioni di selezione e proiezione. Per questo motivo anche le varie operazioni di join non sono strettamente necessarie per il potere espressivo dell'algebra relazionale.

6.5

ESEMPI DI INTERROGAZIONI IN ALGEBRA RELAZIONALE



Esempio

Trovare nome e indirizzo di tutti gli impiegati che lavorano per il dipartimento “Ricerca”:

1. Seleziono i dipartimenti che afferiscono al campo della ricerca:

$$Dip_Ricerca \leftarrow \sigma_{Nome_D='Ricerca'}(Dipartimento)$$

2. Congiungo la relazione *Dip_Ricerca* e IMPIEGATO sul numero di dipartimento:

$$Impiegati_Ricerca \leftarrow (Dip_Ricerca \bowtie_{Numero_D=N_D} Impiegato)$$

3. Proietto il risultato sui campi Nome, Cognome ed Indirizzo:

$$Risultato \leftarrow \Pi_{Nome_Batt, Cognome, Indirizzo}(Impiegati_Ricerca)$$

In una singola espressione:

$$\Pi_{Nome, Cognome, Indirizzo}(\sigma_{Nome_D='Ricerca'}(Dipartimento) \bowtie_{Numero_D=N_D} (Impiegato))$$

Questa interrogazione potrebbe essere specificata in modi diversi; ad esempio l'ordine delle operazioni di join e selezione potrebbe essere rovesciato, o il join potrebbe essere sostituito da un join naturale (dopo una ridenominazione degli attributi di join).

Esempio

Per ogni progetto con sede a “Stafford” si elenchi il numero di progetto, il numero del dipartimento che lo controlla nonché il cognome, l'indirizzo e la data di nascita del direttore del dipartimento.

1. Seleziono i progetti con sede a “Stafford”:

$$Progetti_Stafford \leftarrow \sigma_{Sede_P='Stafford'}(Progetto)$$

2. Congiungo *Progetti_Stafford* e DIPARTIMENTI su *Num_D* per ottenere le informazioni relative al dipartimento che controlla il progetto:

$$Dip_Controllo_P \leftarrow Progetti_Stafford \bowtie_{Num_D=Numero_D} (Dipartimento)$$

3. Congiungo *Dip_Controllo_P* e IMPIEGATI su *SSN_Dir=SSN* per ottenere le informazioni relative al direttore del dipartimento:

$$Direttore_Dip_Controllo_P \leftarrow Dip_Controllo_P \bowtie_{SSN_Dir=SSN} (Impiegato)$$

4. Proietto le informazioni che mi interessano:

$$Risultato \leftarrow \Pi_{Numero_P, Num_D, Cognome, Indirizzo, Data_N}(Direttore_Dip_Controllo_P)$$

LO STANDARD SQL

SQL è l'acronimo di **Structured Query Language**, cioè linguaggio strutturato di interrogazione. SQL è oggi il linguaggio standard per i DBMS relazionali commerciali. SQL è un linguaggio completo; comprende istruzioni per la definizione dei dati, l'interrogazione e l'aggiornamento; quindi è sia un DDL sia un DML¹. Inoltre fornisce tutte le funzionalità per definire le viste sulla base di dati, per specificare la sicurezza e le autorizzazioni di accesso, per definire i vincoli di integrità e per specificare i controlli sulle transazioni. Comprende persino le regole per incorporare le istruzioni SQL nei linguaggi di programmazione di uso generale come Java o C/C++.

7.1

TIPI DI DATI E DEFINIZIONE DEI DATI IN SQL



SQL usa i termini **tabella**, **riga** e **colonna** al posto dei termini *relazione*, *tupla* e *attributo* del modello relazionale visto nel Capitolo 4. Il comando SQL fondamentale per la **definizione dei dati** è l'istruzione CREATE, che può essere usata per creare:

- schemi
- tabelle
- domini
- viste
- asserzioni
- trigger

7.1.1 ■ Il concetto di schema

Uno **schema sql** è identificato da un **nome di schema** e contiene un identificatore di autorizzazione per indicare l'utente che ne è proprietario, nonché **descrittori** per ciascun elemento nello schema. I suoi **elementi** comprendono le tabelle, i vincoli, le viste, i domini e altri costrutti (quali ad esempio le autorizzazioni) che lo descrivono. Uno schema è creato attraverso l'istruzione CREATE SCHEMA come nel listato 7.1.

```
1 CREATE SCHEMA Azienda;
```

Codice 7.1: Esempio di utilizzo della clausola CREATE SCHEMA

In generale, non tutti gli utenti sono autorizzati a creare gli schemi e i rispettivi elementi.

7.1.2 ■ Il comando CREATE TABLE in SQL

Il comando **CREATE TABLE** è usato per specificare una nuova relazione assegnandole un nome e specificandone attributi e vincoli. Gli attributi sono specificati per primi e a ciascun attributo è assegnato un nome, un tipo di dati per indicare il suo dominio di valori e gli eventuali vincoli sui valori come, ad esempio, NOT NULL.

¹Vedi la Sezione 2.5

Di solito lo schema SQL in cui vengono dichiarate le relazioni è specificato implicitamente dall'ambiente in cui sono eseguite le istruzioni CREATE TABLE. In alternativa si può esplicitamente associare il nome dello schema ai nomi delle relazioni separandoli con un punto. Ad esempio scrivendo:

```
1 CREATE TABLE Azienda.Impiegato
```

anziché:

```
1 CREATE TABLE Impiegato
```

Le relazioni dichiarate attraverso le istruzioni CREATE TABLE sono chiamate **tabelle di base** per indicare il fatto che queste sono effettivamente memorizzate come un file dal DBMS. Le tabelle di base si distinguono dalle **relazioni virtuali**, create attraverso l'istruzione CREATE VIEW, che possono corrispondere o meno a un file fisico. In SQL gli attributi di una tabella di base sono considerati *ordinati nella sequenza in cui sono specificati* nell'istruzione CREATE TABLE. La **sintassi** dell'istruzione CREATE TABLE è la seguente:

```
1 CREATE TABLE table_name
2 (
3     column1 datatype,
4     column2 datatype,
5     column3 datatype,
6     ...
7 );
```

Codice 7.2: Sintassi della clausola CREATE TABLE

Esempio

L'esempio seguente crea una tabella chiamata IMPIEGATO che contiene cinque colonne: *IdImpiegato*, *Nome*, *Stipendio*, *Dipartimento* e la tabella DIPARTIMENTO che contiene le colonne: *IdDipartimento*, *NomeDipartimento*, *City*, *Responsabile*.

```
1 CREATE TABLE Impiegato
2 (
3     IdImpiegato int,
4     Nome varchar(225),
5     Stipendio int,
6     Dipartimento int
7 );
8
9 Create table Dipartimento
10 (
11     IdDipartimento int,
12     NomeDipartimento varchar(255)
13     City varchar(255)
14     Responsabile int
15 );
```

Alla fine dell'esecuzione di questi due comandi si ottengono le tabelle vuote:

Impiegato	IdImpiegato	Nome	Stipendio	Dipartimento
Dipartimento	IdDipartimento	NomeDipartimento	City	Responsabile

7.1.3 Domini e tipi di dati per gli attributi in sql

I principali tipi di dati disponibili per gli attributi includono tipi numerici, stringhe di caratteri, stringhe di bit, booleani, data e ora.

- I tipi di dati **numerici** comprendono:
 - Numeri interi di diverse dimensioni (INTEGER o INT e SMALLINT)
 - Numeri reali a virgola mobile con diversa precisione (FLOAT o REAL e DOUBLE PRECISION)

Possono essere dichiarati anche numeri formattati usando DECIMAL(*i*,*j*) o DEC(*i*,*j*) oppure NUMERIC(*i*,*j*), in cui la *i*, la *precisione*, è il numero totale di cifre decimali e *j*, la *scala*, è il numero di cifre dopo la virgola.

- I tipi di dati per la rappresentazione di **stringhe di caratteri**
 - Per la lunghezza fissa: `CHAR(n)` o `CHARACTER(n)`, in cui `n` è il numero di caratteri.
 - Per la lunghezza variabile: `VARCHAR(n)` o `CHAR VARYING(n)` o `CHARACTER VARYING(n)`, in cui `n` è il numero massimo di caratteri.
- Un tipo di dati **booleano** può assumere i valori tradizionali `TRUE` e `FALSE`. In SQL, a causa della presenza di valori `NULL`, viene usata la logica a tre valori², quindi il terzo valore possibile per un tipo di dati booleano è `UNKNOWN`.
- I tipi di dati per **data** e **ora** sono
 - `DATE`: ha dieci posizioni e i suoi componenti sono `YEAR`, `MONTH` e `DAY` nella forma `YYYY-MM-DD`.
 - `TIME`: ha otto posizioni, con i componenti `hour`, `minute` e `second` nella forma `HH:MM:SS`.
- Un tipo di dati `TIMESTAMP` comprende entrambi i campi `DATE` e `TIME`.
- Un altro tipo di dati correlato al tempo è `INTERVAL` che specifica un **intervallo**, cioè un valore relativo che può essere usato per aumentare o diminuire il valore assoluto di una data, ora o timestamp.

È possibile specificare direttamente il tipo di dati di ogni attributo; in alternativa può essere **dichiarato un dominio** e si può utilizzare il suo nome nella dichiarazione degli attributi. La sintassi per la creazione di un dominio è la seguente:

```
1 CREATE DOMAIN DomainName AS datatype;
```

Codice 7.3: Definizione di un dominio in SQL

7.2

SPECIFICAZIONE DEI VINCOLI DI BASE



Per **vincoli di base** si intendono quei vincoli che vengono specificati in SQL durante la fase di creazione di una tabella. Tra questi ci sono i *vincoli di chiave* e di *integrità referenziale*, le limitazioni sui *domini degli attributi* e sui valori `NULL` e anche i *vincoli sulle singole tuple* all'interno di una relazione.

7.2.1 ■ Specificazione di vincoli e di valori predefiniti per gli attributi

Poiché SQL accetta i valori `NULL` come valori degli attributi³, si può specificare un **vincolo NOT NULL** se il valore `NULL` non è consentito per un particolare attributo. Questo vincolo è sempre specificato implicitamente per gli attributi che fanno parte della *chiave primaria* di ogni relazione. Il vincolo `NOT NULL` obbliga quindi un campo a contenere per forza un valore.

Esempio

Il codice SQL seguente assicura che il campo “`IdImpiegato`”, “`Nome`” e “`Dipartimento`” non accettino valori `NULL` quando viene creata la tabella “`Impiegato`”:

```
1 CREATE TABLE Impiegato
2 (
3   IdImpiegato int NOT NULL,
4   Nome varchar(255) NOT NULL,
5   Stipendio int,
6   Dipartimento int NOT NULL
7 );
```



È possibile anche definire un **valore di default** per un attributo aggiungendo la clausola **DEFAULT**<valore> alla sua definizione. Il valore predefinito viene inserito ogni volta che non viene fornito un valore esplicito per quell'attributo. Se non è indicata alcuna clausola predefinita, il valore predefinito di default è `NULL` per gli attributi che non fanno vincolo `NOT NULL`.

²Vedi la Tabella 6.2.

³Vedi Sezione 4.2.1.

Esempio

Il codice SQL seguente imposta un valore predefinito per il campo “Stipendio” quando si viene a creare la tabella IMPIEGATO:

```
1 CREATE TABLE Impiegato
2 (
3     IdImpiegato int NOT NULL,
4     Nome varchar(255) NOT NULL,
5     Stipendio int DEFAULT 1800,
6     Dipartimento int NOT NULL
7 );
```



Un altro tipo di vincolo può limitare i valori dei domini o degli attributi usando la clausola **CHECK** dopo la definizione di un attributo o di un dominio.

Esempio

Il codice SQL seguente crea un vincolo CHECK sul campo “Dipartimento” quando si viene a creare la tabella Impiegato. Il vincolo assicura che il numero del dipartimento non sia maggiore di 10.

```
1 CREATE TABLE Impiegato
2 (
3     IdImpiegato int NOT NULL,
4     Nome varchar(255) NOT NULL,
5     Stipendio int DEFAULT 1800,
6     Dipartimento int CHECK (Dipartimento ≤ 10)
7 );
```



La clausola CHECK può essere usata anche insieme all’istruzione CREATE DOMAIN:

```
1 CREATE DOMAIN Num AS Integer CHECK (Num ≥ 0 AND Num ≤ 9);
```



7.2.2 Vincoli di chiave e di integrità referenziale

La clausola **PRIMARY KEY** specifica uno o più attributi che costituiscono la chiave primaria di una relazione. Se la chiave primaria è formata da un *unico attributo* la clausola può seguire direttamente l’attributo.

Esempio

Il seguente codice SQL crea una chiave primaria sul campo “IdImpiegato” quando si crea la tabella Impiegato:

```
1 CREATE TABLE Impiegato
2 (
3     IdImpiegato int PRIMARY KEY,
4     Nome varchar(225),
5     Stipendio int,
6     Dipartimento int
7 );
```



Per definire un **vincolo di chiave primaria** su più attributi si usa la sintassi seguente:

```
1 CREATE TABLE Impiegato
2 (
3     IdImpiegato int,
4     Nome varchar(225),
5     Stipendio int,
6     Dipartimento int,
7     CONSTRAINT PK_IMP PRIMARY KEY (IdImpiegato, Nome)
8 );
```



La clausola **UNIQUE** assicura che tutti i valori in una tabella siano differenti. Sia i vincoli **UNIQUE** che **PRIMARY KEY** garantiscono l'unicità di una colonna o un insieme di colonne. Un vincolo di chiave primaria ha un vincolo di **UNIQUE** implicito⁴. In una tabella si possono avere più vincoli **UNIQUE** ma un singolo vincolo di chiave primaria.

```
1 CREATE TABLE Impiegato
2 (
3     IdImpiegato int PRIMARY KEY,
4     Nome varchar(225) UNIQUE,
5     Stipendio int,
6     Dipartimento int
7 );
```



L'integrità referenziale è specificata attraverso la clausola **FOREIGN KEY**. Un vincolo di integrità referenziale può essere violato quando vengono inserite o cancellate alcune tuple oppure quando viene modificato il valore di un attributo di una chiave esterna o di una chiave primaria. L'azione predefinita che SQL prevede in caso di violazione di integrità è **rifutare l'operazione di aggiornamento** che causerebbe la violazione.

```
1 CREATE TABLE Dipartimento
2 (
3     IdDipartimento int,
4     NomeDipartimento varchar(255)
5     City varchar(255)
6     Responsabile int
7 );
8
9 CREATE TABLE Impiegato
10 (
11     IdImpiegato int PRIMARY KEY,
12     Nome varchar(255),
13     Stipendio int,
14     Dipartimento int,
15     CONSTRAINT FK_Dip FOREIGN KEY (Dipartimento) REFERENCES Dipartimento(IdDipartimento)
16 );
```



7.2.3 ■ Specificazione di vincoli sulle tuple

Oltre ai vincoli di chiave e di integrità referenziale si possono specificare altri **vincoli di tabella** usando ulteriori clausole **CHECK** alla fine dell'istruzione **CREATE TABLE**. Questi vincoli possono essere **vincoli di tupla**, perché vengono applicati a ciascuna tupla *singolarmente* e controllati ogni volta che viene inserita o modificata una tupla.

Esempio

Si ha:

```
1 CREATE TABLE Persona
2 (
3     IdPersona int NOT NULL,
4     Cognome varchar(255) NOT NULL,
5     Nome varchar(255),
6     Anni int,
7     City varchar(255),
8     CONSTRAINT CHK_Persona CHECK (Anni ≥ 18 AND City='Boggibonzi')
9 );
```



⁴Per garantire il vincolo di unicità.

7.3.1 Il comando DROP

Il comando DROP può essere usato per eliminare elementi di schema **denominati**, come tabelle, domini o vincoli. È possibile anche eliminare uno schema.

Esempio

Ad esempio, se un intero schema non è più necessario, si può utilizzare il comando **DROP SCHEMA**. Esistono due opzioni relative al *comportamento di eliminazione*:

1. **cascade**: elimina a cascata tutte le tabelle contenute nello schema;
2. **restrict**: elimina lo schema solo se non contiene elementi, in caso contrario il comando DROP non viene eseguito.

```
1 DROP SCHEMA Azienda CASCADE;
```



Se una relazione base non è più necessaria, la relazione e la sua definizione possono essere eliminate usando il comando **DROP TABLE**.

7.3.2 Il comando ALTER

La definizione di una tabella base o di altri tipi di elementi dello schema denominati può essere modificata usando il comando **ALTER**. Per le tabelle base, le possibili **operazioni di modifica** comprendono l'*aggiunta* o l'*eliminazione* di una colonna, la *modifica* di una definizione di colonna e l'*aggiunta* o *eliminazione* di vincoli di tabella. Per aggiungere una colonna in una tabella si usa la sintassi seguente:

```
1 ALTER TABLE table_name
2 ADD column_name datatype;
```



Esempio

Il seguente codice SQL aggiunge una colonna “Email” alla tabella Clienti:

```
1 ALTER TABLE Clienti
2 ADD Email varchar(255);
```



Per rimuovere una colonna da una tabella si usa la sintassi seguente:

```
1 ALTER TABLE table_name
2 DROP COLUMN column_name CASCADE/RESTRICT;
```



Anche l'eliminazione di una colonna può avvenire con le opzioni **cascade** e **restrict**. Se si è scelta l'opzione CASCADE, tutti i vincoli e le viste che si riferiscono alla colonna vengono autonomamente eliminati dallo schema, insieme ad essa. Se si specifica l'opzione RESTRICT, il comando ha successo solo se non ci sono viste o vincoli che fanno riferimento alla colonna.

Esempio

Il seguente esempio elimina la colonna “Email” dalla tabella Clienti:

```
1 ALTER TABLE Clienti
2 DROP COLUMN Email CASCADE;
```



È possibile anche modificare una definizione di colonna eliminando la clausola che specifica il valore predefinito esistente oppure definendo una nuova clausola che specifica un valore predefinito.

```
1 ALTER TABLE table_name
2 ALTER COLUMN column_name DROP DEFAULT;
3 ALTER TABLE table_name
4 ALTER COLUMN column_name SET DEFAULT <newvalue>;
```



Si possono cambiare i vincoli specificati su una tabella aggiungendone di nuovi o eliminandone alcuni. Per aggiungere un vincolo si scrive:

```
ALTER TABLE table_name CREATE CONSTRAINT constraint_name;
```



Esempio

Per aggiungere un vincolo di foreign key si usa la seguente sintassi:

```
1 ALTER TABLE table_name
2 ADD column_name dataType,
3 CONSTRAINT fk_Name FOREIGN KEY(fk_ColumnName) REFERENCES pkTableName(pk_TableColumnName);
```



Per eliminare un vincolo:

```
1 ALTER TABLE table_name
2 DROP CONSTRAINT constraint_name;
```



mentre per abilitare o disabilitare un vincolo:

```
1 ALTER TABLE table_name SET CONSTRAINT constraint_name DISABLED/ENABLED;
```

Codice 7.4: Abilitazione e disabilitazione di un vincolo

Per cambiare il tipo di dato di una colonna in una tabella si usa la sintassi seguente:

```
1 ALTER TABLE table_name
2 ALTER COLUMN column_name datatype;
```



INTERROGAZIONI FONDAMENTALI IN SQL

8.1

LA STRUTTURA SELECT-FROM-WHERE



Come già visto nel Capitolo 6, SQL ha un'istruzione fondamentale per recuperare le informazioni da una base di dati: l'istruzione **SELECT**. A differenza del modello relazionale, SQL consente a una tabella di avere due o più tuple identiche in tutti i valori dei loro attributi (a meno di vincoli di unicità). In generale, quindi, una **tabella sql** non è un insieme di tuple, poiché un insieme non consente di avere ripetizioni, piuttosto è un **multinsieme**, talvolta chiamato **sacca** di tuple.

La forma fondamentale dell'istruzione **SELECT**, talvolta chiamata **blocco select-from-where**, è formata dalle tre clausole **SELECT**, **FROM** e **WHERE** e ha la forma seguente:

```
1 SELECT <elenco attributi>
2 FROM <elenco tabelle>
3 WHERE <condizione>;
```

Codice 8.1: Sintassi della clausola **SELECT**

Dove *<elenco attributi>* è un elenco dei nomi degli attributi i cui valori devono essere reperiti dall'interrogazione, *<elenco tabelle>* è un elenco dei nomi delle relazioni necessarie per eseguire l'interrogazione e *<condizione>* è un'espressione condizionale (booleana) che identifica le tuple che devono essere reperite dall'interrogazione.

Si prenda come esempio la tabella 8.2. È possibile selezionare le colonne “NomeCliente” e “Città” prendendo solo i clienti provenienti dalla Germania scrivendo:

```
1 SELECT NomeCliente, City
2 FROM Clienti
3 WHERE Paese='Italia';
```

Codice 8.2: Prima query in SQL

ottenendo come risultato la tabella 8.1.

NomeCliente	Città
Gunther Steiner	Faenza

Tabella 8.1: Risultato della query 8.2

IdCliente	NomeCliente	Indirizzo	Città	Paese
1	Gunther Steiner	Via della Birra 57	Faenza	Italia
2	Anarita Gonzalez	Avenida Colonial 33	Lima	Perù
3	Antonio Moreno	Via Matador Cavani	Montevideo	Uruguay
4	Francois De Rua	Rua Lisboa	Porto	Portogallo

Tabella 8.2: Tabella **CLIENTI**

Questa interrogazione coinvolge solo la tabella **Clienti** elencata nella clausola **FROM**. L'interrogazione *seleziona* le tuple di **Cliente** che soddisfano la condizione della clausola **WHERE**, poi *proietta* il risultato sugli attributi **NomeCliente** e **Città** elencati nella clausola **SELECT**. Una semplice interrogazione SQL con un solo nome di relazione nella clausola **FROM** è quindi simile alla coppia di operazioni *selezione-proiezione* dell'algebra relazionale.

8.1.1 ■ Assegnazione di alias

La tabella che si ottiene come risultato dell'interrogazione **SELECT** normalmente possiede un'intestazione delle colonne che riporta i nomi degli attributi; se si vuole modificare tale intestazione, occorre rinominare la colonna sfruttando la clausola **AS**.

In altri casi può capitare che lo stesso nome può essere usato per due o più attributi appartenenti a relazioni diverse. In questo caso, se c'è un'interrogazione che si riferisce ad attributi omonimi, si deve **qualificare** il nome dell'attributo con il nome della relazione per evitare ambiguità o utilizzare un **alias**. La sintassi sarà:

```
1 SELECT table1.column_name AS alias1, table2.column_name AS alias2
2 FROM table1, table2;
3 ...
```

Codice 8.3: Sintassi per l'assegnazione di alias alle tabelle

8.1.2 ■ Utilizzo dell'asterisco

Se in una **SELECT** manca la clausola **WHERE**, questa indica che non viene usata alcuna condizione per la selezione delle tuple; quindi tutte le tuple della relazione specificata nella clausola **FROM** sono considerate valide e costituiscono il risultato dell'interrogazione. Se nella clausola **FROM** è specificata più di una relazione e non c'è una clausola **WHERE**, viene eseguito il prodotto cartesiano, ottenendo tutte le possibili combinazioni delle tuple di queste relazioni. Per recuperare tutti i valori degli attributi delle tuple selezionate basta specificare un **asterisco (*)** che sta per *tutti gli attributi*.

```
1 SELECT *
2 FROM table_name;
```

Codice 8.4: Utilizzo dell'asterisco in SQL

8.2

LE TABELLE COME INSIEMI



SQL di solito considera una tabella come un multinsieme: *tuple con identici valori degli attributi possono comparire più di una volta* in una tabella e nel risultato di un'interrogazione. SQL non elimina automaticamente le tuple duplicate nei risultati delle interrogazioni. Una tabella SQL con una chiave deve però necessariamente essere un insieme. Quando si desidera eliminare le tuple duplicate dal risultato da un'interrogazione SQL, si utilizza la parola chiave **DISTINCT** nella clausola **SELECT**, il che significa che solo le tuple distinte compariranno nel risultato. La sintassi del **SELECT DISTINCT** è:

```
1 SELECT DISTINCT column1, column2, ...
2 FROM table_name;
```

Codice 8.5: La clausola **SELECT DISTINCT**

8.2.1 ■ Operazioni insiemistiche

Alcune operazioni dell'algebra relazionale sono state incorporate direttamente in SQL. Ci sono le operazioni insiemistiche di

1. unione: **UNION**
2. differenza: **EXCEPT**
3. intersezione: **INTERSECT**

Le relazioni che ne derivano sono insiemi di tuple, cioè *le tuple duplicate sono eliminate dal risultato*. Poiché tali operazioni si applicano solo alle *relazioni compatibili all'unione*¹, ci si deve assicurare che le due relazioni su cui si applica l'operazione abbiano gli stessi attributi e che gli attributi appaiano nello stesso ordine in entrambe le relazioni. La sintassi degli operatori insiemistici sarà:

```
1 SELECT column_name(s) FROM table1
2 [UNION][EXCEPT][INTERSECT]
3 SELECT column_name(s) FROM table2;
```

Codice 8.6: Gli operatori insiemistici in SQL

SQL ha anche le corrispondenti operazioni multinsieme, che vengono eseguite usando la parola chiave **ALL** (**UNION ALL**, **EXCEPT ALL**, **INTERSECT ALL**) e i loro risultati sono multinsiemi.

¹Vedi 6.2.1



8.3.1 La clausola WHERE

Nella clausola WHERE vanno messe condizioni che restituiscono valori VERO o FALSO. Queste tipologie di condizioni sono chiamate **condizioni booleane**. Le condizioni che si usano per **selezionare righe da una tabella** spesso coinvolgono il confronto del valore di un campo con una determinata costante o un altro valore. La tabella 8.3 mostra alcuni operatori che possono essere usati nelle queries.

Tabella 8.3: Operatori di confronto

Operatore	Significato	Esempio
=	Uguale	5 = 5, 'Junior' = 'Junior'
<	Minore	4 < 5, 'Ann' < 'Zebedee'
<=	Minore o uguale	4 <= 5
>	Maggiore	5 > 4
>=	Maggiore o uguale	5 >= 5
<>	Diverso	'Junior' <> 'Senior'

Si possono inoltre combinare due o più espressioni booleane per creare condizioni più interessanti attraverso gli **operatori logici**: AND, OR, NOT.

IS NULL

SQL consente inoltre di eseguire interrogazioni per verificare se il valore di un attributo è **NULL**. Per confrontare il valore di un attributo con NULL, anziché usare = o <>, SQL usa **IS** o **IS NOT**.

```

1 SELECT column_names
2 FROM table_name
3 WHERE column_name IS [NOT] NULL;
```

Codice 8.7: Utilizzo della clausola IS [NOT] NULL

8.3.2 Confronti di sottostringhe e operatori aritmetici

L'operatore **LIKE** in SQL all'interno della clausola WHERE serve a cercare un *pattern* specifico in una colonna. Questo può essere usato per il confronto di stringhe. Le **stringhe parziali** sono specificate utilizzando due caratteri riservati² (che possono essere anche usati in modo combinato):

- il **segno di percentuale** (%): sostituisce un numero arbitrario di zero o più caratteri;
- il **segno di sottolineatura** (_ , *underscore*): sostituisce un singolo carattere.

La sintassi dell'operatore LIKE è:

```

1 SELECT column1, column2, ...
2 FROM table_name
3 WHERE columnN LIKE pattern;
```

Codice 8.8: L'operatore LIKE

²Detti anche **wildcards**

Esempio

Di seguito vengono elencati differenti operatori LIKE unito ai caratteri riservato (%) e (_).

Operatore LIKE	Descrizione
WHERE Name LIKE 'a%'	Cerca ogni valore che inizia con “a”
WHERE Name LIKE '%a'	Cerca ogni valore che finisce con “a”
WHERE Name LIKE '%or%'	Cerca ogni valore che contiene la stringa “or”
WHERE Name LIKE '_r%'	Cerca ogni valore che contiene “r” nella seconda posizione
WHERE Name LIKE 'a_%'	Cerca ogni valore che inizia con “a” e ci sono almeno due caratteri
WHERE Name LIKE 'a%o'	Cerca ogni valore che inizia con “a” e finisce con “o”

Tabella 8.4: Esempio di utilizzo dell'operatore LIKE

Per quanto riguarda ancora le stringhe, l'operazione di concatenazione || può essere utilizzato in un'interrogazione per giustapporre i due valori stringa.

8.3.3 IN e BETWEEN

Un altro operatore di confronto che può essere usato per comodità è **BETWEEN** che seleziona i valori all'interno di un dato range. I valori possono essere numerici, stringhe o date. L'inclusione è da considerarsi inclusiva: gli estremi iniziali e finali sono compresi. Corrisponde quindi ad un AND. La sua sintassi è:

```
1 SELECT column_name(s)
2 FROM table_name
3 WHERE column_name BETWEEN value1 AND value2;
```

Codice 8.9: Sintassi della clausola BETWEEN

Allo stesso modo della keyword BETWEEN è possibile specificare vari possibili valori da selezionare mediante la keyword **IN** (che corrisponde quindi ad un OR):

```
1 SELECT column_name(s)
2 FROM table_name
3 WHERE column_name IN (value1, value2, ... );
```

Codice 8.10: La clausola IN

oppure mediante una query innestata³:

```
1 SELECT column_name(s)
2 FROM table_name
3 WHERE column_name IN (SELECT STATEMENT);
```



Gli operatori aritmetici Altre funzionalità permettono l'utilizzo dell'aritmetica nelle interrogazioni. Gli operatori aritmetici standard per l'addizione (+), la sottrazione (-), la moltiplicazione (*) e la divisione (/) possono essere applicati ai valori numerici o agli attributi con domini numerici.

8.4

GESTIRE I RISULTATI DELLE INTERROGAZIONI



8.4.1 Ordinare i risultati: la clausola ORDER BY

SQL consente all'utente di **ordinare** le tuple del risultato di un'interrogazione rispetto ai valori di uno o più attributi, usando la clausola **ORDER BY**. La clausola ordina le tuple in senso ascendente di default secondo la relazione d'ordine definita per il tipo di dato delle colonne che seguono la clausola ORDER BY. Per ordinare le tuple in ordine discendente si usa la parola chiave **DESC**. La sintassi sarà:

```
1 SELECT column1, column2, ...
2 FROM table_name
3 ORDER BY column1, column2, ... [ASC/DESC];
```

Codice 8.11: La clausola ORDER BY

³Vedi Capitolo 9

8.4.2 ■ Funzioni di aggregazione in SQL

Nella sezione 6.4.1 del Capitolo 6 sono state introdotte le funzioni di aggregazione. All'interno del comando `SELECT` possono essere usate funzioni predefinite che agiscono sui valori contenuti in insiemi di righe della tabella e che per questo motivo si chiamano **funzioni di aggregazione**. Le funzioni di aggregazione agiscono sui valori di una singola colonna e restituiscono un solo valore come il massimo o il minimo dei valori considerati. Le righe considerate per l'aggregazione sono quelle prodotte dall'interrogazione e quindi, in presenza di una clausola `WHERE`, sono le sole righe che soddisfano la condizione specificata. Le funzioni di aggregazione possono essere utilizzate solo come argomento scritto accanto a `SELECT` oppure nella clausola `HAVING`.

- **COUNT**: conta il numero di righe selezionate dall'interrogazione. La sintassi del linguaggio richiede di specificare come argomento della funzione il nome di un attributo oppure il carattere `*`: nel primo caso non vengono conteggiate le righe che hanno valore `NULL` nella colonna dell'attributo specificato; nel secondo caso, indicando l'asterisco, la funzione **`count(*)`** calcola il numero delle righe della tabella, incluse quelle con campi di tipo `NULL`. In sostanza **`COUNT(*)`** serve per ottenere la cardinalità di una relazione.

```
1 SELECT COUNT(column_name)
2 FROM table_name
3 WHERE condition;
```

Codice 8.12: La clausola `COUNT`

La stessa funzione nella versione **`count(distinct x)`** consente di ottenere il numero dei valori diversi tra loro nella colonna *x* che soddisfano alla condizione specificata nella `WHERE`.

- **SUM**: restituisce la somma di tutti i valori contenuti in una colonna specificata come argomento della funzione: naturalmente l'attributo utilizzato nel calcolo deve essere di tipo numerico; anche con questa funzione si può usare, se necessario, la parola `DISTINCT` prima del nome dell'attributo, se si vogliono eliminare dal calcolo della somma i valori duplicati presenti nella colonna specificata; i valori nulli sono trascurati dalla funzione `SUM`.

```
1 SELECT SUM(column_name)
2 FROM table_name
3 WHERE condition;
```

Codice 8.13: La funzione `SUM`

- **MAX**: restituisce il valore più grande della colonna selezionata.

```
1 SELECT MAX(column_name)
2 FROM table_name
3 WHERE condition;
```

Codice 8.14: Funzione `MAX`

- **MIN**: restituisce il valore più piccolo della colonna selezionata.

```
1 SELECT MIN(column_name)
2 FROM table_name
3 WHERE condition;
```

Codice 8.15: Funzione `MIN`

Esempio

Utilizzando un attributo numerico come lo stipendio, si possono calcolare i valori di minimo e massimo degli stipendi di tutti i dipendenti:

```
1 SELECT Min(Stipendio), MAX(Stipendio)
2 FROM Impiegato;
```



- **AVG**: funziona in maniera analoga della funzione `SUM`. Calcola la media dei valori (numerici) contenuti in una determinata colonna di una tabella, con l'eventuale aggiunta dell'opzione `DISTINCT`.

```
1 SELECT AVG(column_name)
2 FROM table_name
3 WHERE condition;
```

Codice 8.16: Funzione `AVG`

Queste funzioni possono essere usate nella clausola `SELECT` o nella clausola `HAVING`.

8.4.3 ■ Le clausole HAVING e GROUP BY

In molti casi, prima di applicare le funzioni di aggregazione, bisogna creare *sottogruppi di tuple in una relazione*, dove i sottogruppi si basano sui valori di un attributo.

Ad esempio, si può voler trovare lo stipendio medio dei dipendenti di ciascuna Sede o il numero di dipendenti che lavorano su ogni progetto. In questi casi è necessario **suddividere** la relazione in **gruppi** che non si sovrappongano. Ogni gruppo sarà costituito dalle tuple che hanno lo stesso valore per uno o più attributi, chiamati **attributi di raggruppamento**. SQL ha una clausola **GROUP BY** particolare per questo scopo. Questa clausola specifica gli attributi di raggruppamento che *dovrebbero comparire anche nella clausola SELECT*, così che il valore che risulta dall'applicazione di ogni funzione di aggregazione a un gruppo di tuple appare insieme al valore dell'attributo o degli attributi di raggruppamento.

Esempio

Elenchiamo id, nome e stipendio di ogni impiegato per ciascun dipartimento:

```
1 SELECT IdImpiegato, Nome, Stipendio
2 FROM Impiegati
3 GROUP BY Dipartimento;
```



La struttura del comando SELECT con raggruppamenti può essere ulteriormente ampliata con la clausola **Having** con la quale è possibile sottoporre al controllo di una o più condizioni i gruppi creati con la clausola GROUP BY. La condizione scritta dopo HAVING in genere controlla il valore restituito dalle funzioni di aggregazione.

Esempio

Il seguente esempio mostra l'uso del comando SELECT per ottenere, per i soli dipartimenti con più di due impiegati, il numero degli impiegati e la somma degli stipendi.

```
1 SELECT Dipartimento, Count(*) as NumImpiegati, Sum(Stipendi) as SommaStipendi
2 FROM Impiegato
3 GROUP BY Dipartimento
4 HAVING Count(IdImpiegato)>2
```



Se l'istruzione SELECT contiene la clausola WHERE, i valori vengono raggruppati **dopo aver operato la selezione sulle righe** che rispettano le condizioni scritte nella WHERE. Sintetizzando in un unico quadro le diverse clausole che possono apparire nel comando SELECT.

```
1 SELECT Elenco colonne da mostrare
2 FROM Tabelle da cui estrarre le righe
3 WHERE Condizioni sulle congiunzioni o sulle righe estratte
4 GROUP BY Campi da considerare per i raggruppamenti
5 HAVING Condizioni sui raggruppamenti
6 ORDER BY Ordinamenti sulle colonne elencate nella clausola select;
```



È opportuno fare le seguenti precisazioni:

- Le uniche clausole che devono comparire obbligatoriamente in un comando SELECT sono le prime due dell'elenco, le altre sono facoltative;
- Le clausole utilizzate devono essere elencate rispettando l'ordinamento del precedente elenco;
- Le interrogazioni che si possono costruire con le operazioni relazionali si esprimono con le clausole SELECT, FROM, WHERE;
- Il DBMS esegue il comando elaborando le clausole nel seguente ordine: 1.From, 2. Where, 3. Group by, 4. Having, 5. Select, 6. Order by

QUERY AVANZATE: LE SUBQUERIES

9.1

INTERROGAZIONI NIDIFICATE



Alcune interrogazioni richiedono di estrarre alcuni valori esistenti nella base di dati per poi usarli in una condizione di confronto. Tali interrogazioni possono essere formulate usando **interrogazioni nidificate**, che sono blocchi completi SELECT-FROM-WHERE posti all'interno di clausole FROM, WHERE e HAVING. Non a caso SQL sta per **structured query language** proprio ad indicare la possibilità di utilizzare interrogazioni, chiamate **esterne** composte da altre sottointerrogazioni o **subquery**.

In generale si è visto che l'argomento della clausola WHERE si basa su *condizioni composte da predicati semplici* in cui ciascun predicato rappresenta un semplice confronto fra due valori. SQL ammette anche l'uso di predicati con una struttura più complessa, in cui si confronta un valore (ottenuto come risultato di un'espressione valutata sulla singola riga) con il risultato dell'esecuzione di un'interrogazione SQL, definita direttamente nel predicato interno alla clausola WHERE.

Va osservato però il fatto che se in un predicato si confronta un attributo con il risultato di una query sorge un **problema di disomogeneità** dei termini del confronto. Infatti, se *nel membro a destra si ha un multinsieme*, a sinistra si ha *il valore di un attributo per la particolare riga*. Tale problema viene risolto da SQL tramite l'utilizzo di alcune parole chiave come **IN** (e **NOT IN**) che confronta un valore v con un insieme o un multinsieme di valori V e il cui valore è TRUE se v è uno degli elementi di V .

Se un'interrogazione nidificata restituisce un singolo attributo e una singola tupla, il risultato sarà un singolo valore, in questo caso la subquery si dirà **scalare**. In questi casi, come operatore di confronto è possibile usare $=$ anziché IN. Il numero e il tipo di attributi della SELECT interna **devono però sempre coincidere con quelli valutati nella condizione where**. La sintassi generale sarà quindi:

```
1 SELECT ...
2 FROM ...
3 WHERE (attr1, ..., attrN) [NOT] IN
4 (
5     SELECT (expression1, ..., expressionN)
6     FROM ..
7     WHERE ..
8 );
```



Si considerino le tabelle nello schema 9.1 e si supponga di voler conoscere il nome del direttore dell'impiegato Kris Weaben. Le **interrogazioni nidificate** permettono di risolvere il problema nel seguente modo:

```
1 -- Step 3 : risalire al nome del manager
2 select Nome
3 from Impiegati
4 where SSN=
5 -- Step 2 : recupero identificativo responsabile
6 ( select Responsabile
7   from Dipartimenti
8   where Numero_D =
9   -- Step 1 : recupero del numero di dipartimento
10  ( select Dipartimento
11    from Impiegato
12    where Nome= 'Kris Weaben' ) );
```



Esempio

Si considerino le seguenti tabelle:

Studenti	Id_Studente	Nome
	V001	Carlo
	V002	Vladimiro
	V003	Giuseppe
	V004	Federico

Voti	Id_Studente	Voti_Totali
	V001	95
	V002	100
	V003	85
	V004	80

Tabella 9.1: Tabelle STUDENTI e VOTI

Vogliamo scrivere una interrogazione per identificare tutti gli studenti che hanno voti migliori dello studente con id = V002, senza sapere il voto di V002. Per risolvere il problema saranno necessarie due query.

- Una query per ottenere il voto di 'V002':

```
1 SELECT *
2 FROM Voti
3 WHERE Id_Studente = 'V002';
```



- Una seconda query che identifica gli studenti che hanno un voto maggiore del risultato della prima query:

```
1 SELECT S.Studenti_Id, S.Nome, V.Voti_Totali
2 FROM Studenti AS S JOIN Voti AS V
3 ON S.Id_Studente = V.Id_Studente
4 WHERE V.Voti_Totali > [voto di 'V002']
```



Unendo le due query si ottiene la subquery:

```
1 SELECT S.Studenti_Id, S.Nome, V.Voti_Totali
2 FROM Studenti AS S JOIN Voti as V
3 ON S.Id_Studente = V.Id_Studente
4 WHERE V.Voti_Totali > (
5     SELECT Voti_Totali
6     FROM Voti
7     WHERE Id_Studente = 'V002');
```



9.1.1 Le clausole ANY e ALL

Innestare una subquery dopo la clausola WHERE richiede una espressione booleana che possa essere valutata come TRUE o FALSE. Le due operazioni possibili sono:

1. Verifica dell'appartenenza di un valore o insieme di valori all'insieme V;
2. Verifica della cardinalità del risultato della subselect (se è vuoto o contiene righe).

Oltre all'operatore IN possono essere usati altri operatori di confronto per confrontare un singolo valore v (di solito un nome di attributo) con un insieme o un multi-insieme V . L'operatore **=ANY** restituisce TRUE se il valore v è uguale a un *qualsiasi* valore nell'insieme V ed è quindi equivalente a IN.

Altri operatori che possono essere usati con ANY e SOME includono $>$, $<$, $>=$, $<=$ e $<>$. Anche la parola chiave **ALL** può essere usata insieme a ciascuno di questi operatori. Ad esempio la condizione di confronto ($v > \text{ALL}(V)$) restituisce TRUE se il valore v è maggiore di tutti i valori dell'insieme V .

Se la subquery può restituire più di un valore si usano le forme:

- **<op> ANY**: la relazione **<op>** vale per **almeno uno** dei valori;

```
1 SELECT column_name(s)
2 FROM table_name
3 WHERE column_name operator ANY(
4     SELECT column_name
5     FROM table_name
6     WHERE condition
```



- **<op> ALL**: la relazione <op> vale per **tutti** i valori;

```

1 SELECT column_name(s)
2 FROM table_name
3 WHERE column_name operator ALL(
4     SELECT column_name
5     FROM table_name
6     WHERE condition);

```



9.1.2 ■ SELECT innestate nella clausola FROM

Una query SELECT può essere annidata in un'altra query all'interno della clausola FROM. La SELECT annidata può restituire una tabella con più righe e colonne ed eventualmente può essere battezzata con un nome. La sintassi generale sarà:

```

1 SELECT <attr1>, ... ,<attrN>, nome_tabella.attr1, ...
2 FROM
3 (
4     SELECT <attr1>, ... ,<attrK>
5     FROM <tabella>
6     WHERE <condizione booleana>
7 ) AS nome_tabella
8 WHERE nome_tabella.attr1 ...

```



9.2

INTERROGAZIONI NIDIFICATE CORRELATE



Quando una condizione nella clausola WHERE di una interrogazione nidificata fa riferimento ad attributi di una relazione dichiarata nell'interrogazione esterna, si dice che le due interrogazioni sono **correlate**. Si può capire meglio il concetto di interrogazione correlata considerando che l'interrogazione nidificata è valutata una volta *per ciascuna tupla* (o combinazioni di tuple) dell'interrogazione esterna.

9.2.1 ■ Quantificazione esistenziale: la funzione EXISTS

Mediante la funzione **EXISTS** è possibile verificare se il risultato di una subquery restituisce almeno una tupla. Facendo uso di **NOT EXISTS** il predicato è vero se la subquery non restituisce alcuna tupla. Le funzioni EXISTS e NOT EXISTS sono frequentemente usate nelle interrogazioni nidificate correlate.

Nell'esempio 9.2.1, l'interrogazione nidificata si riferisce agli attributi *Num_D* della relazione Dipartimento dell'interrogazione esterna. Adesso quindi **il risultato della query innestata dipende dal dipartimento specifico**, e la semantica diventa: *per ogni tupla del blocco esterno considera il valore di Dipartimento.Num_D e risolvi la query innestata*. Se nel risultato dell'interrogazione nidificata esiste almeno una tupla, si selezioni quella tupla di Dipartimento e si proietti il campo Num_D.

In generale EXISTS(I) restituisce **true** se c'è almeno una tupla nel risultato dell'interrogazione I, altrimenti restituisce **false**. Al contrario NOT EXISTS(I) restituisce **true** se non ci sono tuple nel risultato dell'interrogazione I, altrimenti restituisce **FALSE**.

Esempio

Si considerino le tabelle nello schema 9.1. Si vogliono trovare le sedi con almeno un impiegato:

```

1 SELECT S.Num_D
2 FROM Dipartimento AS S
3 WHERE EXISTS
4 (
5     SELECT *
6     FROM Imp as I
7     WHERE S.Num_D = I.Dipartimento
8 );

```



Esempio

Si trovino i nomi degli impiegati che non lavorano in nessun progetto:

```
1 SELECT Nome, Cognome
2 FROM Impiegati AS I
3 WHERE NOT EXISTS (
4     SELECT *
5     FROM Progetti
6     WHERE I.SSN = P.Impiegato
7 );
```



Esempio

Sia PERSONE una tabella composta da quattro colonne:

Nome	Cognome	Età	Residenza
Mario	Rossi	21	Roma
Giovanni	Bianchi	22	Milano
Paolo	Rossi	23	Pescara
Giuseppe	Verdi	19	Bologna
Antonio	Ferrari	21	Napoli
Maria	Rossetti	18	Milano

Voglio verificare se esistono nella tabella delle persone con lo stesso cognome ma nome diverso.

```
1 SELECT *
2 FROM Persone AS T1
3 WHERE EXISTS (
4     SELECT *
5     FROM Persone AS T2
6     WHERE T2.Cognome = T1.Cognome AND T1.Nome <> T2.Nome
7 );
```



Il risultato della query sarà:

Nome	Cognome	Età	Residenza
Mario	Rossi	21	Roma
Paolo	Rossi	23	Pescara

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Figura 9.1

JOIN E PRODOTTO CARTESIANO IN SQL

Il concetto di **tabelle collegate via join** è stato già visto nella sezione 6.3 del Capitolo 6. Congiungere le tabelle permette agli utenti di effettuare la ricerca di informazioni mettendo in relazione i dati presenti in tabelle diverse.

Consideriamo due tabelle che abbiano colonne contenenti dati in comune. Se due tabelle soddisfano questa condizione possono essere correlate tramite una operazione di join. Questa caratteristica rappresenta una delle maggiori potenzialità dei sistemi relazionali: i collegamenti tra i dati vengono implementati dinamicamente al momento dell'esecuzione delle query basandosi sul contenuto di colonne omogenee senza dover utilizzare dei puntatori diretti. Ciò rende sempre possibile collegare tra di loro informazioni contenute in tabelle diverse ma accomunate da un identico valore in una particolare colonna. Grazie a questa caratteristica i modelli dei dati di un sistema relazionale possono crescere al crescere delle esigenze del sistema che rappresentano.

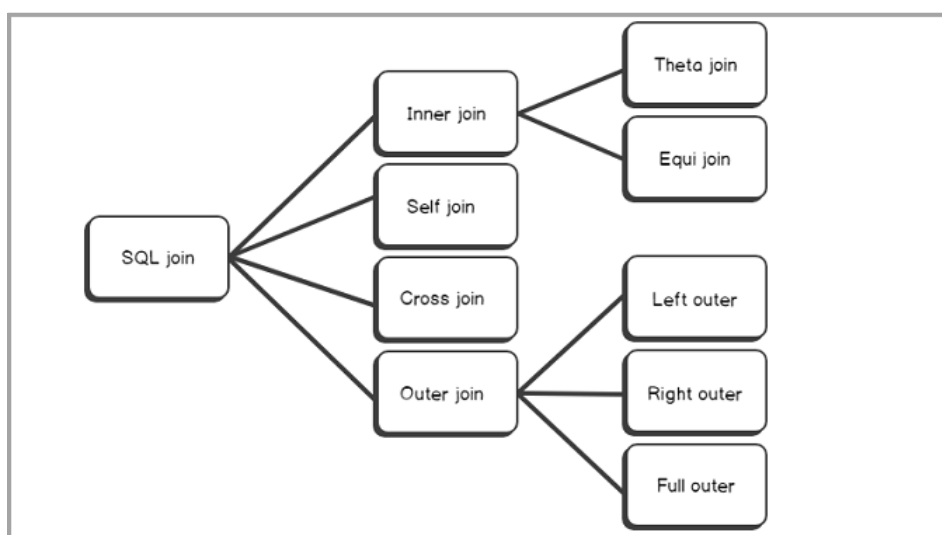


Figura 10.1: Elenco delle operazioni di join in SQL

10.1

LA CLAUSOLA JOIN



Una clausola di **JOIN** permette di combinare le righe di due o più tabelle, sulla base della condizione specificata dalla clausola **ON** (in caso contrario effettua il prodotto cartesiano). La sua sintassi è:

```
1 SELECT column_names
2 FROM table_name1 JOIN table_name2
3 ON <condizione di join>
4 WHERE <condizione di selezione>
```



Il risultato di una join, anche detta **INNER JOIN** sarà una tabella che contiene solo le righe che hanno valori corrispondenti in entrambe le tabelle.

10.1.1 La clausola NATURAL JOIN

La **NATURAL JOIN** o anche detta **equi join** è un caso particolare di operazione di join. Infatti non necessita di specificare alcuna **condizione di join**. Sarà il DBMS a verificare la presenza di colonne con lo stesso nome e dominio e ad effettuare una congiunzione tra le tuple compatibili. In caso contrario viene effettuato un prodotto cartesiano.

10.1.2 Le operazioni di OUTER JOIN

Quando vengono correlate mediante una join due tabelle con colonne contenenti dati in comune, è possibile che un valore presente in una delle due colonne non compaia nell'altra.

Esempio

Si considerino le tabelle CLIENTI e ORDINI mostrate in 10.1.

Clienti	id_cliente	nome	Ordini	id_ordine	quantità	cliente
	1	John		1	200	10
	2	Robert		2	500	3
	3	David		3	300	6
	4	John		4	800	5
	5	Betty		5	150	8

Tabella 10.1: Esempio di outer join: CLIENTI e ORDINI

Effettuando una operazione di join su *id_cliente* e *id_ordine* si ottiene la seguente tabella:

id_cliente	nome	id_ordine	quantità	cliente
3	David	2	500	3
5	Betty	4	800	5

In alcuni casi però è importante non perdere nella join la nozione di **mancata corrispondenza**.

Una join che preservi questa informazione è detta **outer join**. La outer join può mantenere i valori per cui non esiste corrispondenza per una, per l'altra o per entrambe le tabelle. I tre casi vengono rispettivamente definiti:

- **LEFT (OUTER) JOIN**: restituisce le righe della tabella di sinistra, e le righe che hanno trovato il match della tabella di destra;
- **RIGHT (OUTER) JOIN**: restituisce tutte le righe della tabella di destra e le righe che hanno trovato il match della tabella di sinistra;
- **FULL (OUTER) JOIN**: restituisce tutte le righe quando viene trovato un match e quelle escluse sia della tabella di sinistra che di destra.

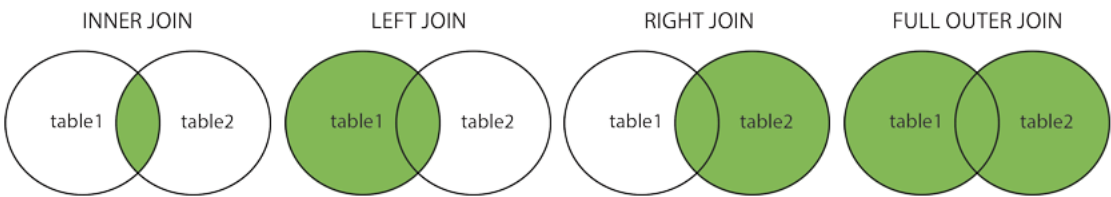


Figura 10.2: Risultato delle varie operazioni di join in SQL

10.2

IL CROSS JOIN O PRODOTTO CARTESIANO

Per ottenere il **prodotto cartesiano**, ovvero una combinazione tra tutti i valori degli attributi specificati nella clausola SELECT basta separare da una virgola le relazioni presenti nella clausola FROM:

```
1 SELECT column_name1, column_name2
2 FROM table_name1,table_name2;
```

In alternativa è possibile usare l'operazione **CROSS JOIN** senza specificare alcuna *condizione di giunzione*:

```

1 SELECT *
2 FROM table_name1 CROSS JOIN table_name2

```



10.2.1 Il self-join

Un caso molto particolare di join è quello che utilizza una singola tabella, correlandola con se stessa; si ha in questo caso una **self join**. La tecnica suppone di avere due copie assolutamente identiche della stessa tabella ed effettua successivamente una natural join sulla colonna da congiungere. Per mettere in pratica una self join è necessario usare gli alias, ovvero ridenominare le tabelle.

Si consideri l'associazione esistente tra un oggetto e le sue parti (che sono ancora oggetti). Per esempio, l'oggetto camicia è composto da 1 colletto, 2 maniche, 10 bottoni, 1 taschino e altri oggetti. È stata introdotta, di fatto, un'associazione ricorsiva sull'entità *Oggetto*, di attributi *ID*, *Descrizione*, *Qta*, che è rappresentata nel modello relazionale da una tabella con il seguente schema:

OGGETTI (ID, Descrizione, Qta, ComponenteDi)

Nel caso degli oggetti che compongono una camicia si dovrebbe considerare una tabella del tipo:

Id	Descrizione	Qta	ComponenteDi
1	Camicia	1	NULL
2	Manica	2	1
3	Colletto	1	1
4	Bottone	10	1
5	Taschino	1	1

Tabella 10.2: Tabella OGGETTI

Dalla tabella si vede che l'oggetto identificato dall'*Id* = 4, il bottone, è una componente dell'oggetto camicia e che in una camicia ci sono 10 bottoni. Naturalmente nella tabella Oggetti ci saranno oltre all'oggetto *Camicia* anche altri oggetti composti, quali giacca, cappotto e altri, con le relative componenti.

Se si vogliono ottenere le informazioni delle diverse parti che compongono gli oggetti, assieme alle informazioni sugli oggetti composti, occorre congiungere la tabella OGGETTI con se stessa, cioè operare un **self-join**. Per esprimere la condizione che una riga di OGGETTI deve essere abbinata ad una riga, sempre di OGGETTI, con valore di *Id* in una uguale a quello di *ComponenteDi* nell'altra, occorre usare gli **alias** per il nome della tabella, cioè indicare con due nomi diversi la stessa tabella.

```

1 create table oggetti
2 (
3   id int primary key,
4   descrizione varchar(25),
5   quantity int,
6   componentedi int
7 );
8
9 insert into oggetti values
10 (1, 'camicia', 1, null),
11 (2, 'manica', 2, 1),
12 (3, 'colletto', 1, 1),
13 (4, 'bottone', 10, 1),
14 (5, 'taschino', 1, 1);
15
16 select *
17 from oggetti as parte join oggetti as composto
18 on parte.componentedi = composto.id;

```



id	descrizione	qta	componentedi	id	descrizione	qta	componentedi
2	manica	2	1	1	camicia	1	null
3	colletto	1	1	1	camicia	1	null
4	bottone	10	1	1	camicia	1	null
5	taschino	1	1	1	camicia	1	null

DML: DATA MANIPULATION LANGUAGE

In SQL, per modificare la base di dati si possono usare tre comandi: **INSERT**, **DELETE** e **UPDATE**.

11.1

IL COMANDO INSERT



L'istruzione **INSERT INTO** serve per inserire, anche detto “popolare”, nuove righe in una tabella. È possibile usare questa istruzione in due modi:

1. Specifica delle colonne e dei valori da inserire.

```
1 INSERT INTO <nome tabella> [elenco attributi]
2 VALUES (lista valori);
```

Codice 11.1: Sintassi della clausola **INSERT INTO**

Se si omette l'elenco degli attributi si intende che i valori sono inseriti nell'ordine in cui sono dichiarati nello schema della tabella. Se indico i nomi delle colonne, posso indicare un sottoinsieme di valori per cui l'ordine non conta. Posso inserire più righe alla volta.

Esempio

Si consideri la tabella:

ESAMI (Matricola, NomeStudente, Data, Corso, Voto, Lode)

È possibile fare un inserimento di una riga all'interno della tabella come segue:

```
1 INSERT INTO Esami
2 VALUES ('N961234', 'Ciro Mertens', '2021-10-10', 'BDD', 25, NULL);
3 INSERT INTO Esami (Nome_Studente, Data, Corso, Matricola, Voto)
4 --Lode viene messa a NULL
5 VALUES ('Edinson Esposito', '2021-11-9', 'BDD', N891111, 30);
```



2. Inserimento di dati calcolati mediante **subselect**:

```
1 INSERT INTO table_name [(column1, column2, column3, ..)]
2 (
3     SELECT (expression)
4 );
```



La lista degli attributi deve avere lo stesso numero e lo stesso tipo della lista delle espressioni estratte con la **SELECT**. In questo modo si inseriscono tante righe contemporaneamente.

Esempio

Si consideri la tabella

CORSI (NomeCorso, Anno, Descrizione, IdDocente)

```
1 INSERT INTO Corsi
2 (
3     SELECT C.NomeCorso,(C.AnnoCorso+1),C.Descrizione,C.IdDocente
4     FROM Corsi AS C
5     WHERE C.AnnoCorso=2022
6 );
```



11.2

IL COMANDO DELETE



Il comando **DELETE** elimina le tuple da una relazione. Include una clausola **WHERE** per selezionare le tuple da cancellare. A seconda del numero di tuple selezionate dalla condizione nella clausola **WHERE**, nessuna, una o più tuple possono essere cancellate da un singolo comando **DELETE**. Se la clausola **WHERE** è omessa allora è come se ci fosse la clausola

WHERE TRUE

e vengono eliminate tutte le righe della tabella.

La sintassi è:

```
1 DELETE FROM table_name
2 WHERE condizione
```

Codice 11.2: Sintassi della clausola DELETE

Esempio

La seguente interrogazione elimina tutte le tuple della tabella **EMPLOYEE** che soddisfano alla condizione **lname = 'Brown'**.

```
1 DELETE FROM Employee
2 WHERE lname = 'Brown'
```



È possibile specificare una query annidata all'interno della clausola **WHERE**:

```
1 DELETE FROM Employee
2 WHERE dno IN (
3     SELECT dnumber
4     FROM Department
5     WHERE dname='Research');
```

Codice 11.3: Esempio di utilizzo della clausola **WHERE** con query annidata

11.3

IL COMANDO UPDATE



Il comando **UPDATE** è usato per modificare i valori degli attributi di una o più tuple selezionate. Come nel comando **DELETE**, la clausola **WHERE** del comando **UPDATE** seleziona le tuple da modificare in una singola relazione. Tramite la clausola **SET** si specificano gli attributi su cui sono fatte le modifiche. Le operazioni vengono eseguite in questo ordine: prima viene scelta la tabella, poi si selezionano le righe e successivamente avviene la modifica degli attributi con la **SET**. La sintassi è:

```
1 UPDATE table_name [alias]
2 SET <nome attributo> = <espressione>[,<nome attributoN>><espressioneN>]
3 [WHERE <condizione>]
```

Codice 11.4: Sintassi della clausola UPDATE

Esempio

Si consideri la tabella, per modificare l'IdDocente per il corso di Basi di Dati dell'anno 2022 si avrà:
CORSI(NomeCorso, Anno, Descrizione, IdDocente)

```
1 UPDATE Corsi
2 SET IdDocente = 1
3 WHERE (Anno = 2022 AND NomeCorso='BDD');
```



Esempio

È possibile avere delle query UPDATE con subselect. Ad esempio:

```
1 UPDATE Corsi
2 SET IdDocente = --subquery scalare
3 (
4   SELECT IdDocente
5   FROM Docenti
6   WHERE Nome='Mara' AND Cognome='Sangiovanni'
7 )
8 WHERE (AnnoCorso = 2022 AND NomeCorso = 'BDD');
```



ASSERZIONI E TRIGGER

12.1

LE ASSERZIONI



Una base di dati, descrivendo i dettagli del mini-mondo che rappresenta, deve soddisfare una serie di restrizioni che garantiscano la coerenza dei dati che contiene (vedi 4.3). I vincoli possono dividersi in quattro categorie:

- **Vincoli di dominio:** ovvero i vincoli su un singolo attributo;
- **Vincoli di ennupla:** ovvero i vincoli su una singola riga;
- **Vincoli intrarelazionali:** ovvero i vincoli che coinvolgono più righe di una stessa tabella e i vincoli di primary key e unicità;
- **Vincoli interrelazionali:** ovvero i vincoli di foreign key.

I vincoli interrelazionali, però, possono coinvolgere colonne di tabelle diverse tabelle. L'SQL permette di definire vincoli di controllo su più tabelle, includendoli nella definizione di una o dell'altra delle tabelle, o magari duplicandoli in tutte le tabelle interessate. Per evitare gli svantaggi che derivano da queste due notazioni, ovvero la scarsa leggibilità nel primo caso e la duplicazione di codice nel secondo, è definito in SQL l'oggetto **asserzione**, che corrisponde ad un vincolo di integrità definito separatamente dalle tabelle cui è riferito. Le asserzioni rappresentano quindi vincoli non associati a nessun attributo o tabella in particolare. Appartengono direttamente alla base di dati e permettono di esprimere vincoli particolari, come vincoli su più tabelle o vincoli che richiedono una cardinalità minima o massima per una tabella.

12.1.1 ■ Sintassi delle asserzioni

La sintassi di una **assertion** è la seguente:

```
1 CREATE ASSERTION constraint_name
2 CHECK boolean_condition
```

Codice 12.1: Sintassi assertion

oppure, usando subquery:

```
1 CREATE ASSERTION constraint_name
2 CHECK [NOT] EXISTS
3 (
4     SELECT ...
5     FROM ...
6     WHERE ...
7 )
```



La maggior parte dei vincoli usa la forma con NOT EXISTS per controllare che una condizione non si verifichi (si cerca nella subquery la **violazione nel vincolo**).

Esempio

Per verificare che non ci siano stagisti nella sede di Boston si può avere:

```
1 CREATE ASSERTION no_stagisti_sede_boston
2 CHECK NOT EXISTS
3 (
4   SELECT 'stagista a Boston'
5   FROM Impiegato AS I,Dipartimento AS D
6   WHERE I.NumeroDipartimento =D.idDipartimento
7   AND I.TipoContratto='Stagista'
8   AND d.Sede='Boston'
9 );
```



Esempio

Si considerino le tabelle IMPIEGATO e DIPARTIMENTO (vedi Tabella 12.1).

Tabella 12.1

IMPIEGATO	Nome	Cognome	Dipartimento	Stipendio
-----------	------	---------	--------------	-----------

DIPARTIMENTO	Nome	Città
--------------	------	-------

```
1 CREATE ASSERTION limite100 (
2 CHECK (
3   (SELECT COUNT( DISTINCT dipartimento) FROM Impiegato) +
4   (SELECT COUNT( nome) FROM dipartimento)<100)
5 );
```



Ad esempio, se il numero di valori contenuti nell'attributo *Nome* della tabella DIPARTIMENTO è superiore a 100 e la tabella IMPIEGATO è vuota, il vincolo non risulta soddisfatto.

Esempio

Creare un vincolo che imponga che nella tabella IMPIEGATO ci sia almeno una riga.

```
1 CREATE ASSERTION almenoUna (
2 CHECK (1 ≤ (SELECT COUNT (*) FROM impiegato)));
```



12.2

I TRIGGER



Un **trigger** è una regola che segue il modello **ECA (event-condition-action)**: quando un evento occorre, controlla una condizione, se la condizione è soddisfatta esegui una azione. Può essere quindi visto come una procedura o una funzione per la manutenzione della base di dati. A differenza delle procedure standard che sono chiamate in maniera esplicita, un trigger non viene mai invocato. La loro attivazione, infatti, è scatenata da una serie di eventi di diversa natura che avvengono nella base di dati. Questi eventi scatenano una serie di operazioni che servono per mantenere la coerenza della base di dati.

Per motivi didattici la sintassi mostrata in questo capitolo è quella proprietaria di Oracle¹.

12.2.1 Parametri di un trigger

Nel momento in cui si va a definire un trigger, bisogna definire i suoi **parametri**:

1. Quale evento scatena la reazione?
 - Operazioni di inserimento
 - Operazioni di cancellazione
 - Operazioni di aggiornamento

¹Vedi il Capitolo 13

2. Quando voglio scatenare la reazione? Prima o dopo l'evento?
 - Dopo: c'è stato, ad esempio una operazione di aggiornamento e voglio propagare l'evento;
 - Prima: Ci sarà, ad esempio, una cancellazione e voglio eseguire dei comandi prima della cancellazione.
3. Condizioni di filtraggio che mi permettono di definire precisamente quale sia l'evento che scatena la reazione;
4. Operazione globale / per ciascuna riga;
5. Reazione

Come detto in precedenza, i possibili eventi che possono scatenare un trigger sono:

```
1 INSERT ON <tabella>
2 DELETE ON <tabella>
3 UPDATE OF <attributo> ON <tabella>
```



Inoltre possiamo specificare **quando scatenare** la reazione attraverso i parametri:

```
1 AFTER
2 BEFORE
3 INSTEAD OF
```



seguiti dall'operazione che scatena il trigger.

Per specificare invece la precisione o **granularità** possiamo specificare di attivare un trigger:

- **FOR EACH ROW** modificata;
- **FOR EACH STATEMENT** che comporta una modifica.

```
1 CREATE [OR REPLACE ] TRIGGER trigger_name
2 {BEFORE | AFTER | INSTEAD OF }
3 {INSERT [OR] | UPDATE [OR] | DELETE}
4 [OF col_name]
5 ON table_name
6 [REFERENCING OLD AS o NEW AS n]
7 [FOR EACH ROW]
8 WHEN (condition)
9 DECLARE
10 Declaration-statements
11 BEGIN
12 Executable-statements
13 EXCEPTION
14 Exceptions-handling-statements
15 END;
```

Codice 12.2: Sintassi generale per la definizione di un trigger

12.2.2 ■ Modifica dei Trigger

Un trigger è un elemento dello schema della base di dati, in quanto tale può subire:

- **Cancellazione:**

```
1 DROP trigger_name
```



- **Disabilitazione:**

```
1 ALTER TRIGGER trigger_name
2 [ENABLED/DISABLED]
```



- **Modifica:**

```
1 CREATE [OR REPLACE] definition
```



12.2.3 La clausola BEFORE

La clausola BEFORE viene utilizzata quando vengono effettuati, ad esempio, degli inserimenti e si vogliono controllare i dati inseriti.

Si vogliono creare degli id autoincrementanti, consideriamo la tabella 12.2.

AutoincrementId	Nome	Cognome
-----------------	------	---------

Tabella 12.2

Per iniziare si crea un particolare tipo di tabella che contiene una **sequenza**:

```
1 CREATE SEQUENCE my_id
2 START WITH 10000
3 INCREMENT BY 1
```



successivamente si definisce un trigger:

```
1 CREATE TRIGGER incrementa_id
2 BEFORE INSERT ON Esempio
3 FOR EACH ROW
4 BEGIN
5 New.AutoincrementId = my_id.Nextval
6 END
```



12.2.4 La clausola INSTEAD OF

La clausola **INSTEAD OF** non viene utilizzata sulle tabelle bensì esclusivamente per operazioni di manipolazione sulle viste². Infatti, nel caso in cui si vogliano fare operazioni di scrittura bisogna sempre propagare le informazioni nelle tabelle sottostanti.

Consideriamo la tabella 12.3 dove gli attributi *CF*, *Nome* e *Cognome* sono attributi totali mentre *DataN* e *Residenza* sono parziali.

Tabella 12.3

<u>Matricola</u>	CF	Nome	Cognome	DataN	Residenza
------------------	----	------	---------	-------	-----------

Quando si crea una vista, questa deve contenere il campo chiave, deve includere gli attributi totali e deve essere basata su una tabella o su particolari combinazioni di tabelle.

```
1 CREATE VIEW Studenti2 AS
2 SELECT Matricola, CF, Nome, Cognome
3 FROM Studenti
```



Tutte le operazioni di aggiornamento (INSERT, DELETE, UPDATE) si ripercuotono sulla tabella originale.

È possibile quindi pensare di scorporare la tabella 12.3 in due tabelle STUDENTE e ANAGRAFE ottenendo così una relazione 1:1 tra STUDENTE e ANAGRAFE e definire successivamente una vista che congiunga le due tabelle.

Tabella 12.4: Divisione della tabella 12.3 in due

Studenti	Matricola	Nome	Cognome	Anagrafe	Matricola	CF	Residenza	DataN
----------	-----------	------	---------	----------	-----------	----	-----------	-------

```
1 CREATE VIEW info AS
2 SELECT *
3 FROM Studente INNER JOIN Anagrafe
```



Il seguente trigger utilizza la sintassi del linguaggio procedurale PL/pgSQL (vedi il Capitolo 15):

```
1 CREATE TRIGGER Info_Stud
2 INSTEAD OF INSERT ON Info OR DELETE ON Info OR UPDATE OF Attributo On ...
3 FOR EACH ROW
```

²Vedi ??.


```

4 BEGIN
5 IF INSERTING (TG_OP = 'INSERT')
6 INSERT INTO Studente (Matricola, Nome, Cognome)
7 VALUES (NEW.Matricola, NEW.Nome, NEW.Cognome)
8 INSERT INTO Anagrafe (Matricola, CF, Residenza, DataN)
9 VALUES (NEW.Matricola, NEW.CF, NEW.Residenza, NEW.DataN)
10 END IF
11 IF DELETING (TG_OP = 'DELETE')
12 DELETE FROM Anagrafe WHERE Matricola = OLD.Matricola
13 DELETE FROM Studente WHERE Matricola = OLD.Matricola
14 END IF
15 IF UPDATING (TG_OP = 'UPDATE')
16 ...
17 END IF
18 END

```

Codice 12.3: Esempio di trigger in PostgreSQL che utilizza la clausola `INSTEAD OF`

LINGUAGGI PROCEDURALI PER SQL: ORACLE PL/SQL

13.1

INTRODUZIONE



Nella costruzione di un database spesso l'attenzione è concentrata sui dettagli concettuali e logici dello schema al fine di ottimizzare lo sfruttamento delle potenzialità degli strumenti standard di manipolazione ed interrogazione dei dati. È spesso vero però che SQL da solo non garantisce le caratteristiche di proceduralità necessarie per la implementazione di operazioni complesse sui dati.

Per implementare tali procedure i vari DBMS rendono disponibili dei linguaggi procedurali che cercano di estendere le potenzialità del linguaggio SQL. Infatti, se ogni istruzione SQL è eseguita in maniera singola dal DBMS, grazie all'estensione dei linguaggi procedurali si sposta parte del carico computazionale sul server del DBMS aumentandone le performance. In particolare, PL/SQL è un linguaggio procedurale reso disponibile nell'ambiente Oracle (è **embedded**) e fornisce una sintassi per la programmazione strutturata che consente, oltre alle tipiche operazioni di un linguaggio di programmazione, la interazione con un database mediante costrutti SQL e l'utilizzo di astrazioni funzionali in procedure/ funzioni e packages.

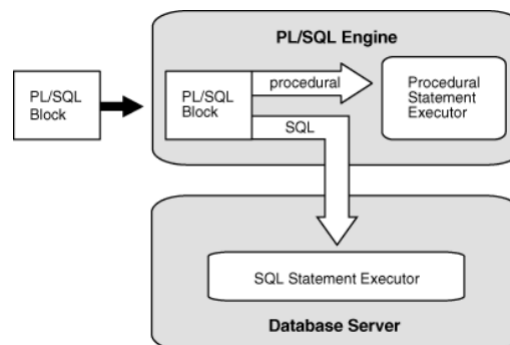


Figura 13.1: PL/SQL engine

PL/SQL rappresenta un linguaggio di programmazione interpretato per la macchina astratta definita dal sottostante interprete **SQLplus**. Dato quindi un programma PL/SQL, memorizzato in un file `PLSQLprog.sql`, la sua attivazione è possibile mediante il seguente comando SQLplus:

```
SQLplusPrompt > start PLSQLprog
```

13.2

STRUTTURA DI BASE DI UN BLOCCO PL/SQL



Un blocco PL/SQL è caratterizzato dalla seguente struttura fondamentale:

- 1 [Header]
- 2 [DECLARE
- 3 <Costanti>
- 4 <Variabili>

```

5 <Gestori di eccezioni/errori>]
6 BEGIN
7 <Istruzioni PL/SQL>
8 [ EXCEPTIONS
9 <Gestione eccezioni/Errori>]
10 END
11 / --Viene usato per eseguire la procedura

```

Codice 13.1: Struttura di un blocco PL/SQL

La struttura quindi contiene sezioni opzionali (**Header**, **Declare**) e una notazione standard per la definizione del blocco (**BEGIN...END**). Costanti e variabili sono analoghe ai corrispondenti usi di essi nella programmazione strutturata. Le regole di visibilità degli identificatori seguono esattamente le regole usate in un ambiente di programmazione a *scope statico* (ad es. il Pascal).

La struttura standard di un blocco PL/SQL è costituita da tre parti ben distinte:

1. **Declare:** sezione di dichiarazione, dove si dichiarano le variabili e i cursori usati nel programma;
2. **Begin:** sezione di esecuzione. In questo punto si scrivono le istruzioni che il programma deve eseguire;
3. **Exception:** dove si scrive il codice per gestire le eccezioni e gli errori che vengono lanciati.

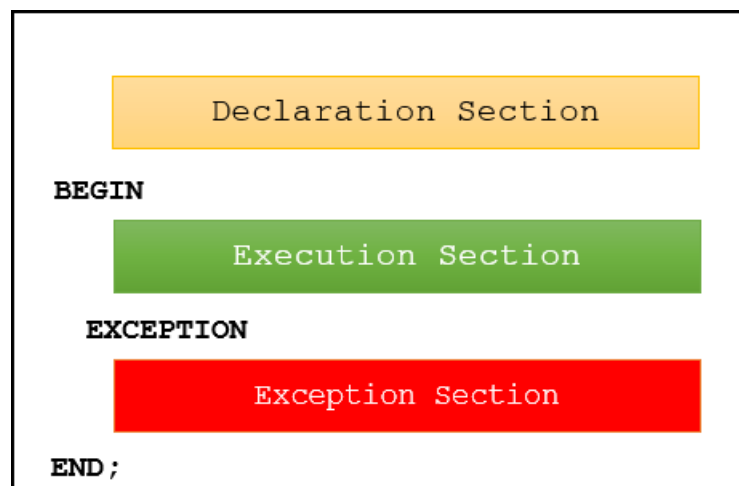


Figura 13.2: Struttura di un blocco PL/SQL

È possibile inoltre **annidare dei blocchi**, ovvero definire un blocco che a sua volta contiene altri blocchi PL/SQL:

```

1 DECLARE
2   istruzioni;
3 BEGIN
4   istruzioni;
5   istruzioni;
6   /* inizio blocco annidato*/
7   DECLARE
8     istruzioni;
9   BEGIN
10    istruzioni;
11    istruzioni;
12  EXCEPTION
13    istruzioni;
14  END;
15  /* fine blocco annidato*/
16 EXCEPTION
17   istruzioni;
18 END;

```

Codice 13.2: Blocchi annidati in PL/SQL

Esempio

Il seguente listato permette di stampare il messaggio `Hello World` all'interno della console.

```
1 DECLARE
2   l_message VARCHAR2(100) := 'Hello World!';
3 BEGIN
4   DBMS_OUTPUT.put_line(l_message);
5 EXCEPTION
6   WHEN OTHERS THEN
7     DBMS_OUTPUT.put_line(SQLERRM);
8 END;
```

Codice 13.3: Hello World.sql

13.2.1 Dichiarazione delle variabili

La sintassi generale per la dichiarazione delle variabili e delle costanti è la seguente:

`<nome_variabile> <tipo_variabile> [:= valore_legale];`

dove la dichiarazione del tipo può essere opzionalmente seguita da una assegnazione di inizializzazione. Una variabile dichiarata **può essere usata solo nel blocco all'interno del quale è stata dichiarata** ed eventualmente **nei sotto-blocchi annidati**.

Esempi di dichiarazioni di variabili sono le seguenti:

```
1 DECLARE
2   data_nascita DATE;
3   nome VARCHAR2(80);
4   stage BOOLEAN;
5   ruolo VARCHAR2(20) := 'Impiegato';
6 BEGIN
7   ...
8   ...
9   ...
10 END;
```



La variabile `data_nascita` è di tipo `DATE` mentre la variabile `nome` è di tipo `VARCHAR2` con una dimensione massima di 80 byte. La variabile `ruolo` è stata inizializzata con un valore di default. È possibile definire delle variabili con tipi di dati complessi, che dipendono dinamicamente dal tipo di dato estratto dalle tabelle. Supponendo di aver definito una tabella nel seguente modo:

```
1 CREATE TABLE PRODOTTO(
2   ID NUMBER,
3   DESCRIZIONE VARCHAR(20),
4   FORNITORE VARCHAR(20),
5   COSTO NUMBER
6 );
```



Quando si desidera salvare all'interno di una variabile i dati recuperati da una query che interroga la tabella si usa il comando **SELECT INTO**:

```
1 DECLARE
2   descrizione_prodotto VARCHAR2(80);
3   fornitore_prodotto VARCHAR2(80);
4   costo_prodotto NUMBER;
5 BEGIN
6   SELECT descrizione
7   INTO descrizione_prodotto /* questa e' una assegnazione */
8   FROM prodotto
9   WHERE id = 1;
10 END;
```

Codice 13.4: Esempio di utilizzo della `SELECT INTO`

Le variabili dichiarate nel blocco `DECLARE` devono avere lo stesso *datatype* delle colonne della tabella, ovvero bisogna assicurarsi che il tipo dato del campo e quello della variabile siano compatibili se non equivalenti e che nel tempo non cambi. In caso contrario il rischio è che il programma possa andare in errore.

```

1 DECLARE
2     descrizione_prodotto PRODOTTO.DESCRIZIONE%TYPE;
3     fornitore_prodotto PRODOTTO.FORNITORE%TYPE;
4     costo_prodotto PRODOTTO.COSTO%TYPE;
5     record_prodotto PRODOTTO%ROWTYPE;
6 BEGIN
7     SELECT descrizione
8     INTO descrizione_prodotto /* questa e' una assegnazione */
9     FROM prodotto
10    WHERE id = 1;
11     SELECT *
12     INTO record_prodotto /* questa e' una assegnazione */
13     FROM prodotto
14    WHERE id = 1;
15 END;

```

Codice 13.6: Dichiarazione di una variabile di tipo ROWTYPE

Il modo più sicuro di fare ciò è quello di utilizzare le cosiddette **dichiarazioni ancorate** che sono dichiarazioni in cui si fa esplicito riferimento al tipo di dato della colonna interessata.

```

1 DECLARE
2     descrizione_prodotto PRODOTTO.DESCRIZIONE%TYPE;
3     fornitore_prodotto PRODOTTO.FORNITORE%TYPE;
4     costo_prodotto PRODOTTO.COSTO%TYPE;
5 BEGIN
6     SELECT descrizione
7     INTO descrizione_prodotto /* questa e' una assegnazione */
8     FROM prodotto
9    WHERE id = 1;
10 END;

```

Codice 13.5: Esempio di dichiarazioni ancorate

È possibile anche dichiarare una variabile in grado di memorizzare tutto il record di una tabella:

13.3

BLOCCO DI UN PROGRAMMA



13.3.1 ■ Assegnazioni ed espressioni

Assegnazioni ed espressioni su valori o variabili dichiarate nel blocco di definizione seguono regole sintattiche analoghe a quelle di un linguaggio di programmazione strutturato. L'istruzione è molto semplice e può essere fatta in due modi.

Il primo metodo consiste con l'operazione di assegnazione diretta, tipicamente usata per assegnare un valore di default, usando l'operatore: `:=`

```

1 DECLARE
2     l_costo_prodotto NUMBER;
3 BEGIN
4     l_variabile := 0; /* assegnazione diretta */
5 END;

```

Codice 13.7: Assegnazione diretta in PL/SQL

Spesso, tuttavia, bisogna assegnare ad una variabile un valore proveniente da una query.

```

1 DECLARE
2     l_costo_prodotto NUMBER;
3 BEGIN
4     l_costo_prodotto := 0; /* assegnazione diretta */
5     SELECT costo
6     INTO l_costo_prodotto /* assegnazione tramite query */
7     FROM prodotto
8    WHERE id = 1;
9 END;

```

Codice 13.8: Assegnazione tramite SELECT INTO

Per fare un'operazione del genere bisogna essere certi, però, che la query restituisca un unico valore, altrimenti l'assegnazione andrà in errore generando una eccezione che causerebbe l'interruzione del programma. Per questo motivo, potrebbe aver senso gestire nel blocco le eccezioni che potrebbero essere generate dal programma quando in una operazione di assegnazione non viene restituito un solo valore.

```
1 DECLARE
2     l_costo_prodotto NUMBER;
3 BEGIN
4     l_costo_prodotto := 0;
5
6     SELECT costo
7     INTO l_costo_prodotto
8     FROM prodotto
9     WHERE id = 1;
10
11 EXCEPTION
12     WHEN no_data_found THEN
13         /* se la query non restituisce righe, vengono eseguite queste istruzioni */
14         l_costo_prodotto := NULL;
15     WHEN too_many_rows THEN
16         /* se la restituisce piu' di una riga, vengono eseguite queste istruzioni */
17         l_costo_prodotto := NULL;
18     WHEN others THEN
19         l_costo_prodotto := NULL;
20 END;
```

Codice 13.9: Gestione delle eccezioni in PL/SQL

13.3.2 ■ Costrutti condizionali

I costrutti condizionali seguono la stessa struttura presente all'intero dei linguaggi di programmazione strutturata:

```
IF condizion1 THEN
    istruzione1;
ELSIF condizione2 THEN
    istruzione2;
ELSE
    istruzione3;
END IF;
```

```
1 IF l_salary BETWEEN 10000 AND 20000 THEN
2     give_bonus(l_employee_id, 1000);
3 ELSIF l_salary > 20000 THEN
4     give_bonus(l_employee_id, 500);
5 ELSE
6     give_bonus(l_employee_id, 0);
7 END IF;
```

Codice 13.10: Esempio di costrutto condizionale in PL/SQL

13.3.3 ■ Costrutti iterativi

All'interno di PL/SQL esistono tre tipologie di costrutti iterativi:

1. **Ciclo for:** termina dopo un predeterminato numero di volte;
2. **Ciclo loop semplice:** viene eseguito fino a quando non si termina esplicitamente il ciclo;
3. **Ciclo while:** termina quando si verifica una determinata condizione.

For loop:

- È eseguito un numero predefinito di volte;
- Vengono specificati upper/lower bounds e la variabile di loop;
- La variabile di loop viene automaticamente incrementata o decrementata;

```
1 FOR loop_variabile IN [REVERSE] lower_bound .. upper_bound LOOP
2     statements
```

```
3 END LOOP;
```



dove `loop_variabile` è la variabile che regola il loop. È possibile utilizzare una variabile che già esiste o se ne utilizza una apposta per il loop. La variabile è incrementata di 1 ogni volta che si passa attraverso il loop. Se non definita al di fuori del loop, la variabile **non è più visibile all'uscita del for**.

La clausola opzionale `REVERSE` indica se la variabile deve essere incrementata o decrementata. In ogni caso, il lower bound deve essere specificato prima dell'upper bound.

```
1 FOR v_counter2 IN 1..5 LOOP
2   DBMS_OUTPUT.PUT_LINE(v_counter2);
3 END LOOP;
4
5 FOR v_counter2 IN REVERSE 1..5 LOOP
6   DBMS_OUTPUT.PUT_LINE(v_counter2);
7 END LOOP;
```

Codice 13.11: Esempio di ciclo for in PL/SQL

Simple loop:

La sintassi per un loop semplice è la seguente:

```
1 LOOP
2   statements
3 END LOOP;
```



Per terminare il ciclo si può usare:

- `EXIT` per terminare il loop immediatamente
- `EXIT WHEN` per terminare il loop quando una determinata condizione occorre.

```
1 v_counter := 0;
2 LOOP
3   v_counter := v_counter + 1;
4   EXIT WHEN v_counter = 5;
5 END LOOP;
6
7 v_counter := 0;
8 LOOP
9   -- after the CONTINUE statement is executed, control returns here
10  v_counter := v_counter + 1;
11  IF v_counter = 3 THEN
12    CONTINUE; -- end current iteration unconditionally
13  END IF;
14  EXIT WHEN v_counter = 5;
15 END LOOP;
16
17 v_counter := 0;
18 LOOP
19   -- after the CONTINUE statement is executed, control returns here
20   v_counter := v_counter + 1;
21   CONTINUE WHEN v_counter = 3; -- end current iteration when v_counter = 3
22  EXIT WHEN v_counter = 5
23 END LOOP;
```

Codice 13.12: Esempio di loop semplice in PL/SQL

Ciclo while

La sintassi del ciclo while è la seguente:

```
1 WHILE condition LOOP
2   statements
3 END LOOP;
```



Nel ciclo while la condizione è controllata all'inizio di ogni iterazione (è possibile quindi che il loop non venga mai eseguito). Si presuppone che nel corpo del loop ci sia una variazione sulla variabile di condizione.

```
1 v_counter := 0;
2 WHILE v_counter < 6 LOOP
3     v_counter := v_counter + 1;
4 END LOOP;
```

Codice 13.13: Esempio di ciclo while in PL/SQL

13.4

BLOCCHI NON ANONIMI: PROCEDURE E FUNZIONI



13.4.1 ■ Creare una procedura in PL/SQL

Una **procedura** PL-SQL è un blocco di codice, identificato da un nome univoco, che viene salvata (in gergo si dice compilata) direttamente nel database Oracle. Una procedura PL-SQL può essere richiamata da altri blocchi PL/SQL, da altre procedure e da altre funzioni utilizzando direttamente il suo nome, rendendola di fatto riutilizzabile al momento del bisogno.

La sintassi per creare una procedura PL-SQL è la seguente:

```
1 CREATE OR REPLACE PROCEDURE <nome_procedura> (<lista di parametri>) IS
2     <dichiarazioni>
3 BEGIN
4     <sequenza di istruzioni>
5 EXCEPTION
6     <gestione eccezione>
7 END <nome procedura>;
```

Codice 13.14: Sintassi per la definizione di una procedura in PL/SQL

Dove:

- CREATE PROCEDURE, obbligatoria, indica che si sta definendo una procedura;
- La clausola OR REPLACE, opzionale, serve per ricreare la procedura nel caso in cui essa sia già stata definita in precedenza;
- <nome_procedura>, obbligatoria, indica il nome con cui la procedura sarà identificata;
- <lista_parametri>, opzionale, consente di inserire una serie di parametri che possono essere passati alla procedura.

Esempio

La procedura che stampa a video gli articoli con un costo superiore ad un determinato valore sarà:

```
1 CREATE OR REPLACE PROCEDURE stampa_articoli_costosi(p_soglia IN NUMBER) IS
2     /* dichiarazione del cursore */
3 CURSOR elenco_prodotto IS
4     SELECT descrizione, fornitore, costo
5     FROM PRODOTTO
6     WHERE costo ≥ p_soglia ;
7 BEGIN
8     FOR rec_elenco_prodotto IN elenco_prodotto LOOP
9         dbms_output.put_line('Prodotto: ' || rec_elenco_prodotto.descrizione);
10    END LOOP;
11 END;
```

Codice 13.15: Esempio di procedura in PL/SQL

I parametri di una procedura vanno dichiarati con questa sintassi:

```
1 <nome_parametro> [IN | OUT | IN OUT] <tipo_dato>
```



dove **tipo_dato** identifica il tipo di parametro che la procedura si aspetta. Le clausole IN, OUT, IN OUT, opzionali, specificano il modo nel quale il parametro è utilizzato. Per default, l'utilizzo di un parametro è in modalità IN.

- IN implica che il parametro può essere utilizzato nella procedura ma non può essere modificato;
- OUT implica che ad esso può essere assegnato un valore nella procedura ma non può essere utilizzato;

- IN OUT implica che il parametro può essere utilizzato sia in lettura che scrittura.

Una procedura può essere richiamata utilizzando il comando **CALL** nel seguente modo:

```
1 CALL nome_procedura(<lista parametri>);
```



oppure all'interno di un altro blocco PL/SQL in questo modo:

```
1 BEGIN
2     nome_procedura(<lista parametri>);
3 END;
```



Una procedura può essere eliminata tramite il comando **DROP** nel seguente modo:

```
1 DROP nome_procedura;
```



13.4.2 ■ Creare una funzione in PL/SQL

Una **funzione** PL-SQL si differenzia da una procedura solo per la presenza di una clausola **RETURN** che specifica il tipo di dato restituito. A differenza di una procedura, infatti, **una funzione restituisce sempre un valore**.

La sintassi della definizione di una funzione è la seguente:

```
1 CREATE OR REPLACE FUNCTION <nome_funzione> (<lista di parametri>) RETURN <tipo dato> IS
2 <dichiarazione>
3 BEGIN
4     <sequenza di istruzioni>
5     RETURN <variabile>
6 EXCEPTION
7     <gestione eccezione>
8 END <nome funzione>;
```

Codice 13.16: Definizione di una funzione in PL/SQL

Esempio

Supponiamo di voler scrivere un programma PL-SQL che conta il numero di articoli che hanno un valore superiore ad un determinato importo e, allo stesso tempo, vogliamo stampare a video questa lista. È possibile utilizzare una funzione PL/SQL definita così:

```
1 CREATE OR REPLACE FUNCTION conta_articoli_costosi(p_soglia IN NUMBER) RETURN NUMBER IS
2     /* dichiarazione del cursore */
3     CURSOR elenco_prodotti IS
4         SELECT descrizione, fornitore, costo
5         FROM PRODOTTO
6         WHERE costo ≥ p_soglia ;
7
8     /* dichiarazione variabili */
9     l_counter NUMBER := 0;
10 BEGIN
11     l_counter := 0;
12     FOR rec_elenco_prodotti IN elenco_prodotti LOOP
13         dbms_output.put_line('Prodotto: ' || rec_elenco_prodotti.descrizione);
14         l_counter := l_counter + 1;
15     END LOOP;
16     /* restituisci il valore al blocco chiamante */
17     RETURN l_counter;
18 END;
```





Spesso una SELECT può restituire molte righe attraverso una interrogazione. In questi casi non è possibile salvare il tutto all'interno di una variabile. Per questo motivo il DBMS alloca un buffer di memoria per contenere il risultato che potrà essere letto mediante un **cursore**. Essi possono essere di sola lettura o, se l'implementazione lo consente, possono essere usati per modificare o cancellare le righe.

13.5.1 Dichiarazione dei cursori

La dichiarazione di un cursore è la seguente:

```
1 CURSOR <cursor_name> IS
2 <SQL_statement>;
```



dove <cursor_name> è l'identificatore del cursore e <SQL_statement> è l'operazione che si intende effettuare sui dati.

Esempio

Esempi di dichiarazione di cursori sono:

```
1 CURSOR Tabella1_Cursor IS
2   SELECT e, f
3   FROM Tabella1
4   WHERE e<f;
5
6 CURSOR Cursor_name IS
7   SELECT *
8   FROM Prodotto
9   WHERE costo ≥ 50.000;
```

Codice 13.17: Dichiarazione di cursori in PL/SQL

13.5.2 Cursori semplici: schemi di iterazione

Dato un cursore, il suo utilizzo è esemplificato dalla seguente sequenza di istruzioni:

```
1 DECLARE
2   ...
3   CURSOR cursor_name IS
4   SELECT *
5   FROM Prodotto
6   WHERE Costo ≥ 50.000;
7   ...
8 BEGIN
9   ...
10  OPEN cursor_name; /* Apertura del cursore, viene calcolata la query
11  e viene allocata la memoria intermedia per le tuple in uscita */
12  counter := 0;
13  LOOP
14    FETCH cursor_name INTO prd; --Viene caricata una tupla in Prd
15    EXIT WHEN cname%NOTFOUND; --Condizione di uscita dal ciclo
16    ProdId := Prd.PID;
17    Cst := Prd.Costo;
18    De := Prd.Descrizione;
19    Ricavo := Cst * 1.25;
20    counter := counter+1;
21  END LOOP;
22  CLOSE cursor_name; --deattivazione del cursore
23  ...
24 END
25 /
```

Codice 13.18: Utilizzo dei cursori in PL/SQL

13.5.3 Cursors vincolati

Un cursore come `cursor_name` del Listato 13.18 può dipendere da alcune costanti (nell'esempio il valore 50000). Per rendere più generale la nozione di un cursore ad esso possono essere associati uno o più parametri rispetto ai quali attivazioni diverse possono operare su valori diversi.

L'astrazione ottenuta è simile quindi a quella di un'astrazione funzionale come la procedura in un ambiente di programmazione strutturata. Tali cursori, detti **parametrici** o **vincolati**, hanno una dichiarazione più complessa, che include il *nome*, il *tipo* ed il numero di parametri. Ad esempio:

```
1  CURSOR Parametric_Cursor (Soglia Prodotto.Costo%TYPE) IS
2  SELECT *
3  FROM Prodotto
4  WHERE Costo >= Soglia;
```

Codice 13.19: Esempio di utilizzo di un cursore parametrico

In questo caso, la condizione WHERE nella query SQL può assumere configurazioni diverse a fronte di diversi valori del parametro `Soglia`. Sono quindi possibili invocazioni diverse (cioè con risultati diversi sulla stessa istanza del database) dello stesso cursore:

```
open Parametric_Cursor(30000);
...
open Parametric_Cursor(50000);
```

13.6

GESTIONE DELLE ECCEZIONI

Quando all'interno del nostro blocco PL/SQL si verifica un errore viene scatenata un'eccezione (Vedi tabella 13.1). Le eccezioni possono essere distinte in due tipi:

- Eccezioni di sistema;
- Eccezioni utente.

Tabella 13.1: Tipologie di eccezioni

Category	Definer	Has Error Code	Has Name	Raised Implicitly	Raised Explicitly
Internally defined	Runtime system	Always	Only if you assign one	Yes	Optionally
Predefined	Runtime system	Always	Always	Yes	Optionally
User-defined	User	Only if you assign one	Always	No	Always

13.6.1 Eccezioni di sistema

Gli errori più generici che si possono verificare durante l'esecuzione di blocchi PL/SQL vengono gestiti dalle **eccezioni di sistema**.

Esse si scatenano automaticamente quando si verificano specifici errori previsti da Oracle. Le eccezioni utente invece devono essere esplicitamente definite e poi attivate dal programmatore.

Vediamo di seguito la sintassi delle eccezioni:

```
1  BEGIN
2  <istruzione>
3  EXCEPTION
4  WHEN <tipo_eccezione_1> THEN
5  <istruzione>
6  WHEN <tipo_eccezione_1> THEN
7  <istruzione>
8  WHEN OTHERS THEN
9  <istruzione>
10 END;
```

Codice 13.20: Sintassi delle eccezioni in PL/SQL

Dato che non sempre è possibile determinare tutte le eccezioni che si possono verificare a seguito dell'esecuzione del nostro codice, il PL/SQL mette a disposizione la clausola **WHEN OTHERS** la quale gestisce, appunto, tutte le eccezioni non esplicitamente trattate dalle clausole che la precedono.

Una clausola WHEN può anche gestire più eccezioni, avremo quindi qualcosa del genere:

```
WHEN tipo_eccezione1 OR tipo_eccezione2 THEN ... ..
```



Tabella 13.2: Eccezioni predefinite

Nome	Descrizione
CURSOR_ALREADY_OPEN	Il programma sta tentando di aprire un cursore già aperto. Un cursore deve essere chiuso prima di poter essere utilizzato.
DUP_VAL_ON_INDEX	Il programma sta tentando di salvare valori duplicati all'interno di una colonna che ha un vincolo di unicità.
INVALID_CURSOR	Il programma sta tentando di eseguire operazioni illegali su un cursore come ad esempio la chiusura di un cursore non aperto.
INVALID_NUMBER	In uno statement SQL, la conversione di una stringa di caratteri in un numero fallisce perché la stringa non rappresenta un numero valido.
NO_DATA_FOUND	Uno statement SELECT INTO restituisce un insieme vuoto di record oppure il programma si riferisce ad un elemento eliminato presente in un blocco innestato o un elemento non inizializzato. Le funzioni aggregate come SUM e AVG restituiscono sempre un valore quindi una funzione aggregata non lancia mai questo tipo di eccezione. Alla chiamata dell'operatore FETCH ci si aspetta al massimo di non leggere nessuna riga, quindi quando succede, non viene lanciata nessuna eccezione nel caso.
TOO_MANY_ROWS	Uno statement SELECT INTO che restituisce più di una riga.
VALUE_ERROR	Si è verificato un errore di tipo aritmetico, o di troncamento o di conversione o di dimensione.
ZERO_DIVIDE	Il programma vuole fare una divisione per zero.

13.6.2 ■ Eccezioni utente

PL/SQL permette di definire delle exception proprie dell'utente. Come per qualsiasi altra dichiarazione, essa deve avvenire nella sezione di dichiarazione. Avremo quindi:

```
1 DECLARE
2 mia_eccezione EXCEPTION;
3 BEGIN
4 <istruzione>
5 END;
```



Le eccezione definite dall'utente non scattano automaticamente ma devono essere scatenate tramite il comando **RAISE** seguito dal nome dell'eccezione, quindi, nel nostro caso avremo:

```
1 RAISE mia_eccezione;
```



Esempio

Esempio di eccezione utente:

```
1 DECLARE
2 mia_eccezione EXCEPTION;
3 BEGIN
4 <istruzione>
5 IF <condizione> THEN
6 RAISE mia_eccezione;
7 END IF;
8 EXCEPTION
9 WHEN <tipo_eccezione_1> THEN <istruzione>;
10 WHEN <tipo_eccezione_2> THEN <istruzione>;
11 WHEN mia_eccezione THEN <istruzione>;
12 WHEN OTHERS THEN <istruzione>;
13 END;
```



Esempio

È possibile associare un'eccezione ad un codice di errore attraverso il “pragma” EXCEPTION_INIT:

```
1 CREATE OR REPLACE PROCEDURE process_balance( balance_in IN NUMBER) --parametro di input
2 IS
3 e_balance_too_low EXCEPTION; --definizione dell'eccezione
4 PRAGMA EXCEPTION_INIT (e_balance_too_low, -20000); --aggancia il codice numerico all'eccezione
5 BEGIN
6 IF balance_in <1000
7 THEN
8 RAISE e_balance_too_low;
9 END IF;
10 END;
```



13.7

SQL DINAMICO



Il **Dynamic SQL** consente di eseguire comandi SQL creati dinamicamente (prodotti a runtime) all'interno del corpo procedurale del linguaggio associato.

La differenza principale con l'*SQL statico* sta nel fatto che, mentre nell'*SQL statico* il programmatore definisce staticamente le istruzioni SQL nel momento in cui definisce le funzioni o le procedure, nell'*SQL dinamico* **il codice SQL viene generato a runtime**. Infatti, usando l'elaborazione basata su stringhe, è possibile “confezionare” un'*istruzione variabile* che viene completata successivamente a runtime.

```
1 CREATE PROCEDURE CANCELLA_TABELLA (table_name IN VARCHAR2)
2 AS
3 sql_istr VARCHAR2(100);
4 BEGIN
5 sql_istr := 'DROP TABLE' || table_name;
6 EXECUTE IMMEDIATE (sql_istr);
7 EXCEPTION <gestione delle eccezioni>
8 END;
9 /
```

Codice 13.21: Esempio di SQL dinamico in PL/SQL

Come si può vedere, nel momento in cui si scrive il codice non si conosce ancora il nome della tabella da eliminare. Sarà poi possibile utilizzare la procedura in altri blocchi:

```
1 BEGIN
2 CANCELLA_TABELLA('T');
3 CANCELLA_TABELLA('T_2');
4 END;
5 /
```

Codice 13.22: Utilizzo di una funzione dinamica

13.7.1 ■ Modalità di interazione

Tutte le istruzioni SQL sono dati di tipo stringa che vengono prima salvate in variabili di tipo VARCHAR, preparate e poi eseguite. Per segnalare al DBMS che quella variabile è un comando vengono messe a disposizione due **modalità di interazione**:

1. **Modalità standard:** la gestione dell'interrogazione avviene in due fasi:
 - **PREPARE:** vi è una fase di preparazione del comando;
 - **EXECUTE:** in cui il comando è mandato in esecuzione.
2. **Esecuzione immediata:** l'interrogazione è eseguita immediatamente mediante una **EXECUTE IMMEDIATE**:
 - mediante un parametro di tipo stringa che contiene il comando;
 - mediante un comando specificato direttamente come parametro.

13.7.2 ■ Prepare ed Execute

Il comando **PREPARE** analizza l'istruzione SQL e ne prepara una traduzione

```
1 PREPARE <nome_comando> FROM <comando_SQL>
```



dove <nome_comando> è il nome associato da **PREPARE** alla traduzione del comando SQL. Il comando SQL può contenere dei parametri in ingresso rappresentati da ?. Ad esempio:

```
1 PREPARE comando_sql
2 FROM 'SELECT nome FROM studente WHERE matricola=?'
```

Codice 13.23: Esempio di utilizzo del comando **PREPARE**

L'esecuzione del comando **PREPARE** associa alla variabile `comando_SQL` la traduzione dell'interrogazione con un parametro in ingresso che rappresenta la matricola dello studente. L'esecuzione della query avviene infine tramite il comando **EXECUTE**:

```
1 EXECUTE <nome_comando> [INTO <variabiliTarget>]
2 [USING <lista_parametri>]
```



In questo caso <variabiliTarget> contiene l'elenco dei parametri in cui deve essere scritto il risultato del comando mentre <lista_parametri> specifica i valori che devono essere assunti dai parametri variabili.

13.7.3 ■ Execute Immediate

Nella seconda modalità la **PREPARE** non è eseguita esplicitamente con un comando specifico, bensì viene effettuata contestualmente all'esecuzione. Il comando che si utilizza è:

```
1 EXECUTE IMMEDIATE <nome comando> [INTO <listaTarget>][USING <lista_parametri>]
```



Esempio

Esempio di SQL dinamico in PL/SQL:

```
1 CREATE OR REPLACE FUNCTION single_number_value (table_in IN VARCHAR2, column_in IN VARCHAR2,
2   where_in IN VARCHAR2) RETURN NUMBER
3 IS
4   l_return NUMBER --variabile di output
5   Comando VARCHAR(100);
6 BEGIN
7   Comando := 'SELECT ' || column_in || ' FROM ' || table_in;
8   Comando := Comando || ' WHERE ' || where_in; --Mettere spazi per formattare correttamente il comando
9   EXECUTE IMMEDIATE Comando INTO l_return;
10  RETURN l_return;
11 END;
```



Tabella 13.3: Funzioni per la manipolazione delle stringhe in PL/SQL

Funzione	Esempio	Risultato	Descrizione
ASCII	ASCII('A')	65	Restituisce il codice ASCII di un carattere.
CHR	CHR('65')	'A'	Converte un valore numerico al suo carattere corrispondente.
CONCAT	CONCAT('A','BC')	'ABC'	Concatena due stringhe.
INITCAP	INITCAP('hi there')	'Hi There'	Trasforma il primo carattere di ogni parola in maiuscolo.
INSTR	INSTR('This is a playlist','is')	3	Cerca una sottostringa e restituisce la sua posizione.
LENGTH	LENGTH('ABC')	3	Restituisce il numero di caratteri o la lunghezza di una stringa.
LOWER	LOWER('Abc')	'abc'	Restituisce una stringa con tutti i caratteri minuscoli.
REPLACE	REPLACE('JACK AND JOND','J','BL')	'BLACK AND BLOND'	Sostituisce tutte le occorrenze di una sottostringa.
SUBSTR	SUBSTR('Oracle Substring',1,6)	'Oracle'	Estrae una sottostringa.

Esempio

Si scriva una procedura che riceve in input il nome di una tabella, una stringa di nomi di colonne separati da ';' ed una stringa contenente un nome. Si vuole creare un vincolo UNIQUE, con il nome dato in input, sulle colonne e sulla tabella date.

```

1  CREATE PROCEDURE crea_unique_constr (tab_name IN VARCHAR(50), colonne IN VARCHAR(1000), constr_name
    IN VARCHAR(100))
2
3  DEFINE
4  comando VARCHAR(1000)
5  tabella_non_esiste EXCEPTION
6  BEGIN
7  IF check_nome_tabella(tab_name)= 0 --controllo che la tabella esista
8  RAISE tabella non esiste
9  ENDIF;
10 comando := 'ALTER TABLE ' || tab_name || 'ADD CONSTRAINT ';
11 comando := comando || constr_name || 'UNIQUE';
12 num INTEGER := 1;
13 colonna_non_in_tab EXCEPTION;
14 uniq = '(';
15 LOOP
16 new_pos = INSTR (colonne, ';', num);
17 EXIT WHEN new_pos=0;
18
19 colname = SUBSTR(colonne, num, new_pos-1);
20 IF check_column_exists (nome_tab, colname) = 0
21 RAISE colonna_non_in_tab;
22 num = new_pos +1;
23 uniq = uniq || colname || ',';
24 END LOOP
25 -- rimuovo l'ultima virgola
26 uniq = RTRIM (uniq,',');
27 uniq = uniq || ')';
28 comando = comando || uniq;
29 EXECUTE IMMEDIATE comando;
30 EXCEPTION
31 WHEN colonna_non_in_tab THEN
32 DBMS_OUTPUT.PUTLINE('Nome colonna errato o non esistente');
33 WHEN tabella_non_esiste THEN
34 -- gestione errore
35 WHEN parametri_vuoti
36 -- bisogna controllare che i parametri siano valorizzati
37 WHEN OTHERS
38 -- generica
39 END;
40 /

```

13.7.4 ■ Cursori per l'SQL dinamico

Un cursore come quelli visti nella Sezione 13.5 possono anche non essere legati obbligatoriamente ad una query particolare. Ciò significa che è possibile usare dinamicamente un cursore tenendolo aperto per una qualsiasi query. Il vantaggio principale di questo tipo di cursori sta nella possibilità di poterli utilizzare per query costruite dinamicamente.

Per poter usare i cursori all'interno dell'SQL dinamico è necessario dichiarare un **puntatore ad un cursore** secondo la sintassi seguente:

```
1 <nome cursore> SYS_REFCURSOR
```



Questi si distinguono in due tipologie:

- **Fortemente tipizzati:** quando si conosce la struttura della tabella su cui si lavora e dei campi restituiti

```
<nome_rif_cur> SYS_REFCURSOR RETURN <tipo>
```

- **Debolmente tipizzati:** altrimenti

```
<nome_rif_cur> SYS_REFCURSOR
```

Per associare un riferimento alla select bisogna fare una OPEN esplicita:

```
1 comando := 'SELECT * FROM ? WHERE' || nomeattr || '=' || nomeval;  
2 cur_for_comando SYS_REF.CURSOR  
3 OPEN cur_for_comando FOR comando
```



È possibile inoltre restituire una variabile di tipo cursore come output di una funzione:

```
1 CREATE OR REPLACE FUNCTION get_direct_reports( in_manager_id IN employees.manager_id%TYPE)  
2 RETURN SYS_REFCURSOR  
3 AS c_direct_reports SYS_REFCURSOR;  
4 BEGIN  
5     OPEN c_direct_reports FOR  
6     SELECT employee_id, first_name, last_name, email  
7     FROM employees  
8     WHERE manager_id = in_manager_id  
9     ORDER BY first_name, last_name;  
10    RETURN c_direct_reports;  
11 END;  
12  
13 DECLARE  
14     c_direct_reports SYS_REFCURSOR;  
15     l_employee_id employees.employee_id%TYPE;  
16     l_first_name employees.first_name%TYPE;  
17     l_last_name employees.last_name%TYPE;  
18     l_email employee.email%TYPE;  
19 BEGIN  
20     --get the ref cursor from function  
21     c_direct_reports := get_direct_reports(46);  
22     --process each employee  
23     LOOP  
24         FETCH  
25         c_direct_reports  
26         INTO  
27         l_employee_id,  
28         l_first_name,  
29         l_last_name,  
30         l_email;  
31         EXIT  
32         WHEN c_direct_reports%NOTFOUND;  
33         DBMS_OUTPUT.put_line(l_first_name || ' ' || l_last_name);  
34     END LOOP;  
35     -- close the cursor  
36     CLOSE c_direct_reports;  
37 END;  
38 /
```





Ogni DBMS conserva un **dizionario dei dati** che contiene tutti i metadati dei vari database:

- le definizioni di tutti gli oggetti di schema di un database, inclusi i valori di default e i vincoli di integrità;
- le informazioni sullo spazio usato e allocato per ogni schema;
- il nome degli utenti, i loro ruoli e privilegi.

Tutte queste informazioni sono conservate in tabelle chiamate **tabelle di sistema**. In realtà solo il DBMS lavora direttamente sulle tabelle, gli utenti hanno accesso alle viste sui metadati.

Oracle fornisce diverse viste integrate che vengono utilizzate per monitorare il database e interrogare il suo stato operativo. Queste viste possono essere utilizzate per monitorare lo stato del database, visualizzare informazioni sugli oggetti dello schema del database e altro ancora. Ogni vista ha un prefisso che indica per quale utente è disponibile:

- **DBA_***: solo per l'admin della base di dati;
- **ALL_***: tutti gli oggetti su cui l'utente è abilitato;
- **USER_***: tutti gli oggetti di cui l'utente è proprietario.

Esempi di viste del dizionario dei dati sono:

- **DBA_TABLES**: informazioni su tutte le tabelle del database corrente.
- **DBA_USERS**: informazioni su tutti gli utenti del database.
- **DBA_DATA_FILES**: informazioni su tutti i file di dati fisici del database.
- **DBA_TABLESPACES**: informazioni su tutti i tablespace del database.
- **DBA_TABLES**: informazioni su tutte le tabelle del database.
- **DBA_TAB_COLS**: informazioni su tutte le colonne, per tutte le tabelle, del database.

Due tabelle importanti all'interno di Oracle sono **ALL_ALL_TABLES** e **ALL_TABLES_COLUMNS** mostrate in 13.4.

ALL_ALL_TABLES	owner	table_name	ALL_TABLES_COLUMNS	owner	table_name	column_name	data_type
-----------------------	-------	------------	---------------------------	-------	------------	-------------	-----------

Tabella 13.4

Le **viste dinamiche delle performance**, anche dette **V\$ Views** sono un insieme di viste che forniscono informazioni di monitoraggio in tempo reale sullo stato attuale della configurazione dell'istanza del database, sulle statistiche di runtime e sulle operazioni. Queste viste vengono aggiornate continuamente mentre il database è in esecuzione. Le informazioni fornite dalle viste dinamiche sulle prestazioni includono informazioni sulle sessioni, l'utilizzo della memoria, l'avanzamento dei lavori e delle attività, lo stato e le statistiche dell'esecuzione SQL e varie altre metriche.

```

1 CREATE FUNCTION tabella_esiste (nome_tab USER_ALL_TABLES.TABLENAME%TYPE)
2 RETURN INTEGER
3 IS
4 DECLARE
5     res INTEGER :=0;
6 BEGIN
7     SELECT count(*) INTO res
8     FROM USER_ALL_TABLES T
9     WHERE TABLENAME = UPPER(nome_tab); --I nomi dei campi nei metadati sono salvati in maiuscolo
10    RETURN res;
11 END;
```

Codice 13.24: Uso delle tabelle di sistema in PL/SQL

INTRODUZIONE A POSTGRESQL

14.1

I TIPI DI DATO IN POSTGRESQL



PostgreSQL supporta i seguenti tipi di dati:

- Tipi booleani.
- Caratteri come **char**, **varchar** e **text**.
- Tipi numerici come numeri interi e a virgola mobile.
- Tipi temporali, come **date**, **time**, **timestamp** e **interval**.
- **UUID** per la memorizzazione di identificatori universali univoci.
- Array per conservare array di stringhe, di numeri ecc..
- **JSON** per conserare dati JSON
- **hstore** per memorizzare coppie chiave-valore
- Tipi speciali come l'indirizzo di rete e i dati geometrici

14.1.1 ■ Tipi booleani

Un tipo di dati booleano può contenere uno dei tre valori possibili: **true**, **false** o **null**. Si usa la parola chiave **boolean** o **bool** per dichiarare una colonna con il tipo di dati booleano.

Quando si inseriscono dei dati all'interno di una colonna di tipo booleano, PostgreSQL li converte in un valore booleano:

- I valori **1**, **yes**, **y**, **t**, **true** vengono convertiti in **true**;
- I valori **0**, **no**, **false**, **f** vengono convertiti in **false**.

14.1.2 ■ Caratteri

PostgreSQL fornisce tre tipi di dati per i caratteri: **CHAR(n)**, **VARCHAR(n)** e **TEXT**.

- **CHAR(n)** rappresenta una sequenza di caratteri a lunghezza fissa con gli spazi. Se si inserisce una stringa più corta della lunghezza della colonna, PostgreSQL inserisce degli spazi. Se si inserisce una stringa più lunga della lunghezza della colonna, PostgreSQL emette un errore.
- **VARCHAR(n)** rappresenta una sequenza di caratteri a lunghezza variabile. Con **VARCHAR(n)** si possono salvare *n* caratteri. PostgreSQL non inserisce spazi quando la stringa memorizzata è più corta della lunghezza della colonna.
- **TEXT** è una stringa di caratteri di lunghezza variabile. In teoria, i dati di tipo **text** sono stringhe di caratteri di lunghezza illimitata.

14.1.3 Numerici

PostgreSQL fornisce due tipi distinti di valori numerici:

- interi:
 - SMALLINT
 - INT
 - SERIAL
- numeri a virgola mobile:
 - FLOAT(n)
 - REAL
 - NUMERIC o NUMERIC(p,s)

14.1.4 Tipi di dati temporali

I tipi di dati temporali consentono di memorizzare dati relativi a date e/o orari. PostgreSQL dispone di cinque tipi di dati temporali principali:

- DATE memorizza solo le date.
- TIME memorizza i valori dell'ora del giorno.
- TIMESTAMP memorizza i valori di data e ora.
- TIMESTAMPTZ è un tipo di dati `timestamp` che tiene conto del fuso orario. È l'abbreviazione di `timestamp` con il fuso orario.
- INTERVAL memorizza intervalli di tempo.

Nome	Descrizione	Min	Max	Sensibilità
<code>timestamp [(p)] [senza fuso orario]</code>	Data, orario senza fuso orario	4713 AC	294276 DC	1 microsecondo
<code>timestamp [(p)] [con fuso orario]</code>	Data, orario e fuso orario	4713 AC	294276 DC	1 microsecondo
<code>date</code>	Data (senza orario)	4713 AC	5874897 DC	1 giorno
<code>time [(p)] [senza fuso orario]</code>	Orario del giorno (senza data)	00:00:00	24:00:00	1 microsecondo
<code>time [(p)] [con fuso orario]</code>	Orario del giorno (senza data) con fuso orario	00:00:00+1559	24:00:00-1559	1 microsecondo
<code>interval [campi] [(p)]</code>	Intervallo di tempo	-178000000 anni	178000000 anni	1 microsecondo

Tabella 14.1: Tipi di dato temporale in PostgreSQL

I tipi `time`, `timestamp`, `interval` accettano un valore opzionale, `p`, che specifica il numero di cifre da inserire nel campo dei secondi. L'intervallo permesso per `p` è da 1 a 6.

Il tipo `interval` ha un parametro opzionale il quale restringe il campo dei valori salvati:

YEAR	MONTH
DAY	HOURL
MINUTE	SECOND
YEAR TO MONTH	DAY TO HOUR
DAY TO MINUTE	DAY TO SECOND
HOUR TO MINUTE	HOUR TO SECOND
MINUTE TO SECOND	

Inserire valori temporali

La data e l'ora sono accettate in quasi tutti i formati ragionevoli, tra cui ISO 8601, SQL-compatibile, Postgres tradizionale e altri. Ogni input letterale riguardante date od orari deve essere racchiuso tra apici singoli, come per le stringhe di testo.

```
1 type [(p)] 'value'
```

La tabella 14.2 mostra alcuni input possibili per il tipo `date`.

Tabella 14.2: Date input

Esempio	Descrizione
1999-01-08	ISO 8601; 8 gennaio in qualsiasi modalità (formato consigliato)
January 8, 1999	Inequivocabile in qualsiasi modalità di inserimento del tipo di dato
1/8/1999	8 gennaio nella modalità MDY; 1 agosto nella modalità DMY
1/18/1999	18 gennaio nella modalità MDY; rifiutato altrimenti
01/02/03	2 gennaio 2003 in modalità MDY; 1 febbraio 2003 in modalità DMY; 3 febbraio 2001 in modalità YMD

14.2

AMMINISTRARE UN DATABASE POSTGRESQL

14.2.1 Comandi principali psql

Il seguente comando permette di collegarsi ad un database attraverso un utente specifico. Dopo aver premuto `ENTER` PostgreSQL chiede di inserire la password:

```
psql -d database -U user -W
```

Cambiare la connessione al database

Una volta connessi a un database, è possibile cambiare la connessione a un nuovo database con un utente specificato da `user`. La connessione precedente verrà chiusa. Se si omette il parametro utente, viene assunto l'utente corrente.

```
\c dbname username
```

Mostrare i database disponibili

Per elencare tutti i database del server di database PostgreSQL corrente, si usa il comando:

```
\l
```

Mostrare le tabelle disponibili

Per elencare tutte le tabelle del database corrente, si usa il comando:

```
\dt
```

Descrivere una tabella

Per descrivere una tabella, ad esempio una colonna, il tipo, i modificatori delle colonne e così via, si utilizza il seguente comando:

```
\d table_name
```

Elencare tutti gli schemi disponibili

Per elencare tutti gli schemi del database attualmente collegato, si usa il comando:

```
\dn
```



Elencare tutte le funzioni disponibili

Per elencare le funzioni disponibili nel database corrente, si usa il comando:

```
\df
```



Elencare tutte le viste disponibili

Per elencare le viste disponibili nel database corrente, si usa il comando:

```
\dv
```



Elencare gli utente e i loro ruoli

Per elencare tutti gli utenti e i ruoli loro assegnati, si usa il comando:

```
\du
```



Eseguire il comando precedente

Per recuperare la versione corrente del server PostgreSQL, si utilizza la funzione `version()` come segue:

```
SELECT version();
```



Se si vuole risparmiare tempo digitando nuovamente il comando precedente, si può usare il seguente comando per eseguire il comando precedente:

```
\g
```



Visualizzare la cronologia dei comandi

Per visualizzare la cronologia dei comandi, si utilizza il comando:

```
\s
```



Se si desidera salvare la cronologia dei comandi in un file, è necessario specificare il nome del file seguendo il comando:

```
\s filename
```



Eseguire un comando psql da un file

Se si desidera eseguire i comandi psql da un file, si utilizza il comando:

```
\i
```



Per conoscere tutti i comandi psql disponibili, si usa il comando:

```
\?
```



14.2.2 Tabelle di sistema

PostgreSQL fornisce tre diverse serie di tabelle di meta-dati che vengono utilizzate per recuperare informazioni sullo stato del database e sulle attività correnti:

- 1. **Statistic collection views:** Sottosistema che raccoglie informazioni dinamiche di runtime su determinate attività del server, come ad esempio informazioni statistiche sulle prestazioni. Alcune di queste tabelle possono essere paragonate alle viste V\$ di Oracle.
- 2. **System catalog tables:** Metadati statici relativi al database PostgreSQL e informazioni statiche sugli oggetti dello schema. Alcune di queste tabelle possono essere paragonate alle tabelle Data Dictionary di Oracle DBA_*.
- 3. **Information schema tables:** Insieme di viste che contengono informazioni sugli oggetti definiti nel database corrente. Lo schema delle informazioni è specificato dallo standard SQL e come tale è supportato da PostgreSQL. Alcune di queste tabelle possono essere paragonate alle tabelle USER_* Data Dictionary di Oracle.

Tabelle del catalogo del sistema

Si tratta di un insieme di tabelle utilizzate per memorizzare i metadati dinamici e statici del database PostgreSQL e possono essere considerate come il dizionario dei dati del database. Queste tabelle sono utilizzate per attività interne di tipo contabile.

Tutte le tabelle del catalogo di sistema iniziano con il prefisso pg_* e si trovano nello schema pg_catalog. Sia le tabelle del catalogo di sistema che le viste del raccoglitore di statistiche si trovano nello schema pg_catalog.

Per visualizzare tutte le tabelle presenti nello schema pg_catalog si userà la seguente interrogazione:

```
1 select * from pg_catalog.pg_tables pt
2 where schemaname = 'pg_catalog'
3 order by tablename asc
```

Codice 14.1: Tabelle di sistema in PostgreSQL

La tabella seguente include alcune delle tabelle comuni del catalogo di sistema.

Nome della tabella	Scopo
pg_database	Contiene informazioni e proprietà su ciascun database del cluster PostgreSQL, come le impostazioni di codifica del database e altre.
pg_tables	Informazioni su tutte le tabelle del database, come gli indici e il tablespace per ogni tabella del database.
pg_index	Contiene informazioni su tutti gli indici del database.
pg_cursors	Elenco dei cursori attualmente disponibili/aperti.

Descrivere una tabella usando information_schema

Il catalogo *information_schema.columns* contiene le informazioni sulle colonne di tutte le tabelle. Per ottenere informazioni sulle colonne di una tabella, si interroga il catalogo *information_schema.columns*. Ad esempio:

```
1 SELECT
2 table_name,
3 column_name,
4 data_type
5 FROM
6 information_schema.columns
7 WHERE
8 table_name = 'order_details';
```

Codice 14.2: Descrizione delle colonne di una tabella

Output:

table_name	column_name	data_type
order_details	orderid	integer
order_details	productid	integer
order_details	quantity	integer



14.3.1 Come confrontare due tabelle

Esistono diversi modi per confrontare il contenuto di due tabelle e trovare le differenze tra di esse.

Confrontare due tabelle usando EXCEPT e UNION

Si creino due tabelle FOO e BAR e si inseriscano alcuni dati:

```
1 CREATE TABLE foo (  
2 ID INT PRIMARY KEY,  
3 NAME VARCHAR (50)  
4 );  
5  
6 INSERT INTO foo (ID, NAME)  
7 VALUES  
8 (1, 'a'),  
9 (2, 'b');  
10  
11 CREATE TABLE bar (  
12 ID INT PRIMARY KEY,  
13 NAME VARCHAR (50)  
14 );  
15  
16 INSERT INTO bar (ID, NAME)  
17 VALUES  
18 (1, 'a'),  
19 (2, 'b');
```

Codice 14.3: Confrontare due tabelle in PostgreSQL

Successivamente, si aggiorni una riga nella tabella BAR:

```
1 UPDATE bar  
2 SET name = 'c'  
3 WHERE  
4 id = 2;
```



Quindi, per trovare le righe nella tabella FOO ma non nella tabella bar, si utilizza la seguente query:

```
1 SELECT  
2 ID,  
3 NAME,  
4 'not in bar' AS note  
5 FROM  
6 foo  
7 EXCEPT  
8 SELECT  
9 ID,  
10 NAME,  
11 'not in bar' AS note  
12 FROM  
13 bar
```



Dove si è usato l'operatore EXCEPT che restituisce le righe della tabella FOO ma non quelle della tabella BAR. Possiamo applicare la stessa tecnica per trovare le righe presenti nella tabella BAR ma non nella tabella FOO:

```
1 SELECT  
2 ID,  
3 NAME,  
4 'not in foo' AS note  
5 FROM  
6 bar  
7 EXCEPT  
8 SELECT  
9 ID,
```

```

10 NAME,
11 'not in foo' AS note
12 FROM
13 foo

```



Infine, si usa l'operatore UNION per combinare i risultati delle query precedenti per ottenere:

- Le righe nella tabella BAR ma non nella tabella FOO;
- Le righe nella tabella FOO ma non nella tabella BAR.

Confrontare due tabelle usando OUTER JOIN

È possibile anche usare una join esterna per confrontare le due tabelle:

```

1 SELECT
2 id,
3 name
4 FROM
5 foo
6 FULL OUTER JOIN bar USING (id, name)
7 WHERE
8 foo.id IS NULL
9 OR bar.id IS NULL;

```

Codice 14.4: Confrontare due tabella con join esterno

Per trovare il numero di righe presenti nella tabella FOO ma non nella tabella BAR e viceversa, si utilizza la funzione COUNT come segue:

```

1 SELECT
2 COUNT (*)
3 FROM
4 foo
5 FULL OUTER JOIN bar USING (id, name)
6 WHERE
7 foo.id IS NULL
8 OR bar.id IS NULL;

```



14.3.2 Come generare un numero randomico

PostgreSQL fornisce la funzione **random()** che restituisce un numero random tra 0 e 1. La seguente istruzione restituisce un valore random tra 0 e 1:

```

1 SELECT random();

```



Output:

```

random
-----
0.867320362944156
(1 row)

```

Per generare un numero casuale compreso tra 1 e 11, si utilizza la seguente istruzione:

```

1 SELECT random() * 10 + 1 AS RAND_1_11;

```



Output:

```

rand_1_11
-----
9.494508291309586
(1 row)

```


Se si desidera generare il numero casuale come numero intero, si applica la funzione `floor()` all'espressione come segue:

```
1 SELECT floor(random() * 10 + 1)::int;
```



Output:

```
floor
-----
9
(1 row)
```

In generale, per generare un numero casuale tra due numeri interi `l` e `h`, si utilizza la seguente istruzione:

```
1 SELECT floor(random() * (h-l+1) + l)::int;
```



È possibile sviluppare una funzione definita dall'utente che restituisca un numero casuale compreso tra due numeri `l` e `h`:

```
1 CREATE OR REPLACE FUNCTION random_between(low INT ,high INT)
2 RETURNS INT AS
3
4 BEGIN
5 RETURN floor(random()* (high-low + 1) + low);
6 END;
7 language 'plpgsql' STRICT;
```



14.4

FUNZIONI IN POSTGRESQL



14.4.1 Funzioni standard

Le **funzioni aggregate** eseguono un calcolo su un insieme di righe e restituiscono una singola riga. PostgreSQL fornisce tutte le funzioni aggregate standard di SQL come segue:

- `AVG()`: restituisce il valore medio;
- `COUNT()`: restituisce il numero di valori;
- `MAX()`: restituisce il valore massimo;
- `MIN()`: restituisce il valore minimo;
- `SUM()`: restituisce la somma di tutti i valori o quelli distinti.

Spesso si utilizzano le funzioni aggregate con la clausola `GROUP BY` nell'istruzione `SELECT`. In questi casi, la clausola `GROUP BY` divide l'insieme dei risultati in gruppi di righe e le funzioni aggregate eseguono un calcolo su ciascun gruppo, ad esempio il massimo, il minimo, la media, ecc.

È possibile utilizzare le funzioni aggregate come espressioni solo nelle seguenti clausole:

- `SELECT`;
- `HAVING`.

La funzione `ARRAY_AGG`

`array_agg` è una funzione aggregata di PostgreSQL che accetta un insieme di valori e restituisce un array all'interno del quale ogni elemento dell'insieme è rappresentato da un elementodell'array. La sintassi della funzione è la seguente:

```
1 ARRAY_AGG(expression [ORDER BY [sort_expression {ASC | DESC}], [ ... ]])
```



La clausola `ORDER BY` è una clausola opzionale. Specifica l'ordine delle righe che vengono elaborate nell'insieme che stabilisce l'ordine degli elementi nell'array dei risultati. La funzione `array_agg` viene spesso utilizzata con la clausola `GROUP BY`.

La funzione **STRING_AGG**

string_agg è una funzione aggregata di PostgreSQL che concatena un elenco di stringhe e inserisce un separatore tra di esse. La funzione non aggiunge il separatore alla fine della stringa. La sua sintassi è la seguente:

```
1 STRING_AGG ( expression, separator [order_by_clause] )
```

14.4.2 ■ Funzioni per la gestione delle stringhe

Tabella 14.3:

Funzione	Descrizione	Esempio	Risultato
ASCII	Restituisce il codice ASCII di un carattere, di un codice Unicode o di un carattere UTF-8	ASCII('A')	65
CHR	Converte un codice ASCII in un carattere o un codice Unicode in un carattere UTF-8	CHR(65)	'A'
CONCAT	Concatena due o più stringhe	CONCAT('A', 'B', 'C')	'ABC'
CONCAT_WS	Concatena stringhe con un separatore	CONCAT_WS(',', 'A', 'B', 'C')	'A,B,C'
FORMAT	Formatta gli argomento in base ad una stringa di formato	FORMAT('Hello %s', 'PostgreSQL')	'Hello PostgreSQL'
INITCAP	Convertire le parole di una stringa in maiuscole e minuscole	INITCAP ('hI tHERE')	'Hi There'
LEFT	Restituisce i primi n caratteri di una stringa	LEFT ('ABC', 1)	'A'
LENGTH	Restituisce il numero di caratteri in una stringa	LENGTH('hI tHERE')	8
LOWER	Converte una stringa in minuscolo	LOWER('ABC')	'abc'
LPAD	Riempie a sinistra una stringa con un carattere di una certa lunghezza	LPAD('123', 5, '00')	'00123'
LTRIM	Rimuove la stringa più lunga che contiene i caratteri specificati dalla sinistra della stringa di input.	LTRIM('00123')	'123'
POSITION	Restituisce la posizione di una sottostringa in una stringa	POSITION('B' in 'ABC')	3
REPLACE	Sostituisce tutte le occorrenze di una sottostringa	REPLACE('ABC', 'B', 'A')	'AAC'
STRPOS	Restituisce il primo indice iniziale di una sottostringa o zero se non è presente	STRPOS('high', 'ig')	2
STRING_TO_ARRAY	Divide la stringa alle occorrenze del delimitatore e forma i campi risultanti in un array di testo.	string_to_array('xx&yy&zz', '&')	{xx, yy, zz}
SPLIT_PART	Suddivide una stringa sulla base di un delimitatore specifico e restituisce la ennesima sottostringa	SPLIT_PART ('A, B, C', ',', 2)	B
SUBSTRING	Estrae una sottostringa da una stringa	SUBSTRING('ABC', 1, 1)	'A'
SUBSTR	Estrae la sottostringa da una stringa a partire dal un carattere specificato e per un determinato numero di caratteri.	substr('alfabeto',3)	fabeto
UPPER	Trasforma una stringa in maiuscolo	UPPER('Hi There')	HI THERE

14.4.3 ■ Funzioni per la gestione delle date

Tabella 14.4:

Funzione	Tipo di ritorno	Descrizione	Esempio	Risultato
AGE	INTERVAL	Calcola l'età tra due timestamp e restituisce un risultato “simbolico” che utilizza gli anni e i mesi.	age(timestamp, '2001-04-10', timestamp '1957-06-13')	43 years 9 mons 27 days
CURRENT_DATE	DATE	Restituisce la data corrente	current_date	22 gennaio 2024
DATE_PART	double precision	Restituisce un campo di un timestamp o di un intervallo, ad esempio anno, mese, giorno, ecc.	date_part('hour', timestamp '2001-02-16 20:38:40')	20
DATE_TRUNC	timestamp	Restituisce un timestamp troncato ad una precisione specifica.	DATE_TRUNC('hour', TIMESTAMP '2017-03-17 02:09:30')	2017-03-17 02:00:00
EXTRACT	double precision	Funziona in maniera analoga della funzione DATE_TRUNC	EXTRACT(YEAR FROM TIMESTAMP '2016-12-31 13:30:15')	2016
ISFINITE	boolean	Controlla se una data, un timestamp o un intervallo è finito o meno.		
LOCALTIME	time	Restituisce l'orario in cui inizia la transazione corrente.	LOCALTIME	09:52:20.751278
NOW	timestampz	Restituisce la data e l'orario con la zona oraria.	now()	2023-02-15 14:04:344453-05
TO_DATE	date	Trasforma una stringa in una data	TO_DATE('20170103', 'YYYYMMDD')	2017-01-03
TO_TIMESTAMP	timestampz	Converte una stringa ad un timestamp	TO_TIMESTAMP('2017-03-31 9:30:20', 'YYYY-MM-DD HH:MI:SS')	2017-03-31 09:30:20-07

LINGUAGGI PROCEDURALI PER SQL: POSTGRES PL/PGSQL

15.1

INTRODUZIONE



PL/pgSQL è un linguaggio per il sistema di database PostgreSQL che, aggiungendo alcuni elementi procedurali come le strutture di controllo, cicli e calcoli complessi, estende il linguaggio SQL. Permette di implementare funzioni complesse e *stored procedures* che non è possibile definire con SQL. È molto simile al linguaggio PL/SQL visto nel capitolo 13. Si veda la sezione 15.11 per osservarne le differenze.

Come anticipato nel Capitolo 13, SQL è il linguaggio che PostgreSQL e la maggior parte degli altri database relazionali utilizzano come linguaggio di interrogazione. È portatile e facile da imparare. Ma *ogni istruzione SQL deve essere eseguita singolarmente* dal server del database. Ciò significa che l'applicazione client deve inviare ogni query al server di database, attendere che venga elaborata, ricevere ed elaborare i risultati, eseguire alcuni calcoli e quindi inviare altre query al server. Tutto questo comporta una comunicazione tra i processi e un sovraccarico di rete se il client si trova su una macchina diversa dal server di database.

Con PL/pgSQL è possibile raggruppare un blocco di calcolo e una serie di query all'interno di un'unica funzione e salvarla all'interno del server del database, avendo così la potenza di un linguaggio procedurale e la facilità d'uso di SQL, ma con un notevole risparmio di *overhead* nella comunicazione client/server.

15.1.1 Il dollar quoting

Le funzioni scritte in PL/pgSQL vengono definite al server eseguendo il comando **CREATE FUNCTION**. Un comando di questo tipo ha normalmente la seguente sintassi:

```
1 CREATE FUNCTION somefunc(integer, text) RETURNS integer
2 AS 'function body text'
3 LANGUAGE plpgsql;
```

Codice 15.1: Dichiarazione di una funzione in PL/pgSQL

Per come è concepita una **CREATE FUNCTION**, il corpo della funzione è semplicemente un *letterale di stringa*. Spesso è utile usare il “**dollar quoting**” (si veda la documentazione di PostgreSQL) per scrivere il corpo della funzione, invece della normale sintassi a virgolette singole. Senza il “dollar quoting”, gli apici singoli o i backslash nel corpo della funzione devono essere evitati raddoppiandoli.

```
1 create function find_film_by_id( id int) returns film as
2 $$
3 select * from film
4 where film_id = id;
5 $$
6 language sql
```

Codice 15.2: Utilizzo del dollar quoting all'interno di una funzione PL/pgSQL

Analogamente alle funzioni, è possibile usare le stringhe all'interno dei doppi dollari anche all'interno delle *stored procedures*:

```
1 create procedure proc_name(param_list)
```

```

2 as
3 $$
4 -- stored procedure body
5 $$
6 language plpgsql;

```

Codice 15.3: Utilizzo del dollar quoting all'interno delle stored procedures in PL/pgSQL

15.1.2 ■ Struttura dei blocchi procedurali

Allo stesso modo di PL/SQL, PL/pgSQL è un linguaggio **strutturato a blocchi**. Il testo completo del corpo di una funzione deve essere un blocco. Un blocco è definito come:

```

1 [ <<label>> ]
2 [ DECLARE
3 declarations ]
4 BEGIN
5 statements
6 END [ label ];

```



Esaminiamo la struttura dei blocchi più in dettaglio:

- Ogni blocco ha due sezioni: una di **dichiarazione** ed un **corpo**. La sezione delle dichiarazioni è opzionale mentre il corpo è obbligatorio. Un blocco è terminato da un punto e virgola dopo la keyword **END**.
- Un blocco può avere un'etichetta opzionale posta all'inizio e alla fine. L'etichetta di blocco viene utilizzata quando si desidera specificarla nell'istruzione **EXIT** del corpo del blocco o quando si desidera qualificare i nomi delle variabili dichiarate nel blocco.
- La sezione di dichiarazione è quella in cui si dichiarano tutte le variabili utilizzate nella sezione del corpo. Ogni dichiarazione nella sezione delle dichiarazioni termina con un punto e virgola.
- La sezione del corpo è quella in cui si inserisce il codice. Ogni istruzione nella sezione del corpo termina con un punto e virgola.
- Tutte le parole chiave non fanno distinzione tra maiuscole e minuscole. Gli identificatori sono implicitamente convertiti in minuscolo a meno che non siano virgolettati, proprio come nei normali comandi SQL.
- I commenti funzionano allo stesso modo del linguaggio SQL. Un doppio trattino (--) inizia un commento che si estende fino alla fine della riga. Un /* inizia un commento a blocchi che si estende fino all'occorrenza corrispondente di */.

L'esempio seguente illustra un blocco molto semplice, chiamato anche **blocco anonimo** per differenziarle dalle **stored procedures** e dalle funzioni.

```

1 do
2 $$
3 <<first_block>>
4 declare
5 order_count integer := 0;
6 begin
7 -- voglio contare il numero di ordini
8 select count(*)
9 into order_count
10 from orders;
11 raise notice 'Il numero di ordini: %',order_count;
12 end first_block
13 $$;

```

Codice 15.4: Esempio di struttura di un blocco PL/pgSQL

Per eseguire un blocco su pgAdmin, bisogna cliccare il bottone **Execute** come mostrato nella figura 15.1:

Si noti che l'istruzione **DO** non appartiene al blocco. Viene utilizzato per eseguire i blocchi anonimi. PostgreSQL ha introdotto l'istruzione **DO** dalla versione 9.0.

15.1.3 ■ Sottoblocchi

Qualsiasi istruzione nella sezione delle istruzioni di un blocco può essere un *sottoblocco*. I sottoblocchi possono essere utilizzati per il raggruppamento della logica o per localizzare le variabili in un piccolo gruppo di istruzioni. Le variabili dichiarate in un

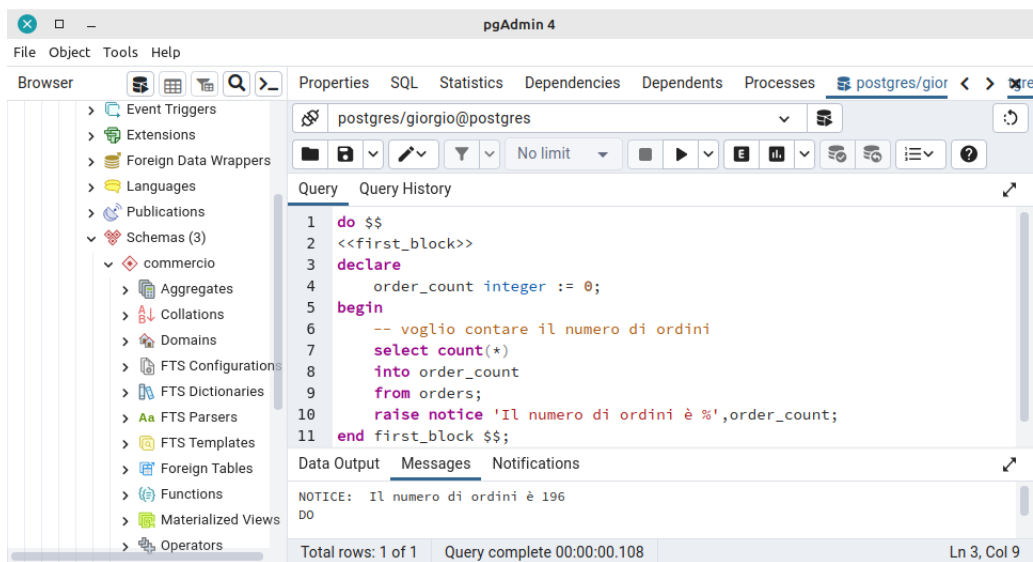


Figura 15.1: Esecuzione di un blocco anonimo su PgAdmin4

sottoblocco mascherano le variabili con nome simile dei blocchi esterni per tutta la durata del sottoblocco; è comunque possibile accedere alle variabili esterne se si qualifica il loro nome con l'etichetta del blocco.

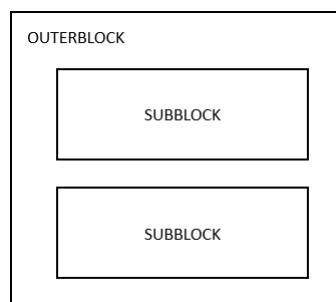


Figura 15.2: Blocchi esterni e sottoblocchi

15.2

VARIABILI E COSTANTI



15.2.1 Variabili

Una variabile è un nome significativo di una posizione di memoria. Una variabile contiene un valore che può essere modificato attraverso il blocco. Una variabile è sempre associata a un particolare tipo di dato. Prima di utilizzare una variabile, è necessario dichiararla nella sezione di dichiarazione del blocco PL/pgSQL. Di seguito la sintassi per la dichiarazione di una variabile:

```
1 variable_name data_type [:= expression];
```



All'interno di questa sintassi:

- Innanzitutto si specifica il nome della variabile. È buona norma assegnare un nome significativo ad una variabile. Ad esempio, invece di nominare una variabile `i`, si dovrebbe usare “*indice*” o “*contatore*”.
- In secondo luogo, si associa un tipo di dato specifico alla variabile. Il tipo di dati può essere qualsiasi tipo di dato valido, come `integer`, `numeric`, `varchar` e `char`.
- In terzo luogo, è possibile assegnare un valore predefinito a una variabile. Se non lo si fa, il valore iniziale della variabile è `NULL`.

All'interno del Listato 15.5 si illustra come dichiarare ed inizializzare le variabili:

```
1 do
2 $$
3 declare
4 counter integer := 1;
5 first_name varchar(50) := 'John';
```

```

6 last_name varchar(50) := 'Doe';
7 payment   numeric(11,2) := 20.5;
8 begin
9 raise notice '% % % has been paid % USD', counter, first_name, last_name, payment;
10 end
11 $$;

```

Codice 15.5: Dichiarazione e inizializzazione delle variabili in PL/pgSQL

Copying data types

L'istruzione `%type` fornisce il tipo di dato di una colonna o di un'altra variabile. Tipicamente si usa `%type` per dichiarare una variabile che deve contenere il valore del database. La sintassi in generale è la seguente:

```

1 variable_name table_name.column_name%type;

```



Per dichiarare una variabile del tipo di una seconda si usa invece la seguente sintassi:

```

1 variable_name variable%type;

```



Si consideri la tabella FILM (Tabella 15.1):

film
film_id
title
description
release_year
language_id
rental_duration
rental_date
length
rating
last_update
special_features
fulltext

Tabella 15.1

Questo esempio utilizza la tecnica della copia dei tipi per dichiarare variabili che contengono valori provenienti dalla tabella dei film:

```

1 do
2 $$
3 declare
4 film_title film.title%type;
5 featured_title film_title%type;
6 begin
7 -- get title of the film id 100
8 select title
9 from film
10 into film_title
11 where film_id = 100;
12
13 -- show the film title
14 raise notice 'Film title id 100: %s', film_title;
15 end;
16 $$

```

Codice 15.6: Utilizzo del parametro `%type` in PL/pgSQL

In questo esempio si dichiarano due variabili:

- La variabile `film_title` ha lo stesso tipo della colonna `title` presente nella tabella 15.1.
- La variabile `featured_title` ha lo stesso tipo della variabile `film_title`.

Utilizzando la tecnica della copia del tipo, si ottengono i seguenti vantaggi:

1. Innanzitutto, non è necessario conoscere il tipo di colonna o di riferimento a cui si fa riferimento.
2. In secondo luogo, se il tipo di dati del nome della colonna (o della variabile) di riferimento cambia, non è necessario modificare la definizione della funzione.

Variabili di tipo riga

Per memorizzare l'intera riga di un insieme di risultati restituito dall'istruzione `SELECT INTO`, si utilizzano le variabili di tipo row o **variabili row**. È possibile dichiarare una variabile con lo stesso tipo di dati della riga di una tabella utilizzando la seguente sintassi:

```
1 row_variable table_name\%ROWTYPE;
2 row_variable view_name\%ROWTYPE;
```



Per accedere al singolo campo della variabile riga, si utilizza la notazione a punti come in questo caso:

```
1 row_variable.field_name
```



```
1 do $$
2 declare
3 selected_actor actor\%rowtype;
4 begin
5 -- select actor with id 10
6 select *
7 from actor
8 into selected_actor
9 where actor_id = 10;
10 -- show the number of actor
11 raise notice 'The actor name is \% \%',
12 selected_actor.first_name,
13 selected_actor.last_name;
14 end;
15 $$
```

Codice 15.7: Utilizzo delle row variables in PL/pgSQL

Variabili di tipo record

PostgreSQL fornisce un “tipo” chiamato **record**, simile al tipo di riga. Un record può contenere solo una riga di un insieme di risultati. A differenza di una variabile di tipo riga, però, una variabile record non ha una struttura predefinita. Per dichiarare una variabile record, si usa un nome di variabile seguito dalla parola chiave `record`, come in questo caso:

```
1 variable_name record;
```



La struttura di una variabile record viene determinata quando l'istruzione `select` o `for` le assegna una riga effettiva. Per accedere a un campo del record, si utilizza la sintassi della notazione a punti, come in questo caso:

```
1 record_variable.field_name;
```



Se si tenta di accedere a un campo di una variabile record prima che sia assegnato, si otterrà un errore. In effetti, un record non è un vero tipo di dati. È solo un **segnaposto**. Inoltre, una variabile record può cambiare la sua struttura quando viene riassegnata.

```
1 do $$
2 declare
3   rec record;
4 begin
5   -- select the film
6   select film_id, title, length
7   into rec
```



```

8   from film
9   where film_id = 200;
10
11   raise notice '% % %', rec.film_id, rec.title, rec.length;
12 end;
13 $$ language plpgsql;

```

Codice 15.8: Utilizzare una variabile di tipo `record` all'interno di una istruzione `select into`

```

1 do $$
2 declare
3   rec record;
4 begin
5   for rec in select title, length
6   from film
7   where length > 50
8   order by length
9   loop
10    raise notice '% (%)', rec.title, rec.length;
11  end loop;
12 end;
13 $$

```

Codice 15.9: Utilizzare una variabile di tipo `record` all'interno di un ciclo `for`

15.2.2 ■ Costanti

A differenza di una variabile, il valore di una costante non può essere cambiato una volta inizializzato. Le costanti rendono il codice molto più leggibile e mantenibile, infatti, si immagini di voler calcolare la seguente formula:

```

1 selling_price := net_price + net_price * 0.1;

```



Cosa significhi quel 0.1 nessuno lo sa. Al contrario, utilizzando una formula del genere, tutti sapranno interpretarla correttamente.

```

1 selling_price := net_price + net_price * vat;

```



In secondo luogo, le costanti riducono il costo di manutenzione in quanto rendono necessaria una modifica solo nella linea della sua definizione piuttosto di tutte le volte in cui essa viene utilizzata.

Per definire una costante in PL/pgSQL si usa la seguente costante:

```

1 constant_name constant data_type := expression;

```



```

1 do $$
2 declare
3   vat constant numeric := 0.1;
4   net_price numeric := 20.5;
5 begin
6   raise notice 'The selling price is %', net_price * ( 1 + vat );
7 end $$;

```

Codice 15.10: Esempio di utilizzo delle costanti in PL/pgSQL

15.3

STATEMENT DI BASE



15.3.1 ■ L'istruzione SELECT INTO

L'istruzione `SELECT INTO` consente di selezionare i dati dal database e di assegnarli a una variabile secondo la seguente sintassi:

```

1 select select_list
2 into variable_name

```

```
3 from table_expression;
```

Codice 15.11: Sintassi dell'istruzione `SELECT INTO`

Se la query restituisce zero righe vengono assegnati dei valori `NULL` ai target. Se la query restituisce più righe, la prima riga viene assegnata alla/e destinazione/i e le altre vengono scartate. (Per gestire più risultati si vedano i cursori).

È possibile controllare la variabile speciale `FOUND` dopo un'istruzione `SELECT INTO` per determinare se l'assegnazione ha avuto successo, ovvero se la query ha restituito almeno una riga. Ad esempio:

```
1 SELECT * INTO myrec FROM emp WHERE empname = myname;
2 IF NOT FOUND THEN
3     RAISE EXCEPTION 'employee \% not found', myname;
4 END IF;
```

■

```
1 do $$
2 declare
3     actor_count integer;
4 begin
5     -- select the number of actors from the actor table
6     select count(*)
7     into actor_count
8     from actor;
9     -- show the number of actors
10    raise notice 'The number of actors: \%', actor_count;
11 end; $$
```

Codice 15.12: Esempio di utilizzo dell'istruzione `SELECT INTO`

Per verificare se il risultato di un record o di una riga è nullo, è possibile utilizzare il condizionale `IS NULL`. Tuttavia, non c'è modo di sapere se sono state scartate altre righe. Il Listato 15.13 mostra un esempio nel quale si gestisce il caso in cui non siano state restituite righe.

```
1 DECLARE
2     users_rec RECORD;
3 BEGIN
4     SELECT INTO users_rec * FROM users WHERE user_id=3;
5
6     IF users_rec.homepage IS NULL THEN
7         -- user entered no homepage, return "http://"
8         RETURN 'http://';
9     END IF;
10 END;
```

Codice 15.13: Record nullo

15.3.2 ■ Non fare nulla

A volte una dichiarazione segnaposto che non fa nulla è utile. Ad esempio, può indicare che un braccio di una catena `if then else` è deliberatamente vuoto. A tale scopo si utilizza l'istruzione `NULL`:

```
1 NULL;
```

Codice 15.14: L'istruzione `NULL`

Ad esempio, i due frammenti di codice seguenti sono equivalenti:

```
1 BEGIN
2     y := x / 0;
3 EXCEPTION
4     WHEN division_by_zero THEN
5         NULL; -- ignore the error
6 END;
```

■

```
1 BEGIN
2     y := x / 0;
3 EXCEPTION
4     WHEN division_by_zero THEN
```

```
5  -- ignore the error
6  END;
```



15.4

STRUTTURE DI CONTROLLO



15.4.1 ■ If statement

L'istruzione if determina quali istruzioni eseguire in base al risultato di un'espressione booleana. PL/pgSQL offre tre forme di dichiarazioni if:

- if then

```
1 DO $$
2 declare
3     selected_film film%\rowtype;
4     input_film_id film.film_id%\type := 0;
5 begin
6     select * from film into selected_film
7     where film_id = input_film_id;
8     if not found then
9         raise notice 'The film \% could not be found', input_film_id;
10    end if;
11 end; $$
```

Codice 15.15: Esempio di blocco if then in PL/pgSQL

- if then else

```
1 do $$
2 declare
3     selected_film film%\rowtype;
4     input_film_id film.film_id%\type := 100;
5 begin
6     select * from film into selected_film
7     where film_id = input_film_id;
8     if not found then
9         raise notice 'The film \% could not be found', input_film_id;
10    else
11        raise notice 'The film title is \%', selected_film.title;
12    end if;
13 end; $$
```

Codice 15.16: Esempio di utilizzo di un blocco if then else

- if then elsif

```
do $$
declare
    v_film film%\rowtype;
    len_description varchar(100);
begin
    select * from film into v_film where film_id = 100;
    if not found then
        raise notice 'Film not found';
    else
        if v_film.length > 0 and v_film.length ≤ 50 then
            len_description := 'Short';
        elsif v_film.length > 50 and v_film.length < 120 then
            len_description := 'Medium';
        elsif v_film.length > 120 then
            len_description := 'Long';
        else
            len_description := 'N/A';
        end if;
        raise notice 'The \% film is \%', v_film.title, len_description;
    end if;
end; $$
```

Codice 15.17: Esempio di blocco if then elsif

15.4.2 Loop statement

L'istruzione **loop** definisce un ciclo incondizionato che esegue ripetutamente un blocco di codice finché non viene terminato da un'istruzione di uscita o di ritorno. Di seguito la sintassi:

```
1 <<label>>
2 loop
3 statements;
4 end loop;
```



In genere, si utilizza un'istruzione **if** all'interno del ciclo per terminarlo in base a una condizione come questa:

```
1 <<label>>
2 loop
3 statements;
4 if condition then
5 exit;
6 end if;
7 end loop;
```



È possibile inserire un'istruzione di **loop** all'interno di un'altra istruzione di **loop**. Quando un'istruzione di **loop** viene inserita all'interno di un'altra istruzione di **loop**, si parla di **loop annidato**:

```
1 <<outer_block>>
2 loop
3 statements;
4 <<inner_block>>
5 loop
6 /* ... */
7 exit <<inner_block>>
8 end loop;
9 end loop;
```



Quando si hanno **loop annidati**, è necessario utilizzare l'etichetta del **loop**, in modo da poterla specificare nelle istruzioni **exit** e **continue** per indicare a quale **loop** si riferiscono.

```
1 do $$
2 declare
3 n integer := 10;
4 fib integer := 0;
5 counter integer := 0 ;
6 i integer := 0 ;
7 j integer := 1 ;
8 begin
9 if (n < 1) then
10 fib := 0 ;
11 end if;
12 loop
13 exit when counter = n ;
14 counter := counter + 1 ;
15 select j, i + j into i, j ;
16 end loop;
17 fib := i;
18 raise notice '%', fib;
19 end; $$
```

Codice 15.18: Calcolo della sequenza di Fibonacci

Output:

```
NOTICE: 55
```



15.4.3 While statement

L'istruzione **while** esegue un blocco di codice finché una condizione non viene valutata come falsa.

```
1 [ <<label>> ]
2 while condition loop
3 statements;
4 end loop;
```



```
1 do $$
2 declare
3 counter integer := 0;
4 begin
5 while counter < 5 loop
6 raise notice 'Counter \% ', counter;
7 counter := counter + 1;
8 end loop;
9 end $$;
```

Codice 15.19: Esempio di ciclo while

15.4.4 For loop

Di seguito viene illustrata la sintassi dell'istruzione **for...loop** che esegue il loop di un intervallo di numeri interi:

```
1 [ <<label>> ]
2 for loop_counter in [ reverse ] from.. to [ by step ] loop
3 statements
4 end loop [ label ];
```



```
1 do $$
2 begin
3 for counter in 1..5 loop
4 raise notice 'counter: \% ', counter;
5 end loop;
6 end; $$
```

Codice 15.20: Esempio di ciclo for

L'esempio seguente itera su 5 numeri da 5 a 1 e mostra ogni numero in ogni iterazione:

```
1 do $$
2 begin
3 for counter in reverse 5..1 loop
4 raise notice 'counter: \% ', counter;
5 end loop;
6 end; $$
```

Codice 15.21: Esempio di ciclo for al contrario

Utilizzo del ciclo for di PL/pgSQL per iterare su un insieme di risultati

La seguente istruzione mostra come utilizzare l'istruzione for loop per iterare su un insieme di risultati di una query:

```
1 do $$
2 declare
3 f record;
4 begin
5 for f in select title, length
6 from film
7 order by length desc, title
8 limit 10
9 loop
10 raise notice '\%(\% mins)', f.title, f.length;
11 end loop;
12 end; $$
```

Codice 15.22: Utilizzo del ciclo for per iterare su un ResultSet

Output:

```
NOTICE: Chicago North(185 mins)
NOTICE: Control Anthem(185 mins)
NOTICE: Darn Forrester(185 mins)
NOTICE: Gangs Pride(185 mins)
NOTICE: Home Pity(185 mins)
NOTICE: Muscle Bright(185 mins)
NOTICE: Pond Seattle(185 mins)
NOTICE: Soldiers Evolution(185 mins)
NOTICE: Sweet Brotherhood(185 mins)
NOTICE: Worst Banger(185 mins)
```



Uso del ciclo for di PL/pgSQL per iterare sull'insieme dei risultati di una query dinamica

La seguente forma dell'istruzione for loop consente di eseguire una query dinamica (vedi Sezione 15.9) e di iterare sul suo insieme di risultati:

```
1 [ <<label>> ]
2 for row in execute query_expression [ using query_param [, ... ] ]
3 loop
4 statements
5 end loop [ label ];
```



```
1 do $$
2 declare
3 -- sort by 1: title, 2: release year
4 sort_type smallint := 1;
5 -- return the number of films
6 rec_count int := 10;
7 -- use to iterate over the film
8 rec record;
9 -- dynamic query
10 query text;
11 begin
12
13 query := 'select title, release_year from film ';
14
15 if sort_type = 1 then
16 query := query || 'order by title';
17 elsif sort_type = 2 then
18 query := query || 'order by release_year';
19 else
20 raise 'invalid sort type %s', sort_type;
21 end if;
22
23 query := query || ' limit $1';
24
25 for rec in execute query using rec_count
26 loop
27 raise notice '% - %', rec.release_year, rec.title;
28 end loop;
29 end; $$
```

Codice 15.23: Utilizzo di ciclo for per iterare sul resultset di una query dinamica

Output:

```
NOTICE: 2006 - Academy Dinosaur
NOTICE: 2006 - Ace Goldfinger
NOTICE: 2006 - Adaptation Holes
NOTICE: 2006 - Affair Prejudice
NOTICE: 2006 - African Egg
NOTICE: 2006 - Agent Truman
NOTICE: 2006 - Airplane Sierra
NOTICE: 2006 - Airport Pollock
NOTICE: 2006 - Alabama Devil
NOTICE: 2006 - Aladdin Calendar
```



15.4.5 Exit

L'istruzione **exit** consente di terminare un ciclo, compresi i cicli incondizionati, i cicli **while** e i cicli **for**:

```
1 exit [label] [when boolean_expression]
```



dove **label** è l'etichetta del ciclo corrente in cui si trova l'uscita o l'etichetta del ciclo esterno. A seconda dell'etichetta, l'istruzione **exit** terminerà il ciclo corrispondente. Se non si utilizza l'etichetta, l'istruzione **exit** terminerà il ciclo corrente. La clausola **when boolean_expression** è usata per specificare una condizione che termina un ciclo. L'istruzione **exit** termina il ciclo se l'espressione booleana è vera.

Oltre a terminare un ciclo, è possibile utilizzare l'istruzione **exit** per terminare un blocco specificato dalle parole chiave **for...loop**. In questo caso, il controllo viene passato all'istruzione successiva alla parola chiave **end** del blocco corrente:

```
1 <<block_label>>
2 BEGIN
3 -- some code
4 EXIT [block_label] [WHEN condition];
5 -- some more code
6 END block_label;
```



```
1 do $$
2 declare
3 i int = 0;
4 j int = 0;
5 begin
6 <<outer_loop>>
7 loop
8 i = i + 1;
9 exit when i > 3;
10 -- inner loop
11 j = 0;
12 <<inner_loop>>
13 loop
14 j = j + 1;
15 exit when j > 3;
16 raise notice '(i,j): (%,%)', i, j;
17 end loop inner_loop;
18 end loop outer_loop;
19 end; $$
```

Codice 15.24: Esempio di utilizzo dell'istruzione **exit** in PL/pgSQL

Output:

```
NOTICE: (i,j): (1,1)
NOTICE: (i,j): (1,2)
NOTICE: (i,j): (1,3)
NOTICE: (i,j): (2,1)
NOTICE: (i,j): (2,2)
NOTICE: (i,j): (2,3)
NOTICE: (i,j): (3,1)
NOTICE: (i,j): (3,2)
NOTICE: (i,j): (3,3)
```



15.4.6 Continue

L'istruzione **continue** salta prematuramente l'iterazione corrente del ciclo e passa a quella successiva. L'istruzione **continue** può essere utilizzata in tutti i tipi di loop, compresi quelli incondizionati, **while** e **for**. Di seguito viene illustrata la sintassi dell'istruzione **continue**:

```
1 continue [loop_label] [when condition]
```



In questa sintassi, la `loop_label` e la condizione `when` sono opzionali. `loop_label` è l'etichetta del ciclo di cui si vuole saltare l'iterazione corrente. Se si omette, l'istruzione `continue` salta l'iterazione corrente del ciclo. Se si specifica l'etichetta di un ciclo, l'istruzione `continue` salta l'iterazione corrente di quel ciclo. La condizione è un'espressione booleana che specifica la condizione per saltare l'iterazione corrente del ciclo. Se la condizione è vera, l'istruzione `continue` salta l'iterazione corrente del ciclo.

```
1 do $$
2 declare
3 counter int = 0;
4 begin
5 loop
6 counter = counter + 1;
7 -- exit the loop if counter > 10
8 exit when counter > 10;
9 -- skip the current iteration if counter is an even number
10 continue when mod(counter,2) = 0;
11 -- print out the counter
12 raise notice '%', counter;
13 end loop;
14 end; $$
```

Codice 15.25: Esempio di istruzione `continue`

15.5

FUNZIONI USER-DEFINED



15.5.1 L'istruzione `Create Function`

L'istruzione `create function` consente di definire una nuova funzione definita dall'utente. Di seguito viene illustrata la sintassi dell'istruzione `create function`:

```
1 create [or replace] function function_name(param_list) returns return_type
2 language plpgsql as $$
3 declare
4 -- variable declaration
5 begin
6 -- logic
7 end; $$
```



```
1 CREATE OR REPLACE FUNCTION find_nth_occurrence(str text, substring text, n integer)
2 RETURNS integer AS $$
3 DECLARE
4 start_pos integer;
5 end_pos integer;
6 count integer;
7 BEGIN
8 start_pos := 1;
9 count := 0;
10 LOOP
11 end_pos := position(substring in substring(str, start_pos));
12 IF end_pos = 0 THEN
13 RETURN 0;
14 END IF;
15 count := count + 1;
16 IF count = n THEN
17 RETURN start_pos + end_pos - 1;
18 END IF;
19 start_pos := start_pos + end_pos;
20 END LOOP;
21 END; $$
22 LANGUAGE plpgsql;
```

Codice 15.26: Trovare l'ennesima occorrenza di una sottostringa in una stringa

La funzione mostrata nel Listato 15.26 cerca la posizione dell'ennesima occorrenza di una sottostringa: prende in ingresso una stringa, una sottostringa e un intero che specifica l'occorrenza desiderata. Il risultato è ottenuto mediante una combinazione delle funzioni `substring` e `position` (vedi sezione 14.4.2) messe insieme all'interno di un blocco `loop`.

Di seguito un esempio di funzione PL/pgSQL:


```

1 create function get_film_count(len_from int, len_to int) returns int language plpgsql
2 as $$
3 declare
4 film_count integer;
5 begin
6 select count(*)
7 into film_count
8 from film
9 where length between len_from and len_to;
10
11 return film_count;
12 end; $$

```

Codice 15.27: Istruzione `create function` in PL/pgSQL

Creare una funzione con PgAdmin4

Innanzitutto, lanciare lo strumento pgAdmin e collegarsi al database. In secondo luogo, aprire lo strumento di query selezionando Strumenti > Strumento di query. In terzo luogo, inserire il codice sopra riportato nello strumento di query e fare clic sul pulsante Esegui per creare la funzione `get_film_count` (Vedi Figura 15.3).

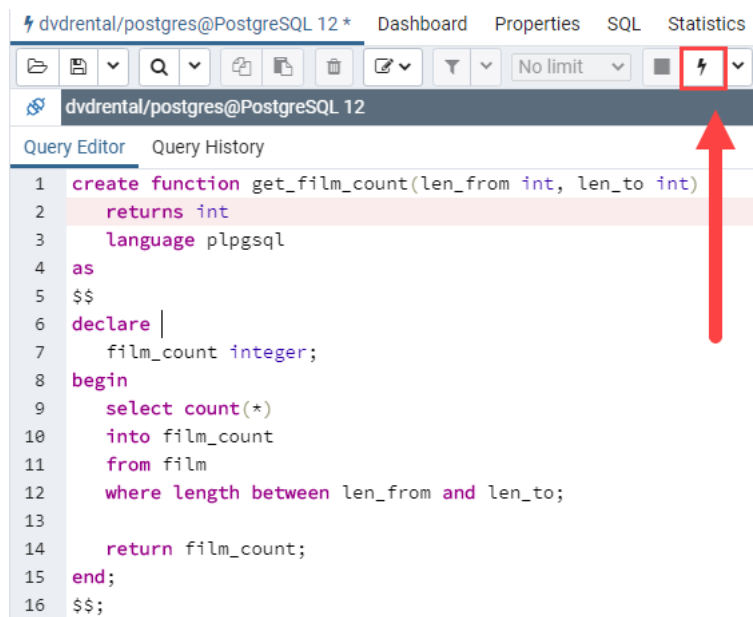


Figura 15.3

Chiamare una funzione

PostgreSQL offre tre modi per chiamare una funzione definita dall'utente:

- Utilizzando la notazione posizionale: specificando gli argomenti nello stesso ordine dei parametri. Ad esempio:

```
select get_film_count(40,90);
```



- Utilizzando la notazione con nome:

```
select get_film_count(
  len_from => 40,
  len_to   => 90
);
```



Nella notazione con nome, si usa il `=>` per separare il nome dell'argomento dal suo valore.

- Utilizzando la notazione mista:

```
select get_film_count(40, len_to => 90);
```



15.5.2 ■ Modalità di passaggio dei parametri

Le modalità di passaggio dei parametri determinano il loro comportamento. PL/pgSQL supporta tre modalità di parametro: **in**, **out** e **inout**. Un parametro assume la modalità **in** per impostazione predefinita, se non viene specificata esplicitamente. La tabella seguente illustra le tre modalità dei parametri:

IN	OUT	INOUT
Default	Specificato esplicitamente	Specificato esplicitamente
Passa un valore ad una funzione	Restituisce un valore dalla funzione	Passa un valore alla funzione e lo restituisce aggiornato
Sono come delle costanti	Sono come delle variabili non inizializzate	Sono come delle variabili non inizializzate
Non è possibile assegnare un valore	Bisogna assegnare un valore	Si dovrebbe assegnare un valore

Parametri IN

La funzione seguente trova un film in base al suo id e restituisce il titolo del film:

```
1 create or replace function find_film_by_id(p_film_id int)
2 returns varchar
3 language plpgsql
4 as $$
5 declare
6 film_title film.title\%type;
7 begin
8 -- find film title by id
9 select title
10 into film_title
11 from film
12 where film_id = p_film_id;
13 if not found then
14 raise 'Film with id \% not found', p_film_id;
15 end if;
16 return title;
17 end; $$
```



Poiché non è stata specificata la modalità per il parametro `p_film_id`, viene presa la modalità **in** per impostazione predefinita.

Parametri OUT

I parametri **out** sono definiti come parte dell'elenco degli argomenti e vengono restituiti come parte del risultato. I parametri **out** sono molto utili nelle funzioni che devono restituire più valori. Per definire i parametri **out**, si fa precedere esplicitamente il nome del parametro dalla parola chiave **out**, come segue:

```
1 out parameter_name type
```



L'esempio seguente definisce la funzione `get_film_stat` che ha tre parametri in uscita:

```
1 create or replace function get_film_stat
2 (
3   out min_len int,
4   out max_len int,
5   out avg_len numeric
6 )
7 language plpgsql
8 as $$
9 begin
10   select min(length),
11          max(length),
12          avg(length)::numeric(5,1)
13   into min_len, max_len, avg_len
14   from film;
15 end; $$
```



La seguente istruzione chiama la funzione `get_film_stat`:

```
1 select get_film_stat();
```



L'output della funzione è un record. Per rendere l'output separato in colonne, si utilizza la seguente istruzione:

```
1 select * from get_film_stat();
```



Parametri INOUT

La modalità inout è la combinazione delle modalità in e out. Significa che il chiamante può passare un argomento a una funzione. La funzione modifica l'argomento e restituisce il valore aggiornato. La seguente funzione `swap` accetta due numeri interi e i loro valori:

```
1 create or replace function swap
2 (
3     inout x int,
4     inout y int
5 )
6 language plpgsql
7 as $$
8 begin
9     select x,y into y,x;
10 end; $$;
```



La seguente istruzione chiama la funzione `swap`:

```
select * from swap(10,20);
```



15.5.3 Funzioni che restituiscono una tabella

Per definire una funzione che restituisce una tabella, si utilizza la seguente forma dell'istruzione `create function`:

```
1 create or replace function function_name ( parameter_list )
2 returns table ( column_list )
3 language plpgsql
4 as $$
5 declare
6 -- variable declaration
7 begin
8 -- body
9 end; $$
```



Invece di restituire un singolo valore, questa sintassi consente di restituire una tabella con un elenco di colonne specificato:

```
returns table ( column_list )
```



15.5.4 Eliminare una funzione

Per rimuovere una funzione definita dall'utente, si utilizza l'istruzione `drop function`:

```
1 drop function [if exists] function_name(argument_list)
2 [cascade | restrict]
```



Se una funzione ha oggetti dipendenti, come operatori o trigger, non è possibile eliminarla. Per eliminare la funzione e i suoi oggetti dipendenti, è necessario specificare l'opzione `cascade`. La funzione `drop` con l'opzione `CASCADE` rimuove ricorsivamente

la funzione, i suoi oggetti dipendenti, gli oggetti che dipendono da questi oggetti e così via. Per impostazione predefinita, l'istruzione `drop function` utilizza l'opzione **RESTRICT**, che rifiuta la rimozione di una funzione quando ha oggetti dipendenti. Per eliminare più funzioni con un'unica istruzione `drop function`, si specifica un elenco di nomi di funzioni separati da virgole dopo la parola chiave `drop function`, come segue:

```
1 drop function [if exists] function1, function2, ... ;
```



15.5.5 ■ L'istruzione Create Procedure

Finora si è visto come definire le funzioni definite dall'utente utilizzando l'istruzione `create function`. Uno svantaggio delle funzioni definite dall'utente, però, è che non possono eseguire **transazioni**. In altre parole, all'interno di una funzione definita dall'utente non è possibile avviare una transazione, eseguire il `commit` o il `rollback`.

PostgreSQL 11 ha introdotto le stored procedure che supportano le transazioni. Per definire una nuova stored procedure, si utilizza l'istruzione `create procedure`. Di seguito viene illustrata la sintassi di base dell'istruzione `create procedure`:

```
1 create [or replace] procedure procedure_name(parameter_list) language plpgsql
2 as $$
3 declare
4 -- variable declaration
5 begin
6 -- stored procedure body
7 end; $$
```



Una stored procedure non restituisce un valore. Non è possibile utilizzare l'istruzione `return` con un valore all'interno di una stored procedure.

Chiamare una stored procedure

Per chiamare una stored procedure si utilizza l'istruzione `CALL` come segue:

```
1 call stored_procedure_name(argument_list);
```



Eliminare una procedura

L'istruzione `drop procedure` rimuove una stored procedure. Di seguito viene illustrata la sintassi dell'istruzione `drop procedure`:

```
1 drop procedure [if exists] procedure_name (argument_list)
2 [cascade | restrict]
```



Per eliminare più stored procedure, si specifica un elenco di virgole di nomi di stored procedure dopo la parola chiave `drop procedure`, in questo modo:

```
1 drop procedure [if exists] name1, name2, ... ;
```



15.6

CURSORI



15.6.1 ■ Gestione dei cursori

Un **cursore** PL/pgSQL consente di incapsulare una query e di elaborare ogni singola riga alla volta.

In genere, si usano i cursori quando si vuole dividere un insieme di risultati di grandi dimensioni in parti ed elaborare ogni parte individualmente. Se si elabora tutto in una volta, si potrebbe avere un errore di overflow della memoria.

Inoltre, è possibile sviluppare una funzione che restituisca un riferimento a un cursore. Questo è un modo efficace per restituire un insieme di risultati di grandi dimensioni da una funzione. Il chiamante della funzione può elaborare il set di risultati in base al riferimento al cursore.

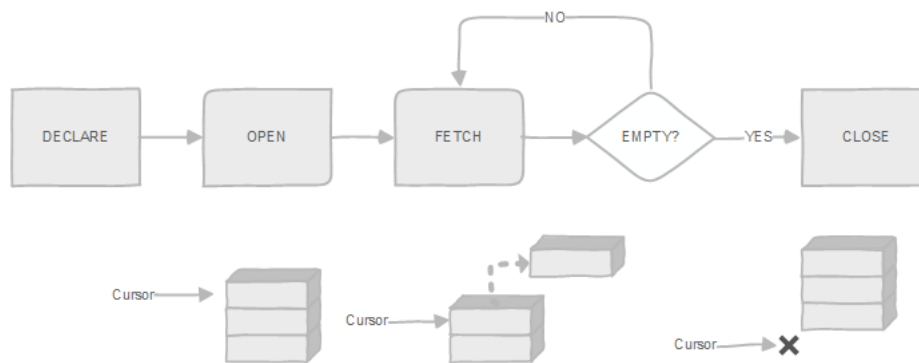


Figura 15.4: Utilizzo di un cursore in PL/pgSQL

Il diagramma seguente illustra come utilizzare un cursore in PostgreSQL:

Nel diagramma in figura 15.4 sono mostrati i seguenti passi:

- Innanzitutto, dichiarare un cursore.
- Quindi, aprire il cursore.
- Quindi, recuperare le righe dall'insieme dei risultati in una destinazione.
- Dopodiché, verificare se sono rimaste altre righe da recuperare. In caso affermativo, passare al punto 3, altrimenti al punto 5.
- Infine, chiudere il cursore.

Dichiarare un cursore

Per accedere a un cursore, è necessario dichiarare una variabile cursore nella sezione di dichiarazione di un blocco. PostgreSQL fornisce un tipo speciale chiamato **REFCURSOR** per dichiarare una variabile cursore.

```
1 declare my_cursor refcursor;
```



È anche possibile dichiarare un cursore che si lega a una query utilizzando la seguente sintassi:

```
1 cursor_name [ [no] scroll ] cursor
2 [( arguments)] FOR query;
```



dove la parola chiave **FOR** può essere sostituita con **IS** per compatibilità con la sintassi¹ Oracle PL/SQL. Se viene specificato **SCROLL**, il cursore sarà in grado di scorrere all'indietro; se viene specificato **NO SCROLL**, le ricerche all'indietro saranno rifiutate; se non compare nessuna delle due specifiche, dipende dalla query se le ricerche all'indietro saranno consentite. **arguments**, se specificato, è un elenco separato da virgole di coppie *nome-tipo di dati* che definiscono i nomi da sostituire ai valori dei parametri nella query data. I valori effettivi da sostituire a questi nomi saranno specificati successivamente, all'apertura del cursore.

Il seguente esempio mostra come dichiarare un cursore:

```
1 declare
2 curs1 REFCURSOR; --Cursore non vincolato
3 curs2 CURSOR FOR SELECT * FROM table; --Cursore vincolato
```

Codice 15.28: Dichiarazione di un cursore in PostgreSQL

15.6.2 Aprire un cursore

I cursori devono essere aperti prima di poter essere utilizzati per recuperare le righe restituite da una interrogazione. Per aprire un cursore non vincolato si utilizza la seguente sintassi:

```
1 OPEN unbound_cursor_variable [ [ NO ] SCROLL ] FOR query;
```



¹Vedi 13.5

Poiché la variabile cursore non vincolata non è vincolata a nessuna query quando viene dichiarata, è necessario specificare la query quando la si apre. Si veda l'esempio seguente:

```
1 open my_cursor for
2 select * from city
3 where country = p_country;
```



PostgreSQL consente di aprire un cursore e di vincolarlo a una **query dinamica**.

```
1 open unbound_cursor_variable[ [ no ] scroll ]
2 for execute query_string [using expression [, ... ] ];
```



Nell'esempio seguente, si costruisce una query dinamica che ordina le righe in base a un parametro `sort_field` e si apre il cursore che esegue la query dinamica.

```
1 query := 'select * from city order by $1';
2 open cur_city for execute query using sort_field;
```



Poiché un cursore vincolato² è già collegato a una query quando viene dichiarato, quando lo apriamo dobbiamo solo passare gli argomenti alla query, se necessario.

```
1 open cursor_variable[ (name:=value,name:=value, ... )];
```



Questa forma di **OPEN** viene utilizzata per aprire una variabile del cursore la cui query è stata vincolata al momento della dichiarazione. Il cursore non può essere già aperto. Un elenco di espressioni di valori di argomenti reali deve comparire se e solo se il cursore è stato dichiarato per accettare argomenti. Questi valori saranno sostituiti nella query.

I valori degli argomenti possono essere passati con notazione posizionale o con nome. Nella *notazione posizionale*, tutti gli argomenti sono specificati in ordine. Nella *notazione con nome*, il nome di ogni argomento viene specificato usando `:=` per separarlo dall'espressione dell'argomento.

15.6.3 ■ Usare i cursori

Dopo aver aperto un cursore, è possibile manipolarlo utilizzando le istruzioni **FETCH**, **MOVE**, **UPDATE** o **DELETE**.

Recuperare la riga successiva

```
1 fetch [ direction { from | in } ] cursor_variable
2 into target_variable;
```



L'istruzione **FETCH** prende la riga successiva dal cursore e le assegna ad una variabile, che può essere un *record*, una *variabile di riga* o un *elenco di variabili* separato da virgole. Se non vengono trovate altre righe, la *target_variable* viene impostata su **NULL**.

Di default, un cursore ottiene la riga successiva se non si specifica esplicitamente la direzione. Per il cursore vale quanto segue:

- **NEXT**
- **LAST**
- **PRIOR**
- **FIRST**
- **ABSOLUTE count**
- **RELATIVE count**
- **FORWARD, BACKWARD**

```
1 FETCH curs1 INTO rowvar;
2 FETCH curs2 INTO foo, bar, baz;
```

Codice 15.29: Utilizzo di un cursore mediante keyword **FETCH**

²Vedi la sezione 13.5.3

15.6.4 Muovere il cursore

Se si desidera spostare solo il cursore senza recuperare alcuna riga, si utilizza l'istruzione `MOVE`. La direzione accetta lo stesso valore dell'istruzione `FETCH`.

```
1 move [ direction { from | in } ] cursor_variable;
```



15.6.5 Cancellare o aggiornare un cursore

Una volta posizionato il cursore, è possibile eliminare o aggiornare la riga identificata dal cursore utilizzando l'istruzione `DELETE WHERE CURRENT OF` o `UPDATE WHERE CURRENT OF`, come segue:

```
1 update table_name
2 set column = value, ...
3 where current of cursor_variable;
4
5 delete from table_name
6 where current of cursor_variable;
```



15.6.6 Chiudere un cursore

Per chiudere un cursore aperto, si utilizza l'istruzione `CLOSE` come segue:

```
1 close cursor_variable;
```



L'istruzione `CLOSE` rilascia le risorse o libera la variabile del cursore per consentirne la riapertura con l'istruzione `OPEN`.

```
1 create or replace function get_film_titles(p_year integer) returns text as $$
2 declare
3 titles text default;
4 rec_film record;
5 cur_films cursor(p_year integer)
6 for select title, release_year
7 from film
8 where release_year = p_year;
9 begin
10 -- open the cursor
11 open cur_films(p_year);
12
13 loop
14 -- fetch row into the film
15 fetch cur_films into rec_film;
16 -- exit when no more row to fetch
17 exit when not found;
18
19 -- build the output
20 if rec_film.title like '%ful%' then
21 titles := titles || ',' || rec_film.title || ':' || rec_film.release_year;
22 end if;
23 end loop;
24
25 -- close the cursor
26 close cur_films;
27
28 return titles;
29 end $$
30 language plpgsql;
```

Codice 15.30: Esempio di utilizzo dei cursori

15.6.7 ■ Restituire un cursore

Le funzioni PL/pgSQL possono restituire i cursori al chiamante. Ciò è utile per restituire più righe o colonne, soprattutto con insiemi di risultati molto grandi. A tale scopo, la funzione **apre il cursore** e **restituisce il nome del cursore** al chiamante. Il chiamante può quindi recuperare le righe dal cursore. Il cursore può essere chiuso dal chiamante, oppure verrà chiuso automaticamente alla chiusura della transazione.

L'esempio seguente mostra un modo di passare un cursore:

```
1 CREATE TABLE test (col text);
2 INSERT INTO test VALUES ('123');
3
4 CREATE FUNCTION reffunc(refcursor) RETURNS refcursor AS $$
5 BEGIN
6 OPEN $1 FOR SELECT col FROM test;
7 RETURN $1;
8 END;$$
9 LANGUAGE plpgsql;
10
11 BEGIN;
12 SELECT reffunc('funcursor');
13 FETCH ALL IN funcursor;
14 COMMIT;
```

Codice 15.31: Funzione che restituisce il nome di un cursore

15.7

GESTIONE DELLE ECCEZIONI



15.7.1 ■ Lanciare errori e riportare messaggi

Per sollevare un messaggio, si usa l'istruzione **raise** come segue:

```
raise level format;
```



L'opzione **level** segue l'istruzione **raise** e specifica la gravità dell'errore. PostgreSQL fornisce i seguenti livelli:

- debug
- log
- notice
- info
- warning
- exception

format è una stringa che specifica il messaggio. La stringa **format** usa dei segnaposto che vengono sostituiti dagli argomenti specificati. Il numero dei segnaposto deve essere lo stesso del numero degli argomenti, altrimenti, PostgreSQL riporterà un errore.

```
1 do $$
2 begin
3 raise info 'information message %', now() ;
4 raise log 'log message %', now();
5 raise debug 'debug message %', now();
6 raise warning 'warning message %', now();
7 raise notice 'notice message %', now();
8 end $$;
```

Codice 15.32: Esempio di utilizzo dell'istruzione **raise**

Output:

```
INFO: information message 2023-01-31 17:02:08.286798+01
WARNING: warning message 2023-01-31 17:02:08.286798+01
NOTICE: notice message 2023-01-31 17:02:08.286798+01
```



Per sollevare un errore si usa il livello `exception` dopo l'istruzione `raise`.

Note

L'istruzione `raise` usa il livello `exception` di default.

Oltre a segnalare un errore, è possibile aggiungere ulteriori informazioni utilizzando la seguente clausola aggiuntiva:

```
using option = expression
```



dove `option` può essere:

- `message`: per impostare il messaggio di errore;
- `hint`: per fornire un suggerimento in modo che sia più facile individuare la causa dell'errore.
- `detail`: fornire informazioni dettagliate sull'errore.
- `errcode`: identifica il codice di errore, che può essere il nome della condizione o direttamente il codice `SQLSTATE` di cinque caratteri. Consulta la tabella dei codici di errore e dei nomi delle condizioni.

```
1 do $$
2 declare
3 email varchar(255) := 'info@postgresqлтutorial.com';
4 begin
5 -- check email for duplicate
6 -- ...
7 -- report duplicate email
8 raise exception 'duplicate email: %', email
9 using hint = 'check the email again';
10 end $$;
```

Codice 15.33: Sollevamento di un errore in PL/pgSQL

Output:

```
[Err] ERROR: Duplicate email: info@postgresqлтutorial.com
HINT: Check the email again
```



15.7.2 La clausola `exception`

Quando si verifica un errore in un blocco, PostgreSQL interrompe l'esecuzione del blocco e anche la transazione circostante. Per recuperare l'errore, è possibile utilizzare la clausola di eccezione nel blocco `begin...end`. Di seguito si illustra la sintassi della clausola di eccezione:

```
1 <<label>>
2 declare
3 begin
4 statements;
5 exception
6 when condition [or condition...] then
7 handle_exception;
8 [when condition [or condition...] then
9 handle_exception;]
10 [when others then
11 handle_other_exceptions;
12 ]
13 end;
```



Come funziona:

- Innanzitutto, quando si verifica un errore tra l'inizio e l'eccezione, PL/pgSQL interrompe l'esecuzione e passa il controllo all'elenco delle eccezioni.

- In secondo luogo, PL/pgSQL cerca la prima condizione che corrisponde all'errore verificatosi.
- In terzo luogo, se c'è una corrispondenza, vengono eseguite le istruzioni `handle_exception` corrispondenti. PL/pgSQL passa il controllo all'istruzione dopo la parola chiave `end`.
- Infine, se non viene trovata alcuna corrispondenza, l'errore si propaga all'esterno e può essere catturato dalla clausola di eccezione del blocco racchiuso. Nel caso in cui non ci sia un blocco di chiusura con la clausola di eccezione, PL/pgSQL interromperà l'elaborazione.

I nomi delle condizioni possono essere `no_data_found` nel caso in cui un'istruzione `select` non restituisca alcuna riga o `too_many_rows` se l'istruzione `select` restituisce più di una riga. Per un elenco completo dei nomi delle condizioni, consultare il sito Web di PostgreSQL.

È anche possibile specificare la condizione di errore tramite il codice `SQLSTATE`. Ad esempio, `P0002` per `no_data_found` e `P0003` per `too_many_rows`.

In genere, si cattura un'eccezione specifica e la si gestisce di conseguenza. Per gestire altre eccezioni oltre a quella specificata nell'elenco, è possibile utilizzare la clausola `when others then`.

15.8

FUNZIONI TRIGGERS



15.8.1 ■ Introduzione ai trigger PostgreSQL

Un trigger PostgreSQL è una **funzione invocata automaticamente** ogni volta che si verifica un evento come un inserimento, un aggiornamento o una cancellazione. Un trigger è una funzione speciale definita dall'utente ed **associata a una tabella**. Per creare un nuovo trigger, si definisce prima una funzione trigger e poi si lega questa funzione trigger a una tabella. La differenza tra un trigger e una funzione definita dall'utente è che un trigger viene invocato automaticamente quando si verifica un evento scatenante.

Tipi di trigger PostgreSQL

PostgreSQL fornisce due tipi principali di trigger:

- Trigger a livello di riga
- Trigger a livello di dichiarazione

Le differenze tra i due tipi sono il numero di volte in cui il trigger viene invocato e il momento in cui viene invocato.

Ad esempio, se si emette un'istruzione `UPDATE` che modifica 20 righe, il trigger a livello di riga verrà invocato 20 volte, mentre il trigger a livello di dichiarazione verrà invocato 1 volta.

È possibile specificare se il trigger viene invocato prima o dopo un evento. Se il trigger viene invocato prima di un evento, può saltare l'operazione per la riga corrente o addirittura cambiare la riga da aggiornare o inserire. Se il trigger viene invocato dopo l'evento, tutte le modifiche sono disponibili per il trigger.

Quando utilizzare i trigger

I trigger sono utili nel caso in cui al database accedano diverse applicazioni e si voglia mantenere la funzionalità trasversale all'interno del database, che viene eseguita automaticamente ogni volta che i dati della tabella vengono modificati. Ad esempio, se si vuole mantenere la cronologia dei dati senza richiedere all'applicazione una logica di controllo per ogni evento come `INSERT` o `UPDATE`.

Inoltre, è possibile utilizzare i trigger per mantenere regole complesse di integrità dei dati che non possono essere implementate altrove se non a livello di database. Ad esempio, quando viene aggiunta una nuova riga nella tabella dei clienti, devono essere create anche altre righe nelle tabelle delle banche e dei crediti.

Lo svantaggio principale dell'uso di un trigger è che bisogna conoscerne l'esistenza e comprenderne la logica per capire gli effetti quando i dati cambiano.

15.8.2 ■ L'istruzione Create Trigger

Una funzione trigger è simile a una normale funzione definita dall'utente. Tuttavia, una funzione trigger **non accetta argomenti** e ha un valore di ritorno **di tipo trigger**.

Di seguito viene illustrata la sintassi della creazione di una funzione trigger:

```
1 CREATE FUNCTION trigger_function()
2 RETURNS TRIGGER
```

```

3 LANGUAGE PLPGSQL
4 AS $$
5 BEGIN
6 -- trigger logic
7 END $$

```



Una funzione trigger riceve dati sull'ambiente che la chiama attraverso una struttura speciale chiamata **TriggerData**, che contiene un insieme di variabili locali. Ad esempio, **OLD** e **NEW** rappresentano gli stati della riga nella tabella prima o dopo l'evento scatenante.

Una volta definita una funzione di trigger, è possibile legarla a uno o più eventi di trigger come **INSERT**, **UPDATE** e **DELETE**.

L'istruzione **CREATE TRIGGER** crea un nuovo trigger. Di seguito viene illustrata la sintassi di base dell'istruzione **CREATE TRIGGER**:

```

1 CREATE TRIGGER trigger_name
2 {BEFORE | AFTER} { event }
3 ON table_name
4 [FOR [EACH] { ROW | STATEMENT }]
5 EXECUTE PROCEDURE trigger_function

```



In questa sintassi vengono specificati:

- Il nome dell'attivazione dopo le parole chiave **TRIGGER**.
- La **tempistica** che provoca l'attivazione del trigger. Può essere **prima** (**AFTER**) o **dopo** (**BEFORE**) il verificarsi di un evento.
- L'**evento** che invoca l'attivazione. L'evento può essere **INSERT**, **DELETE**, **UPDATE** o **TRUNCATE**.
- Il nome della tabella associata al trigger dopo la parola chiave **ON**.
- Il **tipo** di trigger che può essere un:
 1. trigger a livello di riga, specificato dalla clausola **FOR EACH ROW**:
 2. trigger a livello di dichiarazione, specificato dalla clausola **FOR EACH STATEMENT**

Supponiamo che una tabella abbia 100 righe e due trigger che vengono attivati quando si verifica un evento **DELETE**. Se l'istruzione **DELETE** cancella 100 righe, il trigger a livello di riga si attiverà 100 volte, una per ogni riga cancellata. D'altra parte, un trigger a livello di dichiarazione verrà attivato una sola volta, indipendentemente dal numero di righe cancellate.

- Il **nome della funzione di trigger** dopo le parole chiave **EXECUTE PROCEDURE**.

Un primo esempio di trigger in PL/pgSQL

Il listato 15.34 crea una nuova tabella chiamata **EMPLOYEES**:

```

1 DROP TABLE IF EXISTS employees;
2
3 CREATE TABLE employees
4 (
5   id INT GENERATED ALWAYS AS IDENTITY,
6   first_name VARCHAR(40) NOT NULL,
7   last_name VARCHAR(40) NOT NULL,
8   PRIMARY KEY(id)
9 );

```

Codice 15.34: Creazione tabella **EMPLOYEES**

Si supponga che quando il nome di un dipendente cambia, si voglia registrare le modifiche in una tabella separata chiamata **EMPLOYEE_AUDITS**.

```

1 CREATE TABLE employee_audits
2 (
3   id INT GENERATED ALWAYS AS IDENTITY,
4   employee_id INT NOT NULL,
5   last_name VARCHAR(40) NOT NULL,
6   changed_on TIMESTAMP(6) NOT NULL
7 );

```

Codice 15.35: Creazione tabella **EMPLOYEE_AUDITS**

Per prima cosa, si crea una nuova funzione chiamata `log_last_name_changes`:

```
1 CREATE OR REPLACE FUNCTION log_last_name_changes()  
2 RETURNS TRIGGER  
3 LANGUAGE PLPGSQL  
4 AS $$  
5 BEGIN  
6 IF NEW.last_name <> OLD.last_name THEN  
7 INSERT INTO employee_audits(employee_id,last_name,changed_on)  
8 VALUES(OLD.id,OLD.last_name,now());  
9 END IF;  
10  
11 RETURN NEW;  
12 END $$
```

Codice 15.36: Creazione funzione

La funzione inserisce il vecchio cognome nella tabella `EMPLOYEE_AUDITS` includendo l'id del dipendente, il cognome e l'ora della modifica se il cognome di un dipendente cambia.

`OLD` rappresenta la riga prima dell'aggiornamento, mentre `NEW` rappresenta la nuova riga che verrà aggiornata. `OLD.last_name` restituisce il cognome prima dell'aggiornamento e `NEW.last_name` restituisce il nuovo cognome.

In secondo luogo, si lega la funzione di trigger alla tabella dei dipendenti. Il nome del trigger è `last_name_changes`. Prima che il valore della colonna cognome venga aggiornato, la funzione di trigger viene invocata automaticamente per registrare le modifiche.

```
1 CREATE TRIGGER last_name_changes  
2 BEFORE UPDATE  
3 ON employees  
4 FOR EACH ROW  
5 EXECUTE PROCEDURE log_last_name_changes();
```

Codice 15.37: Creazione del trigger

Inseriamo alcune righe all'interno della tabella `EMPLOYEES`:

```
1 INSERT INTO employees (first_name, last_name)  
2 VALUES ('John', 'Doe');  
3  
4 INSERT INTO employees (first_name, last_name)  
5 VALUES ('Lily', 'Bush');
```

Codice 15.38: Inserimento dati

e visualizziamo la tabella:

```
1 SELECT * FROM employees;
```



Supponiamo che Lily Bush cambi il suo nome in Lily Brown. Quindi eseguiamo una `UPDATE`:

```
1 UPDATE employees  
2 SET last_name = 'Brown'  
3 WHERE ID = 2;
```



A questo punto l'operazione di update deve aver fatto scatenare il trigger. Visualizziamo quindi la tabella `EMPLOYEE_AUDITS`:

```
1 SELECT * FROM employee_audits;
```



Se si utilizza lo strumento `psql`, è possibile visualizzare tutti i trigger associati a una tabella utilizzando il comando:

```
\dS employees
```



Argomenti e tipo di ritorno

Quando una funzione PL/pgSQL viene chiamata come trigger, nel blocco di primo livello vengono create automaticamente diverse variabili speciali. Esse sono:

- **NEW**: variabile di tipo **record** che contiene la nuova riga del database per le operazioni **INSERT/UPDATE** nei trigger a livello di riga. Questa variabile è nulla nei trigger a livello di dichiarazione e per le operazioni di **DELETE**.
- **OLD**: variabile di tipo **record** che contiene la vecchia riga del database per le operazioni di **UPDATE/DELETE** nei trigger a livello di riga. Questa variabile è nulla nei trigger a livello di dichiarazione e per le operazioni di **INSERT**.

PostgreSQL fornisce anche altre variabili locali precedute da **TG_**, come **TG_WHEN** e **TG_TABLE_NAME**.

- **TG_OP**: stringa che specifica quale operazione DML ha scatenato il trigger (**INSERT**, **UPDATE**, **DELETE**, **TRUNCATE**);
- **TG_NAME**: variabile che contiene il nome del trigger sparato;
- **TG_WHEN**: stringa che specifica la temporizzazione del trigger a seconda della definizione del trigger;
- **TG_LEVEL**: stringa che specifica la granularità del trigger (se agisce per ogni riga, o istruzione);
- **TG_TABLE_NAME**: specifica il nome della tabella che ha causato l'invocazione del trigger;
- **TG_RELNAME**: specifica il nome della tabella che ha causato l'invocazione del trigger;
- **TG_RELID**: specifica l'id dell'oggetto della tabella che ha causato l'invocazione del trigger;
- **TG_TABLE_SCHEMA**: specifica il nome dello schema della tabella che ha causato l'invocazione del trigger;
- **TG_NARGS**: specifica il numero di argomenti dati alla procedura del trigger;
- **TG_ARGV[]**: specifica gli argomenti dell'istruzione di definizione del trigger.

```
1 CREATE TABLE emp (
2   empname      text NOT NULL,
3   salary       integer
4 );
5
6 CREATE TABLE emp_audit(
7   operation     char(1)  NOT NULL,
8   stamp         timestamp NOT NULL,
9   userid        text     NOT NULL,
10  empname       text     NOT NULL,
11  salary integer
12 );
13
14 CREATE OR REPLACE FUNCTION process_emp_audit()
15 RETURNS TRIGGER AS
16 $$
17 BEGIN
18 --
19 -- Create a row in emp_audit to reflect the operation performed on emp,
20 -- making use of the special variable TG_OP to work out the operation.
21 --
22 IF (TG_OP = 'DELETE') THEN
23 INSERT INTO emp_audit SELECT 'D', now(), user, OLD.*;
24 ELSIF (TG_OP = 'UPDATE') THEN
25 INSERT INTO emp_audit SELECT 'U', now(), user, NEW.*;
26 ELSIF (TG_OP = 'INSERT') THEN
27 INSERT INTO emp_audit SELECT 'I', now(), user, NEW.*;
28 END IF;
29 RETURN NULL; -- result is ignored since this is an AFTER trigger
30 END;
31 $$
32 LANGUAGE plpgsql;
33
34 CREATE TRIGGER emp_audit
35 AFTER INSERT OR UPDATE OR DELETE ON emp
36 FOR EACH ROW EXECUTE FUNCTION process_emp_audit();
```

Codice 15.39: Esempio di utilizzo dell'argomento **tg_op** in un trigger PL/pgSQL

Una funzione di trigger deve restituire o **NULL** o un valore di record/fila che abbia esattamente la struttura della tabella per cui il trigger è stato attivato.

15.8.3 L'istruzione Drop Trigger

Per eliminare un trigger da una tabella, si utilizza l'istruzione **DROP TRIGGER** con la seguente sintassi:

```
1 DROP TRIGGER [IF EXISTS] trigger_name
2 ON table_name [ CASCADE | RESTRICT ];
```



Dove in questa sintassi:

Per prima cosa, si specifica il **nome del trigger** che si vuole eliminare dopo le parole chiave **DROP TRIGGER**.

In secondo luogo, si usa **IF EXISTS** per eliminare condizionatamente il trigger solo se esiste. Il tentativo di eliminare un trigger non esistente senza specificare l'istruzione **IF EXISTS** provoca un errore. Se si usa **IF EXISTS** per eliminare un trigger non esistente, PostgreSQL emette un avviso.

In terzo luogo, si specifica il **nome della tabella** a cui appartiene il trigger. Se la tabella appartiene a uno schema specifico, si può usare il nome qualificato dello schema della tabella, ad esempio, **nome_schema.nome_tabella**.

In quarto luogo, utilizzare l'opzione **CASCADE** se si desidera eliminare automaticamente gli oggetti che dipendono dal trigger. Si noti che l'opzione **CASCADE** eliminerà anche gli oggetti che dipendono da oggetti che dipendono dal trigger.

Quinto, utilizzare l'opzione **RESTRICT** per rifiutare di eliminare il trigger se alcuni oggetti dipendono da esso. Per impostazione predefinita, l'istruzione **DROP TRIGGER** utilizza **RESTRICT**.

Si noti che in SQL standard, i nomi dei trigger non sono locali alle tabelle, quindi l'istruzione diventa semplicemente:

```
1 DROP TRIGGER trigger_name;
```



15.8.4 L'istruzione Alter Trigger

L'istruzione **ALTER TRIGGER** consente di rinominare un trigger. Di seguito viene mostrata la sintassi dell'istruzione:

```
1 ALTER TRIGGER trigger_name
2 ON table_name
3 RENAME TO new_trigger_name;
```



In questa sintassi:

- Per prima cosa, si specifica il nome del trigger che si desidera rinominare dopo la parola chiave **ALTER TRIGGER**.
- In secondo luogo, si specifica il nome della tabella associata al trigger dopo la parola chiave **ON**.
- Terzo, si specifica il nuovo nome del trigger dopo la parola chiave **RENAME TO**.

15.9

SQL DINAMICO CON PL/PGSQL



L'SQL dinamico³ viene utilizzato per ridurre le operazioni ripetitive di interrogazione. Ad esempio, si può usare l'SQL dinamico per aggiungere indici mancanti su tutte le chiavi esterne o per aggiungere funzionalità di verifica dei dati di una determinata tabella senza grandi effetti di codifica. Un altro uso importante dell'SQL dinamico è quello di superare gli effetti collaterali della cache di PL/pgSQL, poiché le query eseguite con l'istruzione **EXECUTE** non vengono memorizzate nella cache.

15.9.1 L'istruzione Execute

L'SQL dinamico si ottiene con l'istruzione **EXECUTE**. L'istruzione **EXECUTE** accetta una stringa e la valuta semplicemente. La sintassi per eseguire un'istruzione è la seguente:

```
1 EXECUTE command-string [ INTO [STRICT] target ] [ USING expression [, ...] ];
```



³Vedi la Sezione 13.7

dove `command-string` è un'espressione contenuta in una stringa (di tipo `text`) contenente il comando da eseguire. Il parametro opzionale `target` è una variabile di tipo record, una variabile di tipo riga, o una lista di variabili semplici separate da una virgola, all'interno del quale conservare il risultato del comando. L'espressione opzionale `USING` fornisce i parametri da inserire all'interno del comando.

Inoltre, non esiste un piano di esecuzione per i comandi eseguiti tramite `EXECUTE`. Al contrario, il comando viene sempre pianificato *ogni volta* che l'istruzione viene eseguita. Pertanto, la stringa di comando può essere creata dinamicamente all'interno della funzione per eseguire azioni su tabelle e colonne diverse.

Nella stringa di comando calcolata non viene effettuata alcuna sostituzione di variabili PL/pgSQL. I valori delle variabili necessarie devono essere inseriti nella stringa di comando mentre viene costruita; in alternativa, è possibile utilizzare i parametri descritti usando la clausola `USING`.

La clausola `INTO` specifica dove devono essere assegnati i risultati di un comando SQL che restituisce righe. Se viene fornita una variabile di riga o un elenco di variabili, questa deve corrispondere esattamente alla struttura dei risultati del comando; se viene fornita una variabile record, questa si configurerà automaticamente per corrispondere alla struttura dei risultati. Se vengono restituite più righe, solo la prima viene assegnata alle variabili `INTO`. Se non vengono restituite righe, alle variabili `INTO` viene assegnato `NULL`. Se non viene specificata la clausola `INTO`, i risultati del comando vengono scartati.

Se viene fornita l'opzione `STRICT`, viene segnalato un errore se il comando non produce esattamente una riga.

La stringa di comando può utilizzare i valori dei parametri, che sono indicati nel comando come `$1`, `$2`, ecc. Questi simboli si riferiscono ai valori forniti nella clausola `USING`. Questo metodo è spesso preferibile all'inserimento di valori di dati nella stringa di comando come testo: evita l'overhead del tempo di esecuzione per la conversione dei valori in testo e viceversa ed è molto meno soggetto ad attacchi di SQL-injection, poiché non sono necessari quoting o escape. Un esempio è:

```
1 EXECUTE 'SELECT count(*) FROM mytable WHERE inserted_by = $1 AND inserted ≤ $2'
2 INTO c
3 USING checked_user, checked_date;
```

Codice 15.40: Esempio di query costruita dinamicamente in PL/pgSQL

Si noti che i simboli dei parametri possono essere utilizzati solo per i valori dei dati; se si desidera utilizzare nomi di tabelle o colonne determinati dinamicamente, è necessario **inserirli testualmente** nella stringa di comando. Ad esempio, se la query precedente dovesse essere eseguita su una tabella selezionata dinamicamente, si potrebbe fare sfruttando gli operatori e le funzioni per la gestione delle stringhe⁴:

```
1 EXECUTE 'SELECT count(*) FROM '
2 || quote_ident(tabname)
3 || ' WHERE inserted_by = $1 AND inserted ≤ $2'
4 INTO c
5 USING checked_user, checked_date;
```



Un approccio più pulito è quello di usare la specifica `%I` della funzione `format()`⁵ per inserire nomi di tabelle o colonne con virgolette automatiche:

```
1 EXECUTE format('SELECT count(*) FROM %I '
2 'WHERE inserted_by = $1 AND inserted ≤ $2', tabname)
3 INTO c
4 USING checked_user, checked_date;
```



L'esempio seguente mostra come utilizzare `EXECUTE` per creare una query SQL dinamica:

```
1 CREATE OR REPLACE FUNCTION dynamic_query(p_table_name text, p_column_name text, p_value text)
2 RETURNS SETOF record AS
3 $$
4 DECLARE
5 query text;
6 BEGIN
7 query := 'SELECT * FROM ' || p_table_name || ' WHERE ' || p_column_name || ' = $1';
8 RETURN QUERY EXECUTE query USING p_value;
9 END;
10 $$ LANGUAGE plpgsql;
```



⁴Vedi 14.4.2

⁵Vedi 14.4.2

In questo esempio, la funzione `dynamic_query` accetta tre parametri: il nome della tabella, il nome della colonna e il valore da cercare. La funzione utilizza la funzione `EXECUTE` per creare una stringa di query SQL dinamica che cerca il valore specificato nella colonna specificata della tabella specificata. La funzione `EXECUTE` viene poi utilizzata per eseguire la query SQL dinamica.

Per evitare possibili attacchi di SQL injection, è importante utilizzare la funzione `EXECUTE` con i parametri utilizzando la clausola `USING`, come mostrato nell'esempio. In questo modo, i valori dei parametri vengono passati in modo sicuro alla query SQL, impedendo l'esecuzione di codice SQL malevolo.

In generale, l'uso di SQL dinamico può essere utile quando è necessario creare query SQL complesse o dinamiche che non possono essere facilmente generate in modo statico. Tuttavia, è importante prestare attenzione alla sicurezza e utilizzare la funzione `EXECUTE` in modo sicuro, evitando così possibili attacchi di SQL injection.

15.9.2 Esecuzione di istruzioni DDL in SQL dinamico

In alcuni casi, è necessario eseguire operazioni a livello di oggetti del database, come tabelle, indici, colonne, ruoli e così via. Ad esempio, per analizzare le tabelle dello schema `car_portal_app`, si potrebbe scrivere il seguente script:

```
1 DO $$
2 DECLARE
3 table_name text;
4 BEGIN
5 FOR table_name IN
6 SELECT tablename FROM pg_tables WHERE schemaname = 'car_portal_app'
7 LOOP
8 RAISE NOTICE 'Analyzing %', table_name;
9 EXECUTE 'ANALYZE car_portal_app.' || table_name;
10 END LOOP;
11 END;
12 $$;
```



In questo esempio si è utilizzata la tabella di sistema `pg_tables` illustrata nella sezione 14.2.2.

15.9.3 Eseguire istruzioni DML in SQL dinamico

Alcune applicazioni possono dover interagire con la base di dati in modo interattivo. Ad esempio, si possono avere dati di fatturazione generati su base mensile ed avere una vista che permette all'utente di filtrare i dati in base a diversi criteri. In questi casi, l'SQL dinamico è molto utile. Ad esempio, nell'applicazione del portale automobilistico, la funzionalità di ricerca è necessaria per ottenere i conti utilizzando il predicato dinamico, come segue:

```
1 CREATE OR REPLACE FUNCTION car_portal_app.get_account (predicate TEXT)
2 RETURNS SETOF car_portal_app.account AS
3 $$
4 BEGIN
5 RETURN QUERY EXECUTE 'SELECT * FROM car_portal_app.account WHERE ' || predicate;
6 END;
7 $$ LANGUAGE plpgsql;
```

Codice 15.41: Esecuzione di un'istruzione DML sfruttando l'SQL dinamico in PL/pgSQL

15.9.4 Il retrieve dinamico

Il retrieve dinamico è una delle operazioni che può essere eseguita grazie all'SQL dinamico. Questa operazione si riferisce alla creazione di una query SQL dinamica che recupera un insieme di dati da una o più tabelle. Invece di creare una query SQL statica che recupera un insieme fisso di colonne da una tabella specifica, il retrieve dinamico consente di creare una query SQL che può recuperare un insieme di colonne dinamico, selezionando solo le colonne specificate in fase di esecuzione.

```
1 CREATE OR REPLACE FUNCTION dynamic_select(p_table_name text, p_columns text)
2 RETURNS SETOF record AS
3 $$
4 DECLARE
5 query text;
6 BEGIN
7 query := 'SELECT ' || p_columns || ' FROM ' || p_table_name;
8 RETURN QUERY EXECUTE query;
9 END;
10 $$
```



```
11 LANGUAGE plpgsql;
```

Codice 15.42: Esempio di retrieve dinamico sfruttando l'SQL dinamico in PL/pgSQL

Nel Listato 15.42 la funzione `dynamic_select` accetta due parametri: il nome della tabella e l'elenco delle colonne da selezionare. La funzione utilizza la funzione `EXECUTE` per creare una stringa di query SQL dinamica che seleziona solo le colonne specificate in fase di esecuzione.

Per eseguire un retrieve dinamico con una lista di parametri contenuti in una singola stringa separati da un simbolo, è possibile utilizzare la funzione `string_to_array` per suddividere la stringa in un array di valori e quindi utilizzare l'operatore `IN`⁶ per creare una query SQL dinamica che recuperi i record corrispondenti.

Ad esempio, supponiamo di avere una tabella "clienti" con una colonna "id" e si vogliano recuperare tutti i record corrispondenti a una lista di ID specificati in una singola stringa separata da virgole. Possiamo utilizzare il seguente codice PL/pgSQL per eseguire un retrieve dinamico:

```
1 CREATE OR REPLACE FUNCTION dynamic_retrieve(p_id_list text)
2 RETURNS SETOF clients AS
3 $$
4 DECLARE
5 id_array text[];
6 query text;
7 BEGIN
8 id_array := string_to_array(p_id_list, ',');
9 query := 'SELECT * FROM clients WHERE id IN (' || array_to_string(id_array, ',') || ')';
10 RETURN QUERY EXECUTE query;
11 END;
12 $$ LANGUAGE plpgsql;
```



In questo esempio, la funzione `dynamic_retrieve` accetta una singola stringa `p_id_list` contenente la lista degli ID separati da virgole. La funzione utilizza la funzione `string_to_array` per suddividere la stringa in un array di valori e l'operatore `IN` per creare una stringa di query SQL dinamica che cerca tutti i record nella tabella "clienti" con gli ID specificati nella lista. La funzione `EXECUTE` viene poi utilizzata per eseguire la query SQL dinamica.

15.9.5 Il sort dinamico

Il sort dinamico, d'altra parte, si riferisce alla creazione di una query SQL dinamica che ordina i dati recuperati da una o più tabelle in base a una colonna specifica. Invece di creare una query SQL statica che ordina i dati in base a una colonna specifica, il sort dinamico consente di creare una query SQL che può ordinare i dati in base a una colonna specifica selezionata in fase di esecuzione.

```
1 CREATE OR REPLACE FUNCTION dynamic_sort(p_table_name text, p_column_name text, p_order text)
2 RETURNS SETOF record AS
3 $$
4 DECLARE
5 query text;
6 BEGIN
7 query := 'SELECT * FROM ' || p_table_name || ' ORDER BY ' || p_column_name || ' ' || p_order;
8 RETURN QUERY EXECUTE query;
9 END;
10 $$ LANGUAGE plpgsql;
```

Codice 15.43: Esempio di sort dinamico sfruttando l'SQL dinamico in PL/pgSQL

15.10

TRANSAZIONI IN PL/PGSQL



15.10.1 Cos'è una transazione?

Le **transazioni** sono un concetto fondamentale di tutti i sistemi di database. Il punto essenziale di una transazione è che raggruppa più fasi in un'unica operazione, che può essere eseguita in un'unica soluzione. Gli stati intermedi tra le fasi non sono visibili alle altre transazioni concorrenti e se si verifica un errore che impedisce il completamento della transazione, nessuna delle fasi influisce sul database.

⁶Vedi ??

Un esempio classico di transazione è il trasferimento bancario da un conto a un altro. Una transazione completa deve garantire l'equilibrio tra i conti del mittente e del destinatario. Ciò significa che se il conto del mittente trasferisce un importo X, il destinatario riceve un importo X, né più né meno.

Come anticipato nella sezione 1.3.4, una transazioni deve rispondere alle proprietà ACID:

- L'atomicità garantisce che la transazione venga completata in modo completo o nullo.
- La consistenza garantisce che le modifiche ai dati scritti nel database siano valide e seguano regole predefinite.
- L'isolamento determina il modo in cui l'integrità della transazione è visibile alle altre transazioni.
- La durabilità assicura che le transazioni che sono state impegnate vengano memorizzate nel database in modo permanente.

15.10.2 ■ Iniziare una transazione

Per avviare una transazione, si utilizza la seguente istruzione:

```
1 BEGIN TRANSACTION;
```



oppure:

```
1 BEGIN WORK;
```



o solo:

```
1 BEGIN;
```



15.10.3 ■ Committare una transazione

Per rendere la modifica visibile ad altre sessioni (o utenti) è necessario eseguire il **commit** della transazione utilizzando l'istruzione COMMIT WORK oppure COMMIT TRANSACTION; o semplicemente COMMIT;.

Dopo l'esecuzione dell'istruzione COMMIT, PostgreSQL garantisce anche che la modifica sarà duratura in caso di crash.

```
1 -- tabella di esempio
2 DROP TABLE IF EXISTS accounts;
3
4 CREATE TABLE accounts (
5 id INT GENERATED BY DEFAULT AS IDENTITY,
6 name VARCHAR(100) NOT NULL,
7 balance DEC(15,2) NOT NULL,
8 PRIMARY KEY(id)
9 );
10 -- start a transaction
11 BEGIN;
12
13 -- inserisco una nuova riga nella tabella
14 INSERT INTO accounts(name,balance)
15 VALUES('Alice',10000);
16
17 -- committo il cambiamento
18 COMMIT;
```

Codice 15.44: Esempio di Transazione in PostgreSQL

```
1 -- start a transaction
2 BEGIN;
3
4 -- deduct 1000 from account 1
5 UPDATE accounts
6 SET balance = balance - 1000
7 WHERE id = 1;
8
9 -- add 1000 to account 2
10 UPDATE accounts
```

```

11 SET balance = balance + 1000
12 WHERE id = 2;
13
14 -- select the data from accounts
15 SELECT id, name, balance
16 FROM accounts;
17
18 -- commit the transaction
19 COMMIT;

```

Codice 15.45: Trasferimento di denaro tra due conti mediante una transazione

15.10.4 ■ Ripristinare una transazione

Per annullare la modifica della transazione corrente, si utilizza una delle seguenti istruzioni:

```

1 ROLLBACK WORK;

```



o

```

1 ROLLBACK TRANSACTION;

```



o sinteticamente:

```

1 ROLLBACK;

```



Supponiamo di voler trasferire 1500 dollari dal conto di Bob a quello di Alice. Tuttavia, per errore si invia il denaro al conto di Jack invece che a quello di Alice. Si vuole quindi annullare l'intera transazione.

```

1 -- begin the transaction
2 BEGIN;
3
4 -- deduct the amount from the account 1
5 UPDATE accounts
6 SET balance = balance - 1500
7 WHERE id = 1;
8
9 -- add the amount from the account 3 (instead of 2)
10 UPDATE accounts
11 SET balance = balance + 1500
12 WHERE id = 3;
13
14 -- roll back the transaction
15 ROLLBACK;

```

Codice 15.46: Esempio di rollback di una transazione

15.10.5 ■ Usare le transazioni all'interno delle procedure

Nelle procedure e nei blocchi anonimi (comando DO), è possibile utilizzare le transazioni utilizzando i comandi COMMIT e ROLLBACK.

```

1 CREATE OR REPLACE FUNCTION transfer_funds(p_from_account_id INTEGER, p_to_account_id INTEGER, p_amount
  NUMERIC)
2 RETURNS VOID AS
3 $$
4 DECLARE
5 -- Variabili per il recupero dei dati dell'account
6 from_account_balance NUMERIC;
7 to_account_balance NUMERIC;
8 BEGIN
9 -- Inizia la transazione
10 BEGIN
11 -- Ottieni il saldo dell'account di origine
12 SELECT balance INTO from_account_balance FROM accounts WHERE id = p_from_account_id FOR UPDATE;

```

```

13
14 -- Controlla se ci sono abbastanza fondi
15 IF from_account_balance < p_amount THEN
16 RAISE EXCEPTION 'Non ci sono abbastanza fondi nell''account di origine'';
17 END IF;
18
19 -- Ottieni il saldo dell''account di destinazione
20 SELECT balance INTO to_account_balance FROM accounts WHERE id = p_to_account_id FOR UPDATE;
21
22 -- Aggiorna i saldi degli account
23 UPDATE accounts SET balance = balance - p_amount WHERE id = p_from_account_id;
24 UPDATE accounts SET balance = balance + p_amount WHERE id = p_to_account_id;
25
26 -- Commit della transazione
27 COMMIT;
28 EXCEPTION
29 -- In caso di errore, rollback della transazione
30 WHEN OTHERS THEN
31 ROLLBACK;
32 RAISE;
33 END;
34 END;
35 $$
36 LANGUAGE plpgsql;

```

Codice 15.47: Esempio di transazione all'interno di una procedura

In questo esempio, la procedura “**transfer_funds**” trasferisce una determinata quantità di fondi da un account di origine a un account di destinazione. La procedura utilizza una transazione per garantire che il trasferimento di fondi sia effettuato in modo atomico e la gestione delle eccezioni per mantenere l'integrità dei dati in caso di errori.

La procedura inizia una transazione utilizzando la clausola **BEGIN**. All'interno della transazione, la procedura recupera i saldi degli account di origine e di destinazione e controlla se ci sono abbastanza fondi nell'account di origine per eseguire il trasferimento. Se non ci sono abbastanza fondi, la procedura solleva un'eccezione utilizzando la clausola **RAISE EXCEPTION**.

Se tutto va bene, la procedura aggiorna i saldi degli account di origine e di destinazione utilizzando le clausole **UPDATE**. Infine, la procedura esegue un commit della transazione utilizzando la clausola **COMMIT** per confermare le modifiche apportate ai dati.

In caso di errori, la procedura solleva un'eccezione utilizzando la clausola **RAISE**, che interrompe l'esecuzione della transazione e esegue un rollback per annullare tutte le modifiche apportate ai dati.

15.11

PORTING DA ORACLE PL/SQL



15.11.1 ■ Le differenze tra PL/SQL e PL/pgSQL

Questa sezione spiega le differenze tra il linguaggio PL/pgSQL di PostgreSQL e il linguaggio PL/SQL di Oracle, per aiutare gli sviluppatori che effettuano il porting di applicazioni da Oracle® a PostgreSQL.

PL/pgSQL è simile a PL/SQL per molti aspetti. È un linguaggio imperativo strutturato a blocchi e tutte le variabili devono essere dichiarate. Assegnazioni, cicli e condizionali sono simili. Le principali differenze da tenere presenti nel passaggio da PL/SQL a PL/pgSQL sono:

- Se un nome utilizzato in un comando SQL può essere sia il nome di una colonna di una tabella utilizzata nel comando sia un riferimento a una variabile della funzione, PL/SQL lo tratta come un nome di colonna. Per impostazione predefinita, PL/pgSQL lancia un errore lamentando l'ambiguità del nome. È possibile specificare `plpgsql.variable_conflict = use_column` per modificare questo comportamento in modo che corrisponda a quello di PL/SQL. Spesso è meglio evitare queste ambiguità, ma se si deve eseguire il porting di una grande quantità di codice che dipende da questo comportamento, impostare `variable_conflict` può essere la soluzione migliore.
- In PostgreSQL il corpo della funzione deve essere scritto come una stringa letterale. Pertanto, è necessario utilizzare il *dollar quoting* o l'escape delle virgolette singole nel corpo della funzione.
- I nomi dei tipi di dati spesso devono essere tradotti. Ad esempio, in Oracle i valori stringa sono comunemente dichiarati di tipo `VARCHAR2`, che è un tipo non standard SQL. In PostgreSQL, si utilizza invece il tipo `VARCHAR` o `TEXT`. Allo stesso modo, si può sostituire il tipo `NUMBER` con `NUMERIC`, o usare un altro tipo di dati numerici se ce n'è uno più appropriato.
- I cicli `for` con la clausola **REVERSE** funzionano in modo diverso: PL/SQL conta alla rovescia dal secondo numero al primo, mentre PL/pgSQL conta alla rovescia dal primo numero al secondo, richiedendo lo scambio dei limiti del ciclo durante il porting.

- Esistono varie differenze di notazione per l'uso delle variabili cursore.

ESERCIZI ED APPROFONDIMENTI

16.1

ALGEBRA RELAZIONALE



Esercizio

Sia dato lo schema relazionale costituito dalle tabelle (le chiavi primarie sono sottolineate):

RIVISTA (CodR, NomeR, Editore) ARTICOLO (CodA, Titolo, Argomento, CodR)

Tabella 16.1: Schema RIVISTA, ARTICOLO

Trovare il codice e il nome delle riviste che hanno pubblicato almeno un articolo di argomento “motociclismo”.

Svolgimento: Seleziono tutti gli articoli che hanno il motociclismo come argomento:

$$Articoli_{motociclismo} \leftarrow \sigma_{Argomento='Motociclismo'}(Articolo)$$

Effettuo una θ -join tra $Articoli_{motociclismo}$ e $Rivista$ per avere tutte le riviste che contengono questi articoli e proietto su $CodR$ e $NomeR$:

$$Riviste_{motociclismo} \leftarrow \Pi_{CodR, NomeR}(Rivista \bowtie_{CodR=CodR'} \rho_{X' \leftarrow X}(Articoli_{motociclismo}))$$

Esercizio

Sia dato lo schema relazionale costituito dalle tabelle (le chiavi primarie sono sottolineate):

VELISTI (Vid, VNome, Esperienza, DataNascita) PRENOTAZIONI (Vid, Bid, Data)
BARCHE (Bid, BNome, Colore)

Tabella 16.2: Schema relazionale VELISTI, PRENOTAZIONI, BARCHE

Trovare i nomi dei velisti che hanno prenotato almeno una barca rossa oppure una barca verde.

Svolgimento: L'insieme delle barche rosse è:

$$BarcheRosse \leftarrow \sigma_{Colore=Rosso}(Barche)$$

Mentre quello delle barche verdi:

$$BarcheVerdi \leftarrow \sigma_{Colore=Verdi}(Barche)$$

L'insieme contenente le informazioni sulle prenotazioni e sulle barche prenotate è dato dalla θ -join tra PRENOTAZIONI e BARCHE, si ha quindi:

$$PrenotazioniBarcheRosse \leftarrow Prenotazioni \bowtie_{Bid=Bid'} \rho_{X' \leftarrow X}(BarcheRosse)$$

$$PrenotazioniBarcheVerdi \leftarrow Prenotazioni \bowtie_{Bid=Bid'} \rho_{X' \leftarrow X}(BarcheVerdi)$$

mentre l'insieme delle prenotazioni e dei velisti che le hanno effettuate è dato dalla θ -join tra VELISTI e PRENOTAZIONI:

$$VelistiPrenotazioniRosse \leftarrow Velisti \bowtie_{Vid=Vid'} \rho_{X' \leftarrow X}(PrenotazioniBarcheRosse)$$

$$VelistiPrenotazioniVerdi \leftarrow Velisti \bowtie_{Vid=Vid'} \rho_{X' \leftarrow X}(PrenotazioniBarcheVerdi)$$

Si ha così l'insieme ricercato:

$$\Pi_{VNome}(\text{VelistiPrenotazioniRosse} \cup \text{VelistiPrenotazioniVerdi})$$

Esercizio

Sia dato lo schema relazionale 16.1.

- Trovare il codice e il nome delle riviste che pubblicano articoli di motociclismo oppure di auto.
- Trovare il codice e il nome delle riviste che pubblicano articoli sia di motociclismo sia di auto.

Svolgimento: Come visto nell'esercizio precedente, l'insieme delle riviste che pubblicano articoli di motociclismo si otteneva facendo una selezione tra gli argomenti all'interno di ARTICOLO per poi fare una θ -join con RIVISTA. In questo caso si dovrà modificare semplicemente la condizione di selezione all'interno della σ iniziale:

$$\Pi_{CodR, NomeR}(\text{Rivista} \bowtie_{CodR=CodR'} \rho_{X' \leftarrow X}(\sigma_{Argomento='Motociclismo' \vee Argomento='Auto'}(\text{Articolo})))$$

Analogamente, l'insieme delle riviste che pubblicano articoli sia di motociclismo sia di auto sarà:

$$\begin{aligned} \text{Articoli}_{Motociclismo} &\leftarrow \sigma_{Argomento='Motociclismo'}(\text{Articolo}) \\ \text{Articoli}_{Auto} &\leftarrow \sigma_{Argomento='Auto'}(\text{Articolo}) \end{aligned}$$

Questi due articoli si trovano nella stessa rivista quando hanno lo stesso codice rivista, quindi prendiamo l'insieme di tutti gli articoli che stanno nella stessa rivista:

$$\text{StessaRivista} \leftarrow \text{Articoli}_{Motociclismo} \bowtie_{codR=codR'} \rho_{X' \leftarrow X}(\text{Articoli}_{Auto})$$

per avere le informazioni relative alla rivista si farà:

$$\Pi_{CodR, NomeR}(\text{StessaRivista} \bowtie_{CodR=CodR'} \rho_{X' \leftarrow X}(\text{StessaRivista}))$$

Esercizio

Sia dato lo schema 16.2. Trovare i nomi dei velisti che hanno fatto almeno due prenotazioni.

Svolgimento: Un velista ha effettuato almeno due prenotazioni quando nell'insieme PRENOTAZIONI compaiono due tuple che fanno riferimento allo stesso *Vid* ma hanno *Bid* o *Data* diverse (potrebbe prenotare la stessa barca in giorni diversi o barche diverse il giorno stesso). Si effettua quindi una self-join sull'insieme PRENOTAZIONI:

$$\text{Prenotazioni}_{StessoVelista} \leftarrow \text{Prenotazioni} \bowtie_{\theta} \rho_{X' \leftarrow X}(\text{Prenotazioni})$$

con $\theta = Vid = Vid' \wedge Bid \neq Bid' \vee Data \neq Data'$. Per ottenere informazioni sul velista si effettua una join naturale e si proietta tutto su *VNome*:

$$\Pi_{VNome}(\text{Velisti} \bowtie \text{Prenotazioni}_{StessoVelista})$$

Esercizio

Sia dato lo schema relazionale 16.2. Trovare il codice dei velisti che non hanno mai prenotato barche.

Svolgimento: Si ragiona in logica negata: preso l'insieme di tutti i velisti, si elimineranno quelli che hanno effettuato almeno una prenotazione (il loro id sarà quindi presente in PRENOTAZIONI) per ottenere l'insieme dei velisti che non hanno mai effettuato una prenotazione. Si ha:

$$\text{TuttiVelisti} \leftarrow \Pi_{Vid, VNome}(\text{Velisti})$$

mentre

$$\text{AlmenoUnaPrenotazione} \leftarrow \Pi_{Vid, VNome}(\text{Velisti} \bowtie \text{Prenotazioni})$$

ed infine:

$$\text{NessunPrenotazione} \leftarrow \text{TuttiVelisti} \setminus \text{AlmenoUnaPrenotazione}$$

Esercizio

Sia dato lo schema relazionale 16.2. Trovare il codice dei velisti che non hanno mai prenotato barche rosse.

Svolgimento: Come nell'esercizio precedente si ragiona in logica negata: dall'insieme di tutti i velisti si elimineranno quelli che hanno prenotato barche rosse.

$$BarcheRosse \leftarrow \sigma_{Colore='Rosso'}(Barche)$$

L'insieme dei velisti che hanno prenotato barche di colore rosso è ottenuto facendo:

$$VelistiPrenotazioniBarcheRosse \leftarrow \Pi_{Vid}(Prenotazioni \bowtie BarcheRosse)$$

Infine:

$$TuttiVelisti \leftarrow \Pi_{Vid}(Velisti)$$

e quindi:

$$NessunaBarcaRossaPrenotata \leftarrow TuttiVelisti \setminus VelistiPrenotazioniBarcheRosse$$

Esercizio

Sia dato lo schema relazionale dato dalle tabelle:

SPETTACOLO (CodS, Titolo, Compagnia, Durata)

CARTELLONE (Data, OraInizio, CodS, NomeTeatro)

Tabella 16.3: Schemi SPETTACOLO, CARTELLONE

Trovare il nome delle compagnie che hanno tenuto spettacoli il 15.10.2003, ma non il 16.10.2003.

Svolgimento: L'insieme delle compagnie che hanno fatto spettacoli in data 15.10.2003 si ottiene facendo una join tra spettacolo e la selezione fatta su $Data = 15.10.2003$ in CARTELLONE:

$$Spettacoli_{15.10.03} \leftarrow \Pi_{Compagnia}(Spettacolo \bowtie_{CodS=CodS} \sigma_{Data=15.10.03}(Cartellone))$$

L'insieme delle compagnie che hanno fatto spettacoli il 16.10.2003 si ottiene in modo del tutto analogo. Il risultato è ottenuto quindi facendo una sottrazione tra i due insiemi appena ottenuti:

$$Spettacoli_{15.10.03} \setminus Spettacoli_{16.10.03}$$

Esercizio

Sia dato lo schema relazionale costituito dalle tabelle:

PITTORE (CodP, NomeP, DataNascita, Nazione) QUADRO (CodQ, Titolo, CodP)

ESPOSIZIONE (CodQ, DataInizio, DataFine, NomeGalleria)

Tabella 16.4: Schemi PITTORE, QUADRO, ESPOSIZIONE

Trovare il codice e il titolo dei quadri che sono stati esposti almeno due volte nella stessa galleria.

Svolgimento: Un quadro esposto almeno due volte avrà più di una tupla nella tabella ESPOSIZIONE contenente lo stesso $CodQ$, lo stesso $NomeGalleria$ e diverse date di inizio esposizione. Quindi, facendo una self-join su ESPOSIZIONE, si ottiene:

$$AlmenoDueEsposizioni \leftarrow Esposizione \bowtie_{\theta} \rho_{X' \leftarrow X}(Esposizione)$$

con $\theta = CodQ = CodQ' \wedge NomeGalleria = NomeGalleria' \wedge DataInizio <> DataInizio'$. Per ottenere informazioni rispetto al quadro si farà una ulteriore join:

$$\Pi_{Titolo, CodQ}(AlmenoDueEsposizioni \bowtie_{CodQ=CodQ} Quadro)$$

Esercizio

Sia dato lo schema relazionale costituito dalle tabelle:

VETTURE (Targa, Modello, DataImmatricolazione) REVISIONE (Targa, DataRev, Esito)

Tabella 16.5: Schemi tabelle VETTURE, REVISIONE

Tra tutte le vetture immatricolate dopo il 31/12/2003, selezionare la targa di quelle vetture che sono state revisionate al massimo una volta.

Svolgimento: Una macchina è stata revisionata più di una volta se nella tabella REVISIONE compaiono più tuple aventi la stessa targa e date diverse:

$$VettureConPiRevisioni \leftarrow \Pi_{Targa}(Revisione \bowtie_{\theta} \rho_{X' \leftarrow X}(Revisione))$$

con $\theta : Targa = Targa' \wedge DataRev <> DataRev'$.

L'insieme delle vetture immatricolate dopo il 31/12/2003 si ottiene facendo una selezione:

$$VettureImmatricolateDopo_{31.12.03} \leftarrow \Pi_{Targa}(\sigma_{DataImmatricolazione > 31.12.2003}(Vetture))$$

L'insieme da cercare sarà quindi:

$$VettureImmatricolateDopo_{31.12.03} \setminus VettureConPiRevisioni$$

Esercizio

Dato lo schema:

R(Impiegato, Stipendio)

trovare il minimo stipendio in R.

Svolgimento: Si prende l'insieme di tutti gli stipendi:

$$TuttiStipendi \leftrightarrow \Pi_{Stipendio}(R)$$

Prendo l'insieme degli stipendi più grandi degli altri. Per farlo viene fatta una join tra la relazione R e se stessa, ridenominando tutti gli attributi della copia di R. La condizione di join seleziona quei valori dell'attributo Stipendio che sono maggiori di almeno un valore della sua controparte rinominata.

$$StipendiMassimali \leftarrow \Pi_{Stipendio}(R \bowtie_{Stipendio > Stipendio'} \rho_{X' \leftarrow X}(R))$$

Il minimo stipendio si ottiene per sottrazione grazie al principio di “complementarietà”.

$$TuttiStipendi \setminus StipendiMassimali$$

Esercizio

Si consideri il seguente schema relazionale.

Partite(Data, Stadio, Squadra1, Squadra2, Gol1, Gol2)

Tabella 16.6

Si trovino tutte le squadre che hanno vinto il 16 ottobre 2022.

Svolgimento: Seleziono tutte le squadre di casa che hanno vinto in data 16/10/22:

$$Vincenti1 \leftarrow \Pi_{Squadra1}(\sigma_{Data='2022-10-16' \wedge Gol1 > Gol2}(Partite))$$

e faccio l'analogo sulle squadre ospiti:

$$Vincenti2 \leftarrow \Pi_{Squadra2}(\sigma_{Data='2022-10-16' \wedge Gol2 > Gol1}(Partite))$$

facendo l'unione dei due insiemi appena ottenuti si calcola l'insieme di tutte le squadre che hanno vinto il 16 ottobre 2022:

$$Vincenti1 \cup Vincenti2$$

Esercizio

Trovare le squadre che non hanno mai vinto allo stadio “Maradona”.

Svolgimento: Seleziono tutte le squadre che hanno giocato in casa al “Maradona” e hanno vinto:

$$Vincenti1_{Maradona} \leftarrow \Pi_{Squadra1}(\sigma_{Stadio='Maradona' \wedge Gol1 > Gol2}(Partite))$$

e faccio l'analogo sulle squadre ospiti:

$$Vincenti2_{Maradona} \leftarrow \Pi_{Squadra2}(\sigma_{Stadio='Maradona' \wedge Gol2 > Gol1}(Partite))$$

facendo l'unione dei due insiemi si ottiene l'insieme delle squadre che hanno vinto almeno una volta al “Maradona”.

$$AlmenoUnaVinta_{Maradona} \leftarrow Vincenti1_{Maradona} \cup Vincenti2_{Maradona}$$

Prendiamo adesso l'insieme di tutte le squadre facendo una rapida unione tra l'insieme delle squadre che giocano in casa e quelle che giocano in trasferta (non è detto che i due insiemi siano uguali):

$$TutteLeSquadre \leftarrow \Pi_{Squadra1}(Partite) \cup \Pi_{Squadra2}(Partite)$$

Facendo la sottrazione tra l'insieme di tutte le squadre e l'insieme delle squadre che hanno vinto almeno una volta si ottiene quindi l'insieme delle squadre che non hanno mai vinto al “Maradona”:

$$SquadreSemprePerso_{Maradona} \leftarrow TutteLeSquadre \setminus AlmenoUnaVinta_{Maradona}$$

Esercizio

Trovare le squadre che non hanno mai fatto goal.

Svolgimento: Individuo le squadre che hanno fatto goal in casa almeno una volta:

$$AlmenoUnGoal1 \leftarrow \Pi_{Squadra1}(\sigma_{Gol1>0}(Partite))$$

e faccio lo stesso per le squadre che hanno giocato in trasferta:

$$AlmenoUnGoal2 \leftarrow \Pi_{Squadra2}(\sigma_{Gol2>0}(Partite))$$

Facendo l'unione dei due insiemi si ottiene così l'insieme delle squadre che hanno segnato almeno un goal:

$$AlmenoUnGoal \leftarrow AlmenoUnGoal1 \cup AlmenoUnGoal2$$

Prendo l'insieme di tutte le squadre:

$$TutteLeSquadre \leftarrow \Pi_{Squadra1}(Partite) \cup \Pi_{Squadra2}(Partite)$$

L'insieme delle squadre che non hanno mai segnato un goal sarà quindi:

$$SquadreScarse \leftarrow TutteLeSquadre \setminus AlmenoUnGoal$$

Esercizio

Trovare tutte le squadre che non hanno mai vinto in trasferta.

Svolgimento: L'insieme delle squadre che hanno vinto in trasferta sarà:

$$AlmenoUnaVittoria_{trasferta} \leftarrow \Pi_{Squadra2}(\sigma_{Gol2>Gol1}(Partite))$$

Prendo l'insieme di tutte le squadre:

$$TutteLeSquadre \leftarrow \Pi_{Squadra1}(Partite) \cup \Pi_{Squadra2}(Partite)$$

L'insieme ricercato sarà:

$$TutteLeSquadre \setminus AlmenoUnaVittoria_{trasferta}$$

Esercizio

Sia:

$$S \leftarrow \text{Studente (Matricola, Nome, Cognome)}$$

Vogliamo trovare le coppie (Nome, Cognome) di studenti *omonimi* e visualizzarli come (*matr1, matr2, Nome, Cognome*).

Svolgimento: Dovendo operare sulla stessa tabella si userà la tecnica del “self join”. Rinominiamo quindi la seconda tabella:

$$Studente2 \leftarrow \rho_{Matr2, Nome2, Cognome2 \leftarrow Matricola, Nome, Cognome}(Studente)$$

Adesso possiamo fare una join:

$$\Pi_{Matricola, Matr2, Nome, Cognome}(\sigma_{Nome=Nome2, Cognome=Cognome2 \wedge Matricola <> Matr2}(Studente \times Studente2))$$

o equivalentemente:

$$\Pi_{Matricola, Matr2, Nome, Cognome}(Studente \bowtie_{\theta} Studente2)$$

con

$$\theta = \begin{cases} \text{Nome} = \text{Nome2} \\ \text{Cognome} = \text{Cognome2} \\ \text{Matricola} <> \text{Matr2} \end{cases}$$

Una terza soluzione possibile sarebbe stata questa:

$$Studente2 \leftarrow \rho_{Matr2 \leftarrow Matricola}(S) \\ \sigma_{Matricola <> Matr2}(Studente \bowtie Studente2)$$

che è corretta dal punto di vista dell'algebra relazionale ma non immediatamente traducibile in SQL, dove va necessariamente creata una View di appoggio.

Esercizio

Sia

$$I \leftarrow \text{Impiegati(Id, Impiegato, Progetto)}$$

Data la relazione I, trovare tutti gli impiegati che lavorano allo stesso progetto in cui lavora Rossi.

Svolgimento: Prima di poter effettuare una join è necessario rinominare gli attributi:

$$Impiegati2 \leftarrow \rho_{id1,imp1,prog1 \leftarrow id,impiegato,progetto}(I)$$

sarà quindi:

$$\Pi_{id1,imp1}(\sigma_{progetto=prog1 \wedge impiegato="Rossi" \wedge imp1 <> "Rossi"}(I \times Impiegati2))$$

16.2

ESERCIZI SQL



16.2.1 ■ Attività sportive studentesche

Esercizio

Le informazioni relative alle attività sportive studentesche devono essere analizzate in una base di dati. Gli studenti, dei quali si conservano le informazioni anagrafiche, frequentano gli Istituti superiori, e possono partecipare a una o più manifestazioni sportive (specialità sportive diverse, giornate diverse, campionati che durano per mesi o gare di un giorno). Per ogni attività sportiva le scuole indicano un professore che svolge la funzione di riferimento e di allenatore: ogni professore segue una sola manifestazione, ma una stessa manifestazione può essere eseguita da professori diversi di scuole diverse.

Definire il modello del database e rappresentare le seguenti interrogazioni:

- Numero degli studenti che partecipano a una determinata manifestazione sportiva.
- Elenco anagrafico degli allenatori di un'attività sportiva.
- Elenco delle scuole (denominazione) con il numero di studenti che partecipano alle attività sportive.
- Elenco delle scuole (denominazione, indirizzo, telefono) con studenti che partecipano a una determinata manifestazione sportiva.
- Elenco dei professori (cognome e nome) e scuole (denominazione) di appartenenza in ordine alfabetico.
- Numero degli studenti partecipanti di una determinata scuola per ciascuna delle manifestazioni sportive

Svolgimento: Le entità che possono essere individuate nel problema sono:

- **Istituto**, per gli istituti di appartenenza di studenti e professori;
- **Studente**, per gli studenti delle diverse scuole che partecipano alle manifestazioni sportive;
- **Professore**, per rappresentare i docenti che svolgono funzioni di allenatore;
- **Manifestazione**, per le attività sportive.

Gli attributi di **Istituto** sono: codice dell'istituto, denominazione, indirizzo, telefono. Gli attributi di **Professore** sono: codice del professore, cognome, nome. Gli attributi di **Studente** sono: codice dello studente, cognome, nome, data di nascita, classe. Gli attributi di **Manifestazione** sono: codice della manifestazione, descrizione, luogo, data di inizio per i tornei e i campionati o di svolgimento per le gare di una giornata.

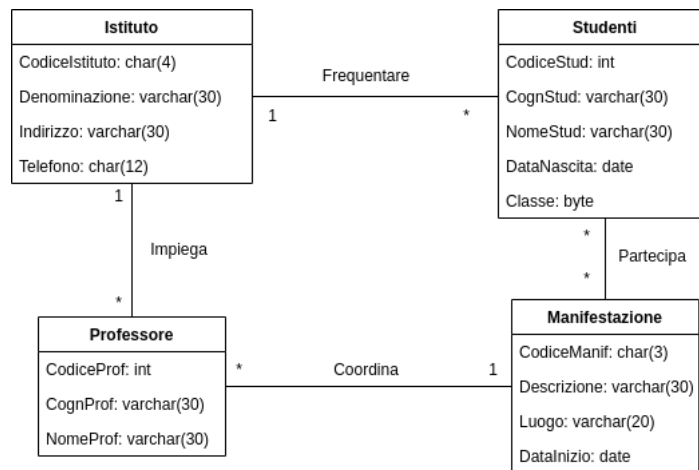
Tra l'entità **Studente** e **Istituto** esiste un'associazione molti a uno, perché gli studenti che partecipano alle manifestazioni possono provenire da più istituti e ogni studente frequenta un solo istituto.

Tra l'entità **Studente** e **Manifestazione** esiste un'associazione molti a molti, perché ogni studente partecipa a una o più manifestazioni e, ovviamente, a ogni manifestazione partecipano più studenti.

Tra l'entità **Manifestazione** e **Professore** si stabilisce un'associazione uno a molti, perché ogni manifestazione può avere più professori come coordinatori, provenienti da scuole diverse; il testo del problema inoltre pone il vincolo che ogni professore possa seguire una sola manifestazione.

Tra l'entità **Istituto** e **Professore** esiste un'associazione uno a molti, perché ogni istituto può avere molti docenti che si occupano delle manifestazioni sportive e ogni docente è titolare di insegnamento in un solo istituto.

Figura 16.1: Modello concettuale per le attività sportive studentesche



Utilizzando le regole di derivazione si passa dal modello concettuale al modello logico, cioè alle tabelle: le associazioni uno a molti si rappresentano introducendo le chiavi esterne nelle tabelle STUDENTI e PROFESSORI per indicare l'istituto di appartenenza; nella tabella PROFESSORI viene aggiunta anche un'altra chiave esterna per l'associazione con l'entità MANIFESTAZIONE; l'associazione molti a molti genera una nuova tabella ISCRIZIONI che contiene le chiavi primarie di STUDENTI e MANIFESTAZIONE più eventuali attributi dell'associazione, per esempio la data di iscrizione.

ISTITUTI (CodiceIstituto, Denominazione, Indirizzo, Telefono)
 STUDENTI (CodiceStud, CognStud, NomeStud, DataNascita, Classe, CodiceIstituto)
 MANIFESTAZIONI (CodiceManif, Descrizione, Luogo, DataInizio)
 PROFESSORI (CodiceProf, CognProf, NomeProf, CodiceIstituto, CodiceManif)
 ISCRIZIONI (CodiceStud, CodiceManif, DataIscrizione)

Tabella 16.7: Modello logico per le attività sportive studentesche

1. Numero degli studenti che partecipano a una determinata manifestazione sportiva.

- (a) Seleziono le manifestazioni sulla base della descrizione ricercata:

$$Temp1 \leftarrow \sigma_{Descrizione='xxx'}(Manifestazioni)$$

- (b) Congiungo Temp1 con ISCRIZIONI mediante una join su CodiceManif:

$$Temp2 \leftarrow Temp1 \bowtie Iscrizioni$$

- (c) Conteggio le righe di Temp2

$$\langle \rangle \tilde{\mathfrak{f}}_{Count(*)}(Temp2)$$

```

1 select Count(*)
2 from Iscrizioni natural join
3 (select * from Manifestazioni where Descrizione= 'xxx');
```



2. Elenco anagrafico degli allenatori di un'attività sportiva.

- (a) Seleziono le attività sportive ricercate:

$$Temp1 \leftarrow \sigma_{Descrizione='xxx'}(Manifestazioni)$$

- (b) Congiungo Temp1 con PROFESSORI per ricavare le informazioni sugli allenatori:

$$Temp2 \leftarrow Temp1 \bowtie Professori$$

- (c) Proietto su CognProf, NomeProf e ordino:

$$\Pi_{CognProf, NomeProf}(Temp2)$$

```

1 select CognProf, NomeProf
2 from Professore natural join
3 (select *
```

```

4  from Manifestazioni
5  where Descrizione='xxx')
6  order by CognProf, NomeProf;

```



3. Elenco delle scuole (denominazione) con il numero di studenti che partecipano alle attività sportive.

- (a) Congiunzione di ISTITUTI e STUDENTI su *CodiceIstituto* per avere le informazioni dell'istituto di appartenenza per ogni studente:

$$Temp1 \leftarrow Istituti \bowtie Studenti$$

- (b) Raggruppamento per denominazione conteggio per sapere quanti studenti partecipano alle manifestazioni studentesche per ogni istituto:

$$Temp2 \leftarrow Denominazione \bowtie_{Count(*)} (Temp1)$$

- (c) Proiezione di Temp2 su *Denominazione* e *Conteggio*

$$\rho_{ElencoScuole(Denominazione, NumStudenti)}(\Pi_{Denominazione, Count(*)}(Temp2))$$

```

1  SELECT Denominazione, Count(*) AS NumStudenti
2  FROM Istituto JOIN Studenti
3  GROUP BY Denominazione;

```



4. Elenco delle scuole (denominazione, indirizzo, telefono) con studenti che partecipano a una determinata manifestazione sportiva.

- (a) Selezione la manifestazione sportiva in MANIFESTAZIONI:

$$Temp1 \leftarrow \sigma_{Descrizione=[manifestazione]}(Manifestazioni)$$

- (b) Congiungo Temp1 con Iscrizioni su *CodiceManif*:

$$Temp2 \leftarrow Temp1 \bowtie Iscrizioni$$

- (c) Congiungo Temp2 e Studenti su *CodiceStud*:

$$Temp3 \leftarrow Temp2 \bowtie Studenti$$

- (d) Congiungo Temp3 e Istituti su *CodiceIstituto*:

$$Temp4 \leftarrow Temp3 \bowtie Istituti$$

- (e) Raggruppamento Temp4 per *Denominazione*

$$Temp5 \leftarrow Denominazione \bowtie (Temp4)$$

- (f) Proietto Temp5 su *Denominazione*, *Indirizzo* e *Telefono*:

$$\Pi_{Denominazione, Indirizzo, Telefono}(Temp5)$$

```

1  SELECT Denominazione, Indirizzo, Telefono
2  FROM (((
3  Select *
4  FROM Manifestazioni
5  WHERE Descrizione = [manifestazione] ) JOIN Iscrizioni) JOIN Studenti) JOIN Istituti
6  GROUP BY Denominazione

```



5. Elenco dei professori (cognome e nome) e scuole (denominazione) di appartenenza in ordine alfabetico.

- (a) Congiungo Istituti e Professori su *CodiceIstituto*:

$$Temp1 \leftarrow Istituti \bowtie Professori$$

(b) Proietto Temp1 su *CognProf*, *NomeProf*, *Denominazione* ordinato su *CognProf*, *NomeProf*:

$$\Pi_{CognProf, NomeProf, Denominazione}(Temp1)$$

```
1 SELECT P.CognProf, p.NomeProf, I.Denominazione
2 FROM Istituti as i JOIN Professori ad P
3 ORDER BY P.CognProf, P.NomeProf
```



6. Numero degli studenti partecipanti di una determinata scuola per ciascuna delle manifestazioni sportive.

(a) Selezione l'istituto per *Denominazione*=*[scuola]*:

$$Temp1 \leftarrow \sigma_{Denominazione=[scuola]}(Istituto)$$

(b) Congiungo Temp1 e Studenti su *CodiceIstituto*:

$$Temp2 \leftarrow Temp1 \bowtie Studenti$$

(c) Congiungo Temp2 e Iscrizioni su *CodiceStud*:

$$Temp3 \leftarrow Temp2 \bowtie Iscrizioni$$

(d) Congiungo Temp3 e Manifestazioni su *CodiceManif*:

$$Temp4 \leftarrow Temp3 \bowtie Manifestazioni$$

(e) Raggruppo Temp4 per *Descrizione* ed effettuo un conteggio:

$$Temp5 \leftarrow_{Descrizione} \mathfrak{F}_{Count(*)}(Temp4)$$

(f) Proietto Temp5 su *Descrizione* e *Conteggio*:

```
1 SELECT Descrizione, Count(*) as NumStudenti
2 FROM Istituto JOIN Studenti JOIN Iscrizioni JOIN Manifestazioni
3 WHERE Denominazione=[scuola]
4 GROUP BY Descrizione;
```



16.2.2 ■ Rappresentanti e fatture ai clienti

Esercizio

Si vogliono organizzare le informazioni relative ai clienti, ai loro rappresentanti e alle fatture emesse. Ogni rappresentante ha tanti clienti, mentre a un cliente corrisponde un solo rappresentante. Per semplicità si supponga di registrare solo la data e l'importo totale di ogni fattura, senza specificare le righe di dettaglio sugli articoli venduti.

Definire il modello del database e rappresentare le seguenti interrogazioni:

- Elenco dei clienti di un rappresentante;
- Numero dei clienti affidati a un determinato rappresentante;
- Cognome e nome del rappresentante di un determinato cliente;
- Elenco delle fatture di un codice cliente prefissato, in ordine di data;
- Fatturato raggruppato per codice rappresentante;
- Numero delle fatture emesse raggruppate per codice rappresentante;
- Nome del cliente e data di emissione per fattura con importo massimo;
- Nome del rappresentante corrispondente alla fattura con importo massimo;
- Codice e nome dei clienti per i quali il fatturato totale è superiore a una cifra prefissata;
- Codice e nome dei rappresentanti per i quali il numero delle fatture è superiore a un numero prefissato.

Svolgimento: Le entità che possono essere individuate nel problema sono:

1. CLIENTE, per i clienti ai quali vengono emesse le fatture;
2. RAPPRESENTANTE, per gli agenti ai quali sono assegnati i clienti;

3. FATTURA, per le vendite effettuate dai rappresentanti;

Gli attributi di CLIENTE sono: codice del cliente, ragione sociale, indirizzo, telefono, partita IVA.

Gli attributi di RAPPRESENTANTE sono: codice del rappresentante, cognome, nome, zona assegnata.

Gli attributi di FATTURA sono: numero della fattura, data di emissione, importo.

Tra l'entità CLIENTE e FATTURA esiste un'associazione uno a molti, perché per lo stesso cliente possono essere emesse molte fatture, ma ciascuna fattura è ricevuta solo da un cliente.

Tra l'entità RAPPRESENTANTE e CLIENTE si stabilisce un'associazione uno a molti perché ogni rappresentante ha tanti clienti, mentre a un cliente corrisponde un solo rappresentante.

I dati del problema si rappresentano graficamente mediante il modello UML in Figura 16.2.

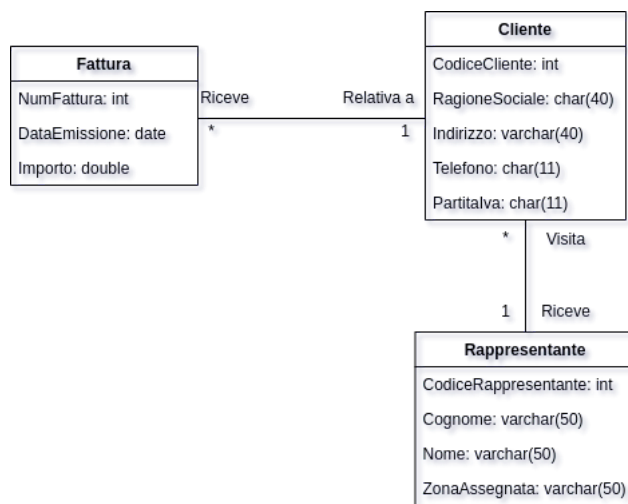


Figura 16.2: Schema concettuale del problema

Lo schema concettuale viene tradotto nello schema logico seguendo le regole viste nel capitolo 5. Si introducono quindi le chiavi esterne delle entità forti all'interno delle entità deboli ottenendo così lo schema 16.8.

Tabella 16.8: Modello logico del problema

RAPPRESENTANTI	(<u>CodiceRap</u> , CognomeRap, NomeRap, Zona)
FATTURE	(<u>NumeroFatt</u> , DataFatt, Importo, CodiceCli)
CLIENTE	(<u>CodiceCli</u> , RagioneSociale, Indirizzo, Telefono, PartitaIVA, CodiceRap)

1. Elenco dei clienti di un rappresentante:

(a) Selezione del rappresentante:

$$Temp1 \leftarrow \sigma_{CodiceRap=[Rappresentante\ da\ controllare]}(Rappresentanti)$$

(b) Congiungo Temp1 e Clienti su *CodiceRap*:

$$Temp2 \leftarrow Temp1 \bowtie Cliente$$

(c) Proietto Temp2 su *CodiceCli*, *RagioneSociale*, *Indirizzo*, *Telefono*:

$$\Pi_{CodiceCli, RagioneSociale, Indirizzo, Telefono}(Temp2)$$

```

1 SELECT CodiceCli, RagioneSociale, Indirizzo, Telefono
2 FROM (
3 SELECT *
4 FROM Rappresentanti
5 WHERE CodiceRap=[rappresentante da controllare]) JOIN Cliente

```



2. Numero dei clienti affidati a un determinato rappresentante:

(a) Selezione del rappresentante:

$$Temp1 \leftarrow \sigma_{CodiceRap=[Rappresentante\ da\ controllare]}(Rappresentanti)$$

(b) Congiungo Temp1 e Clienti su *CodiceRap*:

$$Temp2 \leftarrow Temp1 \bowtie Cliente$$

(c) Conto le righe di Temp2:

```
1 SELECT Count(*)
2 FROM (
3 SELECT *
4 FROM Rappresentanti
5 WHERE CodiceRap=[rappresentante da controllare]) JOIN Cliente;
```



3. Cognome e nome del rappresentante di un determinato cliente:

(a) Selezione il cliente da CLIENTI:

$$Temp1 \leftarrow \sigma_{CodiceCli=[cliente\ specifico]}(Clienti)$$

(b) Congiungo Temp1 e Rappresentanti:

$$Temp2 \leftarrow Temp1 \bowtie Rappresentanti$$

(c) Proietto Temp2 su *CognomeRap* e *NomeRap*:

$$\Pi_{CognomeRap, NomeRap}(Temp2)$$

```
1 SELECT CognomeRap, NomeRap
2 FROM
3 (SELECT *
4 FROM Clienti
5 WHERE CodCli=[Cliente specifico]) JOIN Rappresentanti;
```



4. Elenco delle fatture di un codice cliente prefissato, in ordine di data:

(a) Selezione delle fatture per *CodCliente*:

$$Temp1 \leftarrow \sigma_{CodCliente=[cliente\ da\ controllare]}(Fatture)$$

(b) Proiezione di Temp1 su * ordinato per data fattura:

```
1 SELECT *
2 FROM Fattura
3 WHERE CodCliente= [cliente da controllare]
4 ORDER BY DataFatt;
```



5. Fatturato raggruppato per codice rappresentante:

(a) Congiungo FATTURATO e CLIENTI:

$$Temp1 \leftarrow Fatture \bowtie Clienti$$

(b) Raggruppamento di Temp1 per *CodiceRap* e calcolo somma Importo

$$Temp2 \leftarrow \rho_{Count(Importo) \leftarrow Fatturato} (CodiceRap \bowtie_{Count(Importo)} (Temp1))$$

(c) Proiezione di Temp2 su *CodiceRap*, *Fatturato*:

$$\Pi_{CodiceRap, Fatturato}(Temp2)$$

```
1 SELECT CodiceRap, Sum(Importo) as Fatturato
2 FROM Fatture JOIN Rappresentanti
3 GROUP BY CodiceRap;
```



6. Numero delle fatture emesse raggruppate per codice rappresentante:

- (a) Congiungo FATTURATO e CLIENTI:

$$Temp1 \leftarrow Fatture \bowtie Clienti$$

- (b) Raggruppamento di Temp1 per *CodiceRap*

$$Temp2 \leftarrow \rho_{NumFatture \leftarrow Count(*)}(CodiceRap \tilde{\bowtie} Count(*) (Temp1))$$

- (c) Proiezione di Temp2 su *CodiceRap* e *NumFatture*:

$$Temp3 \leftarrow \Pi_{CodiceRap}(Temp2)$$

```
1 SELECT CodiceRap, Count(*) as NumFatture
2 FROM Fatture JOIN Clienti
3 GROUP BY CodiceRap;
```



7. Nome del cliente e data di emissione per fattura con importo massimo:

- (a) Calcolo il massimo importo tra le fatture:

$$ValMax = \langle \rangle \tilde{\bowtie}_{Max(Importo)}(Fatture)$$

- (b) Selezione le fatture con importo uguale a ValMax:

$$Temp1 \leftarrow \sigma_{Importo=ValMax}(Fatture)$$

- (c) Congiungo Temp1 e Clienti:

$$Temp2 \leftarrow Temp1 \bowtie Clienti$$

- (d) Proietto Temp2 su *CodiceCli*, *RagioneSociale*, *DataFatt*, *Importo*:

$$Temp3 \leftarrow \Pi_{CodiceCli, RagioneSociale, DataFatt, Importo}(Temp2)$$

```
1 Select CodiceCli, RagioneSociale, DataFatt, Importo
2 FROM Clienti JOIN (SELECT * FROM Fatture
3 WHERE Importo = (SELECT Max(Importo) FROM Fatture));
```



8. Nome del rappresentante corrispondente alla fattura con importo massimo:

```
1 SELECT NomeRap, CognomeRap
2 FROM (SELECT * FROM Fatture
3 WHERE Importo = (SELECT Max(Importo) FROM Fatture)) JOIN Clienti JOIN Rappresentanti;
```



9. Codice e nome dei clienti per i quali il fatturato totale è superiore a una cifra prefissata:

- (a) Congiungo CLIENTI e FATTURE:

$$Temp1 \leftarrow Clienti \bowtie Fatture$$

- (b) Raggruppo per *CodiceCli* e conto l'importo massimo:

$$Temp2 \leftarrow \rho_{SommaImporti \leftarrow Sum(Importo)}(CodiceCli \tilde{\bowtie} Sum(Importo) (Temp1))$$

- (c) Selezione di Temp2 per *SommaImporti* > [cifra prefissata]

```
1 SELECT CodiceCli
2 FROM Clienti JOIN Fatture
3 GROUP BY CodiceCli
4 HAVING Sum(Importo)>[cifra prefissata];
```



10. Codice e nome dei rappresentanti per i quali il numero delle fatture è superiore a un numero prefissato:

(a) Congiungo Fatture e Cliente:

$$Temp1 \leftarrow Fattura \bowtie Cliente$$

(b) Congiungo Temp1 e Rappresentanti:

$$Temp2 \leftarrow Temp1 \bowtie Rappresentanti$$

(c) Raggruppo Temp2 per *CodiceRap* e conto il numero delle fatture:

$$Temp3 \leftarrow \rho_{NumFatture \leftarrow Count(*)}(CodiceRap \bowtie Count(*) (Temp2))$$

(d) Selezione di Temp3 per *NumFatture* > [numero prefissato]

(e) Proiezione di Temp4 su *CodiceRap*, *CognomeRap*, *NomeRap*

```
1 SELECT CodiceRap, CognomeRap, NomeRap
2 FROM (Fattura JOIN Cliente) JOIN Rappresentanti
3 GROUP BY CodiceRap
4 HAVING Count(*) > [numero prefissato];
```



16.2.3 Interrogazioni semplici

Esercizio

Si consideri lo schema relazionale PARTITE (Tabella 16.6). Traduciamo in SQL l'espressione calcolata per trovare le squadre che non hanno mai vinto in trasferta.

Svolgimento: L'insieme delle squadre che hanno vinto in trasferta era:

$$AlmenoUnaVittoria_{trasferta} \leftarrow \Pi_{Squadra2}(\sigma_{Gol2 > Gol1}(Partite))$$

Mentre l'insieme di tutte le squadre:

$$TutteLeSquadre \leftarrow \Pi_{Squadra1}(Partite) \cup \Pi_{Squadra2}(Partite)$$

L'insieme ricercato era quindi:

$$TutteLeSquadre \setminus AlmenoUnaVittoria_{trasferta}$$

Traducendo in SQL:

```
1 (
2 (SELECT P.Squadra1 FROM Partite AS P)
3 UNION
4 (SELECT P.Squadra2 FROM Partite AS P)
5 )
6 EXCEPT
7 (
8 SELECT P.Squadra2
9 FROM Partite AS P
10 WHERE P.Gol2 > P.Gol1
11 )
```



Esercizio

Si considerino le tabelle 16.9. Trovare tutti gli studenti che hanno fatto tutti gli esami fatti dallo studente con matricola 100.

Studenti	Matricola	Nome	Cognome
	100	Thomas	Andersen
	200	Robin	Hobb
	300	Emmett	Brown
	400	Robin	Hobb

Esami	Matr_Stud	Voto	Corso
	100	28	BDD
	100	30	PR1
	200	25	BDD
	300	30	PR1
	400	19	ALG

Tabella 16.9: Tabella STUDENTI e ESAMI

Svolgimento: Si ha:

```
1 SELECT *
2 FROM Studenti AS S
3 WHERE NOT EXISTS
4 -- Ragionando in logica negata non deve esistere un esame fatto da 100 e non da uno studente generico
5 (
6 SELECT E.Corso --Cerco gli esami fatti dalla matricola 100
7 FROM Esami AS E
8 WHERE E.Matr_Stud = 100
9 AND E.Corso NOT IN
10 -- Ognuno degli esami fatti da 100 deve essere nell'insieme degli esami fatto dallo studente generico
11 (
12 SELECT E2.Corso --Esami sostenuti dallo studente generico
13 FROM Esami AS E2
14 WHERE E2.Matr_Stud = S.Matricola
15 )
16 )
```

